

TIS Project Reference Booklet

Documentation version: 3.1

Generated: 2026-09-04 15:33 Middle East Daylight Time

Branch: dev

Git commit SHA: f089229

Source of truth: Markdown files under docs/ are authoritative. This PDF is a generated snapshot and must not be edited manually.

Use the table of contents and PDF bookmarks to move directly to each source document. Major document headings are available as child bookmarks.

How to Use This Handbook

Start with the TIS KMS Navigation Guide to choose the smallest useful reading path for your role or task. Use this booklet when a portable, reviewable snapshot is more convenient than browsing the Markdown sources.

Navigate

Use the table of contents for document-level page numbers. In PDF viewers that support outlines, open the bookmarks panel to browse every source document and its major headings.

Verify

The title page records the documentation version, generated timestamp, branch, and Git commit SHA. The generated manifest records source hashes and each document's starting PDF page.

Source Of Truth

Markdown files under docs/ remain authoritative. This PDF is generated and must never be edited manually. When source documents change, run the approved KMS synchronization command to regenerate this snapshot and its manifest.

Table of Contents

TIS Documentation Index 8

TIS KMS Navigation Guide 11

TIS AI Project Context 16

TIS Documentation Update Policy 44

TIS Master Context 47

TIS Project State 72

TIS Change History 95

TIS Architecture Decision Records 143

TIS Engineering Handbook 145

TIS Module Map 147

TIS Repository Architecture 169

TIS User And System Flows 181

TIS Database Architecture Overview 209

TIS Development Standards 225

TIS UI UX Design Philosophy 229

TIS Product Roadmap 234

TIS Rejected Architectural Decisions 239

TIS Visual Documentation Guide 242

TIS AI Optimization Guide 246

TIS Project Governance 250

TIS Knowledge Lifecycle 254

TIS Documentation Automation 257

TIS Knowledge Impact Assessment Standard 262

TIS Self-Evolving Workflow	265
TIS Documentation Dependency Map	268
TIS AI Coding Workflow	271
TIS Future Automation Roadmap	274
Separate Next.js Landing Website	277
Separate SaaS Identity And Operational Users	278
Paddle Payment Architecture	279
Webhook-Only Payment Confirmation	281
Delayed Tenant Provisioning After Verified Payment	283
Documentation As Source / Knowledge Management System	284
Landing Page Visual System Strategy	286
Workspace Classification Foundation	287
Commercial State And Entitlement Resolution	289
Review-Only SaaS Demo Requests Before Provisioning	291
Demo Workspace Provisioning And Commercial Source Links	292
Seven-Day Demo Lifecycle And Access Enforcement	293
Demo-To-Paid Workspace Conversion	294
Demo Customer Communications And Approval Orchestration	295
Centralized AI Entitlements And Usage Accounting	296
ADR 0016 - Pre-Deploy Database Migration Boundary	297
Platform Owner Demo Operations And Access Profiles	298
Unified Commercial Access And Capacity Authority	300
Secure Promo Code Definition And Governance	302
Promo Redemption And Commercial Grants	304
Existing Workspace Conversion Read-Only Audit	306

Controlled Existing Workspace Owner Alignment And Activation-Required Conversion	307
Existing Workspace Paid Activation Through Paddle	309
Promo Expiry Recovery And Paid Continuation	311
Capacity-Based Packaging And Common Customer Feature Baseline	312
Versioned, Constraint-Based Smart Timetable Generation	314
Academic Calendar History	319
AI Entitlements History	320
M8B9 Demo Operations, Notifications, And Testing	321
Common Customer Demo Feature Baseline	322
KMS v3 Engineering Handbook	323
KMS v3 Phase 3B Engineering Layers	325
KMS v3 Phase 3C Governance And AI Traceability	327
KMS v3 Phase 3D Lifecycle Foundation	329
Production Memory Stability Guardrails	331
Automatic KMS Synchronization Enforcement	333
KMS Phase 7A Navigation Foundation	335
KMS Phase 7B Professional PDF Navigation	336
KMS Phase 7D Navigation And Catalog Enforcement	337
Claude Code KMS Configuration	339
Engineering Handbook History	340
Landing CTA Consolidation And Demo Domain Policy	341
M8 Landing Integration Open Account	342
Landing Page History	344
Platform Owner Knowledge Center	345
KMS Phase 7C Knowledge Center Navigation	347

Platform Knowledge Center History	349
KMS Foundation	350
Provisioning History	352
TIS Module History	353
SaaS Onboarding History	354
Subscription History	363
Smart Timetable Stage 2 Version Foundation	369
Smart Timetable Stage 3 Readiness And Slot Semantics	370
Smart Timetable Stage 3.5 Composed Timeline	371
Smart Timetable Stage 4 Version Publication	372
Smart Timetable Stage 5.1 Generator	373
On-Demand Timetable Generation Workflow	374
Smart Timetable Stage 5.2 Simplified Workflow And Published Visibility	375
Explicit Planning Subject Demand Foundation	376
Planning Subject Demand Consumer Transition	377
Atomic Curriculum Adjustment Apply	378
Read-Only Curriculum Adjustment Preview	379
Curriculum Transfer and Subjects Display Correction	380
Guided Curriculum Adjustment UI	381
Reduce-Only Curriculum Adjustment	382
Teacher Scheduling Rules	383
Advanced Timetable Release 1 Reconciliation And Solver Adapter	385
Structured Timetable Conflict Evidence	386
Timetable Lesson Requirement Projection	387
Professional Timetable Lifecycle UX	388

Planning And Subject Delete Dependency Guards	389
Return-To Reopen-On-Load UI Pattern	390
Planning Subject Requirement Removal (Single And Bulk)	391
Workforce Planning History	393
Workspace Classification History	395
TIS Landing Page Source of Truth	396
TIS Landing Page Master Content	397
TIS Location Data Roadmap	405

TIS Documentation Index

docs/README.md

TIS Documentation

This folder is the source of truth for the TIS Knowledge Management System (KMS).

Markdown files are authoritative. The PDF booklet is a generated snapshot and must never be edited manually.

Repository enforcement begins with root `AGENTS.md` and `.kms-impact.yml`. `scripts/check_kms_impact.py` validates the declaration, changed paths, declared Markdown updates, and generated artifacts. It never rewrites documentation.

Choose A Reading Path

Open the [KMS Navigation Guide](KMS_NAVIGATION.md) to select a focused reading path by role or task. It covers human and AI onboarding, SaaS and subscriptions, operational modules, database work, Platform Owner tools, the public landing website, location data, design, architecture decisions, and review/KIA work.

For a fast baseline, read:

- 1 [AI Project Context](AI_PROJECT_CONTEXT.md)
- 2 [TIS Master Context](TIS_MASTER_CONTEXT.md)
- 3 [Project State](PROJECT_STATE.md)
- 4 [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md)

Then follow the relevant task path rather than loading the entire handbook.

Core Documents

- 1 [KMS Navigation Guide](KMS_NAVIGATION.md): role-based and task-based routes through the KMS.
- 1 [AI Project Context](AI_PROJECT_CONTEXT.md): compact onboarding file for future Codex and ChatGPT conversations.
- 1 [TIS Master Context](TIS_MASTER_CONTEXT.md): long-term product, architecture, SaaS, workflow, roadmap, and critical-rules source of truth.
- 1 [Project State](PROJECT_STATE.md): living status for branch strategy, priorities, milestones, known issues, and next work.
- 1 [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md): non-negotiable KMS and Knowledge Impact Assessment policy.
- 1 [Change History](CHANGE_HISTORY.md): chronological summary of meaningful changes.

Engineering Handbook

Use the [Engineering Handbook Index](engineering/README.md) for the full engineering document map.

Primary engineering references:

- 1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

- 1 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- 1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
- 1 [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)
- 1 [Product Roadmap](engineering/PRODUCT_ROADMAP.md)

Decision And History Documents

- 1 [ADR Index](adr/README.md): Architecture Decision Records for major accepted decisions.
- 1 [Rejected Decisions](engineering/REJECTED_DECISIONS.md): significant alternatives and why they were declined.
- 1 [Module History Index](history/README.md): module-based history preserving deeper before/after context.

Current module history areas:

- 1 [Subscriptions](history/subscriptions/README.md)
- 1 [Landing Page](history/landing-page/README.md)
- 1 [Academic Calendar](history/academic-calendar/README.md)
- 1 [Workforce Planning](history/workforce-planning/README.md)
- 1 [SaaS Onboarding](history/saas-onboarding/README.md)
- 1 [Provisioning](history/provisioning/README.md)
- 1 [Platform Knowledge](history/platform-knowledge/README.md)
- 1 [Engineering Handbook](history/engineering-handbook/README.md)

Supporting Documents

- 1 [Location Data Roadmap](location-data-roadmap.md): location data roadmap and related implementation notes.
- 1 [Landing Page Source of Truth](marketing/landing_page_source_of_truth.md): boundary between the public Next.js landing website and the FastAPI application portal.
- 1 [Landing Page Master Content](marketing/tis_landing_page_master_content.md): approved marketing foundation and landing page content direction.

Generated Snapshot

Generated files:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf
- 1 static/docs/docs_manifest.json

The PDF is generated from approved Markdown docs. It includes a handbook-use page, a page-numbered table of contents, source-document bookmarks, and child bookmarks for major headings. The manifest records the generated timestamp, documentation version, branch, commit SHA, included sources, source hashes, and each source document's starting PDF page.

Platform owners can use the protected Knowledge Center at /platform/knowledge to search the manifest-backed library, filter by category, module, or freshness, and open a source at its recorded booklet page. The owner utility remains read-only and never links directly to the static PDF path.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Complete read-only validation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

The command reuses the dependency-light generator and strict impact checker. It must not require LaTeX, Playwright, Chromium, network calls, or system fonts.

Knowledge Impact Assessment

Every future implementation report must include:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until KIA is assessed. If included source docs change, regenerate the PDF.

Update `.kms-impact.yml` during every task. CI validates it on pull requests and dev; deployment from master depends on the same KMS gate.

Phase Boundary

Phase 2A and Phase 2B establish KMS governance, ADRs, module history, AI context, and PDF/manifest generation.

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation with self-evolving documentation workflow, KIA standard, dependency map, AI coding workflow, and future automation roadmap. The Platform Owner Knowledge Center is implemented as a read-only owner utility. Do not add a regenerate button or app-side Markdown rewriting unless explicitly approved.

TIS KMS Navigation Guide

docs/KMS_NAVIGATION.md

Use this guide to choose the smallest useful reading path for the work in front of you. Markdown under docs/ remains authoritative; this file organizes that knowledge but does not replace module, architecture, decision, or history documents.

Start Here

For any engineering task, begin with:

- 1 [AI Project Context](AI_PROJECT_CONTEXT.md) for the compact system orientation and current guardrails.
- 2 [TIS Master Context](TIS_MASTER_CONTEXT.md) for durable product, architecture, SaaS, and workflow truth.
- 3 [Project State](PROJECT_STATE.md) for current priorities, completed milestones, known issues, and next work.
- 4 [Documentation Update Policy](DOCUMENTATION_UPDATE_POLICY.md) for KIA and completion requirements.
- 5 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md) before changing implementation code.

Then select the task path below. Read relevant ADRs and module history before editing.

New Human Developer

Read in this order:

- 1 [Engineering Handbook Index](engineering/README.md)
- 2 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 3 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
- 4 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
- 5 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- 6 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
- 7 [Project Governance](engineering/PROJECT_GOVERNANCE.md)

New AI Coding Conversation

Read in this order:

- 1 [AI Project Context](AI_PROJECT_CONTEXT.md)
- 2 [AI Optimization Guide](engineering/AI_OPTIMIZATION_GUIDE.md)
- 3 [AI Coding Workflow](engineering/AI_CODING_WORKFLOW.md)
- 4 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
- 5 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 6 Relevant [ADRs](adr/README.md) and [module history](history/README.md)

Also follow the repository-level instructions in root AGENTS . md.

SaaS Accounts And Onboarding

Read:

- 1 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- 1 [ADR 0002: Separate SaaS Identity and Operational Users](adr/0002-separate-saas-identity-and-operational-users.md)
- 1 [ADR 0005: Delayed Tenant Provisioning](adr/0005-delayed-tenant-provisioning-after-verified-payment.md)
- 1 [SaaS Onboarding History](history/saas-onboarding/README.md)
- 1 [Provisioning History](history/provisioning/README.md)

Guardrail: SaaS accounts, platform identities, and operational users are separate security and ownership concepts.

Subscriptions, Billing, And Paddle

Read:

- 1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 1 [User and System Flows](engineering/USER_AND_SYSTEM_FLOWS.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- 1 [ADR 0003: Paddle Payment Architecture](adr/0003-paddle-payment-architecture.md)
- 1 [ADR 0004: Webhook-Only Payment Confirmation](adr/0004-webhook-only-payment-confirmation.md)
- 1 [ADR 0005: Delayed Tenant Provisioning](adr/0005-delayed-tenant-provisioning-after-verified-payment.md)
- 1 [Subscriptions History](history/subscriptions/README.md)

Guardrail: provider-confirmed state remains authoritative for paid entitlements, payment completion, proration, invoices, and lifecycle reconciliation.

AI Entitlements

Read:

- 1 [ADR 0015: Centralized AI Entitlements And Usage Accounting](adr/0015-centralized-ai-entitlements-and-usage-accounting.md)
- 1 [AI Entitlements History](history/ai-entitlements/README.md)
- 1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

Guardrail: routes never manipulate AI usage directly; permissions, commercial state, registry policy, and successful-use consumption remain separate.

Operational Academic Modules

For teachers, subjects, sections, workforce planning, timetables, calendars, observations, and reports, read:

- 1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)
- 1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)
- 1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
- 1 [Workforce Planning History](history/workforce-planning/README.md)

1 [Academic Calendar History](history/academic-calendar/README.md)

Guardrail: preserve tenant isolation, branch scope, role permissions, and active academic-year context.

Database, Migrations, And Tenant Isolation

Read:

1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)

1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)

1 [ADR 0002: Separate SaaS Identity and Operational Users](adr/0002-separate-saas-identity-and-operational-users.md)

1 [Project State](PROJECT_STATE.md)

Guardrail: do not modify models, migrations, tenant boundaries, or `tis.db` without explicit approval and focused validation.

Platform Owner And Knowledge Center

Read:

1 [TIS Module Map](engineering/TIS_MODULE_MAP.md)

1 [Project Governance](engineering/PROJECT_GOVERNANCE.md)

1 [ADR 0006: Documentation as Source](adr/0006-documentation-as-source-knowledge-management-system.md)

1 [Documentation Automation](engineering/DOCUMENTATION_AUTOMATION.md)

1 [Knowledge Impact Assessment Standard](engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md)

1 [Knowledge Lifecycle](engineering/KNOWLEDGE_LIFECYCLE.md)

1 [Platform Knowledge History](history/platform-knowledge/README.md)

1 [Engineering Handbook History](history/engineering-handbook/README.md)

Guardrail: KMS Markdown is reviewed source material, the PDF is generated, and protected documentation must remain owner-only.

Public Landing Website

Read:

1 [Landing Page Source of Truth](marketing/landing_page_source_of_truth.md)

1 [Landing Page Master Content](marketing/tis_landing_page_master_content.md)

1 [ADR 0001: Separate Next.js Landing Website](adr/0001-separate-nextjs-landing-website.md)

1 [ADR 0007: Landing Page Visual System Strategy](adr/0007-landing-page-visual-system-strategy.md)

1 [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)

1 [Landing Page History](history/landing-page/README.md)

Guardrail: the public website lives in `tis-landing-website/`; legacy FastAPI landing files are not its source of truth.

Location Data

Read:

- 1 [Location Data Roadmap](location-data-roadmap.md)
- 1 [Repository Architecture](engineering/REPOSITORY_ARCHITECTURE.md)
- 1 [Database Architecture Overview](engineering/DATABASE_ARCHITECTURE_OVERVIEW.md)

Guardrail: preserve runtime memory limits, local dataset boundaries, and record-level manual fallbacks.

UI And Visual Design

Read:

- 1 [UI/UX Design Philosophy](engineering/UI_UX_DESIGN_PHILOSOPHY.md)
- 1 [Visual Documentation Guide](engineering/VISUAL_DOCUMENTATION_GUIDE.md)
- 1 [Development Standards](engineering/DEVELOPMENT_STANDARDS.md)
- 1 the relevant module or landing-page path above

Architecture Decisions And Alternatives

Use:

- 1 [ADR Index](adr/README.md) for accepted architectural and product decisions.
- 1 [Rejected Decisions](engineering/REJECTED_DECISIONS.md) for significant alternatives that should not be reopened without new evidence.
- 1 [Documentation Dependency Map](engineering/DOCUMENTATION_DEPENDENCY_MAP.md) to determine which KMS records a change should propagate through.

Review, Release, And KIA

Read:

- 1 [Project State](PROJECT_STATE.md)
- 1 [Change History](CHANGE_HISTORY.md)
- 1 [Project Governance](engineering/PROJECT_GOVERNANCE.md)
- 1 [Self-Evolving Workflow](engineering/SELF_EVOLVING_WORKFLOW.md)
- 1 [Knowledge Impact Assessment Standard](engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md)
- 1 [Documentation Automation](engineering/DOCUMENTATION_AUTOMATION.md)

Use `python scripts/kms.py sync` after reviewed documentation changes and `python scripts/kms.py check` for complete read-only validation.

Document Types

- 1 **Core context:** durable truth, current state, policy, and chronological change summary.
- 1 **Engineering handbook:** modules, repository boundaries, flows, data architecture, standards, design, governance, and roadmap.

- 1 **ADRs:** accepted architectural and product decisions.
- 1 **Rejected decisions:** important alternatives and why they were declined.
- 1 **Module history:** deeper before-and-after records for a specific area.
- 1 **Marketing documents:** approved public positioning and landing implementation boundaries.
- 1 **Generated artifacts:** the PDF snapshot and manifest; never edit them manually.

Using The Generated Booklet

The generated PDF includes a "How to Use This Handbook" page and a page-numbered table of contents. Open the PDF viewer's bookmarks or outlines panel to browse every source document and its major headings. Source starting pages are also recorded in `static/docs/docs_manifest.json`.

Markdown remains authoritative. PDF navigation is generated from the fixed source order and must be refreshed through `python scripts/kms.py sync` whenever included source documents change.

Navigation Integrity

Every Markdown link in this guide must:

- 1 use a relative POSIX-style path,
- 1 resolve to a Markdown file inside `docs/`,
- 1 point to an existing authoritative document included in the booklet source list,
- 1 avoid external URLs, root-relative paths, parent-directory escapes, and direct generated-artifact links.

`python scripts/kms.py check` enforces these rules. References to repository files outside `docs/`, such as `root AGENTS.md`, must remain code paths rather than clickable links in this guide.

TIS AI Project Context

docs/AI_PROJECT_CONTEXT.md

Talent Assessment Cycle And Frozen Population Foundation

Milestone M4 implements `TalentAssessmentCycle` as one `SchoolGroup`-wide canonical authority bound to a `TalentProgram`, `AcademicYear`, and exact `FrameworkVersion`. A `Cycle` deliberately has no `branch_id`. Its lifecycle is one-way draft -> open -> closed: there is no reopen, rollback, deletion, post-Open population mutation, late-entry exception, or assessor assignment. Draft metadata uses expected-revision stale-write protection. The explicit `population_effective_at` may change only in Draft and is required for preview and Open; no hidden current-time rule is used.

Draft preview dynamically resolves `Students` with an effective-dated `StudentAcademicPlacement` at that exact instant, in the `Cycle` `AcademicYear` and the enabled `Program` annual configuration's eligible KG/1-12 grades. `Student.status` is current mutable state with no effective-dated history, so it never gates this historical-instant derivation - only stable tenant/ identity alignment between `Student` and `Placement` is enforced; a `Student`'s present-day status change never silently alters an already-defined historical population. Opening requires an `ActiveProgram` and exact `ActiveFramework`, locks and revalidates the `Cycle`, derives the complete `SchoolGroup` population, inserts frozen members, stores the full count and SHA-256 population fingerprint, and marks the `Cycle` `Open` in one transaction. Each member preserves `Student` and `Placement` identity plus `AcademicYear`, frozen `Branch`, grade, section, nullable `PlanningSection` provenance, and effective/frozen timestamps. Current `Placement`, annual configuration, `PlanningSection`, and later `Framework` retirement cannot reinterpret an `Open/Closed` population.

M4 adds dedicated Administrator-only-by-default permissions: `talent_assessment_cycles.view`, `.manage`, `.view_population`, and `.govern`. Branch-scoped `.manage` holders may author Draft metadata but cannot `Open` or `Close`; lifecycle governance requires `.govern` plus organization/global scope and always freezes the complete `SchoolGroup` population. Identifiable preview and frozen reads require `.view_population`. Branch-scoped readers receive only members whose resolved preview `Branch` or frozen historical `Branch` is authorized, an explicitly filtered subset count, and no full count or fingerprint. Organization/global population readers receive the complete population, count, and fingerprint. Metadata reads never disclose integrity metadata merely through `.view`.

Operational create/update/Open/Close events use the bounded append-only `TalentAssessmentAudit`, separate from `Framework` configuration audit, with actor, scope, `Program/Year/Framework` context, before/after JSON, correlation id, and `Open` population provenance. APIs are under `/api/talent/assessment-cycles`; no UI exists.

Deferred: `Student` Assessment, assessor assignment, competency results, `ReviewCandidate` instances/evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, development plans, and late population exception workflow. Live PostgreSQL execution remains a follow-up.

Talent Rubric, KPI, And Review Candidate Policy Foundation

Milestone M3 completes the deterministic semantic configuration subordinate to each `TalentProgramFrameworkVersion`. Migration

`20260904_003_talent_rubric_kpi_candidate_policy_foundation` adds one optional `Framework` rubric, configurable ordered rubric levels, exact `Framework`- competency/level descriptors, optional KPI configuration and weighted competency inputs, and an optional bounded `ReviewCandidate` Policy with ordered declarative rules. Rubric level names and codes are `Program/Framework` configuration, never a global vocabulary. A qualitative `PerformingArts` `Framework` can activate without KPI or numeric rubric values.

KPI configuration supports only `weighted_level_average`: enabled inputs are exact `Framework` Competencies with positive basis-point weights totaling 10,000, all levels require explicit integer values inside an explicit result scale, and interpretation text is required. `ReviewCandidate` Policy supports only `all/any` composition of `rubric_level_at_or_above` rules and, when an enabled KPI exists, `kpi_at_or_above` rules. These are stored

policy semantics only; M3 does not evaluate evidence or create candidate records. No script, SQL, Python expression, generic JSON logic, universal score, or cross-Program averaging is accepted.

M3 Governance Closure formally approves `weighted_level_average` as one bounded, optional, Framework-specific KPI calculation primitive: it is never a universal Talent Score, is never cross-Program normalized, is enabled only through explicit Framework configuration, and requires numeric rubric values only while it stays enabled. Additional calculation primitives require future governed Product Owner approval and an explicit model/constraint change; the `calculation_method` CHECK constraint stays a closed enum, never a generic expression/scripting mechanism. Qualitative Programs (for example Performing Arts) remain fully valid and activatable with no KPI and no numeric rubric levels.

`rubric_level_at_or_above` candidate-policy rules are defined by the configured Rubric Level order (`TalentRubricLevel.display_order`, lowest proficiency first) - a rule targeting a level means that level or any level ordered higher. This ordering never depends on `numeric_value`, so qualitative Programs with no numeric levels still get correct, meaningful thresholds. `display_order` is the single authority for both presentation order and this semantic proficiency rank: it is dense, unique per rubric, and fully reindexed on every add/remove/reorder, so no divergence between a level's displayed position and its proficiency rank is possible. Reordering levels is Draft-only and always bumps the Framework revision/semantic fingerprint since order is part of the M3 semantic payload; Active/Retired order is immutable. Candidate-policy rule evaluation itself remains deferred to a later assessment milestone.

Every M3 mutation requires `expected_revision`, is restricted to a Draft Framework, increments its revision, recomputes its semantic fingerprint over the complete ordered M2+M3 configuration, and appends an M3-specific action to the existing `TalentConfigurationAudit`. Audit identity now targets the specific M3 child resource that changed - `rubric`, `rubric_level`, `rubric_descriptor`, `kpi_configuration`, `kpi_component`, `review_candidate_policy`, or `review_candidate_rule` - matching the M2 `framework_competency` precedent, instead of only the containing `framework_version`. Create operations audit the new child row's post-flush id; `reorder_rubric_levels` (the one M3 bulk operation) appends one audit row per reordered level so each level's own audit identity is preserved. The widened `resource_type` CHECK constraint is applied by the M3 migration itself (both the SQLite table-rebuild and PostgreSQL NOT VALID/VALIDATE pattern already used elsewhere in `db_migrations.py`), since the audit table was originally created by the separate, already-applied M2 migration. Active and Retired Framework M3 state is immutable. Framework cloning creates independent rubric, level, descriptor, KPI, component, policy, and rule rows, mapping Framework Competencies through durable `TalentCompetency` lineage. Composite scoped foreign keys enforce SchoolGroup, Program, Framework, FrameworkCompetency, rubric, and level alignment.

The bounded JSON routes remain under `/api/talent/programs`: reads use `talent_programs.view`, Draft mutations use `talent_programs.manage`, and the existing `talent_programs.govern` plus organization/global scope requirement continues to control activation/retirement. No new permission or entitlement logic was introduced.

Not implemented: assessment/evidence, scoring/results, Review Candidate instances, candidate-policy evaluation, Official Identification, Educator Input, Learner Profile, analytics/Talent Map, AI, Development & Support, Learning Style, or UI. Live PostgreSQL execution remains a follow-up; SQLite migration, FK, lifecycle, service, API, permission, and regression coverage is in `tests/test_talent_rubric_kpi_candidate_policy.py` plus M1/M2 suites.

Student & Academic Placement Foundation

Student is a canonical SchoolGroup-owned identity distinct from User: `models.py/migration` `20260904_001_student_academic_placement_foundation` add students (first/father/last name, gender, status, audit timestamps), `student_external_identifiers` (namespace/value/source cross-system linking with tenant-scoped uniqueness and an activate/deactivate lifecycle), effective- dated `student_academic_placements`, and append-only `student_audits`. There is no separate Enrollment entity; `StudentAcademicPlacement` is the authoritative enrollment record, resolved by half-open [`effective_from`, `effective_to`) interval semantics with overlap rejection and deterministic point-in-time resolution. `academic_grade.py` (extracted from

routers/planning.py's prior inline normalization, existing callers unchanged) is the shared canonical KG/1-12 grade vocabulary used by both Planning and Student.

Every placement carries an immutable historical `grade_level/section_name` snapshot captured from the linked `PlanningSection` at creation or transition time (or supplied directly when no section link exists). A later `PlanningSection` rename/renumber never rewrites already-recorded placement history; deleting the linked `PlanningSection` only nulls its FK on referencing placements and preserves the snapshot. `transition_placement` closes the current placement and opens a new one atomically, preserving full history; `correct_placement` revalidates scope/snapshot/overlap and governs an in-place correction of the same row rather than creating parallel history. Every mutation writes an append-only `StudentAudit` row (before/after JSON, actor, action) in the same transaction; audits are never updated or deleted. Tenant/branch-scoped composite foreign keys reject cross-tenant `Branch`, `AcademicYear`, or `PlanningSection` references (`invalid_scope`).

routers/students.py (/api/students) is backed by `student_academic_service.py`. Branch-scope enforcement always runs after the permission check: an in-scope Student ID with an out-of-scope existing or target placement branch is rejected 403 without revealing existence; a foreign-tenant Student ID returns a uniform 404. The final canonical permission mapping (Checkpoint A) replaces the interim implementation, which gated all Academic Placement mutation through the broad, commonly-granted `planning.edit_section` permission alone with no Student-specific permission at all. `permission_registry.py` now has a dedicated students group: `students.view` (Student and Placement-history reads), `students.create`, `students.edit` (identity, non-status fields), `students.activate_deactivate` (status-only updates), `students.manage_identifiers` (External Identifiers), and `students.manage_placements` (create/end/transition/correct Academic Placement). See "Student And Talent Program Permission Governance" below for the shared default-assignment rationale.

Not implemented: Student UI, import/merge, Talent/Rubric assessment linkage, and analytics. PostgreSQL validation (constraints, concurrent placement writes) has not been executed against live PostgreSQL; only SQLite-backed `tests/test_student_academic_foundation.py` coverage exists.

Talent Program & Framework Foundation

`TalentProgram` is a durable `SchoolGroup/organization`-owned identity, not a Subject: migration `20260904_002_talent_program_framework_foundation` adds `talent_programs` (draft/active/retired lifecycle), `talent_program_academic_year_configurations` (per-year enable flag plus eligible KG/1-12 grade set), `talent_program_framework_versions` (Framework Version lifecycle with deterministic sequential version allocation, a revision counter, and a semantic fingerprint), `talent_competencies` (organization-scoped competency lineage), `talent_framework_competencies` (version-specific competency membership snapshots - label/description/order captured per Framework Version independent of later competency edits), and append-only `talent_configuration_audits`.

Exactly one Active Framework Version exists per Program: activating a new version requires explicit supersession of the current active version (`supersession_required` otherwise) and atomically retires it; an active Framework's content becomes immutable (`immutable_framework`). Every Draft mutation and activation call is stale-write protected: the caller's `expected_revision` (and, for activation, `expected_fingerprint`) must match current state or the call fails `stale_framework`.

Governance is split into two tiers. Draft authorship - Program create/edit, Program annual configuration, Framework Draft edits (including competency membership add/update/reorder/remove within a Draft), and Competency create/edit - is permission-gated and organization-scope-independent. Framework activate/retire and Program lifecycle transitions additionally require organization/global access scope (`auth.ACCESS_SCOPE_ORGANIZATION/ACCESS_SCOPE_GLOBAL`, checked through `_organization_authorized()` in `routers/talent_programs.py`) on top of the permission check; a Branch-scoped actor can never activate, retire, or transition a Program/Framework regardless of permission grant. This organization-scope gate was already correct in the interim implementation and is unchanged by Checkpoint A.

Checkpoint A fixed a real defect: Draft authorship was gated only by the same broad `planning.edit_section` permission already default-granted to Editor/User roles, so any Branch-scoped actor holding that common permission received full organization-wide authorship of Programs, Framework Drafts, and Competencies with zero Branch-relationship constraint (TalentProgram has no branch dimension to constrain against). `permission_registry.py` now has a dedicated `talent_programs` group: `talent_programs.view` (Program, Framework

1 including history - and Competency reads), `talent_programs.manage` (all

Draft authorship listed above), and `talent_programs.govern` (Framework activate/retire, Program lifecycle transition; checked in addition to, not instead of, the unchanged organization-scope gate).

`routers/talent_programs.py (/api/talent/programs)` is backed by `talent_program_service.py`; foreign-tenant Program IDs return a uniform 404.

Not yet implemented: Student Assessment, Review/Identification workflow, Learner Profile, analytics/Talent Map, and AI. PostgreSQL validation has not been executed against live PostgreSQL; only SQLite-backed `tests/test_talent_program_framework_foundation.py` coverage exists.

Student And Talent Program Permission Governance

Both new `students.*` and `talent_programs.*` permission keys follow the same established default-assignment pattern already used for the sensitive users group: `permission_registry.DEFAULT_ROLE_PERMISSIONS[ROLE_ADMINISTRATOR]` grants every permission key not in `DEVELOPER_ONLY_PERMISSION_KEYS` automatically, while Editor and User roles only receive the curated `_EDITOR_LIKE_PERMISSIONS` subset. Neither new group was added to that subset, so - exactly like `users.*` today - Student and Talent Program management are Administrator-only by default across every tenant; Limited remains read-only-elsewhere and gets neither group. No migration or seed change was needed: a newly introduced permission key with no per-tenant `RolePermission` override row simply falls back to the code-level default at read time (`auth.get_allowed_permission_keys`), matching how every prior permission addition (e.g. `timetable.manage_teacher_rules`, `curriculum.adjust`) was introduced as a pure `permission_registry.py` edit. Whether either group should ever be extended to Editor/User by default is an explicit Product Owner decision to be made later through the existing Role Permissions configuration UI, not a default this closure task changed.

Planning Subject Requirement Removal (Single And Bulk)

A Planning requirement is classified removable (untouched explicit `PlanningSubjectDemand`), permanent (Curriculum-Adjustment-touched - `updated_by_user_id` set - shown with a "Protected" explanation, never a hidden action), fallback (no explicit row at all - resolved only from the Subject catalog), or `not_found`. Both removable and fallback are now removable: an explicit row is deleted, while a fallback requirement is removed by creating an explicit `is_active=False/weekly_periods=0` suppression row. Unlike a Curriculum Adjustment retirement (which stamps `updated_by_user_id`), this suppression row only sets `created_by_user_id` (the acting admin, for audit) and leaves `updated_by_user_id` NULL - so it stays classified removable/setup-only rather than becoming permanent. A row only becomes permanent by genuinely being touched by Curriculum Adjustment, never merely by being suppressed from this action. When its section is otherwise dependency-free, section deletion automatically removes only these inactive zero-period setup suppressions in the same transaction; admins do not need to expose or clear an implementation artifact. This closed a real gap where "Remove demand" only ever appeared for explicit-row requirements, so the same teacher's subjects could inconsistently show the action for reasons that had nothing to do with the teacher.

`POST /planning/subject-requirements/remove` (`routers/planning.py`) handles both single removal ("Remove Subject Requirement") and checkbox-selected bulk removal (supported across every section on the page at once) atomically. Each expanded Planning section exposes a section-scoped **Select all** control beside a trash-icon

action whose label reflects the current selection (singular or the exact plural count); the page-level action mirrors that state. Every target is validated and scope-checked before anything is mutated, and any protected/invalid/ out-of-scope target blocks the whole request with a named reason. Teacher assignments, timetable data, and Curriculum Adjustment audit history are never touched. The prior round's GET /planning/subject-demand/delete/{demand_id} route remains in place for removing active setup requirements.

Claude Code KMS Configuration

Claude Code uses root CLAUDE.md as its repository entry point, .claude/skills/tis-kms/SKILL.md as the reusable TIS execution workflow, and .claude/agents/tis-kms-developer.md as the bounded specialized subagent. These files mirror, but do not replace, root AGENTS.md or the authoritative KMS. They require the same first reads, worktree preservation, tenant and deployment boundaries, task-level .kms-impact.yml, proportional validation, KMS sync/check, and no-commit/no-push default.

Return-To Reopen-On-Load UI Pattern

TIS is server-rendered FastAPI/Jinja, not an SPA, so a redirect after an action normally collapses any open <details>/accordion. The sanctioned fix reuses the existing return_to convention: redirect_utils.py (moved out of main.py, existing callers unchanged) provides safe_redirect_path() - rejecting http(s):// and // targets - plus fragment-preserving redirect_with_notice/redirect_with_error. A globally-loaded shared script, static/js/reopen-on-load.js, reads a target element id from window.location.hash (or an inline window.__tisReopenTargetId for a non-redirect response) on page load and reopens it if it is a <details>, scrolling it into view; a missing/stale id is a harmless no-op. Planning's "Remove demand" action is the first consumer - do not invent a page-local variant of this pattern; give the element a stable id and pass return_to through the existing convention instead.

Planning And Subject Delete Dependency Guards

Deleting a Planning section or a Subject now runs a dependency check before any mutation and blocks with a customer-safe, per-category explanation instead of raising an unhandled IntegrityError or deleting silently. Planning section delete checks TeacherSectionAssignment, PlanningSubjectDemand, TimetableEntry, TeacherSchedulingRuleTarget, CalendarEvent/CalendarEventSectionTarget, and section-scoped SubjectDistributionRule; TeacherSectionAssignment is now a blocker like every other category rather than being auto-deleted. The sole cleanup exception is an inactive zero-period PlanningSubjectDemand with no updated_by_user_id: this setup-only Remove Subject Requirement suppression is not a blocker and is deleted atomically with an otherwise deletable section. Active setup demand, Curriculum-Adjustment-touched history, and every other dependency still block. Subject delete and Bulk Delete consolidate the equivalent check across Teacher.subject_code, TeacherSubjectAllocation, TeacherSectionAssignment, PlanningSubjectDemand, TimetableEntry, CurriculumAdjustmentAudit, and SubjectDistributionRule; Bulk Delete is atomic, so any blocked Subject in the selection deletes none of the batch and names every blocked Subject with its reason.

Planning subject demand is split into removable and permanent, using the existing PlanningSubjectDemand.updated_by_user_id column as the signal: Curriculum Adjustment always stamps it, the one-time setup backfill never does, so an untouched row is pure scaffolding and can be hard-deleted through the new GET /planning/subject-demand/delete/{demand_id} action ("Remove demand" on the Planning page), after which the section or subject becomes deletable. A row Curriculum Adjustment has ever touched, active or retired, is genuine history and remains a permanent blocker. Published/archived timetable placements follow the same principle without a new action: only mutable Draft entries can be removed. No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Professional Timetable Lifecycle UX

The timetable workspace presents Draft, Validate, Approve, and Publish without a new durable state. Validation and approval remain one backend transition to `publication_ready`; the active pointer identifies the immutable current published version. Version facts expose origin, source, created/generated, validation, approval, publication, mutability, and active state. Authorized users can create a working Draft from current published or immutable history without changing the active publication. Lifecycle and mutation failures retain legacy messages and add canonical conflicts, with an explicit refresh action for stale revisions.

Structured Timetable Conflict Evidence

`timetable_conflicts.py` defines the canonical internal conflict contract used by readiness, feasibility payloads, generation terminal payloads, and the independent validator. Stable machine codes coexist with legacy source codes/messages for API and UI compatibility. Conflicts classify HARD/SOFT severity and DURABLE, RECALCULABLE, or TRANSIENT evidence without adding a universal conflict table.

Public serialization is authorization-safe: it exposes only already-authorized same-scope entity references, redacts unauthorized references, never emits internal Lesson Requirement identities or solver clauses, and represents internal correlation only as presence flags. Infeasible, timeout, cancellation, stale authority, and solver/execution failure remain distinct outcomes. Existing safe human-readable generation and feasibility messages remain unchanged.

Timetable Lesson Requirement Projection

`timetable_requirement_projection.py` is the read-only domain boundary from Planning authority to timetable scheduling requirements. It resolves exact SchoolGroup/Branch/Academic Year scope, Planning section and subject identity, explicit-first PlanningSubjectDemand, legacy `Subject.weekly_hours` fallback only when no explicit row exists, and the existing Planning/HRT teacher authority. Explicit zero or inactive rows remain authoritative suppression evidence.

Each projected requirement has an internal deterministic identity over its scope and governing Planning source plus a source fingerprint covering effective periods, active state, and teacher assignment authority. Schema-v5 immutable snapshots capture both values. Readiness and snapshot creation consume the same projection; the problem builder carries it to generation and the independent validator verifies the captured requirement contract. Planning remains the only editable academic-demand authority, and no Lesson Requirement table or CRUD API exists.

Timetable Solver Adapter Boundary

The timetable Workflow worker consumes the solver-neutral `TimetableSolver` contract in `timetable_solver.py`. Release 1 binds that contract only to the existing OR-Tools CP-SAT implementation; this boundary does not authorize a second solver, relaxed constraints, or a parallel generation architecture. Infeasibility diagnosis is part of the same adapter contract. The independent domain validator and atomic fingerprint/revision persistence gates remain outside the solver and continue to decide whether a candidate may be saved.

Teacher Scheduling Rules

Administrators with `timetable.manage_teacher_rules` configure teacher-specific timing policy in Timetable Settings without changing Planning demand or teacher allocation. The normal form asks only for teacher, rule, days, numbered periods, and optional Current/New sections with Select All. **Schedule within these periods** constrains existing eligible lessons to an allowed window without creating occupancy or demand; **Must teach these periods** requires occupancy in every selected slot; **Unavailable** prohibits all teacher lessons in those slots. Existing first/last, grade-target, and preference rules remain compatible but are not exposed in the simplified normal form.

Schema-v5 immutable snapshots capture canonical resolved teacher rules in the constraint fingerprint. Rule changes mark unpublished Drafts stale, clear Draft approval, and require regeneration while published history remains untouched. Readiness and the problem builder reject deterministic workload, allowed-window capacity, target, lock, slot, and contradictory hard-rule conflicts; CP-SAT enforces hard timing and scores existing preferences; the

independent validator repeats hard-rule and window checks. With no rules, generation behavior is backward compatible.

When more than one Schedule-within rule targets the same teacher demand, their configured slots form one allowed-window union for that demand. They are not independent whole-workload restrictions and must not be intersected. The problem builder computes this combined authority before CP-SAT and performs exact teacher-slot matching plus deterministic checks for window/unavailability capacity, Must-teach compatibility, daily coverage, hard maximum-per-day, minimum-day, and double-block feasibility. Proven conflicts appear as readiness blockers with required-versus-available guidance instead of reaching a generic solver-infeasible result. Grouped activities count one teacher occupancy.

If CP-SAT still proves the complete model infeasible after deterministic checks, the Workflow reruns bounded diagnostic profiles without changing the real run. It distinguishes base demand/collisions, locks, grouped activities/resources, Subject Distribution Rules, Teacher Scheduling Rules, and subject/teacher-rule interaction. The proven family, current lock count, and grouped-activity count are stored in the Generation Run's customer-safe failure details. A diagnostic timeout is reported as inconclusive rather than mislabeling a constraint family. Diagnostic profiles use the dedicated `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` Workflow setting, defaulting to 60 seconds per attempted profile and independent of the main solver timeout. Profiles for locks or grouped activities are skipped when their captured counts are zero; successful isolation also reports safe model/rule/window counts where available.

The configuration UI keeps section selection literal: Select All synchronizes every visible Current/New section, editing restores the exact saved targets, and saved summaries name selected grade-section labels instead of collapsing them to an any-class label. Manual Draft placement rejects hard Unavailable and Schedule-within violations immediately. Complete Draft validation additionally requires every Must-teach slot to contain an eligible lesson in its configured section scope; the same validation blocks approval and publication. Soft preferences remain non-blocking.

Guided Curriculum Adjustment UI

Administrators with `curriculum.adjust` can enter a five-step, page-based workflow from Planning or Subjects. The UI selects Current/New scope, prepares the existing read-only preview, separates blockers from warnings, requires an explicit eligible teacher decision for every section, and submits the reviewed fingerprint to the atomic apply endpoint. A changed Planning state refreshes the preview without exposing revision terminology. Success links to Timetable and regeneration, but never starts generation automatically.

The entered transfer period count is authoritative: preview computes each section's source and target result from its effective `PlanningSubjectDemand`, rejects non-positive or excessive transfers, and apply persists exactly that reviewed result. Subjects presents those same Current/New Planning values for the active branch/year: a uniform grade-subject demand is shown directly, while section differences display as **Varies** with a section breakdown. `Subject.weekly_hours` remains a catalog/default value and is not rewritten by scoped adjustments.

The same workflow supports `reduce_only`: no target subject or teacher reassignment is required, the requested reduction is applied to each selected source demand, and all existing preview, fingerprint, transaction, Draft invalidation, lock, rule, and publication safeguards remain in force. Subject Details shows current effective Planning periods and offers a permission-gated **Adjust Weekly Periods** link prefilled for that grade-specific subject. Multi-grade reduction is intentionally not combined because subject codes and the atomic review/audit contract are single-subject identities.

Atomic Curriculum Adjustment Apply

Stage 4 adds `curriculum_adjustment_apply_service.py` and permissioned `POST /planning/curriculum-adjustments/apply`. The service requires a Stage 3 fingerprint and an explicit target-teacher decision, including intentional unassigned, for every affected section. In one owned transaction it locks and revalidates exact-scope Current/New Planning authority, rejects active generation, stale previews, qualification/capacity failures, rule conflicts, and invalid Draft locks, then updates source/target demands and assignments and writes one durable audit. Zero source demand is retired and its active section rule is inactivated. The

current unpublished Draft is preserved but marked stale, approval is cleared, and later regeneration is required. Published versions and the active pointer are never mutated. `curriculum.adjust` is a dedicated apply permission.

Read-Only Curriculum Adjustment Preview

Stage 3 adds `curriculum_adjustment_preview_service.py` and permissioned JSON route `POST /planning/curriculum-adjustments/preview`. The service previews source-demand reduction and transfer to a target subject for one grade, selected Current/New sections, or every active source use in one exact SchoolGroup/branch/year. It reads explicit-first PlanningSubjectDemand, current Planning/HRT teachers, teacher loads and capacity, normalized Subject Scheduling Rules, grouped legacy configuration, and the current mutable Draft Timetable revision. It returns per-section before/ after periods, teacher options, blockers/warnings, Draft stale/regeneration impact, and a deterministic fingerprint that can be rebuilt before a future apply action. It writes nothing, never assigns a teacher, and never changes timetable history.

Explicit Planning Subject Demand Foundation

Migration `20260828_004_planning_subject_demands_foundation` adds `planning_subject_demands`, an explicit branch/year and Planning-section scoped weekly subject-demand foundation. The migration idempotently backfills matching grade subjects for Current/New Planning sections only. Composite foreign keys prevent section or subject scope drift, while a partial unique index permits at most one active demand for a section and subject and preserves retired rows. `planning_subject_demand_service.py` resolves explicit active or retired rows before using legacy `Subject.grade + Subject.weekly_hours` fallback. Operational Planning, teacher required/allocated/remaining load, timetable readiness/workspace/snapshot/ generation input, Subject Scheduling Rule arithmetic, and required-hours reports consume that section demand. An explicit inactive or zero row suppresses demand. Legacy fallback remains only when a section-subject has no explicit row. Published timetable history remains immutable and unchanged.

Tenant Report Branding

Tenant-facing operational reports and exports use `tenant_report_branding.py` as their shared logo authority. It resolves only configured branch/organization logo slots through the existing tenant-scoped branding resolver. A tenant with no configured logo receives a clean text header with no image; tenant artifacts must never fall back to a TIS product logo. Timetable PDF/XLSX exports, academic-calendar PDF exports, and observation PDFs consume this rule. TIS branding remains valid in the application shell, login/product surfaces, public/platform-owned materials, and explicitly internal platform-owner reports.

Subject Scheduling Rules UI

Timetable Settings now presents a grade-first Subject Scheduling Rules section: `planning_scope_service.py` is the reusable operational selector authority. The main and copy-rule grade selectors, timetable workspace section filters, and calendar/grouped section choices derive only from Current/New PlanningSection rows in the selected branch and academic year, never from the global grade range or a catalog-only Subject row. An administrator selects one planned grade, can search its compact subject list, and sees each Planning weekly total, current pattern, and configured/default status without raw subject codes. The native dialog stays closed on page load and opens only after Edit has populated the selected Subject + Grade and read-only Planning total; Cancel/Close restores the closed state. Its compact Session Structure keeps the aligned double-block and single-session steppers above one full-width live total. Two-period Separate Sessions and Consecutive Double Block quick choices remain available. Multiple primary conditions can be combined in a balanced grid; minimum teaching days, strictness, and section overrides remain progressively disclosed. Legacy subject-code mappings and Swimming JSON remain available only inside a secondary Advanced / Legacy area. `subject_distribution_rules_ui.py` re-validates the fixed two-period block arithmetic authoritatively on save using the Stage 1 validator. Saving creates or updates only the intended Grade+Subject (`scope_level="grade"`) row, or a specific section override when a section is selected from the panel's Section

Overrides list. Reset To Default removes only the grade-level row so resolution falls back to the next hierarchy tier; Copy Rules From Grade matches subjects across grades by name (since subject codes are grade-specific) and skips any subject whose arithmetic does not fit the target grade instead of copying it blindly. No changes were made to the CP-SAT solver, independent validator, snapshot immutability, or grouped-activity/Swimming JSON authority.

Smart Timetable Academic Quality Rules

Branch/year Timetable Settings store explicit subject-code mappings for core, spread, ICT, double-period, and grouped Swimming behavior. Schema-v3 snapshots carry the normalized rules into the problem builder. CP-SAT requires daily core coverage when weekly demand reaches the teaching-day count, maximizes distinct-day spread for shorter core and configured spread subjects, supports hard ICT one-per-day, and keeps configured Swimming sections simultaneous without weakening section, teacher, or shared-resource protection. The independent validator repeats every new hard check. Regeneration diversity remains configurable with a 25 percent default.

Subject Distribution Rules Generation Wiring

Schema-v3 snapshots now resolve and embed the effective Subject Distribution Rule (section over grade over branch default, None for legacy fallback) on every Planning demand at snapshot creation time, so a later rule change never alters an already-created snapshot. The problem builder carries that resolved rule per demand, exposes true physical period adjacency on every slot, and runs the arithmetic/feasibility validator as a final pre-solve defense, failing cleanly with `distribution_rule_invalid` rather than reaching CP-SAT. CP-SAT enforces an exact partition into intentional two-period blocks and single-period sessions. Explicit block-start and single-session decisions channel every occupied period to exactly one session: sessions may touch, so three consecutive periods can be one double plus one single and four can be two touching doubles, but session membership never overlaps. Physical timeline interruptions still break a double. Daily session/load channeling strengthens hard daily-coverage propagation without relaxing its meaning. CP-SAT also enforces generalized daily-coverage/spread/max-per-day/min-teaching-days hard-or-soft behavior per resolved rule, and exempts declared blocks from the consecutive-avoidance penalty; demands without a resolved rule keep the exact legacy `quality_rules_json` code-list behavior. The independent validator mirrors every new hard check (existence of the same exact non-overlapping session partition via true adjacency, hard daily coverage, hard max/day, hard min teaching days) alongside the unchanged grouped-activity and collision checks. Readiness now blocks genuinely invalid or infeasible normalized configurations before generation while keeping the existing core-subject warning path for legacy-only scopes. Regeneration is unchanged: the same constraint-building path applies during regenerate, so hard distribution rules remain valid while the diversity requirement reshuffles unlocked placements.

Subject Distribution Rules Foundation

A normalized `subject_distribution_rules` table (migration 20260828_003_subject_distribution_rules_foundation) now exists alongside `quality_rules_json` without replacing it. Rows are scoped `branch_default`, `grade`, or `section` per branch/academic year, with block/single counts, min teaching days, max periods per day, daily-coverage/spread/consecutive preferences, an optional min day gap, and a hard/soft strictness flag. `subject_distribution_rules.py` resolves section-over-grade-over-branch-default precedence with field-level inheritance for nullable fields, and returns None when no normalized row exists so the caller keeps today's `quality_rules_json` behavior unchanged. `subject_distribution_validator.py` is a pure arithmetic and feasibility check confirming $\text{block_count} * \text{block_length} + \text{single_count}$ equals the authoritative Planning weekly total and that day/period limits are feasible. `timetable_slot_service.py` marks each composed teaching period with true physical adjacency to its next period, false whenever a Break, Prayer, or other non-teaching item is composed between them, so intentional-block enforcement can rely on genuine continuity rather than raw period-index arithmetic.

Smart Timetable Stage 5.2

Stage 5.2 presents Configure, Generate Draft, Review/Edit Draft, and Publish to Users while retaining the Stage 2-5.1 version and solver internals. The primary workspace resolves one exact-scope mutable unpublished Draft Timetable, including a freshly created draft, and uses Approve Draft, Draft Approved, Published, and Timetable History language. Generation never approves a draft. Approval records the reviewing administrator and exact draft state; placement, lock, regeneration, and stale-authority changes invalidate it. Publication requires approval and repeats validation before atomically changing the active pointer. Generated Draft workflow actions are grouped together on the primary workspace; delete, archive, comparison, and version-specific exports remain in History.

Delete Draft Timetable permanently removes an eligible never-published draft and preserves the active pointer. Timetable History may permanently delete every never-published, non-active candidate regardless of origin or internal draft/archive state; dependent unpublished versions are removed child-first while snapshots, generation runs, and audit history are preserved. Protected published lineage and active generation are reported as blockers. The assignable `timetable.delete_versions` permission controls these actions and is granted to Administrators by default. Mutable working lessons may be moved or atomically swapped between canonical teaching slots, with lock, class, and teacher conflicts failing the whole operation. Publication uses explicit first-publish or replacement confirmation and customer-facing Published to Users language. Authorized managers may copy the current Published Timetable into an editable draft or create a fresh empty draft from current Planning/configuration authority. Neither action changes the active published pointer before explicit publication. `timetable_visibility_service.py` resolves official reads strictly through `TimetableActiveVersion`. `/my-timetable` matches scoped `User.user_id` to `Teacher.teacher_id` and shows only that teacher's published lessons. View-only users are redirected away from management drafts and history.

Smart Timetable Stage 5.1

Stage 5.1 implements automatic branch/year timetable generation with Google OR-Tools CP-SAT 9.15.6755 in a separate task dependency environment. HTTP requests capture schema-v3 immutable inputs, commit durable PostgreSQL `TimetableGenerationRun` rows, and use the server-side Render client to start one on-demand Workflow task with only the run public ID. The task claims that exact row, leases and heartbeats it, supports cooperative cancellation, solves hard constraints, and submits candidates to an OR-Tools-independent validator. Render automatic retry is disabled so TIS terminal state and Generate Again remain the only retry authority.

Generate creates a new unpublished `publication_ready` generated version only in one atomic transaction after current-fingerprint revalidation. Regenerate preserves locked placements, excludes the exact source, and requires 25 percent of unlocked placements to differ, rounded up. An infeasible diversity target fails explicitly rather than returning a nearly identical result. Neither flow changes published history or the active pointer. The generated result remains the logical Draft successor of the fresh source draft; the source is retained for provenance but omitted from normal History presentation. Freshness fingerprints cover Planning, canonical timetable configuration, stable generation constraints, and locks; generation-only source metadata is retained in snapshots but excluded from authority comparison. Safe component-level diagnostics identify real mismatches. `timetable.generate` is independent of publish authority. Stage 5.2 The old polling worker is optional/local only; production requires no always-on solver service. Immediate dispatch failure is terminal and customer-safe, while delayed task start is reported as waiting for compute. Stage 5.2 supplies the simplified workflow, Delete Working Timetable, and teacher My Timetable.

Smart Timetable Stage 3.5

`timetable_slot_service.py` is the single clock-time authority. For every working day it composes the configured shift start, teaching-period count/duration, and applicable non-teaching blocks into ordered teaching/block items. After-period blocks are the default and shift every later period; legacy fixed-time blocks are inserted only at a valid live boundary and otherwise surface a correction blocker. The calculated end, settings preview, timetable grid, readiness, assignment validation, snapshots/fingerprints, and XLSX/PDF exports consume this projection. The additive placement columns preserve existing blocks as `fixed_time`. Stage 3.5 itself added no solver, generation endpoint, availability, or resources; Stage 5.1 now consumes that canonical timeline.

Smart Timetable Stage 4

The timetable page now exposes exact-scope version history and distinguishes Draft, Publication Ready, Active/Published, Superseded, and Archived views. Opening a historical version never changes the active pointer. Operational default resolution remains: newest mutable draft derived from active, otherwise active, otherwise newest mutable scope draft. Immutable versions require an explicit copied draft.

Draft lesson locks are explicit future-regeneration commitments. Placement and lock edits use `edit_revision`; lock edits refresh the version snapshot/fingerprint. Solver-independent draft validation is separate from Stage 3 generation readiness. Atomic publication revalidates freshness/completeness, locks the selected version and active pointer, checks pointer revision, supersedes the old active version, and makes the selected version immutable through the active pointer. Stage 4 itself added no solver or worker; Stage 5.1 now builds generation on this boundary.

Smart Timetable Stage 3

`timetable_slot_service.py` is the single composed-timeline authority for teaching, non-teaching, and invalid slots. Full coverage disables a slot; a block wholly between periods removes no slot; partial overlap is invalid; all-day and every-day rules expand deterministically. Blocks never shift or shorten teaching periods.

`TimetableReadinessService` is read-only, solver-independent, and exact-scope. It combines configuration, Planning demand, explicit/HRT teacher resolution, existing teacher capacity, slot sanity, locks, and Stage 2 fingerprints. Generation-ready means only that a future solve may be attempted, never that feasibility is proven.

Capacity-Based Packaging And Common Customer Features

Starter, Professional, and Enterprise AI now represent organization scale, not progressive access to normal customer modules. Their ceilings remain 1/5/25, 5/20/100, and 25/100/500 for active branches, operational staff users, and teachers. Paid, active promo, and active customer-demo authority share the normal customer baseline only after source/lifecycle, permission, tenant, branch, and academic-year checks succeed.

`saas/customer_feature_policy.py` classifies that baseline and explicitly excludes the Developer-only audit export. `saas.entitlement_service.evaluate_feature_access` is the shared source-aware decision used by Advanced Reporting. Promo activation and migration

`20260819_001_capacity_based_customer_feature_baseline` persist the same baseline without changing exact promo capacities or immutable promo history. Standard, Full, and Custom demos retain the baseline; Custom controls only legitimate scope, safety, experimental, and consumption policy.

Enabled AI features are commercially available across paid tiers, promo, and active demo with `ai.use`.

`saas/ai_consumption_policy.py` separately resolves execution allowance. Existing reservations, counters, idempotency, and operation events remain unchanged, and globally disabled AI definitions remain unavailable.

Stage 5 SchoolGroup Authorization And Capacity Presentation

Direct operational SchoolGroup creation and deletion are global platform actions. They require a Platform identity plus `schools.manage_all_schools` and the corresponding `schools.create` or `schools.delete` permission. Tenant roles cannot receive those create/delete permissions, and platform identity is checked again in the handlers so stale permission state cannot cross the boundary. Tenant users with `schools.edit` may update only their linked SchoolGroup; arbitrary IDs fail before validation or mutation. Approved paid, demo, promo, and conversion provisioning remain separate and unchanged.

School Management presents branch usage through `saas/commercial_authority_service.py`, so paid and promo workspaces use the same authoritative source-aware counts without exposing source internals. Branch create, promo assignment, and entitlement remain atomic under the existing SchoolGroup lock. The individual last-active-branch safeguard now counts only the target SchoolGroup.

`scripts/audit_schoolgroup_provenance.py` is a PostgreSQL-only, repeatable-read, read-only report for active

internal sandboxes that have neither tenant provenance nor explicit workspace entitlement; it never remediates candidates.

M4C Existing Workspace Paid Activation

M4C lets a verified tenant owner activate an existing customer workspace in provisioning through Paddle without recreating onboarding or tenant data. The durable `ExistingWorkspacePaidActivation` aggregate is anchored to `SchoolGroup`, workspace UUID, SaaS account, tenant-owner link, selected plan, billing interval, branch snapshot, quote fingerprint, checkout lineage, payment attempt, and a `SchoolGroup`-anchored subscription contract. Checkout and payment rows use strict pending-organization versus paid-activation contexts.

Professional and Enterprise AI quotes include every active operational branch; Paddle quantity remains active branch count. Starter remains unavailable for existing workspaces until restricted-branch enforcement is proven across all operational entry points. Preparation creates no `PendingOrganization`, `ProvisioningJob`, `SchoolGroup`, or pre-payment entitlement. Organization Account's **Choose a Plan** action always opens plan selection while the activation is editable. A saved draft or `checkout_ready` selection is highlighted but may be replaced by another currently eligible plan, which rebuilds the authoritative quote. Plan browsing and draft replacement make no Paddle API call and create no `PaymentAttempt`. Once checkout is `checkout_started`, `payment_processing`, or in a manual-review/inconsistent state, plan replacement fails closed. Billing identity is stored against `SchoolGroup` and explicitly associates any reused Paddle customer with the workspace. The association also snapshots the provider address and business used by that workspace, so a shared SaaS account cannot make another workspace's mutable customer defaults authoritative.

Only a verified matching `transaction_completed` event can activate access. In one transaction TIS revalidates identity, quote, capacity, selected branches, provider customer/address/business, price, quantity, currency, interval, transaction, and subscription identity; then it confirms the contract and subscription, creates paid workspace and branch entitlements plus the paid `TenantProvisioningLink`, and changes lifecycle to `active`. Classification remains `customer`. `transaction.paid` is processing only, browser return grants nothing, and conflicting or stale evidence fails closed. Returned and reused transactions must be billed, automatic-collection transactions whose item, price, quantity, subtotal, interval, currency, customer, address, business, checkout URL, and full custom-data lineage match the current quote. PostgreSQL lock acquisition refreshes mapped state before duplicate webhook decisions. Existing promo, demo-to-paid, and `PendingOrganization` checkout paths remain separate.

Organization Account presentation distinguishes commercial activation from payment processing. A coherent `activation_required` workspace with no current payment attempt displays **Activation required. Payment processing** requires a current, unexpired existing-workspace `PaymentAttempt` in `checkout_started` or `payment_processing`; failed, cancelled, and expired attempts display recovery states. Workspace lifecycle provisioning alone is never payment evidence.

M4B Controlled Existing Workspace Conversion

M4B converts an explicitly audited internal sandbox into a customer workspace that still requires commercial activation. The durable conversion operation is the ownership-claim record: it stores the exact workspace tuple, normalized intended-owner email, approved M4A hash, canonical parameter hash, immutable branch/commercial snapshots, setup state, actors, stage, status, and failure code. Append-only conversion events contain only allowlisted redacted details.

Preparation creates no account and sends no email. The intended owner registers and verifies through the normal TIS Account flow, explicitly claims the approved workspace, and is aligned through the shared operational-owner identity rules. No tenant ownership is granted before verification. The setup review accepts only legal name, an IANA timezone, and educational program; existing display name, country, branches, academic and operational records remain unchanged.

Final execution is PostgreSQL-only and locks the operation and `SchoolGroup`. It re-audits immediately, requires exact identity and unchanged branch/dependency and sandbox-entitlement evidence, then ends the internal entitlement and sets `customer / provisioning` in one commit. It creates no active entitlement or tenant source. M1 resolves this

coherent state as `activation_required`, keeps Organization Account access, and blocks operations until M3 promo activation. Existing-workspace Paddle activation is deliberately deferred; new-onboarding Paddle behavior is unchanged.

M4A Existing Workspace Conversion Audit

M4A adds a generic read-only evidence boundary before any existing internal sandbox can be considered for customer conversion. The service resolves one explicit `SchoolGroup.id`, workspace UUID, exact name, and normalized intended owner email. It inventories workspace metadata, identities, account links, tenant/provisioning evidence, entitlements, paid/demo/promo records, and every reflected branch foreign-key descendant. Logical branch references without a database foreign key are reported separately for manual review.

The production CLI requires PostgreSQL and starts a repeatable-read, read-only transaction. It outputs deterministic sanitized JSON or text, records the transaction settings, and provides a stable SHA-256 snapshot hash. Exit 0 means the audit is coherent, exit 2 requires manual review, exit 3 means the supplied workspace identity does not match, and exit 1 is an execution or configuration failure. Provider references remain presence flags. The CLI always rolls back and performs no conversion, branch change, owner alignment, entitlement or tenant-link mutation, Paddle call, or email. M4A never approves hard deletion or write conversion; incomplete traversal, identity mismatch, duplicate ownership, non-sandbox commercial authority, or missing schema evidence fails closed for manual review. Recommended archival IDs include only active branches with zero direct, transitive, logical, or soft-deleted dependencies and are withheld when unmodeled branch foreign keys exist.

M3 Promo Redemption And Commercial Grants

M3 turns an approved promo definition into commercial authority only after an authenticated, verified organization owner completes a resume-safe activation. The workflow supports either a completed `PendingOrganization` or a separately aligned existing `SchoolGroup` with a valid tenant-owner account link; it never fabricates pending, payment, subscription, contract, or demo records.

Successful activation creates one immutable `PromoRedemption`, one immutable capacity/plan/scope snapshot in `PromoGrant`, one promo `WorkspaceEntitlement`, explicit branch entitlements and assignments, and a promo-sourced `TenantProvisioningLink`. The link requires exactly one source: paid contract, approved demo request, or promo grant. M1 resolves promo access from that chain. Pending sessions grant nothing, expired or ambiguous grants fail closed, and no Paddle API or billing record participates.

When eligible branches exceed promo capacity, the owner selects exactly the covered branches. Every preserved workspace branch receives explicit evidence: selected branches receive an assignment and active entitlement, while every unselected branch, including an operationally inactive branch, receives an inactive entitlement without consuming grant capacity or changing operational status. Reactivation locks and rechecks M1 capacity, then activates the branch, assignment, and entitlement in one commit. A PostgreSQL-only, dry-run-first reconciliation command can add only deterministically missing inactive evidence; ambiguous or contradictory chains require manual review. Staff and teacher records are never selected or deactivated, and their authoritative counts must fit the grant.

Organization Account resolves the authoritative commercial source before choosing its customer presentation. Paid authority continues through the Paddle-backed subscription portal, demo authority keeps its dedicated journey, and promo authority uses `commercial_portal_service` to compose the central commercial authority with only attributable immutable promo metadata. The promo page shows the grant plan, safe status and dates, masked reference, and authoritative branch/system-user/teacher capacity. Operational promo access ends exactly at `PromoGrant.effective_to`; `grace_period_days` is only a recovery window for Organization Account and paid continuation and never extends operational access.

An expired or recovery-period promo owned by a verified tenant owner can enter the existing-workspace paid activation flow. Checkout preparation and pending payment do not change promo authority. Only verified provider completion locks and revalidates the tenant, quote, capacity, promo source, and paid evidence, then atomically ends the promo

entitlement, relinks the existing tenant source to the paid contract, creates paid workspace/branch entitlements, and preserves the SchoolGroup, workspace UUID, branches, users, teachers, data, branding, and immutable promo history. Active-promo early conversion remains blocked because promotional value is not paid credit.

`commercial_badge_service` converts centralized commercial access into one reusable, source-aware Promo, Demo, or Paid badge view model. Organization Account and commercial templates render that prepared identity and never infer authority from a historical contract, grant, plan, or subscription row.

M2 Promo Code Foundation

M2 adds secure, definition-only Starter, Professional, and Enterprise AI promo administration under `/saas-admin/promo-codes`. Exact positive branch, system-user, and teacher capacities must pass the existing SubscriptionPlan ceilings and shared plan-capacity validator. Scope supports global, organization, pending organization, account email, and email domain targets, with coherent reinforcing restrictions and optional existing-branch snapshots.

Raw codes use at least 100 bits of secure randomness and are shown exactly once in a `Cache-Control: no-store` response. TIS stores only an HMAC-SHA256 lookup hash produced with the dedicated `TIS_PROMO_CODE_HMAC_SECRET`, a key ID, and safe display prefix/suffix. Missing promo security configuration fails only promo generation/future lookup, with no insecure fallback. Platform Developers need `promo_codes.view` or `promo_codes.manage`; only Platform Owners activate or terminally revoke. Row locks and allowlisted durable audit protect lifecycle transitions. The M2 definition remains non-authoritative until M3 completes a valid redemption; definition creation itself creates no entitlement, tenant access, Paddle object, or M1 commercial-authority source.

Subscription Capacity Presentation And Billing Entry

Customer subscription pages distinguish confirmed paid branch quantity from the active plan's branch ceiling. Every active branch remains a Paddle quantity unit; the plan ceiling describes only the largest organization the plan can support. System-user and teacher counts remain included eligibility ceilings and never become provider quantity. Review Capacity shows current paid branches, required active branches, additional billed branches, the plan ceiling, and the resulting minimum eligible plan without describing unused plan ceiling as prepaid capacity. The approved commercial feature matrix is common across the three self-service plans, so customer plan cards continue to show plan identity, three capacity ceilings, current/eligibility state, and permitted plan-change actions rather than a feature ladder.

An authorized operational organization owner or user with `subscriptions.manage_billing` sees Billing & Subscription under System Configuration. The bridge revalidates the operational user, selected tenant, SaaS account-user link, and billing authority, then opens the existing Organization Account subscription page. If SaaS account authentication is required, only an allowlisted internal continuation is preserved through password or social sign-in. Multiple-organization selection remains explicit, restricted customers retain permitted recovery access, and external return destinations fail back to Organization Account. The public landing action is labelled Organization Sign In and continues to use centralized `/saas/login`; it never defaults an organization owner into operations.

Organization Account Sign-In Boundary

Public and onboarding SaaS Sign In authenticate through `/saas/login` and then resolve the customer journey centrally. An activated organization owner or a linked operational user with account-management permission lands on `/saas/account`, which is the Organization Account Overview. That overview exposes only permitted Organization Profile, Branches, Billing & Subscription, and Account & Security sections. `/login` is never the default destination for an authorized organization account manager; only the explicit Enter TIS Platform action enters the operational authentication flow, where commercial access and operational permissions are checked again.

Incomplete onboarding still resumes its authoritative setup step. Restricted or suspended account managers remain in Organization Account with permitted billing/recovery access and no active Enter TIS action. A linked operational user without account-management permission follows the existing role-based operational destination and receives no

organization-owner or billing controls. When one SaaS identity manages multiple organizations, `/saas/account` requires organization selection before rendering the selected account overview. The selected organization UUID is retained in an HTTP-only cookie, revalidated against the account's tenant links and permissions on every request, and then supplied to the existing entitlement resolver so Subscription Management uses the selected tenant without weakening isolation. Password login, social login, already-authenticated sign-in, SaaS root restoration, and post-verification sign-in all use this same resolver.

Initial Checkout Retry And Lineage Safety

The diagnosed Secure Payment failure was a local readiness rejection before any Paddle API request. `_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state even though it could be prepared safely. The verified Professional Annual catalog mapping remains USD 790 per active branch; two branches therefore produce Paddle quantity 2 and an authoritative annual total of USD 1,580.

Initial Secure Payment authority is the current server-built quote plus its plan selection, checkout session, and payment attempt lineage. A plan, interval, branch quantity, capacity estimate, provider price, or quote fingerprint change supersedes unfinished local sessions and attempts. Late webhooks for those attempts are retained for review but cannot activate a subscription or workspace.

Retry Secure Payment revalidates otherwise eligible unpaid onboarding state, normalizes legacy pre-checkout billing status, and creates a fresh checkout when the prior transaction is missing, incomplete, non-billed, or mismatched. Compatible active Paddle customer addresses are reused by country. Only an automatic, billed, launchable transaction for the current customer and quote may be released to the public payment launcher. Provider diagnostics and readiness failures remain logged server-side with tracebacks; customers receive one structured safe alert without provider or internal details.

Organization Billing Identity

Organization billing identity is explicit persisted organization state, not an alias for the authenticated user's login identity. `OrganizationBillingProfile` owns the confirmed billing email, legal/billing name, contact, optional company and tax identifiers, and the supported billing address fields. Initial checkout requires this profile. Authorized organization owners or users with `subscriptions.manage_billing` may update it from Subscription Management; changing it does not change login credentials, and changing login credentials does not change billing identity.

`PaymentCustomer` persists the mapped Paddle customer plus `provider_address_id` and `provider_business_id`. TIS synchronizes explicit billing changes to the mapped customer, reuses or updates one tenant-attributable active Paddle Business, updates the active subscription's customer/address/business identity, and includes the mapped business in new initial transactions. Ambiguous or failed synchronization fails closed with customer-safe guidance and server-side diagnostics. Existing financial documents are never silently rewritten; updated details primarily apply to future billing documents.

Billing Contact is read-only until an authorized user explicitly selects Edit. Valid local changes remain saved if Paddle synchronization fails. A dedicated, tenant-scoped retry reuses the stored profile and existing customer, address, and business mappings; an already synchronized retry is a provider-call-free no-op. Provider failures are logged by safe step (customer, address, business, or active-subscription identity) with provider status and error code, while the customer sees no provider identifiers or diagnostics. Once a saved profile is pending or failed, new plan and quantity mutations fail closed until synchronization succeeds; cancellation and legacy subscriptions that have no saved billing profile retain their existing behavior.

Paddle transaction states remain distinct. `paid` records that money was received and is displayed as `Payment received` - processing; it does not complete subscription reconciliation. `completed` remains the authoritative processed-payment signal and is displayed as `Paid`.

Organization Profile Save And Pending Logo Safety

Organization Profile accepts National, International, and Both through a controlled selector and normalizes those values to the existing uppercase codes. Missing or invalid customer input returns the same page with preserved form values. Pending organization logos reuse the established image decoder and 4 MB limit, allow PNG/JPG/WEBP only, enforce minimum dimensions, and use an opaque unique filename plus atomic file replacement. A database or later save failure rolls back organization changes and removes the newly written file; an empty upload retains the current logo.

Pending logos currently use the application-local `static/uploads/saas/pending_logos` directory and store only a relative public path in `PendingOrganization`. This is not durable on an ephemeral production filesystem. Persistent disk or object storage requires a separate deployment and storage-architecture decision; this correction does not introduce one. The initial Account Setup state is compact: one setup title, one explicit POST start action, and the existing eight-step journey.

Branch Setup never totals nullable estimates in Jinja. The service normalizes legacy/missing display values to zero in Python, while browser saves still require explicit non-negative whole numbers for every active branch. New pending branches receive explicit zero defaults.

After a pending logo is saved, Organization Profile and the shared School Workspace identity area show it beside the organization name while retaining the official TIS logo as platform branding. Checkout keeps the pending reference. Paid and demo provisioning already converge on `create_workspace_records()`, which copies the validated pending image into the established primary `SchoolGroupLogo` slot. The operational shell renders that organization-owned asset through its protected route and keeps the TIS logo separate. A referenced pending file that is missing at activation now fails provisioning instead of silently activating without the customer logo.

Post-Verification Sign-In Safety

Successful email verification redirects by HTTP 302 to the GET `/saas/login` page. The form submits by POST to `/saas/auth/login`. A newly verified account with no School Workspace Setup record continues to the GET `/saas/account` dashboard, where the existing explicit start action may create the setup record by POST. Login continuations accept only known customer GET destinations; POST-only, malformed, traversal, and external targets fall back to Account Setup. A defensive GET request to `/saas/auth/login` redirects to the normal sign-in page instead of exposing FastAPI's raw 405 response.

Public Subscription Entry

The landing hero Subscribe Now action scrolls to the public pricing section. Starter, Professional, and Enterprise AI then enter the public TIS Account signup route with an allowlisted preferred-plan code. That preference is non-authoritative: it creates no plan selection, checkout, payment attempt, or Paddle object, and it is applied only after School Workspace Setup confirms the plan remains eligible across branches, system users, and teachers. Invalid or undersized preferences are ignored or cleared safely. Custom remains a contact-only path.

Three-Dimension Subscription Capacity

Self-service subscription eligibility is organization-wide across three independent persisted limits: branches, tenant operational staff users, and teacher records. Branch Setup stores required non-negative system-user and teacher estimates on each active onboarding branch; legacy organization totals are assigned to the primary branch only when no branch estimates exist, after which organization totals are derived summaries.

Before confirmed payment, capacity authority is the active onboarding branch count plus the greater of each branch-estimate total and the same workspace's actual active count. After activation, actual active tenant operational users and active teacher records are authoritative. Every distinct active tenant User counts as staff regardless of role, title, position, or internal-test attribution. An operational teacher-user with an active Teacher record consumes one staff slot and one teacher slot. Platform users and account-only SaaS identities do not consume tenant staff capacity. Starter is 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Exceeding any Enterprise AI limit requires the contact-only Custom path and cannot create Paddle checkout.

Active Subscription Management uses that same three-dimension authority. The Review Capacity flow accepts proposed branch, system-user, and teacher totals, selects the minimum eligible plan from all three, and records the capacity snapshot on a generated plan-change preview. Branch growth may therefore produce one combined plan-and-quantity upgrade, while system-user and teacher growth affects plan eligibility only. Paddle quantity always remains the branch quantity; user and teacher counts never become billable units.

Plan downgrades remain scheduled for the next billing boundary and are checked against all three live operational counts both before submission and again when provider evidence says the downgrade is effective. If capacity no longer fits, local activation stops in manual review and the current safe entitlement state is preserved. Quantity reductions receive the same effective-date capacity check. Cancellation remains provider-scheduled at paid-period end, preserves tenant data and access through that date, and may be reversed while provider lifecycle rules allow it.

M1 enforces post-activation growth through `saas/commercial_authority_service.py`. The facade composes, rather than duplicates, workspace classification/lifecycle, `WorkspaceEntitlement`, `TenantProvisioningLink`, `SubscriptionContract`, `PaymentSubscription`, commercial state/access, demo lifecycle, and plan capacity. Paid branch capacity is the confirmed subscription quantity capped by the plan branch ceiling. Known teacher IDs are normalized and deduplicated; each blank legacy teacher row counts independently. Customer Demo and Internal Sandbox remain explicitly unmetered in M1, while missing or contradictory authority fails closed.

Capacity-increasing branch, user, teacher, academic-year, and provisioning paths lock the owning `SchoolGroup`, recount, evaluate the proposed final state, mutate, and commit in one transaction. Existing over-capacity workspaces keep their data and access but cannot further increase an exceeded dimension.

Returning Customer Journey And Commercial Expiry

Returning SaaS-account login resolves authoritative onboarding, demo request, account-to-workspace, workspace classification, lifecycle, and subscription evidence. Incomplete onboarding resumes, pending demos open request status, active demo or paid tenants enter operational login, unpaid customers reach subscription setup, and expired customers receive branded expiry guidance. Operational login and protected requests run the commercial guard before branch, academic-year, or workspace page work; expected expiry never becomes a 500.

Paid-workspace access uses the same tenant-link and `SubscriptionContract`-linked `PaymentSubscription` selected by `saas.entitlement_service`. A newer stale, orphaned, or unrelated organization-level subscription row cannot replace that authority, and Subscription Management and operational access consume that same resolution. An unresolved, pending, failed, expired, canceled, or abandoned plan upgrade remains a separate change request and does not revoke the currently active plan. For example, active Professional access remains available while an Enterprise AI upgrade is `payment_pending`. The specialized plan-change `subscription.updated` handler synchronizes provider subscription status while retaining the existing two-signal rule before the target plan becomes authoritative.

For a production subscription whose provider state is authoritative but whose local status is stale, replay the attributable stored, signature-verified Paddle webhook through the existing webhook reconciliation path. Do not repair commercial status fields manually. Confirm that `subscription.updated` synchronizes the current `PaymentSubscription.status`; a pending plan change must still wait for its separate required provider payment signal before the target plan becomes authoritative.

Commercial access distinguishes active, trialing, payment processing, past due, paused, canceled within a paid period, expired, suspended, archived, and inconsistent states. Canceled subscriptions retain current entitlements only through their confirmed current-period end. Ambiguous evidence fails closed and uses verification guidance rather than falsely claiming expiration. Renewal guidance appears only for a genuinely expired commercial state.

Expired demos select real public plans and intervals against the existing organization and actual active operational branches. Checkout and confirmed payment reuse the existing Paddle and conversion authorities, preserving the `SchoolGroup`, workspace UUID, tenant link, users, branches, permissions, and data. The Next.js landing derives Request Demo, Subscribe, Sign In, and Open TIS App from `NEXT_PUBLIC_TIS_APP_BASE_URL`. Customer

communications name the TIS team; internal role names and audit evidence remain unchanged.

M8B9 Demo Operations And Access Profiles

M8B9 adds Platform Owner-only orchestration in `saas/demo_operations_service.py` and policy resolution in `saas/demo_access_service.py`. Owners can expire, reactivate, set an unbounded future expiry, send final-day reminders, invoke the existing lifecycle processor, and choose Standard, Full, or Custom access at workspace or tenant-validated branch scope. M8B8 usage history is never reset; permissions, commercial lifecycle, and feature entitlement remain separate fail-closed checks. Material changes reuse M8B7 communications and every attempt is durably audited.

M8B8 AI Entitlement Foundation

M8B8 centralizes AI access in `saas.ai_entitlement_service`. A controlled registry owns stable feature identifiers and temporary reviewed plan mapping. Customer Demo receives two successful uses per feature; pre-execution reservations block concurrent excess attempts, while failed/no-result work releases capacity without consuming. Durable counters and operation events are scoped by SchoolGroup, feature, and separate internal/demo/paid metric context. Existing tenant resolution, `ai.use` permission, commercial state, demo expiry, and `paid module.ai` entitlement remain distinct authorities. No AI business tool or M8B9 operational control is included.

M8B7 Demo Customer Journey

M8B7 makes normal Platform Owner approval orchestrate the existing independently retryable provisioning service. It adds a durable branded demo email outbox, deduplicated Platform Owner Notification Center events, a shared-shell active-demo indicator, atomic Day 6/expiry communication intents, and coherent expired-demo conversion after authoritative confirmed payment. The same PendingOrganization and then exactly one SchoolGroup are preserved. Production must schedule `python scripts/process_demo_lifecycle.py --apply`; dry-run remains the default. M8B8 and M8B9 are not included.

This is the first file future Codex or ChatGPT coding conversations should load. It is a compact project onboarding reference; detailed source of truth remains in the other Markdown docs.

What TIS Is

TIS is Teacher Information System, a developing SaaS academic operations platform for schools and school groups. It connects teacher information, staffing and workload planning, academic calendars, observations, branch context, SaaS onboarding, billing, provisioning, and future AI-assisted academic decision support.

Public URLs:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>
- 1 Render must set `TIS_PUBLIC_BASE_URL=https://app.tisplatform.com` so transactional emails and background Workspace Activation emails generate production login and static asset URLs instead of local-development fallbacks.

Important routes:

- 1 Operational login: `/login`
- 1 SaaS signup: `/saas/signup`
- 1 SaaS login: `/saas/login`
- 1 SaaS account: `/saas/account`

1 Platform console: /platform

Current Architecture

The operational app is a FastAPI application at the repository root.

The FastAPI web process never creates tables or runs pending migrations during import, startup, middleware, or background threads. Render must run `python scripts/run_migrations.py` as its Pre-Deploy Command; only after that command succeeds may Render activate the new web version. The Start Command is `uvicorn main:app --host 0.0.0.0 --port $PORT`, which constructs the app, performs only lightweight process-local startup checks, and binds without PostgreSQL DDL. This boundary applies consistently to production, local development, and tests.

Key files and folders:

- 1 `main.py`: primary FastAPI app and many route handlers.
- 1 `auth.py`: authentication, roles, platform identity helpers, permissions, sessions.
- 1 `authorization.py`: protected route rules and access-denied handling.
- 1 `permission_registry.py`: permission keys, groups, defaults, and developer-assignable permissions.
- 1 `location_service.py`: global location picker lookup/validation. Must stay memory-conscious and scoped; do not restore full unbounded dataset parsing for normal picker requests.
- 1 `ui_shell.py`: shared app shell/navigation/page metadata.
- 1 `models.py`, `database.py`, `db_migrations.py`: data model, DB setup, local schema repair/migration logic.
- 1 `routers/`: modular operational routes.
- 1 `saas/`: SaaS account, onboarding, payment, billing, and provisioning services/routes.
- 1 `saas/entitlement_service.py`: provider-confirmed commercial entitlement and paid branch-capacity resolution.
- 1 `saas/commercial_authority_service.py`: unified operational commercial access, capacity counters, structured decisions, and tenant row locking.
- 1 `saas/commercial_portal_service.py`: source-aware customer commercial presentation for promo-backed workspaces without duplicating commercial decisions.
- 1 `saas/subscription_portal_service.py`: customer subscription portal view model.
- 1 `saas/subscription_change_service.py`, `saas/subscription_plan_change_service.py`, and `saas/subscription_cancellation_service.py`: provider-authoritative quantity, plan, and cancellation workflows.
- 1 `saas/subscription_lifecycle_service.py`: centralized lifecycle state and allowed-action policy.
- 1 `saas/billing_history_service.py`: provider-sourced billing history and short-lived invoice download resolution.
- 1 `saas/billing_identity_service.py`: explicit organization billing profile validation and Paddle customer/address/business synchronization.
- 1 `saas/payment_lifecycle_reconciliation_service.py`: guarded reconciliation for attributable finalized payment evidence.
- 1 `scripts/sync_paddle_price_ids.py`: environment-specific Paddle initial checkout price mapping sync into `subscription_plan_prices.provider_price_id`.
- 1 `templates/`: Jinja templates.
- 1 `static/`: static assets and generated documentation output.

1 tests/: pytest coverage.

The public marketing website is separate:

1 tis-landing-website/

1 Next.js / Node runtime

1 Source of truth for public landing implementation

1 M8 landing integration exposes two public conversion paths through NEXT_PUBLIC_TIS_APP_BASE_URL: Request a Demo routes to /saas/signup?intent=demo, while Subscribe Now routes to /saas/signup?intent=subscribe.

1 The selected intent is preserved through TIS Account signup and School Workspace Setup, then emphasized on the customer-safe commercial-choice page without removing the customer's ability to choose either path.

1 Customer demo eligibility is reserved once per normalized organization domain. Existing Customer Demo history, including expired or converted demos, remains ineligible for a second demo; internal sandbox history does not consume customer eligibility.

Legacy FastAPI landing files are not the source of truth:

1 templates/landing.html

1 static/landing/landing.css

Engineering Handbook

For deeper onboarding, read:

1 docs/engineering/README.md

1 docs/engineering/TIS_MODULE_MAP.md

1 docs/engineering/REPOSITORY_ARCHITECTURE.md

1 docs/engineering/USER_AND_SYSTEM_FLOWS.md

1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

1 docs/engineering/DEVELOPMENT_STANDARDS.md

1 docs/engineering/UI_UX_DESIGN PHILOSOPHY.md

1 docs/engineering/PRODUCT_ROADMAP.md

1 docs/engineering/REJECTED_DECISIONS.md

1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md

1 docs/engineering/AI_OPTIMIZATION_GUIDE.md

1 docs/engineering/PROJECT_GOVERNANCE.md

1 docs/engineering/KNOWLEDGE_LIFECYCLE.md

1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

1 docs/engineering/SELF_EVOLVING_WORKFLOW.md

1 docs/engineering/AI_CODING_WORKFLOW.md

These files explain module ownership, repository boundaries, end-to-end flows, and what must not be changed casually.

Paddle Initial Checkout Configuration

Initial subscription checkout uses Paddle price IDs stored in `subscription_plan_prices.provider_price_id`. Use `scripts/sync_paddle_price_ids.py` with a structured sandbox or production mapping JSON to configure these values. Paddle credentials and endpoints remain environment variables. Real mapping files are ignored by Git; keep sandbox and live provider price IDs separate. If a mapping is missing, checkout fails closed before Paddle is called and customers receive a support-oriented Secure Payment message while internal diagnostics keep plan code, billing interval, and currency context.

Paddle transaction checkout uses a dedicated public SaaS payment launcher at `/saas/payment`. Configure `PADDLE_CHECKOUT_BASE_URL` to that page so Paddle returns transaction checkout URLs with `_ptxn` appended to the launcher instead of the operational app root. The launcher uses Paddle.js with `PADDLE_CLIENT_TOKEN` and `PADDLE_ENVIRONMENT`; never expose `PADDLE_API_KEY` in HTML or JavaScript.

Domains And Routing

The public website lives at `https://tisplatform.com`. The app portal lives at `https://app.tisplatform.com`.

SaaS account routes are under `/saas`. Platform-owner SaaS administration routes are under `/saas-admin`. Operational tenant workflows use routes such as `/dashboard`, `/teachers`, `/subjects`, `/planning`, `/timetable`, `/academic-calendar`, and `/observations`.

Completed M1-M5 Milestones

M1: SaaS identity foundation and separation between platform, tenant, and SaaS account identities.

M2: SaaS onboarding flow for organization, contacts, branches, academic setup, and review.

M3: Billing and plan foundation with plan catalog, checkout, billing status, and payment service boundaries.

M4: Tenant provisioning foundation with pending organizations, provisioning jobs, retry/run actions, and platform owner oversight.

M5: Platform access and owner controls, including platform owner/developer identities, permissions, and platform console behavior.

Platform Owner pending-organization views use a centralized owner lifecycle projection. The pending queue includes only records still in draft/setup, review, subscription checkout/payment, or incomplete/recoverable workspace activation. An active tenant link plus one coherent confirmed subscription, payment, contract, and active SchoolGroup resolves as Active Tenant and is excluded from pending counts. Completed-provisioning evidence that does not reconcile across those records is excluded from the normal queue and shown as Lifecycle Review Required in retained Organization Records. Historical onboarding fields are not rewritten by this projection.

Completed M7 Subscription Management

M7 includes a normalized entitlement foundation, a customer Subscription Management portal, paid branch-quantity management, upgrades and scheduled downgrades, Paddle-authoritative previews and proration, scheduled cancellation and reversal, a centralized lifecycle/action policy, provider-sourced billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Commercial state fails closed when ownership, provider evidence, or local relationships are ambiguous. TIS does not independently calculate replacement monetary values. Immediate changes require provider-confirmed outcomes; scheduled changes remain pending until provider/webhook evidence reaches the effective boundary.

M8B-1 Workspace Classification Foundation

M8B-1 adds metadata-only workspace classification without changing customer behavior. SchoolGroup now owns an immutable workspace UUID, classification (`internal_sandbox`, `customer_demo`, or `customer_paid`), and lifecycle (`provisioning`, `active`, `suspended`, or `archived`). `PendingOrganization.workspace_intent`, `SaaSAccount.account_purpose`, and `User.is_internal_test_identity` preserve the corresponding pre-provisioning and identity intent.

All records that predate M8B-1 are confirmed test data. The controlled backfill classifies them as internal sandbox/test records; it is dry-run by default, transactional, idempotent, and records completion in `schema_migrations`. The separate diagnostic is read-only and reports tenant, onboarding, and Paddle relationship presence without exposing provider identifiers or secrets. Platform Owners can inspect workspace UUID, classification, and lifecycle in `/platform`; Platform Developers and tenant users do not receive that metadata block.

M8B-1 does not use classification for billing, entitlements, permissions, tenant isolation, reset eligibility, or customer routing. Demo workflows, conversions, Al-Andalus migration, memberships, and commercial-state resolution remain later work.

M8B-2 Commercial State And Entitlement Foundation

M8B-2 adds a read-only commercial decision layer without changing customer access or billing behavior. `WorkspaceEntitlement` is the workspace-level entitlement envelope, `WorkspaceEntitlementValue` reuses the existing normalized entitlement catalog for features and limits, and `BranchEntitlement` optionally records whether a branch inherits, is explicitly active, or is commercially inactive.

The commercial resolver combines persisted workspace classification/lifecycle metadata with one coherent effective workspace entitlement. Customer-paid workspaces additionally require the existing M7 confirmed subscription entitlement resolution and matching `PaymentSubscription`; M8B-2 does not calculate billing state or contact Paddle. Missing, ambiguous, cross-tenant, orphaned, or invalid relationships fail closed to manual review.

These results are visible only to Platform Owners in `/platform`. They are not wired into tenant authorization, feature restrictions, branch behavior, onboarding, Paddle, demo expiration, or customer workflows. Internal sandbox workspaces created outside the migration retain a compatibility-only implicit entitlement so existing development flows remain unchanged.

M8B-3 Demo Request Workflow

M8B-3 adds the first customer-facing commercial choice after completed onboarding: Request Demo or Subscribe Now. Subscribe Now continues into the existing plan-selection and Paddle workflow unchanged. Request Demo validates the verified account and completed organization, contact, academic, and branch setup before creating a `SaaSDemoRequest` in Pending Review.

The request records the requester, pending organization, submission time, customer-demo classification intent, provisioning commercial-state snapshot, and a fail-closed pre-provisioning entitlement snapshot. It does not create a `SchoolGroup`, workspace entitlement, subscription, checkout, or Paddle record. The workflow is separate from the legacy public marketing `DemoRequest` lead table.

Platform Owners have a searchable and sortable review queue. Approval records a review decision only; rejection requires a reason; owner cancellation and customer withdrawal are limited to Pending Review. Every transition creates durable audit and internal-notification events, but M8B-3 sends no email and performs no demo provisioning or activation.

M8B-4 Demo Workspace Provisioning And Activation

M8B-4 adds a Platform Owner-only provisioning action for an approved SaaS demo request. The service fails closed unless the approval review, organization, customer-demo intent, pre-provisioning commercial snapshot, and entitlement snapshot remain coherent and no workspace has already been provisioned.

Demo provisioning reuses the paid provisioning engine's shared workspace-record builder for the SchoolGroup, branches, academic year, owner user, account link, role permissions, and branding. It creates no Paddle, payment, subscription-contract, payment-subscription, checkout, or billing record. A demo-backed TenantProvisioningLink identifies the operational tenant without fabricating a paid contract.

Workspace records, the demo entitlement, tenant link, request linkage, and activation are committed atomically. A failed attempt rolls the workspace changes back, leaves the request Approved and unprovisioned, and records a retryable failure outcome. Successful activation sets the SchoolGroup and demo entitlement active, records activation metadata and audit/internal events, and prevents duplicate provisioning. M8B-4 sends no email and implements no expiration, scheduler, login restriction, or conversion behavior.

M8B-5 Standard Customer Demo Lifecycle

Every customer-demo workspace lasts exactly seven days from the M8B-4 `activated_at` timestamp. Day 6 reminder and Day 7 expiration timestamps are derived from that single authority, calculated with timezone-aware UTC values, and displayed in the onboarding organization's IANA timezone. Missing or inconsistent timestamps fail closed.

`saas.demo_lifecycle_service` is the authoritative read-only resolver and independently callable processor. It resolves Active, Reminder Due, Expired, Suspended, or Manual Review; creates idempotent internal Day 6 notifications for the requesting SaaS account and active Platform Owners; and atomically ends the demo entitlement and suspends the SchoolGroup at expiration. Workspace users, branches, and all tenant data remain preserved.

The operational authentication middleware checks customer-demo commercial access on every protected request, including existing sessions. Active and reminder-due demos continue normally. Expired or ambiguous demos are redirected to the subscription activation experience, while protected API/download requests receive a customer-safe 403. Platform users, paid workspaces, internal sandboxes, public/authentication routes, secure logout, and SaaS subscription routes are unaffected. Access-block auditing is deduplicated.

Run the lifecycle processor safely with `python scripts/process_demo_lifecycle.py --dry-run`; use `--apply` only for scheduled execution after validating the report. M8B-5 sends no email and adds no extension, conversion, archive, deletion, or read-only expired mode.

M8B-6 Demo-To-Paid Conversion

An active Customer Demo may enter the existing subscription checkout without closing or recreating its operational tenant. After `transaction.completed` establishes one confirmed active M7 subscription for the same pending organization, `saas.demo_conversion_service` atomically converts the existing SchoolGroup from `customer_demo` to `customer_paid`.

The conversion preserves the workspace UUID, SchoolGroup, tenant link row, organization, branches, users, permissions, academic data, and all demo/request/audit history. It ends the demo entitlement, creates a paid entitlement linked to the confirmed PaymentSubscription, moves branch entitlement links, and replaces the tenant link's demo source with the confirmed SubscriptionContract. Existing M7, workspace-entitlement, and commercial-state resolvers must then resolve the same tenant as Customer Paid Active.

Conversion Requested, Processing, Completed, and Failed states are durable and retryable. Any conversion failure rolls back workspace mutations while retaining confirmed provider payment records. Expired, invalid, ambiguous, cross-tenant, internal-sandbox, paid, or already-converted workspaces fail closed. Completed conversions no longer enter demo reminder or expiration processing.

Test Workspace Reset Dependency Rule

The Platform Owner-only test workspace reset keeps its existing preflight and single-transaction rollback behavior. Before it deletes scoped operational User records, it deletes SubscriptionChangeRequest rows scoped by the selected workspace's authoritative `school_group_id`. It also deletes selected workspace-entitlement values and

branch-entitlement children before the selected `workspaceEntitlement`, which is removed before the final `SchoolGroup`. This prevents user, account, contract, subscription, and entitlement foreign-key references from blocking removal while preserving records belonging to every other workspace. Pre-analysis and structured deletion diagnostics report the same scoped record set.

The same Platform Owner-only test reset is a clean-room exception for internal M8 testing only. Within its existing single transaction, it deletes the selected pending organization's linked demo request, domain-eligibility reservation, review and audit history, demo provisioning/lifecycle records, and demo-to-paid conversion history before removing their parents. It also resolves the selected organization's domain through the same customer-demo resolver and may remove a detached reservation only when no other organization, request, workspace, or customer account uses that domain and the reservation has no historical manual-review evidence. Analysis exposes a separate list of safe detached reservation IDs; deletion consumes that exact list, flushes before parent deletion, and verifies that none of those IDs remain. A mismatch, remaining row, conflicting ownership, or historical ambiguity blocks or rolls back the reset. This allows the same test email and organization domain to complete a new internal journey after a reset. The normal customer one-demo-per-domain reservation policy is unchanged; global plans, prices, Paddle records, platform configuration, and records from other organizations remain preserved.

Historical detached demo-domain reservations whose owning organization and account were already removed are managed separately through the Platform Owner-only `/saas-admin/demo-eligibility-maintenance` workflow. It scans eligibility rows, reuses the authoritative domain normalization/resolution rules, and fails closed when any matching organization, account, request, tenant-profile workspace, provisioning record, subscription evidence, conversion, or manual-review marker remains. A confirmed deletion locks and re-analyzes one exact eligibility ID, deletes only that ID, flushes, verifies absence, and commits through the owner route. Durable audit records identify the owner, eligibility ID, domain, previous status, timestamp, and maintenance reason. This administrative repair path does not alter normal Customer Demo eligibility or the clean-room reset.

Current SaaS Account Verification State

Phase 1 TIS Account email verification recovery is accepted. Valid verification links now mark the SaaS account email verified/active and redirect to the TIS Account login page with a professional success notice so the customer can continue school workspace setup.

Expired or invalid verification links no longer dead-end. They show a recovery page with a resend verification form. Resend verification handles unverified accounts, already verified accounts, and unknown email addresses with safe customer-facing messaging that does not reveal account existence. Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.

This Phase 1 verification recovery work did not change payment, billing, provisioning, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current SaaS Customer-Facing Language State

Phase 2 TIS Account wording cleanup is accepted for customer-facing account and school workspace setup pages. Customer-visible account/setup pages now avoid presenting "SaaS" and technical identifiers as product language, while internal `/saas` routes, modules, models, and stored statuses remain unchanged.

The customer journey should use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation". Customer templates should label internal billing, payment, onboarding, and activation statuses through customer-safe display labels instead of raw database statuses such as `tenant_active`, provisioning states, checkout session states, provider identifiers, plan IDs, school group IDs, attempt UUIDs, or provider subscription/transaction IDs.

The shared TIS Account customer shell uses an official TIS logo image so customer account/setup forms inherit official branding. The light account shell uses the full-color horizontal logo variant, and transactional account emails use an existing official dark-blue wordmark asset. This wording/logo pass did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work and was not implemented.

Current TIS Account Guided Setup Framework State

Phase 3A shared guided setup framework is accepted for the TIS Account dashboard only. The customer account shell now supports a shared setup console with the official TIS logo, an 8-step journey stepper, a current-step/status area, one primary next action, and concise guidance. The account page uses this framework to answer "What should I do next?" without dashboard statistics.

The 8 customer-facing journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. The display state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

This Phase 3A work did not redesign onboarding forms, subscription/payment pages, or billing/status pages. It did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, or admin views.

Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the Phase 3A shared shell: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. The pages now use consistent guided-wizard sections, a single shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and lower visual clutter while preserving all existing form actions, field names, validation behavior, draft behavior, and onboarding state transitions.

Phase 3B did not redesign subscription/payment/billing/status pages and did not change payment, billing, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, or admin views.

Phase 3C redesigned the customer-facing Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages on top of the Phase 3A shared shell and Phase 3B guided style. These pages now use one shared-shell primary CTA, customer-safe payment/activation labels, clearer browser-return messaging, and explicit guidance that TIS Platform access becomes available after Workspace Activation.

Phase 3C did not change payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, stored statuses, or admin views.

Current Priority

Current priority is automatic KMS synchronization enforcement and reliable post-M7 engineering context:

- 1 Markdown is source of truth.
- 1 PDF is generated snapshot.
- 1 Change history preserves chronological change context.
- 1 ADRs preserve major decisions.
- 1 Module history preserves deeper area-specific evolution.
- 1 Platform Owner Knowledge Center is implemented as a read-only owner utility with manifest-backed document titles, summaries, logical groups, client-side search, category/module/freshness filters, and latest-activity ordering.

- 1 The Knowledge Center uses protected routes for PDF view/download and source-specific pdf_page deep links; it does not link directly to static PDF paths.
- 1 KMS v3.0 Phase 3A adds a true engineering handbook with module map, repository architecture, workflows, and developer onboarding.
- 1 KMS v3.0 Phase 3B adds database architecture, development standards, UI/UX philosophy, product roadmap, and stronger human/AI developer guidance.
- 1 KMS v3.0 Phase 3C adds rejected decisions, visual documentation framework, AI optimization guidance, project governance, and decision traceability.
- 1 KMS v3.0 Phase 3D completes KMS v1.0 lifecycle standards, dependency mapping, AI coding workflow, and future automation roadmap.
- 1 Root AGENTS.md makes KMS onboarding mandatory for Codex tasks.
- 1 .kms-impact.yml is the machine-readable task declaration.
- 1 scripts/kms.py sync is the single local synchronization command; it validates KIA before writing, regenerates the PDF/manifest, then verifies freshness.
- 1 scripts/kms.py check is the single read-only local and CI validation command.
- 1 scripts/check_kms_impact.py validates major-change classification, declared Markdown updates, and generated-artifact freshness.
- 1 GitHub Actions block pull-request integration and production deployment when KMS validation fails.

Smart Timetable Architecture Baseline

ADR 0026 governs timetable evolution. Planning remains authority for section demand, subject requirements, assigned teachers, and HRT fallback; one placement means one teaching period, and Subject.weekly_hours remains the compatibility weekly-period authority. Timetables are durable SchoolGroup/Branch/Academic-Year versions. Active selection is a separate unique exact-scope pointer, not a version Boolean. Active, superseded, and archived history is immutable; drafts are mutable.

Stages 2-5.1 provide models, migrations, snapshots/fingerprints, lock persistence, version services, readiness, comparison, validation/publication, durable generation, and version-aware views/exports. Stage 3.5 supplies one composed per-day timeline and solver-ready teaching slots. Stage 5.1 adds worker-isolated CP-SAT, independent candidate validation, Generate/Regenerate actions, and atomic unpublished results. Existing and published placements remain unchanged unless the separate publication flow is used. Availability, room/resource, preferences, quality scoring, and broader Stage 5.2 UX remain future work.

Critical Rules

- 1 Do not touch SaaS flows unless explicitly approved.
- 1 Do not touch operational logic unless required by the approved task.
- 1 Do not touch database migrations or tis.db unless explicitly approved.
- 1 Do not weaken tenant isolation.
- 1 Do not bypass permissions or platform owner checks.
- 1 Do not merge platform user, SaaS account, and tenant user concepts.
- 1 Do not change landing page implementation unless explicitly approved.
- 1 Do not add a KMS regenerate button until explicitly approved.
- 1 Do not push or commit unless explicitly requested.

- 1 Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or warning-level debug spam on normal requests.

KMS Policy

Every implementation must include a Knowledge Impact Assessment:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

If included docs change, synchronize:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Development Workflow

- 1 Read this file first.
- 2 Read root AGENTS.md, docs/TIS_MASTER_CONTEXT.md, and docs/PROJECT_STATE.md.
- 3 Read docs/engineering/README.md.
- 4 Read docs/engineering/DEVELOPMENT_STANDARDS.md.
- 5 Read docs/engineering/AI_OPTIMIZATION_GUIDE.md.
- 6 Read docs/engineering/AI_CODING_WORKFLOW.md.
- 7 Read relevant engineering docs, ADRs, module history, and supporting docs.
- 8 Inspect code before editing.
- 9 Keep changes scoped.
- 10 Update .kms-impact.yml and affected KMS docs when meaningful behavior, architecture, product state, module map, repository ownership, data model, design philosophy, roadmap, governance, decision traceability, automation, lifecycle, or workflow changes.
- 11 Run scripts/kms.py sync if included source docs changed.
- 12 Run scripts/kms.py check for final read-only validation.
- 13 Run implementation validation.
- 14 Report KIA in final response.

Landing Page Situation

The public landing implementation is in tis-landing-website/. Marketing docs live under docs/marketing/. Do not modify legacy FastAPI landing files unless explicitly approved.

Next Planned Work

Review the automatic KMS enforcement baseline, keep M7 subscription documentation synchronized as follow-up fixes evolve, and later consider an explicit owner-only regenerate action. Any regenerate action may rebuild artifacts from reviewed Markdown only and must not rewrite source prose.

Timetable feasibility authority (2026-08-30)

Deterministic timetable readiness ends at `configuration_complete`. The `timetable_feasibility_verifications` ledger stores a hard-only CP-SAT result by SchoolGroup, branch, academic year, immutable snapshot, and full input fingerprint. Generate is permitted only for an exact verified fingerprint. The validated placements are reused as an optimization hint and as the independently validated timeout fallback; soft objectives do not participate in verification. `stale_input` is an orthogonal Draft lifecycle state, not structural incompleteness: when it is the only finding, current inputs remain eligible for a fresh feasibility verification. The old Draft stays stale and unpublishable; after the current fingerprint is verified, Regenerate creates a new version from that protected source. Historical runs from older authority fingerprints remain auditable but do not drive the current action state. A regeneration source is the current populated, mutable, unpublished Draft with a usable placement arrangement, regardless of whether its origin is manual, generated, or regenerated. An empty starter Draft remains a Generate candidate; active, historical, superseded, and archived versions are never source candidates.

TIS Documentation Update Policy

docs/DOCUMENTATION_UPDATE_POLICY.md

This policy defines the non-negotiable rules for keeping the TIS Knowledge Management System reliable.

Source Of Truth

Markdown files under docs/ are the source of truth.

The PDF booklet at static/docs/TIS_Project_Reference_Booklet.pdf is a generated snapshot. It must never be edited manually. It must be regenerated whenever included Markdown source files change.

Required Knowledge Impact Assessment

Every approved implementation must end with a Knowledge Impact Assessment (KIA).

Required final report template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

A task is not complete until the KIA is included, even when no documentation changes are needed.

Every task must also update .kms-impact.yml. The declaration is the machine-readable form of the KIA and records knowledge_impact, a summary, affected areas, KMS files updated, a no-impact reason, and the explicit major-change override. scripts/check_kms_impact.py validates it against the actual Git diff.

knowledge_impact: yes requires changed authoritative Markdown. knowledge_impact: no requires a specific reason. When a path is conservatively classified as major but the change is genuinely non-behavioral, major_change_override: yes is allowed only with that written reason.

When Documentation Must Be Updated

Update documentation when a task changes:

- 1 product vision or positioning,
- 1 architecture or module boundaries,
- 1 SaaS signup, login, onboarding, billing, payment, or provisioning flows,
- 1 platform owner or permission behavior,
- 1 operational workflows such as calendar, planning, timetable, teachers, subjects, or observations,
- 1 deployment assumptions,
- 1 branch/release/project status,
- 1 landing page source-of-truth or visual strategy,
- 1 roadmap or known issues,
- 1 documentation system behavior.

Which Docs To Update

Use this guide:

- 1 docs/AI_PROJECT_CONTEXT.md: compact onboarding context for future AI coding conversations; update when the high-level project situation changes.
- 1 docs/TIS_MASTER_CONTEXT.md: durable product, architecture, workflow, roadmap, and critical rules.
- 1 docs/PROJECT_STATE.md: current branch strategy, priorities, milestone status, known issues, and next work.
- 1 docs/CHANGE_HISTORY.md: chronological summary of meaningful changes.
- 1 docs/adr/: major architectural or product decisions.
- 1 docs/history/: deeper module-specific before/after history.
- 1 Supporting docs under docs/marketing/ or other folders when those areas change.

ADR Rule

Create or update an ADR when a change affects a major long-term decision, including identity boundaries, payment architecture, provisioning strategy, deployment architecture, public website architecture, documentation governance, or visual system strategy.

Do not create ADRs for routine bug fixes unless they introduce or reverse a significant decision.

Module History Rule

Update docs/history/<module>/ when a meaningful module area changes and the previous documented state should be preserved.

docs/CHANGE_HISTORY.md remains the chronological summary. Module history stores deeper context.

KMS Synchronization

Validate KIA, regenerate the PDF and manifest, and verify freshness with:

```
..\venv\Scripts\python.exe scripts\kms.py sync
```

The generator must remain dependency-light:

- 1 use existing report lab,
- 1 no LaTeX,
- 1 no Playwright or Chromium,
- 1 no external network calls,
- 1 no system font dependency.

Complete read-only validation:

```
..\venv\Scripts\python.exe scripts\kms.py check
```

The check command must never write documentation. GitHub Actions run it for pull requests and dev; the production deployment workflow requires it before triggering deployment from master. The sync command writes generated artifacts only and never rewrites authoritative Markdown.

Navigation And Catalog Integrity

Every included Markdown source must have a usable front-matter title or level-one Markdown heading. Placeholder, empty, punctuation-only, or excessively long titles are invalid.

The approved document categories are:

- 1 core
- 1 engineering
- 1 decisions
- 1 history
- 1 marketing
- 1 supporting

The approved module values are maintained in `kms_catalog.py`. Declared front-matter taxonomy must use the exact approved slug, and any declared category must match the source path's category. New modules require a reviewed catalog update and focused validation.

`docs/KMS_NAVIGATION.md` may link only to existing, listed Markdown sources inside `docs/` through relative POSIX-style paths. External URLs, parent-directory escapes, root-relative links, non-Markdown targets, missing targets, and unlisted authoritative documents fail validation.

The manifest source inventory must exactly match the normalized generator source list in deterministic order. Every source must have a positive integer `pdf_page`, source pages must increase in generator order, and no source page may exceed the generated PDF page count.

Prohibited KMS Content

KMS documentation describes system design only. Never include customer or organization information, personal data, subscription rows, invoices, transactions, real production identifiers, real webhook payloads, credentials, secrets, environment values, database row contents, or test-customer personal details.

Reporting Rule

Every final implementation report must mention:

- 1 whether knowledge was impacted,
- 1 which docs changed,
- 1 whether change history changed,
- 1 whether an ADR was needed,
- 1 whether module history changed,
- 1 whether the PDF was regenerated,
- 1 whether `AI_PROJECT_CONTEXT.md` changed,
- 1 validation results.

App Behavior Rule

The application may later detect documentation freshness and expose docs through owner-only protected routes. It must not silently rewrite Markdown source docs. Source docs are edited through reviewed development work.

TIS Master Context

docs/TIS_MASTER_CONTEXT.md

Planning Subject Requirement Removal (Single And Bulk)

`routers/planning.py:_get_planning_requirement_removal_status` classifies every displayed Planning requirement using the same explicit-first/legacy- fallback authority the rest of Planning already renders through `(resolve_section_subject_demands)`, never a separate derivation: an untouched explicit `PlanningSubjectDemand` row is removable; a row `Curriculum Adjustment` has ever created or modified `(updated_by_user_id set)` is permanent, shown with an explicit "Protected (Curriculum Adjustment history)" explanation rather than a silently hidden action; a requirement resolved only from the Subject catalog with no explicit row at all is fallback; anything else is `not_found`. Both removable and fallback are removable through the same action - an explicit row is deleted, while a fallback requirement is removed by creating an explicit `is_active=False/weekly_periods=0` row. That row deliberately does NOT match a `Curriculum Adjustment` retirement stamp-for-stamp: it sets only `created_by_user_id` (the acting admin, for audit) and leaves `updated_by_user_id` NULL, so it stays classified removable/setup-only rather than immediately becoming permanent. Only a row `Curriculum Adjustment` itself has touched is ever permanent; a leftover setup-only row is automatically cleaned when its otherwise dependency-free section is deleted, preserving the remove requirement -> remove section -> remove subject workflow without exposing an implementation artifact. This closed a genuine gap: previously only explicit-row requirements ever showed a removal action, so identical-looking subject rows for the same teacher could differ in removability for reasons unrelated to the teacher.

POST `/planning/subject-requirements/remove` handles single and checkbox-selected bulk removal atomically, with selection supported across every section on the page in one request. Each expanded section provides a section-scoped **Select all** control and an adjacent trash-icon action. Its label reflects the actual selection as "Remove Subject Requirement" or "Remove N Subject Requirements"; the page-level action mirrors the same state. Every target is scope-validated (SchoolGroup/branch/academic year) and status-classified before any mutation; if any target is protected, invalid, or out of scope, nothing is removed and every blocked target is named with its reason - the same all-or-nothing contract as Subject bulk delete. Confirmation is required before both single and bulk removal. `TeacherSectionAssignment`, `TimetableEntry`, and `CurriculumAdjustmentAudit` rows are never touched. The prior round's GET `/planning/subject-demand/delete/{demand_id}` route remains functional for active untouched setup rows. Section delete itself now removes only inactive zero-period rows with `updated_by_user_id IS NULL`, in the same transaction and exact branch/year scope; active demand, Curriculum Adjustment history, and every other protected dependency still block.

Claude Code Repository Configuration

Claude Code is governed through three repository-native instruction layers: root `CLAUDE.md` provides the concise entry point, `.claude/skills/tis-kms/SKILL.md` defines the reusable KMS-governed working method, and `.claude/agents/tis-kms-developer.md` defines a specialized bounded subagent. They defer to root `AGENTS.md` and the authoritative KMS rather than duplicating project architecture as a competing source of truth. Every Claude Code task must still preserve the existing worktree, update `.kms-impact.yml`, synchronize the KMS when knowledge changes, run final validation, and avoid commits, pushes, deployments, production mutations, and `tis.db` changes unless explicitly approved.

Return-To Reopen-On-Load UI Pattern

`redirect_utils.py` centralizes the open-redirect guard (`safe_redirect_path`) and fragment-preserving redirect helpers previously private to `main.py`; `main.py` now imports them under their original names so every existing caller is unaffected. `static/js/reopen-on-load.js`, loaded globally from `base.html`, reopens whatever `<details>` element matches a target id taken from `window.location.hash` or an inline `window.__tisReopenTargetId`,

scrolling it into view, and is a silent no-op for a missing or stale id. Planning's "Remove demand" action (`routers/planning.py: delete_planning_subject_demand`) is the reference implementation: the section's `<details>` gets a stable id, the link passes `return_to` to that section's fragment, and the route's in-place blocked-render paths set the same reopen target without a redirect. This is not an SPA conversion and adds no browser storage. Subjects and Teachers were not changed; this establishes the pattern for later, incremental adoption elsewhere.

Planning And Subject Delete Dependency Guards

`routers/planning.py:delete_planning_section` and `routers/subjects.py`'s single/Bulk Delete perform a read-only dependency scan before any mutation and block with a specific, customer-safe message naming every blocking category, instead of raising an unhandled database error. Planning section delete checks `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent/CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule`. `TeacherSectionAssignment` is now a blocker like every other dependency rather than being silently deleted alongside the section, since no documented product rule required that cascade. Subject delete/Bulk Delete additionally check `TimetableEntry.subject_code`, `CurriculumAdjustmentAudit` source/target codes, and `SubjectDistributionRule.subject_code`, none of which had an application-level check before, so a Subject referenced only by one of those could previously be deleted and silently orphan that historical reference.

Planning subject demand is classified removable or permanent using the existing `PlanningSubjectDemand.updated_by_user_id` column, set only by Curriculum Adjustment (`curriculum_adjustment_apply_service._set_demand`) and never by the one-time setup backfill migration. A row with no `updated_by_user_id` has never been acted on by an admin - pure setup scaffolding - and can be hard-deleted through the new, permission-gated `GET /planning/subject-demand/delete/{demand_id}` action ("Remove demand" on the Planning page), after which the section or subject becomes deletable. A row Curriculum Adjustment has ever touched, active or retired, is genuine history that TIS preserves permanently and remains an unresolvable blocker with an honest explanation. Timetable placements follow the same principle without a new action: only entries in a mutable Draft can be removed, while published/active/superseded/archived placements are permanent history. Bulk Subject delete is atomic: any blocked Subject in the selection stops the whole batch, and every blocked Subject is named with its specific reason and, where applicable, the Remove-demand action that resolves it. No archive/ closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Guided Late-Stage Curriculum Adjustment

The administrator workflow at `/planning/curriculum-adjustments` is gated by `curriculum.adjust` and consumes the established preview/apply contracts. It supports grade, selected-section, and all-active-use scopes, displays section demand, teachers, capacity, scheduling rules, grouped configuration, and Draft impact, then requires explicit teacher selections and confirmation. Stale review responses return the user to a refreshed preview. Applying does not regenerate the Draft or change published history.

`requested_transfer_periods` is the preview/apply request authority. Per affected section, source after equals source before minus that count and target after equals target before plus it. Partial transfers retain active source demand; only zero retires it. The Subjects list resolves exact-scope Current/New section demand through the explicit-first service and shows either the uniform effective value or **Varies** with section values, without mutating catalog defaults.

`adjustment_type` distinguishes transfer from `reduce_only`. Reduce-only omits the target, changes only source demand, reduces the assigned source teacher's calculated load, and requires no teacher decision payload. The workflow retains one grade, selected sections, or all active uses of one exact subject code; it does not merge grade-specific subjects into one multi-grade transaction.

Atomic Late-Stage Curriculum Adjustment Application

`curriculum_adjustment_apply_service.py` owns the Stage 4 write transaction. Apply requires the exact Stage 3 request and fingerprint plus one explicit target-teacher decision per affected section. Duplicate successful fingerprints resolve through the audit ledger without applying twice. The service locks scoped Planning demand, assignments, rules, timetable versions, and active generation state before rebuilding the preview; any drift or blocker aborts before mutation.

Successful apply changes only the selected Current/New section demands. Source zero is retained as inactive retirement evidence, target demand becomes explicit, source assignment is removed only when retired, and target assignment follows only the confirmed teacher or unassigned decision. Chosen teachers must be exact-scope, target-allocated/qualified, and within final international capacity. Active source section rules are retired with zero demand; unresolved target rule arithmetic fails closed. Invalid source/teacher locks block apply. The unpublished Draft retains its snapshot and placements but becomes stale, loses approval, advances revision, and requires later regeneration. Published history and `TimetableActiveVersion` remain immutable.

Read-Only Late-Stage Curriculum Adjustment Preview

`curriculum_adjustment_preview_service.py` is the Stage 3 analysis boundary for future late-stage subject retirement/reduction/transfer workflows. It accepts only an exact SchoolGroup, branch, and academic year and selects Current/New Planning sections by grade, explicit section IDs, or active source-subject use. The preview uses `PlanningSubjectDemand`, retains missing-row fallback through the existing resolver, and performs no mutation.

Each section reports source/target before and proposed periods, current explicit or HRT-resolved teachers, ranked but uncommitted teacher options, projected capacity, Subject Scheduling Rule arithmetic, grouped legacy warnings, and blockers. A mutable Draft is reported as becoming stale and requiring regeneration only after a future apply; published history is never selected for mutation. The preview fingerprint covers scope, request, resolved section analysis, teacher/rule/configuration inputs, and Draft identity/revision/authority so later confirmation can fail closed after authority changes. The API requires `planning.edit_section`.

Explicit Section Subject Demand Transition

`PlanningSubjectDemand` is the normalized future authority for one section's subject and weekly-period requirement. It is isolated by branch and academic year, supports retained inactive/retired rows, and database constraints prevent cross-scope section or Subject references and duplicate active section-subject demand. Migration `20260828_004_planning_subject_demands_foundation` seeds only Current/New Planning sections from the existing grade-aligned Subject catalog and is idempotent.

Stage 2 makes the explicit-first resolver authoritative for live Planning demand, teacher workload arithmetic, timetable readiness/workspace/snapshot/generation input, Subject Scheduling Rule validation, and required-hours reporting. An explicit inactive or zero row suppresses fallback and represents durable retirement for that section. `Subject.grade` + `Subject.weekly_hours` remains a transitional fallback only where no explicit section-subject row exists. Existing teacher assignment identity, solver behavior, timetable UX, and published history are not rewritten.

Smart Timetable Customer And Published-Visibility Boundary

Stage 5.2 keeps internal versions but presents Configure ? Create New Timetable ? Draft ? Generate/Review/Edit ? Publish. The newest mutable non-active exact-scope version is the current Draft Timetable; generated results continue that logical Draft while source rows remain internal provenance. Technical selection, comparison, historical exports, archive, and permanent cleanup actions live in Timetable History.

Publishing remains the atomic exact-scope active-pointer swap. Deleting an eligible never-published draft removes its placements and version while preserving snapshots, runs, and published history; History bulk cleanup uses dependency-safe child-first deletion and reports protected remnants. Official non-management reads use

`timetable_visibility_service.py`, which accepts only `TimetableActiveVersion`. Teacher identity follows the existing scoped `User.user_id == Teacher.teacher_id` convention.

Smart Timetable Generation Authority

Academic scheduling quality is configured per branch and academic year using authoritative subject codes. Core English, Mathematics, and Science receive hard daily coverage when demand permits and distinct-day soft spread otherwise. Art, Well-being, Social Studies, Reflection, and ICT mappings receive configurable spread and non-consecutive preferences; ICT may enforce one session per day. Explicit Swimming groups synchronize selected sections and account for one common teacher and an optional capacity-limited resource. Independent validation repeats all hard rules.

Stage 5.1 makes Planning-resolved section/subject/teacher demand, including subject-specific Grades 1-2 HRT fallback, the only lesson authority for automatic generation. Schema-v3 snapshots freeze exact SchoolGroup/Branch/Academic-Year scope, canonical composed teaching slots, demands, locks, source revision and regeneration arrangement. OR-Tools is task-runtime-only; web processes commit a run, dispatch only its public ID to an on-demand Render Workflow task, and poll TIS state. The task claims the exact run under PostgreSQL locking and retains lease/heartbeat protection against duplicates and late persistence. Render automatic retries are disabled, and the old infinite worker loop is an optional local fallback only.

Hard-valid candidates must pass an independent validator and a current-input check before atomic persistence as unpublished generated/regenerated publication-ready versions. Generation never changes `TimetableActiveVersion`. Freshness authority uses stable Planning, canonical timetable configuration, generation constraints, and locks; runtime generation metadata is retained in snapshots but excluded from the freshness comparison. Regeneration preserves locks and requires a calculated minimum difference; random seed alone is not diversity authority. Publishing remains a separate permission and transaction.

Capacity-Based Commercial Packaging

ADR 0025 establishes one normal customer feature baseline for coherent active paid, promo, and customer-demo workspaces. Starter, Professional, and Enterprise AI differ by organization capacity and billing context, not by ordinary product modules. Their established 1/5/25, 5/20/100, and 25/100/500 branch/staff/teacher ceilings remain unchanged; Custom remains contact-only above Enterprise.

Commercial source and lifecycle decide whether the workspace may operate. Permissions and tenant/branch/academic-year scope decide what a user may do. Normal feature policy is resolved centrally, and branch entitlement is required for branch-scoped actions. Platform, Developer, System Owner, global audit, promo-admin, and demo-admin capabilities are excluded. AI availability uses the common baseline, while AI consumption uses a separate policy boundary and the existing durable accounting model.

Stage 5 SchoolGroup Management Boundary

Top-level SchoolGroup creation and deletion from operational configuration are platform-global actions, not tenant administration. Each requires Platform identity, `schools.manage_all_schools`, and the matching create/delete permission. Tenant Administrators no longer receive those permissions by default, and stale permission state cannot bypass the identity guard. Tenant SchoolGroup updates remain available only for the user's linked SchoolGroup; foreign IDs fail before mutation.

School Management consumes the unified commercial authority facade for branch capacity presentation. Promo and paid tenants therefore see authoritative used, allowed, and remaining branch capacity while branch mutations retain the existing locked, source-aware enforcement and atomic promo evidence. A read-only PostgreSQL provenance audit can identify active internal sandboxes with missing tenant-link and workspace-entitlement evidence for Platform Owner review; it performs no repair.

Controlled Existing Workspace Conversion Boundary

M4B is the only supported bridge from an audited internal sandbox to a customer workspace awaiting commercial activation. A durable operation binds the exact SchoolGroup identity, normalized intended owner, approved M4A evidence hash, canonical parameters, branch/dependency snapshot, sandbox entitlement snapshot, required setup fields, actors, lifecycle stage, and idempotency key. Conversion events are append-only and redacted. A partial unique database index permits at most one active `tenant_owner` account link per SchoolGroup; migration stops for manual review if legacy duplicates exist.

The operation is also the ownership claim. It does not fabricate an account or password. The intended owner must use ordinary registration and verification, then explicitly claim the workspace. Alignment rejects duplicate, suspended, unverified, platform, or cross-tenant identities and reuses an existing same-tenant operational User when safe. An existing different owner requires a separate Platform Owner transfer approval.

Only legal organization name, IANA timezone, and educational program are collected. Final execution locks and revalidates the operation and workspace, performs a fresh M4A audit, rejects branch or commercial drift, retires the one active internal-sandbox entitlement, and transitions to customer / provisioning without creating a replacement entitlement or TenantProvisioningLink. M1 calls that state `activation_required`; Organization Account remains available while operational access fails closed. M3 promo activation is available. Paddle activation for an already existing workspace is a deferred milestone and must not be inferred from normal new-onboarding checkout support.

Existing Workspace Conversion Audit Boundary

Before a legacy internal sandbox may enter any controlled customer conversion, M4A requires an explicit read-only audit keyed by SchoolGroup ID, workspace UUID, exact organization name, and normalized intended-owner email. The audit reflects the deployed schema, traverses branch foreign-key descendants, identifies unconstrained branch references, and inventories ownership, provisioning, entitlement, subscription, demo, and promo evidence. It emits counts and allowlisted metadata only; provider identifiers are reduced to presence flags and secrets or private payloads are never included.

`scripts/audit_existing_workspace_conversion.py` runs only on PostgreSQL in a repeatable-read, read-only transaction and always rolls back. Deterministic JSON and text formats expose the observed transaction mode, a stable SHA-256 snapshot, conservative archival candidates, and explicit exit codes: 0 coherent, 1 execution/configuration failure, 2 manual review, and 3 workspace identity mismatch. Soft-deleted rows still count as dependencies; an unmodeled branch foreign key blocks archival recommendations. Its result is evidence for a later design decision, not conversion authority. It cannot archive a branch, align an owner, change workspace metadata, create a tenant or commercial source, call Paddle, send email, approve deletion, or perform the future conversion. Any identity conflict, commercial conflict, incomplete schema coverage, or uncertain relationship requires manual review.

Promo Redemption And Activation Authority

Promo access is an explicit third customer commercial source beside paid subscription and customer demo. A verified organization owner may redeem an active, approved, in-window definition from completed onboarding or from an existing workspace that a separate controlled process already classified as customer, left in provisioning, linked to that owner, and left without any commercial source. Active internal sandboxes are never converted implicitly.

Final activation locks the activation session, promo definition, and target workspace; revalidates scope, limits, source compatibility, people usage, and branch selection; then creates immutable redemption/grant evidence, explicit branch access, a promo workspace entitlement, and the promo tenant link in one commit. Exactly one tenant-link source is required: `SubscriptionContract`, `SaaS DemoRequest`, or `PromoGrant`. Pending activation gives no access and Paddle is not involved.

Branch selection is the only reversible capacity selection in M3. Existing branches above the grant require exactly the allowed count; fewer branches are all selected and unused capacity remains available for normal future branch

creation. The final entitlement inventory covers every preserved branch: selected branches have an assignment and active entitlement; unselected active or inactive branches have an explicit inactive entitlement. Operational status is unchanged. Individual and bulk reactivation acquire the workspace lock, enforce grant capacity, and update status, assignment, and entitlement atomically. The `dry-run-first reconcile_promo_branch_entitlements.py` command may add only missing inactive evidence for a coherent active promo source and otherwise fails closed. Staff and teachers are never modified. Any organization-wide excess blocks activation until the owner changes operational usage and resumes.

Billing & Subscription is source-aware after organization selection and permission validation. Paid workspaces retain the existing Paddle subscription portal; demo workspaces retain their dedicated demo journey; promo workspaces render a dedicated Commercial Access page. Promo presentation composes M1 authority for plan, status, capacity usage, and remaining capacity with the attributable grant/redemption for effective dates and a masked reference. Normal operational access ends exactly at the grant's `effective_to`; grace days provide only a customer recovery window and do not extend access.

Expired and recovery-period promo owners may continue through the existing-workspace paid plan and Paddle checkout path. Promo remains the sole authority while checkout is pending, failed, abandoned, or unconfirmed. A verified completed transaction locks and revalidates the existing tenant and atomically replaces promo source evidence with paid contract, subscription, workspace entitlement, and branch entitlement evidence. The SchoolGroup, workspace UUID, branches, users, teachers, academic data, branding, ownership, and immutable promo records are retained. Active-promo early conversion is not supported. Shared commercial badges are generated from centralized authority for Organization Account and full commercial presentation. The normal operational shell does not expose commercial source, plan, or lifecycle identity.

Promo Definition And Governance Boundary

Platform Console now manages definition-only promo offers for the three active self-service plan tiers. A definition selects exact positive capacity within the existing plan ceilings, a controlled scope, validity and one access-expiry policy, redemption-policy metadata, and optional eligible existing branches. Draft, Active, Paused, and Revoked are persisted; Expired is derived. Active material edits require pause and return the versioned definition to Draft.

Raw promo codes are generated by TIS, shown once, and never persisted. Lookup authority is a deterministic HMAC-SHA256 hash under a dedicated secret and key identifier. Platform permission and owner-approval rules are separate from tenant permissions and capacity. M3 consumes only an approved active definition through the separate locked redemption path; M2 definition actions still do not grant access or call Paddle.

Organization Account Sign-In Boundary

Public SaaS Sign In resolves an activated organization owner or authorized account manager to `/saas/account`, the Organization Account Overview, rather than directly to the operational workspace. The overview separates organization profile, branch visibility, billing/subscription, and account security by the linked operational user's existing permissions. Enter TIS Platform is the only customer-account action that enters `/login`, and operational commercial-access and authorization checks still apply there. Restricted account managers retain safe account and billing recovery access without operational entry; incomplete accounts resume onboarding; non-management users retain their approved role-based destination; and multi-organization accounts select an organization before its overview is shown. That selection is an HTTP-only UUID hint that is accepted only after revalidation against the current account links and existing permissions; it cannot grant cross-tenant access.

Customer Journey Continuity

TIS treats demo or subscription expiry as an expected commercial state. Returning routing is derived from SaaS onboarding, request, durable workspace link, lifecycle, and subscription records. Expired access is intercepted before protected workspace services and shown with plan, renewal, support, and sign-out actions. Expired-demo checkout quotes the preserved workspace's active branches and must never create another organization or SchoolGroup.

Customer-visible language uses "TIS team" or "TIS support team"; Platform Owner remains an internal role.

M8B9 Demo Operations, Notifications, And Testing

Customer Demo operations are generic and Platform Owner-only. Immediate expiry is reversible; reactivation and custom expiry reuse the same workspace and require a future date with no maximum. Manual reminders are limited to the final organization-calendar day, and manual lifecycle runs call the production M8B7 processor. Standard preserves M8B8 usage, Full remains permission/expiry bound, and Custom supports registry-controlled workspace and branch policy.

M8B8 AI Entitlements And Commercial Foundation

AI access resolves centrally after tenant scope and `ai.use` permission. Commercial state remains authoritative: active Internal Sandbox is unlimited, active Customer Demo receives its configured successful-use allowance per registered feature, and expired/restricted demos fail closed. Enabled AI availability uses the common customer `module.ai` baseline for paid, promo, and active demo authority; globally disabled definitions remain denied. Usage is durable, auditable, idempotent by operation key, concurrency-safe, and separated into internal, demo, promo, and paid metric contexts. Availability and consumption policy are independent. No executable AI tool or M8B9 owner override/reset operation exists.

M8B7 Demo Customer Journey

M8B7 projects customer Pending, Active, Expired, and Declined presentation from the existing request, provisioning, and lifecycle sources. Approval invokes the separate retryable provisioning service and creates activation communications only after success. Six email types use the existing branded provider through a durable deduplicated outbox; demo events reuse the Platform Owner Notification Center. The shared tenant shell shows active demo status. A coherent expired demo may convert after authoritative confirmed payment by reactivating and relinking the same SchoolGroup and preserving its UUID and all tenant data. Sandbox, checkout-return, and unverified evidence remain non-authoritative.

Documentation version: 3.0

Last major context update: 2026-06-27

Product Identity

TIS stands for Teacher Information System. It is a developing SaaS academic operations platform for schools, school groups, academic leaders, supervisors, and platform owners who need one trusted place for academic staffing, teacher records, planning, calendars, observations, branch context, and future intelligence.

TIS is not only a teacher directory. It is intended to become the operational backbone for academic decision-making across a school or multi-branch organization.

Public product presence:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>
- 1 Operational login: `/login`
- 1 SaaS signup: `/saas/signup`
- 1 SaaS login: `/saas/login`
- 1 SaaS account area: `/saas/account`

Product Vision

TIS helps schools move away from scattered spreadsheets and disconnected operational files. The product vision is to give academic leaders a structured, secure, and tenant-isolated platform where teacher information, teaching loads, staffing needs, observations, calendars, and branch-level context can be managed together.

As the platform matures, TIS should also become the trusted data foundation for AI-assisted academic operations. Future AI features should depend on verified school data, clear permissions, and careful subscription packaging.

Business Goal

The business goal is to establish TIS as a subscription-based SaaS platform for academic operations. The platform should support school onboarding, plan selection, payment, provisioning, account management, and scalable multi-tenant usage.

Near-term business priorities:

- 1 Convert interested schools from public landing page traffic into SaaS accounts.
- 1 Support clear onboarding from signup through school setup.
- 1 Provide reliable billing and payment status visibility.
- 1 Enable platform owners to manage pending organizations and provisioning.
- 1 Preserve trust by keeping tenant data isolated and operational flows stable.

Educational Goal

The educational goal is to reduce administrative friction so schools can make better academic decisions. TIS should help leadership teams identify staffing gaps, workload problems, incomplete academic coverage, observation follow-up needs, and calendar conflicts earlier.

The platform should support educational quality by making academic operations easier to see, review, and improve.

Target Customers

Primary customers:

- 1 Private schools
- 1 Multi-branch school groups
- 1 Academic leadership teams
- 1 Principals and vice principals
- 1 Department heads and supervisors
- 1 School operations and HR-adjacent academic staff

Internal platform users:

- 1 Platform Owner
- 1 Platform Co-Owner
- 1 Platform Developer

Tenant users:

- 1 Administrators
- 1 Coordinators

- 1 Supervisors
- 1 Teachers and academic staff, depending on enabled workflows

FastAPI App Architecture

The operational TIS application is a FastAPI application at the repository root. It uses Python, SQLAlchemy, Jinja templates, static assets, and modular routers.

Core app areas:

- 1 `main.py`: primary FastAPI app, route registration, app-level workflows, startup checks, platform console routes, dashboard and configuration flows.
- 1 `models.py`: main SQLAlchemy models.
- 1 `database.py`: database connection/session setup.
- 1 `db_migrations.py`: local schema migration and repair logic.
- 1 `auth.py`: authentication, role normalization, platform identity helpers, permission helpers, session handling.
- 1 `authorization.py`: route permission rules and access-denied response helpers.
- 1 `permission_registry.py`: permission groups, labels, defaults, developer-assignable permissions, and system owner permissions.
- 1 `role_permission_service.py`: role permission persistence and related helpers.
- 1 `ui_shell.py`: shared application shell context, navigation, visual identity, and page metadata.
- 1 `academic_grade.py`: shared canonical KG/1-12 grade vocabulary and normalization, consumed by Planning and by Student Academic Placement.
- 1 `student_academic_service.py`: Student identity, external identifiers, and effective-dated Academic Placement authority.
- 1 `talent_program_service.py`: Talent Program, annual configuration, Framework Version lifecycle, competency lineage, Framework-specific rubric/descriptors, optional bounded KPI, and deterministic Review Candidate Policy configuration authority.
- 1 `talent_assessment_cycle_service.py`: SchoolGroup-wide Assessment Cycle lifecycle, explicit-time Draft eligibility preview, atomic frozen Student population, and population fingerprint authority.
- 1 `routers/`: modular feature routers for users, teachers, subjects, planning, timetable, academic calendar, observations, students, and talent programs.
- 1 `templates/`: Jinja templates for the operational app and SaaS app pages.
- 1 `static/`: CSS, JavaScript, images, branding assets, and generated public artifacts.
- 1 `tests/`: pytest coverage for tenant isolation, SaaS phases, platform access, permissions, email, branding, and related workflows.

Important operational route families:

- 1 `/login`: operational app login.
- 1 `/dashboard`: tenant operational dashboard.
- 1 `/platform`: platform console for platform identities.
- 1 `/system-configuration`: branch, year, branding, role permissions, and configuration workflows.

- 1 /teachers, /subjects, /planning, /timetable, /academic-calendar, /observations: core academic operations.
- 1 /api/students: Student identity, external identifiers, and Academic Placement JSON API (routers/students.py, students.* permissions). No page/UI exists yet.
- 1 /api/talent/programs: Talent Program, Framework Version, Competency, rubric/level/descriptor, optional KPI, and Review Candidate Policy JSON API (routers/talent_programs.py, talent_programs.* permissions). M3 semantic mutations are Draft-only and expected-revision protected; no page/UI exists yet.
- 1 /api/talent/assessment-cycles: Cycle metadata, authorization-filtered Draft/frozen population, and organization-governed Open/Close API (routers/talent_assessment_cycles.py, dedicated talent_assessment_cycles.* permissions). Cycles have no Branch ownership; identifiable population access is filtered by resolved/frozen Branch context.

Next.js Landing Architecture

The public marketing website lives in `tis-landing-website/`. It is separate from the FastAPI operational portal.

Landing architecture:

- 1 Runtime: Next.js / Node.
- 1 Public website domain: `https://tisplatform.com`.
- 1 Local development URL: `http://localhost:3000`.
- 1 Main page: `tis-landing-website/src/app/page.tsx`.
- 1 App layout: `tis-landing-website/src/app/layout.tsx`.
- 1 Global styles: `tis-landing-website/src/app/globals.css`.
- 1 Logo component: `tis-landing-website/src/components/tis-logo.tsx`.
- 1 Public assets: `tis-landing-website/public/`.
- 1 M8 landing integration uses `NEXT_PUBLIC_TIS_APP_BASE_URL` as the configurable deployed application base URL. Public Open Account CTAs build one shared registration destination at `/saas/signup`.
- 1 The hero Subscribe Now action scrolls to `#pricing`. Starter, Professional, and Enterprise AI enter public signup with an allowlisted preferred-plan code. The preference creates no plan, checkout, payment, or Paddle record and is applied only after existing branch, system-user, and teacher capacity validation. Custom remains contact-only.

The application portal remains separate:

- 1 App domain: `https://app.tisplatform.com`.
- 1 Runtime: FastAPI / Python with a relational database.

Legacy landing files in the FastAPI app are not the source of truth:

- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

Those legacy files must not be modified unless explicitly approved.

Multi-Tenant SaaS Strategy

TIS is designed as a multi-tenant SaaS platform. Tenant isolation is a critical rule. School groups, branches, users, academic years, teachers, planning data, timetable data, observations, and configuration records must remain scoped

to the correct organization and branch context.

The platform distinguishes between:

- 1 Platform identities: platform owners, co-owners, and developers.
- 1 Tenant identities: users belonging to a school group and branch context.
- 1 SaaS account identities: accounts that move through signup, onboarding, billing, and provisioning.

Platform users may inspect or switch organization context through controlled platform workflows. Tenant users must remain inside their authorized school, branch, academic year, and permission scope.

Critical tenant strategy:

- 1 Do not weaken tenant isolation.
- 1 Do not bypass permission checks.
- 1 Do not assume a platform identity is a tenant identity.
- 1 Do not assume a SaaS account is already provisioned into an operational tenant.
- 1 Keep onboarding, billing, and tenant provisioning as distinct stages.

Workspace Classification Foundation

M8B-1 introduces a metadata boundary between workspace identity, workspace classification, and operational tenant state:

- 1 `SchoolGroup.workspace_uuid` is the stable unique workspace identifier.
- 1 `SchoolGroup.workspace_classification` accepts only `internal_sandbox`, `customer_demo`, or `customer_paid`.
- 1 `SchoolGroup.workspace_lifecycle_status` accepts only `provisioning`, `active`, `suspended`, or `archived`.
- 1 `PendingOrganization.workspace_intent` carries pre-provisioning intent.
- 1 `SaaSAccount.account_purpose` distinguishes internal-test and customer account purpose.
- 1 `User.is_internal_test_identity` identifies operational identities attributable to internal test data.

The M8B-1 fields are not authorization, entitlement, payment, or reset gates. Existing flows continue unchanged. Classification conversion is rejected except for the dedicated M8B-6 Customer Demo to Customer Paid service, and the commercial-state service remains the authority for the effective commercial result. Every pre-M8B-1 workspace/onboarding record is confirmed test data and is assigned internal sandbox/test metadata by the controlled one-time backfill. No code may infer customer-paid status from Paddle or onboarding fields outside the approved conversion and commercial-resolution boundaries.

M8B-2 implements that commercial resolver as a read-only foundation. Workspace entitlements are separate from classification and lifecycle, and branch commercial activity is separate from operational `Branch.status`. The resolver recognizes provisioning, active internal sandbox, active customer demo, active customer paid, inactive, suspended, archived, and manual-review outcomes. It does not implement demo expiration or authorization enforcement.

Paid workspace resolution delegates plan capabilities and paid branch quantity to the existing M7 entitlement resolver and requires a matching persisted `PaymentSubscription`. Demo and internal entitlements use explicit workspace values tied to the shared `EntitlementDefinition` catalog. Branches inherit their workspace entitlement unless an optional coherent `BranchEntitlement` says active or inactive. All calculations are read-only and fail closed on ambiguity or tenant mismatch.

M1 adds one operational facade over these existing authorities. Paid capacity uses confirmed `PaymentSubscription.quantity` capped by the plan branch ceiling, plus plan staff and teacher limits. Staff is every distinct active tenant operational User in the SchoolGroup, including an operational owner and teacher-position user; platform and account-only identities are excluded. Teacher people deduplicate by normalized `Teacher.teacher_id`, while each blank legacy identity counts separately. A person represented in both models consumes one staff slot and one teacher slot. Demo and Internal Sandbox are explicitly unmetered in M1 only when their existing lifecycle/access authority resolves.

Every operational capacity increase locks the SchoolGroup, resolves authority, recounts, evaluates the final proposed totals, writes, and commits in one transaction. Branch create/reactivation/bulk status, operational user create/reactivation, teacher create/year-copy, academic-year switching, and paid/demo provisioning final validation use this boundary. Existing over-capacity records and access are preserved, but an exceeded dimension cannot grow further. M1 adds no commercial-state persistence, schema, migration, Paddle, pricing, webhook, onboarding-transition, permission, or feature-packaging change.

M8B-3 introduces a separate SaaS demo-request aggregate after onboarding review. The public landing website enters signup with either a Request a Demo or Subscribe Now intent; the valid intent persists through account and School Workspace Setup and is emphasized, but never locked, at the commercial-choice page. A self-service pricing card may also supply an allowlisted preferred-plan code. That preference is not a plan selection and cannot create checkout or bypass the existing branch, system-user, and teacher capacity checks; invalid or ineligible preferences are discarded before the customer selects a plan. Subscribe Now preserves the existing plan-selection and Paddle path. Request Demo captures immutable commercial/classification/entitlement context and starts in Pending Review without creating an operational workspace.

Customer Demo eligibility is keyed by a normalized organization domain. TIS prefers the pending organization's authoritative domain, uses a work email domain only when no organization identity exists, and requires an official website/domain for public email providers. A unique domain-eligibility reservation prevents concurrent or historical duplicate Customer Demo opportunities. The reservation remains after request review, activation, expiry, cancellation, rejection, or Demo-to-Paid conversion; Internal Sandbox history is excluded. The only exception is the separately guarded Platform Owner test workspace/account clean-room reset, which atomically deletes the selected test organization's linked demo-commercial records and reservation so internal M8 testing can begin again; it never changes customer reset or demo rules.

Only Platform Owners can approve, reject, or cancel requests. Approval creates a review record but does not provision or activate a demo. Rejection requires a reason, and customers may withdraw only while review is pending. Durable request events serve both audit and internal-notification purposes; email delivery remains out of scope.

M8B-4 consumes an approved request through a separate demo-provisioning aggregate. Provisioning reuses the shared operational workspace builder but creates a customer-demo SchoolGroup and explicit demo entitlement without a Paddle object, payment subscription, or subscription contract. `TenantProvisioningLink` accepts exactly one commercial source: a paid `SubscriptionContract`, an approved `SaaS DemoRequest`, or an immutable `PromoGrant`.

The workspace, entitlement, tenant link, request linkage, and activation transition are atomic. Failure rolls back those records while preserving the approved request and a retryable provisioning failure record. Success activates the customer-demo workspace and entitlement, records activation metadata and internal audit events, and prevents a second provisioning attempt. Platform Owners see detailed provisioning outcomes; customers see only Approved, Provisioning In Progress, Demo Active, or a safe support state.

M8B-5 makes the M8B-4 activation timestamp the only demo clock authority. The resolver derives Day 6 and Day 7 boundaries, validates persisted lifecycle metadata, converts values to the organization's timezone only for display, and fails closed on inconsistent ownership, entitlement, timestamps, or timezone.

The idempotent lifecycle processor creates internal reminder notifications on Day 6 and atomically expires demos at Day 7. Expiration ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, updates

the commercial snapshot to suspended, and preserves every operational row. The operational request middleware enforces the resolver for customer-demo tenant users on every request, so an existing authenticated session cannot bypass expiration. Web users receive the preserved-data/subscription page; API and download requests receive a safe 403. Platform administration, paid tenants, and internal sandboxes bypass this demo-only gate.

M8B-6 permits one commercial classification transition: an active, valid Customer Demo may become Customer Paid after a provider-confirmed M7 subscription exists for the same pending organization. The existing paid checkout and webhook remain authoritative; no local conversion occurs from page state, onboarding selection, or an unconfirmed payment attempt.

The dedicated conversion service locks and validates the demo request, provisioning record, SchoolGroup, demo entitlement, tenant link, contract, and payment subscription. One atomic transaction ends the demo entitlement, creates the subscription-backed paid entitlement, relinks branch entitlement rows, switches the tenant link from the preserved demo reference to the confirmed contract, and changes the existing SchoolGroup classification. The same workspace UUID, tenant row, organization, branches, users, permissions, academic data, and audit history remain in place.

The Platform Owner-only internal test-workspace reset is a separate controlled cleanup flow. Its dependency order removes the selected pending organization's linked demo request, domain eligibility reservation, review/event history, provisioning/lifecycle records, and conversion records before their commercial and workspace parents. A detached domain reservation is eligible for clean-room removal only after the reset uses the shared customer-demo domain resolver and confirms that no other pending organization, demo request, tenant workspace, or customer account independently uses the domain and that the reservation is not already marked as historically ambiguous; any conflict or historical ambiguity blocks the reset for manual review. Analysis produces an explicit safe detached-reservation ID list. Deletion uses only those IDs, flushes their removal before demo requests or parent records, and verifies inside the transaction that no selected ID remains; an ID mismatch or failed verification aborts the reset. It also removes `subscription_change_requests` scoped to the selected SchoolGroup before scoped operational User rows, then removes selected `WorkspaceEntitlementValue` and `BranchEntitlement` children before the selected `WorkspaceEntitlement` and final SchoolGroup. This permits a clean internal M8 retest using the same email and organization domain without touching global plan, price, entitlement-definition, Paddle, platform, or other-organization records. The existing validation gates, owner access, one-transaction commit/rollback behavior, and production one-demo-per-domain customer policy remain unchanged.

The separate Platform Owner demo-eligibility maintenance area handles historical detached reservations that no longer have an organization/account target for clean-room reset. `/saas-admin/demo-eligibility-maintenance` performs a read-only scan and marks a row removable only when its exact domain has no matching organization, SaaS account, demo request, tenant-profile workspace, provisioning record, subscription evidence, Demo-to-Paid conversion, or manual-review marker. Deletion requires typed-ID and checkbox confirmation, locks and rechecks the exact row, deletes by eligibility primary key only, flushes, verifies zero remaining rows for that ID, and commits atomically. Blockers preserve the row and rollback. Successful maintenance writes a durable owner audit event. This path never deletes by domain and does not weaken the one-demo-per-domain rule.

The existing M7 subscription resolver, workspace-entitlement resolver, and commercial-state resolver verify the post-conversion result before commit. Failure rolls back every workspace mutation while preserving provider-confirmed subscription records and a retryable failure history. Completed conversions are excluded from demo reminder/expiration processing. Expired, suspended, ambiguous, cross-tenant, internal-sandbox, paid, or already-converted workspaces fail closed.

SaaS Routes And Account Experience

Core SaaS routes:

- 1 `/saas/signup`: public SaaS account creation.
- 1 `/saas/login`: SaaS account login.
- 1 `/saas/account`: SaaS account dashboard.

Related SaaS areas include plan selection, onboarding organization details, contacts, branch setup, academic setup, onboarding review, billing status, checkout summary, checkout return, checkout cancel, sessions, security, and profile pages.

Platform owner SaaS administration exists under `/saas-admin` for pending organizations, demo-request review, payments, and provisioning workflows.

The Platform Owner pending queue is an operational work queue, not a list of every `PendingOrganization` row. It includes draft/setup, review, checkout/payment, and incomplete or recoverable provisioning states only while no completed tenant link, completed provisioning job, or final tenant billing state exists. Confirmed active tenants and retained rejected/completed records remain available under Organization Records. When completed provisioning evidence conflicts with payment, subscription, contract, tenant, or SchoolGroup evidence, the owner lifecycle resolver labels the record Lifecycle Review Required and fails closed instead of presenting it as ordinary pending work. Raw onboarding status remains historical and does not override confirmed operational truth.

TIS Account Email Verification Recovery

The accepted Phase 1 verification recovery improvement strengthens the public TIS Account setup journey without changing payment, billing, provisioning, tenant activation, database schema, migrations, operational modules, or the landing website.

Current verification behavior:

- 1 Valid email verification links mark the account email verified/active and redirect to `/saas/login` with a professional success notice: the customer is told their email has been verified and asked to sign in to continue school workspace setup.
- 1 `/saas/login` is the GET sign-in page and its form submits only by POST to `/saas/auth/login`. Fresh verified accounts with no organization record continue to the GET `/saas/account` dashboard; the existing explicit onboarding-start action remains POST-only.
- 1 Login continuation values are limited to known customer GET destinations. POST-only, malformed, traversal, fragment, and external values fall back to Account Setup. Defensive GET navigation to `/saas/auth/login` redirects to `/saas/login` and never exposes raw FastAPI 405 JSON.
- 1 Expired or invalid verification links show a recovery page instead of a dead-end generic error.
- 1 The recovery page includes a resend verification form.
- 1 Resend verification safely handles unverified accounts, already verified accounts, and unknown email addresses without exposing whether an account exists.
- 1 Password-based accounts that are still pending verification cannot start or continue school workspace setup.
- 1 New customer-facing verification language in this flow uses "TIS Account" and "school workspace setup".

Google and Microsoft sign-in remain future work. OAuth-based verification bypass rules should not be expanded without a separate approved identity decision.

TIS Account Customer-Facing Wording And Branding

The accepted Phase 2 customer-facing wording cleanup improves the public TIS Account and school workspace setup experience without renaming internal `/saas` routes, modules, models, or stored statuses.

Customer-visible account/setup pages should use professional labels including:

- 1 TIS Account
- 1 Account Setup

- 1 Account Dashboard
- 1 School Workspace Setup
- 1 Organization Profile
- 1 Branch Setup
- 1 Academic Setup
- 1 Subscription Setup
- 1 Secure Payment
- 1 Workspace Activation

Customer pages should not expose "SaaS" as product copy, raw database statuses, tenant/provisioning terminology, provider transaction/subscription identifiers, attempt UUIDs, checkout session internals, plan IDs, or school group IDs. Those internal concepts may remain in backend code, admin views, stored data, tests, and documentation where they describe architecture or platform-owner operations.

The shared customer account shell includes the official full-color horizontal TIS logo on the light account background. Transactional TIS Account emails use an existing official dark-blue TIS wordmark URL. This Phase 2 cleanup did not change payment, billing, provisioning behavior, tenant activation behavior, database schema, migrations, operational modules, or the Next.js landing website. Google/Microsoft login remains future work.

TIS Account Guided Setup Framework

The accepted Phase 3A implementation introduces the shared customer setup framework for the TIS Account dashboard only. The account page now behaves like a guided onboarding console rather than a dense admin dashboard.

For a verified account that has not created a pending organization, the initial Account Setup view uses a compact variant: a smaller official logo, one "Start Your School Workspace Setup" title, one supporting sentence, one POST action to `/saas/onboarding/start`, and the existing eight-step journey. Duplicate status, next-action, account/workspace, and guidance panels are omitted in this initial state. Later setup states retain their existing contextual console.

The shared framework provides:

- 1 Official TIS logo/header.
- 1 An 8-step customer journey stepper.
- 1 Current step and status banner.
- 1 One primary next action.
- 1 Main content area.
- 1 Help/guidance area.
- 1 Clear messaging that TIS Platform access becomes available after Workspace Activation.

The journey steps are TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform. Step state is derived from existing account, pending organization, onboarding progress, billing, payment, and activation data. Internal route names, stored statuses, billing/payment/provisioning behavior, database schema, migrations, operational modules, the landing website, and OAuth behavior remain unchanged.

Phase 3B applies this shared framework to the five School Workspace Setup onboarding pages: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup. These pages now use a

consistent guided wizard structure with grouped sections, one shared-shell primary CTA, secondary Back/Save Draft actions, concise guidance, and reduced visual clutter.

The Phase 3B onboarding redesign preserves existing form actions, field names, validation behavior, draft behavior, route names, and onboarding state transitions. Subscription/payment/status pages remain future Phase 3 work.

Organization Profile program input is a required controlled selector whose National, International, and Both labels normalize to the existing uppercase stored codes. Invalid values render inline without creating another pending organization. Pending logo uploads are decoded as PNG/JPG/WEBP, limited to 4 MB, checked for minimum dimensions, written under an opaque UUID filename, and promoted atomically. Empty upload retains the current logo. Replacement commits the new relative path before safely removing the obsolete pending file; rollback removes the newly written file and retains the old database reference. Storage failures are logged with traceback and rendered as customer-safe page errors.

Pending logo storage remains `static/uploads/saas/pending_logos` on the application filesystem. Only the relative path is persisted. An ephemeral Render filesystem is unsuitable for durable customer branding across restarts or deploys unless a persistent disk is explicitly mounted. Object storage or persistent disk adoption is a separate owner-approved architecture and deployment decision.

Organization Profile and the shared School Workspace setup header render the saved pending logo with contain sizing, organization-name alt text, and a neutral no-logo/unavailable placeholder. The official TIS logo remains the separate platform identity. The pending relative path remains authoritative through checkout. Both paid and demo activation use `provisioning_service.create_workspace_records()`, which validates the pending file again and copies it to the primary `SchoolGroupLogo` slot under the established organization-owned branding directory. `SchoolGroup` has no logo column; `SchoolGroupLogo` is the final branding authority. The operational shell and branding settings already consume that record through the protected organization-asset route. Missing source files now block activation rather than silently dropping the logo.

Branch Setup capacity summaries are computed in Python with nullable or incomplete estimate values normalized to zero for display. HTML and server save validation still require explicit non-negative whole numbers for system users and teachers on every active branch. Newly created branch rows receive explicit zero defaults. No schema change is required because current columns already have non-null zero defaults.

Phase 3C applies the same guided framework to Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages. These pages use one shared-shell primary CTA, customer-safe status labels, concise supporting cards, and explicit messaging that browser return from checkout does not itself confirm payment and that TIS Platform access becomes available after Workspace Activation.

The Phase 3C redesign preserves payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, route names, database schema, migrations, operational modules, the landing website, OAuth behavior, and admin views.

M1-M5 Completed Milestone Summary

M1: Identity and SaaS foundation

- 1 Established core SaaS account concepts.
- 1 Added initial signup/login/account flows.
- 1 Clarified separation between platform, tenant, and SaaS account identities.
- 1 Added supporting tests for identity and SaaS phase behavior.

M2: Onboarding foundation

- 1 Added structured onboarding stages for organization information, contacts, branches, academic setup, and review.
- 1 Improved the path from SaaS signup toward a provisionable organization.

- 1 Preserved the distinction between pending SaaS organizations and operational tenants.

M3: Billing and plan foundation

- 1 Added pricing, plan catalog, billing status, checkout summary, return, and cancel flows.
- 1 Created billing/payment service modules to keep payment logic separate from operational academic logic.
- 1 Prepared the platform for subscription-based SaaS packaging.

M4: Provisioning foundation

- 1 Added pending organization and provisioning workflows for platform owners.
- 1 Added provisioning queue concepts and retry/run operations.
- 1 Created a controlled path for turning a pending SaaS organization into an operational school context.

M5: Platform access, permissions, and owner controls

- 1 Strengthened platform identity handling.
- 1 Added platform owner/developer concepts and owner management controls.
- 1 Improved permission boundaries for platform and tenant users.
- 1 Added tests around platform access and role permissions.

M7: Subscription Management And Entitlements

- 1 Added normalized entitlement definitions and plan entitlement values resolved from one confirmed active provider subscription.
- 1 Added a customer Subscription Management portal with lifecycle state and centrally controlled allowed actions.
- 1 Added paid branch-quantity increases and scheduled reductions with operational branch-capacity enforcement.
- 1 Added plan upgrades and scheduled downgrades using Paddle-authoritative previews and proration behavior.
- 1 Added scheduled cancellation and reversal while preserving paid access through the confirmed effective period.
- 1 Added provider-sourced billing history and protected, freshly resolved invoice downloads.
- 1 Added an explicit organization-owned billing profile for the confirmed billing email, legal/billing identity, contact, optional registration/tax identifiers, and supported address fields. Login email and billing email are independent authorities.
- 1 Mapped Paddle customers now retain `provider_address_id` and `provider_business_id`. Authorized billing changes synchronize the existing customer, reuse or update one attributable active Paddle Business, update the active subscription identity, and attach the business to future initial transactions. Historical invoices are not rewritten automatically.
- 1 Billing Contact is read-only by default and uses an explicit edit interaction. Local profile changes survive provider failure; a permission- and tenant-scoped retry reuses persisted Paddle mappings, is idempotent after success, and records safe step-level provider diagnostics. A saved pending or failed billing identity blocks new plan and quantity mutations until synchronized, without blocking cancellation or changing legacy active subscriptions that have no saved profile.
- 1 Paddle paid and completed remain separate lifecycle evidence: paid is customer-visible payment receipt while provider processing continues; completed remains required for final reconciliation.
- 1 Added webhook idempotency, strict provider/local relationship validation, manual-review fail-closed paths, diagnostics, and guarded reconciliation tooling.

- 1 Unified active subscription capacity review and operational enforcement across branches, tenant operational staff users, and teachers. Required upgrades use the highest capacity dimension, Paddle quantity remains branch-only, and scheduled downgrades/reductions revalidate capacity at their effective boundary.
- 1 Subscription presentation separates paid branch quantity from the plan's maximum branch ceiling; unused ceiling is never described as prepaid capacity. The approved common customer feature baseline keeps plan comparison focused on identity, capacity, eligibility, and actions rather than a feature ladder.
- 1 Linked operational organization owners and billing-authorized account managers can enter the existing Organization Account subscription page from System Configuration. Tenant/account linkage is revalidated and only an allowlisted internal billing continuation survives SaaS authentication. The public landing customer entry is labelled Organization Sign In and targets centralized SaaS login.

Paddle And Payment Architecture Summary

Existing customer workspaces in `activation_required` can use a separate SchoolGroup-anchored paid activation flow. It shares Paddle customer/address/business and transaction infrastructure but does not create `PendingOrganization` or run tenant provisioning. Professional and Enterprise AI quote all active operational branches; Starter remains disabled until complete branch-entitlement enforcement is proven. The workspace-customer association persists the address and business used for that SchoolGroup, and a returned or reused Paddle transaction is accepted only after its complete billed transaction and current-quote lineage are revalidated. The existing-workspace **Choose a Plan** entry point remains a selection surface while the current activation is `draft` or `checkout_ready`. Its selected plan is a default, not commercial authority: another eligible plan may replace it and produce a fresh authoritative quote without calling Paddle, creating a `PaymentAttempt`, or changing branches. `checkout_started`, `payment_processing`, and manual-review or inconsistent activations are not silently replaced and fail closed against plan changes. Promo activation and `PendingOrganization` checkout remain separate flows. Verified `transaction.completed` evidence atomically establishes the paid contract, subscription, workspace/branch entitlements, paid tenant link, and active lifecycle. Classification remains customer, and browser return or `transaction.paid` never grants access.

Account presentation does not infer payment from workspace lifecycle. A customer / provisioning workspace with no current paid-activation attempt displays Activation required. Payment processing requires a current unexpired attempt in `checkout-started` or `payment-processing` state; failed, cancelled, and expired attempts use recovery presentation. `PendingOrganization` and demo status resolution is separate.

The payment architecture is organized under the `saas/` package.

Key modules:

- 1 `saas/pricing_service.py`: plan/pricing behavior.
- 1 `saas/payment_service.py`: payment-related service logic.
- 1 `saas/paddle_client.py`: Paddle integration boundary.
- 1 `saas/billing_service.py`: billing status and related account billing behavior.
- 1 `saas/currency_service.py`: currency-related helpers.
- 1 `saas/router.py`: SaaS and SaaS admin routes.
- 1 `saas/entitlement_service.py`: provider-confirmed paid subscription entitlement authority.
- 1 `saas/commercial_authority_service.py`: unified operational commercial-access and three-dimension capacity authority.
- 1 `saas/subscription_portal_service.py`: customer portal composition.
- 1 `saas/subscription_change_service.py`: quantity change previews, submissions, schedules, and webhook reconciliation.

- 1 saas/subscription_plan_change_service.py: plan upgrade/downgrade lifecycle.
- 1 saas/branch_pricing_quote_service.py: shared onboarding and operational three-dimension capacity counts, plan eligibility, and minimum-plan resolution.
- 1 saas/subscription_cancellation_service.py: cancellation and reversal lifecycle.
- 1 saas/subscription_lifecycle_service.py: lifecycle resolver and allowed-action policy.
- 1 saas/billing_history_service.py: Paddle transaction history and invoice access.
- 1 saas/payment_lifecycle_reconciliation_service.py: guarded repair from attributable authoritative evidence.

Architecture rules:

- 1 Keep Paddle-specific details behind service/client boundaries.
- 1 Do not mix payment logic into academic operations.
- 1 Treat payment status, onboarding status, and provisioning status as related but separate concepts.
- 1 Use platform owner admin views for payment/provisioning oversight.
- 1 Avoid changing live SaaS payment flows unless the task explicitly requires it.
- 1 Keep Paddle credentials and endpoints in environment variables.
- 1 Keep Paddle price mappings environment-specific and store the runtime mapping in subscription_plan_prices.provider_price_id.
- 1 Use scripts/sync_paddle_price_ids.py with sandbox or production mapping JSON to configure initial checkout price IDs; never hardcode live Paddle IDs in migrations or source.
- 1 If an initial checkout price mapping is missing, fail closed before calling Paddle and show customers a support-oriented Secure Payment message while retaining internal plan/interval/currency diagnostics.
- 1 Paddle transaction checkout uses the public /saas/payment launcher as the payment-link page. Set PADDLE_CHECKOUT_BASE_URL to https://app.tisplatform.com/saas/payment, configure PADDLE_CLIENT_TOKEN and PADDLE_ENVIRONMENT for Paddle.js, and never expose PADDLE_API_KEY to frontend code.
- 1 Paddle is authoritative for prices, previews, proration, transactions, scheduled changes, and invoice documents. TIS stores workflow/audit state but does not invent monetary outcomes.
- 1 Subscription changes send the complete retained recurring item set. Immediate changes prevent provider mutation on payment failure; scheduled changes do not change local entitlements before verified effective evidence.
- 1 Webhooks are signature-verified and idempotent. Ambiguous, mismatched, or incomplete provider evidence fails closed to manual review.

Tenant Provisioning Summary

Tenant provisioning turns a pending SaaS organization into an operational TIS school context. Provisioning should create or connect the required organization records, branches, users, and initial tenant setup while preserving auditability and data isolation.

Key provisioning concepts:

- 1 Pending organization: SaaS onboarding entity still requiring setup, review, payment, or incomplete/recoverable activation work, with no completed tenant evidence.
- 1 Provisioning job: controlled action to create or update operational tenant structures.

- 1 Platform owner review: human oversight before or during provisioning.
- 1 Retry behavior: failed provisioning work should be recoverable without corrupting tenant data.

Provisioning rules:

- 1 Do not directly create operational tenant data from public signup without the approved provisioning path.
- 1 Keep provisioning idempotent where possible.
- 1 Log or surface errors clearly.
- 1 Do not merge tenant data across school groups.
- 1 Treat active tenant/subscription evidence as authoritative over stale onboarding status in Platform Owner queue presentation.

Landing Page Strategy

The public landing page exists to explain the product, build trust, capture demand, and route interested schools into SaaS signup or demo request flows.

Source of truth:

- 1 The public website implementation is `tis-landing-website/`.
- 1 Marketing content references live in `docs/marketing/`.

Landing priorities:

- 1 Explain the problem of scattered academic operations.
- 1 Present TIS as a connected academic operations platform.
- 1 Show credible platform capabilities.
- 1 Present Request a Demo and Subscribe Now as the two public conversion pathways.
- 1 Route both paths through environment-configured deployed TIS Account signup URLs with the selected intent.
- 1 Keep the landing page separate from operational app templates.

Landing rule:

- 1 Do not change landing page design, copy, or architecture during operational backend tasks unless explicitly approved.

Customer Experience Roadmap

Near-term customer experience:

- 1 Clear signup and login path.
- 1 Smooth onboarding for organization, contacts, branches, and academic setup.
- 1 Transparent billing/plan status.
- 1 Clear provisioning status after checkout or owner review.
- 1 Reliable operational login once provisioned.

Medium-term customer experience:

- 1 Better account self-service.

- 1 Clearer subscription lifecycle.
- 1 More guided onboarding and implementation support.
- 1 Improved platform owner visibility into pending organizations and payment status.

Long-term customer experience:

- 1 AI-assisted academic planning and decision support.
- 1 Subscription-gated advanced analytics.
- 1 Intelligent recommendations based on verified tenant data.
- 1 Richer executive visibility across branches and academic years.

Knowledge Management System

TIS uses a Knowledge Management System (KMS) to preserve source-of-truth documentation, current project state, decision history, module history, and generated snapshots.

KMS source documents:

- 1 docs/AI_PROJECT_CONTEXT.md: first-read compact onboarding context for future Codex and ChatGPT conversations.
- 1 docs/TIS_MASTER_CONTEXT.md: durable product, architecture, workflow, roadmap, and critical rules.
- 1 docs/PROJECT_STATE.md: living project state.
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md: mandatory KMS and Knowledge Impact Assessment rules.
- 1 docs/CHANGE_HISTORY.md: chronological summary of meaningful changes.
- 1 docs/adr/: Architecture Decision Records.
- 1 docs/history/: module-specific history.
- 1 docs/engineering/: engineering handbook with module map, repository architecture, user/system flows, and developer onboarding.

PDF philosophy:

- 1 Markdown files are the source of truth.
- 1 The PDF booklet is a generated snapshot.
- 1 The PDF must never be edited manually.
- 1 The PDF must be regenerated when included Markdown source docs change.
- 1 The PDF should later be served through owner-only protected routes.

Generated KMS artifacts:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf
- 1 static/docs/docs_manifest.json

Automatic enforcement:

- 1 root AGENTS.md defines mandatory Codex KMS behavior,
- 1 .kms-impact.yml records the task-level machine-readable KIA,

1 scripts/kms.py sync validates KIA before writing, regenerates PDF/manifest artifacts, and runs post-generation freshness checks,
1 scripts/kms.py check provides the single read-only local and CI validation command,
1 scripts/check_kms_impact.py compares the declaration with changed files and major-change rules,
1 scripts/generate_docs_pdf.py --check validates source coverage and snapshot freshness without writing,
1 GitHub Actions require validation for pull requests, dev, and master deployment.

Automation never rewrites Markdown. It detects omissions and blocks stale work so reviewed human/AI updates remain authoritative.

Owner-only app access:

1 /platform/knowledge: read-only Platform Owner Knowledge Center.
1 /platform/knowledge/booklet: protected inline PDF view.
1 /platform/knowledge/booklet/download: protected PDF download.

The Knowledge Center is protected by the existing Platform Owner access pattern. It is not public, not a landing page, and does not regenerate or rewrite source docs. Its manifest-backed library presents document titles and summaries, groups sources into Core, Engineering, Decisions, History, Marketing, and Supporting sections, supports client-side category/module/freshness filtering and search, and opens documents at their recorded pdf_page through the protected booklet route.

Engineering handbook:

1 docs/engineering/TIS_MODULE_MAP.md maps product/system modules and guardrails.
1 docs/engineering/REPOSITORY_ARCHITECTURE.md explains repository ownership and risky files.
1 docs/engineering/USER_AND_SYSTEM_FLOWS.md documents end-to-end customer, SaaS, payment, provisioning, operational, platform owner, KMS, and developer flows.
1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md explains data areas, ownership boundaries, and tenant isolation rules.
1 docs/engineering/DEVELOPMENT_STANDARDS.md defines non-negotiable engineering rules.
1 docs/engineering/UI_UX_DESIGN PHILOSOPHY.md defines design direction for operational, platform, SaaS, Knowledge Center, and landing surfaces.
1 docs/engineering/PRODUCT_ROADMAP.md records completed, current, next, and future roadmap.
1 docs/engineering/REJECTED_DECISIONS.md records significant rejected alternatives.
1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md defines future visual documentation standards.
1 docs/engineering/AI_OPTIMIZATION_GUIDE.md guides future AI assistants.
1 docs/engineering/PROJECT_GOVERNANCE.md defines ownership, approvals, quality gates, documentation gates, and traceability.
1 docs/engineering/KNOWLEDGE_LIFECYCLE.md defines documentation, engineering, approval, review, release, and maintenance lifecycle.
1 docs/engineering/DOCUMENTATION_AUTOMATION.md defines current automation and future automation rules.
1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md formalizes KIA.
1 docs/engineering/SELF_EVOLVING_WORKFLOW.md defines the official task-to-deployment workflow.
1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md explains KMS propagation.

- 1 docs/engineering/AI_CODING_WORKFLOW.md defines future AI coding discipline.
- 1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md records future automation opportunities.

KMS v1.0 status:

KMS v3.0 Phase 3D completes the KMS v1.0 lifecycle foundation. Future work should improve automation, search, visuals, route inventories, test strategy docs, deployment runbooks, and deeper module guides only through approved phases.

Development Workflow

Default workflow for approved implementation tasks:

- 1 Inspect the relevant code and docs before editing.
- 2 Keep changes scoped to the approved task.
- 3 Preserve tenant isolation, permission checks, SaaS flows, and landing page boundaries.
- 4 Update tests or add focused tests when behavior changes.
- 5 Complete the Knowledge Impact Assessment and update `.kms-impact.yml`.
- 6 Update relevant Markdown docs.
- 7 Update change history, ADRs, module history, and AI project context when needed.
- 8 Run `scripts/kms.py sync` if included docs changed.
- 9 Run `scripts/kms.py check` for final read-only enforcement.
- 10 Run reasonable implementation validation.
- 11 Report code changes, docs changes, KIA, validation, assumptions, and known issues.

Smart Timetable Versioning Boundary

Stage 3.5 corrects school-day timing. Start time plus teaching-period count and duration plus inserted non-teaching durations produces one per-day ordered timeline and a calculated end. `after_period` blocks shift later teaching periods; preserved `fixed_time` blocks must begin on a live timeline boundary or configuration fails closed. The projection is shared by preview, operational grid, readiness, assignments, snapshots, solver slots, and exports. Existing `break`, `prayer`, and `non_teaching` keys remain valid and the controlled catalog also includes recess, lunch, assembly, whole-school event, advisory, intervention, transition, dismissal preparation, and other. This stage does not generate a timetable.

Stage 4 makes the existing lifecycle operational. Version review is explicit and does not move the active pointer. Active, superseded, and archived versions are read-only and may be reused only through an explicit copied draft. Draft validation checks current fingerprint, complete demand, canonical slots, Planning teacher authority, collisions, stale placements, and locks; it is distinct from generation readiness. Explicit Approve Draft validation records an approval timestamp and actor for the exact draft revision. Generation does not approve; placement, lock, regenerated successor, and stale-authority changes invalidate approval. Publication requires an approved fresh `publication_ready` draft, revalidates it, and atomically swaps the revisioned active pointer while superseding prior active history. The main Draft workspace groups Regenerate, Approve Draft, and Publish Timetable; technical and destructive version controls remain secondary in History. Regeneration requires `ceil(25% * unlocked_placements)` changes from its direct source and fails with a controlled diversity result when no valid alternative reaches that threshold.

Placement actions use separate create/edit/delete permissions. Locks use `timetable.lock_lessons`, publication uses `timetable.publish`, archive uses `timetable.archive_versions`, and explicit selected-version export uses `timetable.export`. Every operation remains branch/year/SchoolGroup scoped.

Stage 3.5 makes the composed per-day slot projection authoritative across the page, assignment service, exports, readiness, and snapshots. Inserted blocks consume no teaching-period number and shift later clock times; ambiguous fixed-time placement is invalid. Readiness is an exact-scope, read-only structural gate; `generation_ready` permits a Stage 5.1 solve attempt and does not guarantee feasibility. Existing versions and stale placements remain preserved.

The operational timetable uses a versioned aggregate scoped by `SchoolGroup`, `Branch`, and `Academic Year`. `TimetableVersion` owns placements and lifecycle; `TimetableActiveVersion` separately selects at most one exact-scope operational baseline. `TimetableInputSnapshot` preserves deterministic Planning/settings/lock authority and component fingerprints. `TimetableGenerationRun` is the Stage 5.1 durable PostgreSQL work queue with progress, attempts, lease/heartbeat, cancellation, safe terminal status, and result linkage.

Planning remains authoritative for section-subject demand, teacher assignments, and HRT fallback. A timetable placement is one teaching period and current `Subject.weekly_hours` is the compatibility required-period value. Existing live placements migrate without normalization into an imported active compatibility version. The current edit route uses a working draft copied from active history; current views and exports resolve that operational draft after edits. Active, superseded, and archived versions cannot be edited in place.

ADR 0026 defines readiness, hard/soft constraint separation, CP-SAT execution, independent validation, regeneration diversity, and publication boundaries. Stage 5.1 implements the hard-constraint generation slice; availability, rooms/resources, cross-campus coordination, preferences, and quality scoring remain unimplemented.

Knowledge Impact Assessment Rule

Every approved implementation must:

- 1 Assess knowledge impact.
- 2 Update relevant Markdown docs.
- 3 Update docs/`CHANGE_HISTORY.md` for meaningful changes.
- 4 Create or update ADRs when major decisions change.
- 5 Update module history when a module's documented state changes.
- 6 Update docs/`AI_PROJECT_CONTEXT.md` when high-level AI onboarding context changes.
- 7 Regenerate the PDF booklet if included docs changed.
- 8 Mention KIA details in the final report.

A task is not complete until the KIA is assessed.

Required final report KIA template:

```
Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:
```

The generated booklet output is:

- 1 `static/docs/TIS_Project_Reference_Booklet.pdf`

Critical Rules Codex Must Follow

- 1 Do not touch SaaS flows unless the task explicitly requires it.
- 1 Do not touch operational logic unless the approved task requires it.

- 1 Do not touch database migrations or `tis.db` unless explicitly approved.
- 1 Do not weaken tenant isolation.
- 1 Do not bypass route permissions or platform owner checks.
- 1 Do not merge platform user, SaaS account, and tenant user concepts.
- 1 Do not change the landing page design or legacy landing files unless explicitly approved.
- 1 Do not add KMS regenerate behavior unless explicitly approved.
- 1 Do not expose the KMS PDF through direct public static links in the app UI.
- 1 Do not push or commit unless explicitly requested.
- 1 Treat production memory as a hard budget. Do not add unbounded full-dataset caches, duplicate production template renders, startup-heavy work, or normal-request diagnostic logging that can trigger Render restarts and 502s.
- 1 Prefer conservative, dependency-light automation.
- 1 Use `reportlab` for the documentation PDF generator.
- 1 Do not require LaTeX, Playwright, Chromium, external network calls, or system font dependencies for PDF generation.
- 1 Always include a Knowledge Impact Assessment in implementation final reports.
- 1 Always keep `.kms-impact.yml` aligned with the task and actual Git diff.
- 1 Never put customer, personal, production, credential, secret, environment, transaction, invoice, webhook payload, or database-row data into KMS documentation.

TIS Project State

docs/PROJECT_STATE.md

M4 Talent Assessment Cycle And Frozen Population Implemented

Migration 20260904_004_talent_assessment_cycle_frozen_population adds the SchoolGroup-wide TalentAssessmentCycle (draft -> open -> closed, no branch_id), frozen population members, and bounded append-only operational audit. Draft preview and Open resolve canonical Academic Placement at the explicit population_effective_at against the Program Academic Year configuration's eligible grades. Open atomically freezes the full organization population with exact Placement/Academic Year/Branch/grade/section context and stores a deterministic full count and fingerprint. No reopen, post-Open mutation, late-entry exception, or assessor assignment exists.

Dedicated talent_assessment_cycles.view/manage/view_population/govern permissions are Administrator-only by default. Branch authors may manage Draft metadata but cannot Open/Close. Governance additionally requires organization/global scope. Population reads are separately permissioned: Branch actors see only authorized preview/frozen Branch members and subset counts, never the full count/fingerprint; organization/global population readers see the canonical whole. Frozen visibility uses the member's historical Branch, not current Student Placement. Focused coverage is in tests/test_talent_assessment_cycle_frozen_population.py.

Student Assessment, assessor assignment, results, Review Candidate instances, Official Identification, analytics, AI, late population exceptions, and UI remain deferred. Live PostgreSQL execution remains outstanding.

M3 Governance Closure: KPI Approval, Rubric Ordering, And Audit Granularity

weighted_level_average is now Product-Owner-approved as one bounded, optional, Framework-specific KPI calculation primitive - never a universal or cross-Program score, enabled only through explicit Framework configuration, and requiring numeric rubric values only while enabled. Qualitative no-KPI Programs remain first-class.

rubric_level_at_or_above candidate rule semantics are defined by the configured Rubric Level order (display_order, lowest proficiency first), independent of numeric_value; display_order was confirmed to safely carry both presentation order and semantic proficiency rank with no divergence risk, so no new field was added. TalentConfigurationAudit.resource_type was widened (model CHECK plus a widening step inside the M3 migration, since the audit table itself was created by the separate already-applied M2 migration) to add rubric, rubric_level, rubric_descriptor, kpi_configuration, kpi_component, review_candidate_policy, and review_candidate_rule; every M3 mutation now audits its specific child resource instead of only framework_version, matching the M2 framework_competency precedent. Candidate-policy rule evaluation remains deferred to a later assessment milestone. PostgreSQL live validation remains outstanding.

Talent Program Deterministic Framework Configuration Implemented Through M3

Migration 20260904_003_talent_rubric_kpi_candidate_policy_foundation adds Framework-versioned configurable rubric levels, exact competency/level descriptors, optional bounded weighted-level KPI configuration, and a bounded declarative Review Candidate Policy. All configuration is Draft-only, expected-revision protected, included in the Framework semantic fingerprint, independently cloned, tenant-scoped by composite foreign keys, and recorded through the existing append-only Talent configuration audit. Active/Retired configuration is immutable. Qualitative Programs can omit KPI and numeric levels entirely. Existing talent_programs.view/manage/govern permissions are reused; governance scope for activation/retirement is unchanged.

No Student Assessment, evidence/result, candidate instance, Official Identification, analytics, AI, entitlement, or UI capability is part of M3. Focused M3 coverage is in `tests/test_talent_rubric_kpi_candidate_policy.py`.

Student & Academic Placement Foundation And Talent Program & Framework Foundation Implemented

Migration `20260904_001_student_academic_placement_foundation` adds Student as a canonical SchoolGroup-owned identity distinct from User, external identifiers, an effective-dated Academic Placement authority (no separate Enrollment entity), and append-only audit. Migration `20260904_002_talent_program_framework_foundation` adds TalentProgram as a durable organization-owned identity distinct from Subject, annual configuration, an immutable-once-active Framework Version lifecycle with deterministic version allocation and one-Active-Framework-per-Program supersession, competency lineage with version-specific framework membership snapshots, and append-only configuration audit. Full architecture and business-rule detail is recorded in `docs/AI_PROJECT_CONTEXT.md` under "Student & Academic Placement Foundation" and "Talent Program & Framework Foundation". `tests/test_student_academic_foundation.py` (13 tests) and `tests/test_talent_program_framework_foundation.py` (15 tests) cover both foundations, including tenant/branch isolation and permission-boundary cases.

Checkpoint A closed the permission-governance gap in both interim implementations. `permission_registry.py` adds dedicated students (`.view`, `.create`, `.edit`, `.activate_deactivate`, `.manage_identifiers`, `.manage_placements`) and talent_programs (`.view`, `.manage`, `.govern`) permission groups, replacing the temporary reuse of the users.* keys (for Student) and the bare, broadly-granted `planning.edit_section` permission (for both Student Academic Placement mutation and Talent Program/Framework Draft authorship - the latter previously let any Branch-scoped Editor/User actor author organization-wide Talent configuration with no Branch-relation constraint). Both new groups are Administrator-only by default, matching the existing users.* precedent; the M2 organization/global-scope gate on Framework activate/retire and Program lifecycle transitions was already correct and is unchanged. `routers/students.py` and `routers/talent_programs.py` now check the dedicated keys; branch-scope-after-permission-check ordering and non-enumerating 404/403 discipline are preserved on every route.

PostgreSQL validation (constraints, concurrent placement/framework writes) has not been executed against live PostgreSQL for these foundations; only SQLite-backed pytest coverage exists. No Student UI, import/merge, Assessment, Review/Identification, Learner Profile, analytics/Talent Map, or AI code exists yet.

Planning Subject Requirement Removal (Single And Bulk) Implemented

Administrators can now remove a Planning subject requirement whether or not it has an explicit `PlanningSubjectDemand` row. Previously "Remove demand" only appeared for a requirement with an untouched explicit row, so a purely legacy-fallback requirement (resolved only from the Subject catalog, with no explicit row at all) silently had no removal action - the same teacher could have some subjects removable and others not, for reasons unrelated to the teacher. `routers/planning.py` classifies every requirement as `removable` (untouched explicit row), `permanent` (Curriculum-Adjustment-touched, shown with an explicit "Protected" explanation instead of a hidden action), `fallback` (no explicit row - now removable by creating an explicit `is_active=False/weekly_periods=0` suppression row; unlike a Curriculum Adjustment retirement, this row sets only `created_by_user_id` and leaves `updated_by_user_id` NULL, so it stays classified `removable/setup-only` rather than immediately becoming `permanent`), or `not_found`.

The customer-facing action is "Remove Subject Requirement" (single, per row) and a checkbox-selected removal action at `POST /planning/subject-requirements/remove`. Bulk removal is atomic: every selected target is validated and scope-checked first, and if any is protected, invalid, or out of scope, nothing is removed and every blocked target is named with its reason. Confirmation dialogs precede both single and bulk removal. Each expanded section now places **Select all** beside a trash-icon action whose label changes to the singular or exact selected count; the page-level action mirrors that selection state. Selection can still span multiple sections in one request. Teacher assignments, timetable

data, and Curriculum Adjustment audit records are never touched by this action. The existing GET `/planning/subject-demand/delete/{demand_id}` route from the prior round remains available, superseded in the rendered UI, for clearing an active untouched setup row. An inactive zero-period setup suppression with no Curriculum Adjustment history no longer blocks an otherwise-empty section: deletion removes that exact scoped artifact and the section atomically. Active demands, permanent demand history, assignments, timetable/calendar/rule dependencies, and all other existing guards still block.

Claude Code KMS Integration

The repository now includes root `CLAUDE.md`, a reusable project skill at `.claude/skills/tis-kms/SKILL.md`, and a specialized subagent at `.claude/agents/tis-kms-developer.md`. This gives Claude Code a native entry point to the same KMS onboarding, scope safeguards, worktree preservation, validation, and completion reporting already required by `AGENTS.md`; it does not change application, database, deployment, or customer behavior.

Return-To Reopen-On-Load UI Pattern Implemented

A shared `redirect_utils.py` module (`safe_redirect_path`, `redirect_with_notice`, `redirect_with_error`, moved out of `main.py` with existing callers unchanged) and a shared `static/js/reopen-on-load.js` (loaded globally from `base.html`) finish wiring the existing `return_to` convention so a server-rendered action inside an expanded element can land the user back on that same element, scrolled into view, instead of collapsing it. Planning's "Remove demand" action is the first and, for now, only consumer: each section's `<details>` carries a stable id, the link passes `return_to` to that section's fragment, and the route's in-place blocked-render outcomes set the same reopen target without a redirect. No SPA conversion, no new browser storage. Subjects and Teachers were not changed; the pattern is available for them to adopt later.

Planning And Subject Delete Dependency Guards Implemented

Planning section delete and Subject delete/Bulk Delete now run a read-only dependency check before any mutation, naming every blocking category in a customer-safe message instead of raising an unhandled `IntegrityError`. Planning section delete checks `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent/CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule`; `TeacherSectionAssignment` is now a blocker instead of being auto-deleted. Subject delete/Bulk Delete additionally check `TimetableEntry`, `CurriculumAdjustmentAudit`, and `SubjectDistributionRule` subject-code references that previously had no application-level check. Bulk Subject delete is atomic: any blocked Subject stops the whole batch and is named with its reason.

Planning subject demand is classified removable or permanent using the existing `updated_by_user_id` column: Curriculum Adjustment always sets it, the one-time setup backfill never does. A never-touched row is pure setup scaffolding and can now be hard-deleted through the new "Remove demand" action on the Planning page (GET `/planning/subject-demand/delete/{demand_id}`), after which the section or subject becomes deletable; a row Curriculum Adjustment has ever touched remains a permanent, honestly-explained blocker. Published/archived timetable placements follow the same principle without a new action. No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced.

Professional Timetable Lifecycle UX Implemented

The workspace now identifies CURRENT PUBLISHED, editable working Drafts, and read-only history with visible Draft ? Validate ? Approve ? Publish guidance and audit facts. Safe actions distinguish copying published/history into a working Draft from starting an empty Draft, and Generate/Regenerate language makes Draft-only effects explicit. The active pointer remains unchanged until approval and publication succeed. Mutation/version errors remain backward-compatible and now add canonical conflict evidence plus stale-revision refresh remediation. No schema, solver, or Workflow behavior changed.

Guided Curriculum Adjustment UI Implemented

Stage 5 adds the permission-gated administrator workflow from Planning and Subjects through scope, change, preview, teacher decisions, final confirmation, and success. It reuses the deployed Stage 3 preview and Stage 4 atomic apply APIs, preserves selections where safe after a refreshed review, and offers timetable/regeneration navigation without automatic generation.

The transfer contract now carries the requested period count instead of deriving an absolute source result from `Subject.weekly_hours`. Subjects also reflects effective Planning weekly periods for Current/New sections, including a section breakdown when values differ.

Curriculum Adjustment now also supports reduction without transfer. Subject Details exposes effective Planning periods and a permissioned prefilled reduction entry point; original catalog periods remain separately editable defaults and are not operational Planning authority.

Atomic Curriculum Adjustment Apply Implemented

Stage 4 implements a service-owned, one-commit curriculum adjustment transaction behind dedicated `curriculum.adjust`. It revalidates the reviewed preview fingerprint under exact-scope locks, rejects active timetable generation and all unresolved teacher/rule/lock conflicts, applies section-specific source/target demand and the explicit teacher decision, retires zero demand and its section rule, marks the unpublished Draft stale, clears approval, and records a durable audit.

Migration `20260829_001_curriculum_adjustment_apply_foundation` adds the audit ledger and duplicate-fingerprint guard. No guided UI or automatic regeneration is included; published timetable versions and the active pointer are untouched.

Curriculum Adjustment Preview Implemented

Stage 3 provides a read-only curriculum adjustment preview service and `POST /planning/curriculum-adjustments/preview`. Grade, selected-section, and all- active-use scopes are exact-tenant and Current/New only. Preview output includes per-section demand transfer, current and suggested teachers, capacity projections, scheduling-rule and grouped-configuration impacts, blockers/warnings, Draft stale/regeneration impact, and a deterministic stale-confirmation fingerprint.

No demand, teacher assignment, timetable version, placement, active pointer, or published history is changed. Apply, teacher confirmation, atomic write, and regeneration workflows remain future stages.

Explicit Planning Subject Demand Foundation Implemented

Migration `20260828_004_planning_subject_demands_foundation` adds normalized `planning_subject_demands` records for section-subject weekly periods. Current/New Planning sections are backfilled from grade-matched Subjects in the same branch and academic year. The operation is idempotent; composite scope foreign keys and an active-row partial unique index protect integrity while allowing retired history.

`planning_subject_demand_service.py` provides exact-scope explicit-first resolution and legacy fallback, including authoritative inactive rows that suppress fallback. Stage 2 now routes Planning calculations, teacher workload, timetable readiness, workspace, snapshots and generation inputs, Subject Scheduling Rule arithmetic, and required-hours reports through that authority. Missing explicit rows retain legacy fallback during transition. Published timetable rows are untouched.

Central Tenant Report Branding Implemented

Tenant-facing operational exports now resolve logo assets through `tenant_report_branding.py`, which delegates to the existing branch/organization- scoped branding slots and returns configured tenant logos only. Timetable

PDF/XLSX, academic-calendar PDF, and observation PDF generation share this authority. When no tenant logo is configured, exports retain their normal report title/header but render no logo; they never substitute the TIS product mark. Application UI, login/product, public/platform, and internal platform-owner branding are unchanged.

Subject Scheduling Rules UI Implemented

The Timetable Settings page presents a grade-first Subject Scheduling Rules table sourced automatically from Planning. `planning_scope_service.py` supplies one branch/year-scoped Current/New PlanningSection source to its main/copy grade selectors, section overrides, the timetable workspace section filters, and directly related calendar/grouped section choices. Global, non-operational placeholder, and Subject-catalog-only grades are excluded. Only the selected grade's compact Subject, read-only Weekly, Current Pattern, Status, and Edit columns are shown, with optional subject search. The modal uses native closed-dialog behavior on page load and opens only after Edit populates the selected Subject + Grade rule; Cancel/Close closes it. Its compact two-column Session Structure aligns double-block/single-session steppers above a full-width automatic total, while conditions use a balanced two-column grid. Two-period presets and progressively disclosed advanced settings remain. A Section Overrides list preserving Stage 1 section-over-grade-over-branch-default precedence, Copy Rules From Grade (name-matched across grades, skipping arithmetic mismatches), and Reset To Default (removes only the grade-level row). New routes `/system-configuration/timetable-settings/subject-rules` (save/reset/clear-override/copy) reuse the existing `timetable.manage_settings` permission, remain tenant/branch/year-scoped, and never mutate Planning weekly totals. `subject_distribution_rules_ui.py` re-validates arithmetic and feasibility authoritatively before any write. Existing legacy subject-code mappings and Swimming/grouped JSON remain unchanged but are collapsed inside an Advanced / Legacy area; Non-Teaching Blocks management is unchanged. Newly saved/updated rules take effect the next time a Draft Timetable snapshot is created, through the unchanged Stage 2 resolution/snapshot pipeline; no already-created snapshot is altered.

Subject Distribution Rules Generation Wiring Implemented

The resolved Subject Distribution Rule (section over grade over branch default, None for legacy fallback) is now embedded per Planning demand in the schema-v3 snapshot at creation time; a later rule change never alters an already-created snapshot. The problem builder carries the resolved rule per demand, exposes true physical period adjacency per slot, and runs the arithmetic/feasibility validator as a final pre-solve defense (`distribution_rule_invalid`). CP-SAT now models explicit double-block starts and single sessions, channels every occupied period to exactly one session, and enforces the configured block and single counts exactly. Separate sessions may touch (including double-plus-single three-period runs and two-double four-period runs), while physical interruptions break doubles and overlapping session membership remains impossible. Redundant daily session/load channeling strengthens daily-coverage propagation. CP-SAT also enforces generalized daily-coverage/spread/max-per-day/ min-teaching-days behavior (hard only when explicitly configured hard), and exempts declared blocks from the consecutive-avoidance penalty; demands without a resolved rule keep the exact legacy `quality_rules_json` behavior. The independent validator mirrors every new hard check. Readiness blocks genuinely invalid/infeasible normalized configurations while keeping the existing legacy warning path. Regeneration's `ceil(25% x unlocked placements)` default, locks, and teacher authority are unchanged; hard distribution rules remain enforced during regenerate through the same constraint-building path.

Subject Distribution Rules Foundation Implemented

Migration `20260828_003_subject_distribution_rules_foundation` adds the normalized `subject_distribution_rules` table (branch/grade/section scope, block/single counts, min teaching days, max periods per day, daily-coverage/spread/consecutive preferences, min day gap, hard/soft strictness). `subject_distribution_rules.py` resolves section-then-grade-then-branch-default precedence with field-level inheritance and returns None for legacy fallback when nothing is configured. `subject_distribution_validator.py` independently checks the Planning-weekly-total arithmetic and day/period feasibility. `timetable_slot_service.py` now marks true physical period adjacency (false across a Break, Prayer,

or other non-teaching item) so a later intentional-block feature can rely on genuine continuity. No existing solver, validator, readiness, or settings-UI behavior changed, and `quality_rules_json` remains the active authority for every tenant.

Smart Timetable Academic Scheduling Quality Implemented

Timetable Settings persist branch/year subject-code mappings and grouped Swimming configuration through migration `20260828_002_smart_timetable_academic_quality_rules`. Immutable snapshots, problem construction, CP-SAT, and independent validation share the normalized authority for core daily coverage, short-demand spread, ICT daily limits, grouped synchronization, teacher/resource safety, and configurable regeneration diversity.

On-Demand Timetable Generation Workflow Implemented

Generate and Regenerate still create the Stage 5.1 durable PostgreSQL run and immutable snapshot, then the web service dispatches only its public ID to a registered Render Workflow task. The task reuses the single-run solver/validator/persistence pipeline, keeps leases, heartbeats, cancellation, staleness, and atomic result guards, and exits. Duplicate task starts are no-ops after claim or completion. Immediate dispatch failure safely terminates an unclaimed run and delayed start receives an honest waiting message. Render retries are explicitly disabled; Generate Again is the retry authority. No schema or solver behavior changed, and the polling worker is optional/local only rather than a production service.

Smart Timetable Stage 5.2 Simplified Workflow Implemented

The manager workspace now emphasizes Create New Timetable, one current Draft Timetable, readiness, Generate or Regenerate, Approve Draft, and Publish Timetable. Generation creates an unapproved draft. Approval records the reviewing administrator only after exact-version validation, every content or authority change invalidates approval, and publication revalidates before changing the active pointer. Regenerate, Approve Draft, and Publish Timetable now occupy one primary Draft workflow area; History retains destructive and technical version actions. Technical lifecycle controls and permanent unpublished cleanup are secondary Timetable History features. Draft deletion uses `timetable.delete_versions` and permanently removes eligible never-published versions without moving the active pointer; historical archive remains a separate History-only action.

Published-only `/my-timetable` and `/published-timetable` views resolve exclusively from `TimetableActiveVersion`. My Timetable fails closed without exact-scope teacher identity and filters to that teacher. `timetable.view-only` users are redirected away from management. Migration `20260828_001_smart_timetable_stage52_draft_approval` adds nullable approval timestamp and actor provenance; solver behavior is unchanged.

Smart Timetable Stage 5.1 Generator Implemented

Authorized administrators can queue Generate or Regenerate from structurally complete current inputs. A stale existing Draft is reported as `stale_input` / Draft Needs Regeneration without becoming configuration-incomplete: it may run a fresh feasibility verification, then regenerate only after the current fingerprint is verified. The stale source remains versioned and unpublishable, and obsolete-run messages from older authority fingerprints do not replace the current state. Regeneration eligibility is based on the current Draft's lifecycle and populated placement arrangement rather than creation origin: eligible manual, generated, and regenerated Drafts regenerate, while empty starters generate and active or historical versions remain excluded. A durable hard-only feasibility Workflow uses the same immutable snapshot/problem builder and independent validator before full optimization. Its validated solution is cached in `timetable_feasibility_verifications` by exact tenant scope and full input fingerprint; only Feasibility Verified enables generation. Full CP-SAT optimization uses that solution as a hint and validated timeout fallback, while any authority fingerprint change invalidates reuse. A separate OR-Tools CP-SAT execution environment uses immutable schema-v3 input, durable lease/heartbeat/recovery state, exact demand and collision constraints, Planning/HRT teacher authority, canonical teaching slots, and fixed locks. A solver-independent validator and final

fingerprint/source-revision check gate atomic creation of one unpublished `publication_ready` version. Regeneration keeps its source unchanged and enforces the approved unlocked-lesson difference. The default regeneration difference is 25 percent of unlocked source placements, rounded up. Infeasible diversity is reported explicitly without weakening hard rules. The active/published pointer is never changed by generation.

The Workflow worker now reaches the existing CP-SAT solver and its diagnostic profiles through `timetable_solver.py`, a solver-neutral adapter boundary. CP-SAT remains the only Release 1 production implementation. Problem construction, independent validation, and atomic persistence remain separate authorities.

Planning-to-timetable demand now passes through `timetable_requirement_projection.py`. The read-only projection preserves explicit-first demand, authoritative zero/retirement, legacy fallback only when no explicit row exists, scoped teacher authority, and deterministic internal identity/provenance. Schema-v5 snapshots freeze the projected requirements; schema-v3/v4 generation snapshots remain readable. No new table, editable demand authority, API, or migration was added.

Timetable conflict evidence now passes through `timetable_conflicts.py`. Readiness exposes additive safe conflicts beside its unchanged blocker/warning fields; feasibility exposes them beside unchanged diagnostics; generation-run payloads derive durable conflicts from existing terminal status/category/message; and the independent validator returns canonical conflicts beside legacy errors. Internal requirement and solver correlation never appears in public payloads. No conflict persistence table or migration was added.

Migration `20260822_001_smart_timetable_stage51_generator` adds progress, attempts, cancellation audit, worker-claim indexing, and one-active-run-per-scope enforcement. The page polls real phases and recovers active work after reload. Stage 5.2 UX and teacher visibility are implemented as documented above.

Smart Timetable Stage 3.5 Composed Timeline Implemented

The timetable now derives every day from one canonical composer: shift start, teaching-period count/duration, and applicable inserted blocks. After-period blocks are first-class; boundary-aligned legacy fixed times remain compatible and ambiguous overlaps block readiness/assignment. Calculated end time, previews, timetable rows, snapshots, staleness fingerprints, and exports share the same projection. Migration `20260821_001_smart_timetable_stage35_composed_timeline` adds placement mode, after-period boundary, and duration metadata without rewriting legacy times. The timeline is now consumed by Stage 5.1; availability and room/resource models remain absent.

Smart Timetable Stage 4 Version Publication Implemented

Administrators can review scoped version history, explicitly open historical versions, copy immutable versions to drafts, lock/unlock draft lessons, validate a specific draft, compare two same-scope versions, archive drafts/superseded history, export the selected version, and publish a complete fresh publication-ready draft.

Publication is one transaction using version row locking, `edit_revision`, active pointer locking/revision, current fingerprint revalidation, and full solver-independent validation. The former active version becomes superseded and remains reviewable/exportable; the new active version is immutable. Stage 3 Ready to Generate remains a separate input-readiness concept. Stage 5.1 now supplies the solver.

Smart Timetable Stage 3 Readiness Implemented

Stage 3 adds one deterministic canonical projection for fixed teaching periods, full-period unavailability, between-period display blocks, and invalid partial overlaps. The projection drives UI availability, assignment validation, snapshots, exports, and the future solver boundary without shifting period times.

`TimetableReadinessService` evaluates one explicit organization/branch/year scope against configuration, Planning demand/allocation, HRT, authoritative teacher capacity, slot sanity, locks, and staleness. Only `generation_ready` is ready, and it means inputs are coherent enough to attempt solving-not that a feasible global

timetable is guaranteed. No solver or Generate endpoint exists.

Smart Timetable Stage 2 Version Foundation Implemented

ADR 0026 Stage 2 is implemented. Timetable placements now belong to durable, SchoolGroup/Branch/Academic-Year-scoped versions with deterministic input snapshots, a separate exact-scope active pointer, lock metadata, version-scoped collision guarantees, and a future generation-run record. Migration 20260819_002_smart_timetable_stage2_version_foundation imports each populated legacy timetable exactly, creates one compatibility active version, records safe staleness evidence without repairing placements, and skips empty settings-only scopes.

The legacy assignment route remains usable through copy-on-write: its first edit creates a mutable working draft from the immutable active version. Current views and XLSX/PDF exports resolve the operational version. Stage 2 adds no solver, worker, Generate/Regenerate endpoint, generation UI, room/resource authority, availability rules, or new teacher-capacity authority. Structural readiness and solver execution remain later stages.

Capacity-Based Packaging Implemented

ADR 0025 is implemented. Nine normal customer entitlement keys are identical for Starter, Professional, and Enterprise AI. Paid, promo, and active demo workspaces resolve those features through one source-aware permission/scope/commercial decision. Advanced Reporting allocation exports now follow this path. The report generators and calculations are unchanged.

Migration 20260819_001_capacity_based_customer_feature_baseline updates plan values and compatible legacy flags, repairs only currently active promo feature values, and reconciles active demo baseline policy. It does not change capacity, priority support, Paddle/payment evidence, immutable promo records, or expired authority. AI availability is common; demo allowances and future provider limits resolve through a separate consumption-policy boundary.

Stage 5 SchoolGroup Boundary Correction Complete

Operational SchoolGroup create/delete actions now require both Platform identity and global SchoolGroup-management capability. Tenant create/delete permissions were removed from role defaults, direct stale-permission requests fail before side effects, and tenant SchoolGroup updates are constrained to the linked workspace. School Management now presents paid and promo branch capacity from unified commercial authority, while existing locked mutation enforcement is unchanged. The individual last-active-branch rule is SchoolGroup-scoped. A sanitized, read-only PostgreSQL provenance audit is available for later Platform Owner review; no production audit or remediation has been run.

Existing Workspace Controlled Conversion

M4B is implemented. Migration 20260806_001_existing_workspace_controlled_conversion adds the conversion operation/event ledger, tenant-profile legal name, append-only event guards, and a partial unique tenant-owner link constraint with duplicate preflight. The generic dry-run-first CLI requires exact workspace, owner, M4A hash, operation, idempotency, and actor parameters; write preparation and final conversion require explicit phrases and PostgreSQL.

The normal TIS registration and email-verification flow establishes the owner identity. A verified owner then claims the prepared operation and completes only legal name, controlled IANA timezone, and educational program. Final conversion re-audits under row locks, detects branch/dependency or commercial drift, preserves every branch and operational record, ends internal sandbox authority, and leaves the customer workspace in coherent `activation_required` state with no active entitlement or tenant source. Organization Account and M3 promo activation remain available; direct operations and existing-workspace Paddle activation remain unavailable. No production conversion has been run.

Existing Workspace Conversion Audit Foundation

M4A is implemented as a read-only prerequisite to any legacy internal-sandbox conversion.

`saas/existing_workspace_conversion_audit_service.py` resolves an explicit workspace/owner tuple, inventories ownership and commercial evidence, and traverses reflected branch dependencies without modifying the session. `scripts/audit_existing_workspace_conversion.py` requires PostgreSQL, starts a repeatable-read read-only transaction, emits deterministic sanitized JSON or text with a stable SHA-256 snapshot, and always rolls back. Exit codes separate coherent (0), execution failure (1), manual review (2), and workspace identity mismatch (3). Reflected model drift, soft-deleted dependencies, owner conflicts, and paid/demo/promo evidence fail closed. Archival candidate IDs are emitted only for active branches proven dependency-free.

No target-specific identifier exists in reusable service or CLI logic. No branch, account, owner, classification, lifecycle, entitlement, tenant link, approval, Paddle, email, or production data is changed. M4A grants neither hard-delete approval nor write-conversion authority; those remain later milestones.

Promo Redemption And Organization Activation

M3 is implemented. Verified organization owners can apply an approved promo from the post-onboarding commercial choice or an eligible existing Organization Account. Activation is resumable and idempotent, uses HMAC lookup without persisting the raw code, and commits the redemption, immutable grant, entitlement, branch assignments, tenant source, lifecycle, and durable audit as one transaction.

Promo-backed workspaces use classification customer, lifecycle active, and M1 source `promo_grant`. Access is limited to selected active branch entitlements and expires from the persisted grant window. Existing aligned organizations need no `PendingOrganization`; onboarding organizations continue through the shared workspace builder. Staff/teacher excess blocks without mutation. Internal sandboxes, incompatible sources, ambiguous links, and expired grants fail closed. No Paddle or billing object is created.

Promo activation now writes explicit branch evidence for the complete preserved workspace inventory: selected branches are assigned and active, while unselected branches are explicitly inactive even when their operational status is inactive. Individual and bulk branch reactivation enforce grant capacity and update the operational branch, assignment, and entitlement in one transaction. A generic PostgreSQL reconciliation CLI defaults to dry-run and may create only safely attributable missing inactive evidence; contradictory chains remain manual review.

Organization Account Billing & Subscription now selects its presentation from the authoritative commercial source. Promo-backed customers receive a first-class Commercial Access page showing their immutable grant plan and period, safe active or expired status, masked promo reference, and M1-resolved branch, system-user, and teacher capacity. Paid subscription and demo pages remain separate. Operational promo access now ends exactly at `effective_to`; grace days are represented as recovery only and do not authorize tenant operations.

Expired and recovery-period promo workspaces can continue with a paid subscription through M4C existing-workspace activation. Promo authority is unchanged until a verified completed Paddle transaction. Completion reuses the tenant and atomically ends promo entitlement authority, relinks the existing tenant source to the paid contract, establishes paid workspace and branch entitlements, and retains promo history. Active promos cannot convert early. A shared source/plan/status badge now identifies Promo, Demo, and Paid authority in Organization Account and the detailed commercial page. The normal operational header remains commercial-status free.

Promo Code Foundation And Platform Management

M2 is implemented as an additive promo-definition layer. `promo_codes` stores only HMAC lookup authority and masked display fragments, exact capacity, controlled scope, validity, lifecycle, replacement, and governance metadata. `promo_code_branch_restrictions` preserves eligible existing-branch snapshots; `promo_code_audit_events` records allowlisted durable action history. Migration `20260805_001_promo_code_foundation` performs no backfill.

Platform Owners and permission-authorized Developers can use the Promo Codes console. Developers may view, create/edit unused drafts, duplicate, pause, and create replacement definitions; activation and terminal revocation remain owner-only. M3 now consumes approved definitions through a separate customer-redemption service; definition management itself remains isolated from activation, M1 authority, Paddle, onboarding, and tenant operations.

Unified Commercial Access And Capacity Authority

M1 establishes `saas/commercial_authority_service.py` as the only operational capacity facade. It composes workspace classification and lifecycle, `WorkspaceEntitlement`, `TenantProvisioningLink`, `SubscriptionContract`, `PaymentSubscription`, commercial state/access, demo lifecycle, and the existing plan-capacity and feature-entitlement services. It does not persist a second commercial status or entitlement model. Missing, invalid, ambiguous, or unsupported authority fails closed.

For active paid workspaces, allowed branches are confirmed subscription quantity capped by the plan ceiling; staff and teacher limits come from the confirmed plan. Staff usage includes every distinct active tenant operational User in the SchoolGroup, including an operational organization owner and users whose position is Teacher. Platform users and account-only SaaS identities do not count. Active teacher people are deduplicated by normalized `Teacher.teacher_id`; blank legacy identities each count separately. A person represented by both records consumes one staff slot and one teacher slot.

Branch, user, teacher, academic-year, and provisioning growth now acquires a tenant row lock, recounts authoritative usage, evaluates the proposed final state, and mutates in the same transaction. Existing over-capacity tenants keep their data and access but cannot further increase an exceeded dimension. Demo and Internal Sandbox authority is explicitly unmetered in M1. No schema, migration, pricing, Paddle, webhook, onboarding-transition, permission, or feature-packaging change is included.

Explicit Organization Billing Identity

Organization Account Billing & Subscription now owns a confirmed billing profile independent from SaaS authentication. The profile stores billing email, legal/billing organization name, contact, optional company/tax identifiers, and the existing supported address structure. Initial Secure Payment confirms this profile; only organization owners or linked users with `subscriptions.manage_billing` may view or update it for an active tenant.

The mapped Paddle customer is reused and synchronized after an authorized billing change. Its active address and one attributable Paddle Business are created or updated and persisted as `PaymentCustomer.provider_address_id` and `PaymentCustomer.provider_business_id`; the active subscription identity and future initial transaction use those mappings. Ambiguous mappings fail closed. Login-email changes never mutate billing email, billing-email changes never mutate authentication, and historical invoices are not silently revised.

Billing Contact now renders saved values read-only until an authorized Edit. Saving commits valid local details even when Paddle cannot be updated. The customer can retry synchronization directly without changing the profile; the retry reuses existing customer/address/business mappings and becomes a no-op after success. Safe server logs identify the failed provider step, error code, and status without rendering diagnostics to customers. A saved profile in pending or failed synchronization state blocks new plan and quantity changes until synchronization succeeds. Cancellation and legacy active subscriptions without a saved profile are not newly blocked.

Provider `paid` is now presented as payment received while processing and is recorded separately from final confirmation. Provider `completed` remains the required reconciliation signal and renders as `Paid`. No webhook completion rule or subscription authority changed.

Subscription Capacity Presentation And Account Billing Access

Subscription Management and Review Capacity now present paid branch quantity separately from the plan branch ceiling. Additional required branches are identified as additional billed quantity, while system-user and teacher counts

remain non-billed plan-eligibility ceilings. Customer plan cards no longer show unapproved feature claims or feature placeholders.

Authorized operational organization owners and explicitly billing-authorized account managers can open the existing Organization Account Billing & Subscription page from System Configuration. The bridge and destination both revalidate tenant/account linkage and permissions. Missing SaaS authentication preserves only the approved internal subscription continuation through sign-in; invalid, unrelated, and external destinations fail closed. The Next.js landing page now labels this centralized customer entry Organization Sign In. Paddle quantity, pricing, entitlement, lifecycle, webhook, and provider-authority behavior are unchanged.

Organization Account Sign-In Routing

Activated organization owners and account-authorized linked users now land on the Organization Account Overview after public/onboarding password or social sign-in, authenticated sign-in restoration, and SaaS root restoration. The overview filters Organization Profile, Branches, Billing & Subscription, and Account & Security by existing ownership and operational permissions. Enter TIS Platform is the only overview action that enters operational login. Restricted account managers retain billing/recovery access without operational entry; incomplete onboarding resumes; non-management users retain role-based routing; and multi-organization identities select the organization first. No schema, permission semantics, commercial-access decision, or operational login behavior changed. The selected UUID is stored only in an HTTP-only cookie and validated against the account's links before the entitlement resolver uses it.

Initial Secure Payment Recovery

The production-equivalent failure was local and preceded Paddle transaction creation:

`_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state. That eligible state is now prepared safely. Professional Annual remains USD 790 per active branch, and the verified two-branch checkout uses quantity 2 with a USD 1,580 annual total.

Secure Payment retry now repairs eligible unpaid legacy/stale checkout state instead of repeatedly rejecting it. Plan, interval, quantity, selection, and quote changes supersede the old checkout session and unfinished attempt, clear their local authority, and produce a fresh transaction from the current server quote. Superseded transaction webhooks remain fail-closed and cannot create a subscription or workspace.

Paddle customer addresses are reused only when active, customer-matched, and country-compatible. New and reused transaction URLs are released only through the billed automatic-collection transaction contract. Provider failures are logged with status/code/traceback and rendered as one customer-safe retry alert. Pricing, plan limits, branch authority, webhook confirmation, checkout architecture, provisioning, and schema are unchanged.

Organization Profile And Initial Account Setup Correction

Organization Profile now handles approved educational-program values consistently and maps invalid input back to the same page with preserved form values. Pending logos are actual-image validated, limited to 4 MB, assigned an opaque unique filename, written atomically, and cleaned up when a later database step rolls back. Empty upload retains the existing logo, successful replacement removes the obsolete file, and storage/unexpected errors are logged server-side while customers receive a safe inline response.

The fresh verified-account page now presents one compact setup title, one explicit POST start action, and the existing eight-step progress indicator. Its smaller header logo and consolidated content remove the repeated status, action, account/workspace, and guidance panels without changing later onboarding states.

Pending organization logos still use `static/uploads/saas/pending_logos`. This local path is not production-durable on an ephemeral Render filesystem; persistent disk or object storage remains an owner-approved follow-up. No schema, migration, payment, billing, operational-module, or Next.js landing change is included; provisioning changed only to validate and preserve the already-established logo promotion.

The saved pending logo now appears in Organization Profile and the existing School Workspace identity area with the organization name and a neutral placeholder when absent. Existing paid/demo workspace creation promotes the image into the primary SchoolGroupLogo slot, whose protected URL is consumed by the operational header and branding settings. The official TIS logo remains separate. Provisioning no longer silently ignores a referenced pending logo whose local file is missing.

Branch Setup totals now come from a Python normalization helper, so placeholder, legacy-null, and mixed-value rows render without Jinja arithmetic errors. Customer saves continue to require explicit non-negative system-user and teacher estimates, and new branches receive zero defaults.

Post-Verification Login Correction

Fixed the fresh-account login destination that previously redirected a successful POST login to the POST-only /saas/onboarding/start action. The browser followed that 302 with GET and received raw HTTP 405 JSON. Fresh verified accounts now open /saas/account; onboarding creation remains behind the existing explicit POST action. Continuations are restricted to known GET destinations, and accidental GET navigation to /saas/auth/login redirects to the normal sign-in page. Verification, password authentication, subscription intent, preferred-plan behavior, and onboarding business rules are unchanged.

Public Subscription Journey

The Next.js hero Subscribe Now CTA now scrolls to the stable pricing section. Starter, Professional, and Enterprise AI share one CTA presentation and enter public TIS Account registration with an allowlisted preferred-plan code. Registration preserves that preference without creating commercial records. After School Workspace Setup, plan selection reuses the existing three-dimension capacity authority; an invalid, inactive, or undersized preference is cleared and the customer reviews eligible plans. Custom remains mail/contact-only. No plan price, capacity limit, Paddle behavior, or AI entitlement changed.

Per-Branch Subscription Capacity

Implemented required system-user and teacher estimates on each onboarding branch, organization-wide live totals, and one three-dimension capacity decision covering branches, tenant operational staff users, and teachers. The lowest eligible self-service plan is recommended while higher eligible plans remain selectable. Starter limits are 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500; exceeding any Enterprise limit routes to contact-only Custom.

Before payment, estimates are compared with actual same-workspace counts and the greater value is authoritative. After activation, every distinct active tenant operational User counts as staff regardless of position, and active teacher people count separately. Capacity changes invalidate quote/checkout lineage and clear only an undersized plan. Paid system-user creation/reactivation, teacher creation/year-copy preflight, and downgrades fail before mutation when the result would exceed capacity.

The active customer portal now presents unified Organization Capacity instead of a branch-only action. Review Capacity compares proposed branches, system users, and teachers with the current plan and opens either the existing branch quantity preview or a required plan-upgrade preview. A branch-triggered upgrade may update plan price and branch quantity together; user and teacher totals never change Paddle quantity. Scheduled plan downgrades and quantity reductions revalidate live capacity at the provider-confirmed effective boundary and enter manual review rather than applying an unsafe local reduction.

Customer Journey And Expired-Access Correction

Implemented returning-account state routing, direct expired-demo subscription selection, operational and SaaS expiry pages, and a pre-workspace commercial guard for both demo and paid subscriptions. Demo checkout uses the existing organization, SchoolGroup, and active operational branch quantity and continues into the existing Paddle conversion workflow. The Next.js landing exposes Request a Demo, Subscribe, Sign In, and Open TIS App through

NEXT_PUBLIC_TIS_APP_BASE_URL. The owner demo page retains all M8B9 operations behind progressive disclosure, and customer communications identify the TIS team. No schema, pricing, Paddle-authority, AI-entitlement, migration, or deployment change is included.

Paid commercial access now consumes the existing contract-linked entitlement authority instead of selecting the newest subscription for an onboarding organization. Active or trialing current subscriptions retain operational access while plan changes await payment/provider confirmation, fail, expire, or are abandoned. Plan-change subscription webhooks synchronize provider status without bypassing two-signal plan confirmation. Restricted pages and APIs use state-specific past-due, paused, expired, suspended, archived, and manual-review guidance. A canceled subscription remains entitled only through its confirmed paid period end.

M8B9 Demo Operations, Notifications, And Testing

Implemented Platform Owner-only demo lifecycle controls, synchronous single/global lifecycle summaries, Standard/Full/Custom access profiles, workspace and tenant-safe branch scope, durable success/failure audits, and M8B7-based customer communications. Migration 20260727_003_m8b9_demo_operations remains in the pre-deploy boundary and M8B8 usage history is retained across profile transitions.

M8B8 AI Entitlements And Commercial Foundation

Completed: centralized AI entitlement decisions, stable feature registry, assignable AI permission, two-successful-use demo allowances per feature, Enterprise AI paid mapping, tenant-safe durable counters and operation ledger, idempotent/concurrency-safe consumption, consistent subscription guidance, and Render-safe migration. No real AI execution route exists yet, and M8B9 inspection/reset/override/lifecycle controls remain unimplemented.

M8B7 Demo Customer Journey

Completed: approval-orchestrated activation with safe retry; durable request, approval, decline, reminder, expiry, and continuation email intents; existing Notification Center integration; shared-shell active-demo indicator; lifecycle communication processing; and authoritative-payment conversion of coherent expired demos using the same workspace. M8B8 and M8B9 remain unimplemented.

Render startup no longer executes SQLAlchemy table creation or pending migrations while importing or starting `main:app`. Render runs that work through `python scripts/run_migrations.py` as a required Pre-Deploy Command before activating the new web version. The web process has no migration worker or schema-readiness middleware and binds independently of PostgreSQL DDL.

Last Updated

Last updated: 2026-07-29

Update this file after every meaningful milestone, active development change, roadmap shift, known issue change, or documentation/KMS change.

Current Branch Strategy

Current working branch assumption: dev.

Branch strategy:

- 1 Development work should happen on dev unless the owner explicitly requests another branch.
- 1 Production/live branch is assumed to be separate from active development.
- 1 Confirm production branch before any deployment, merge, or production-sensitive change.
- 1 Do not push, merge, or commit unless explicitly requested.

- 1 Preserve unrelated local changes.

Production / Live Branch Assumption

The live production branch is assumed to be the branch deployed to the public app environment, while dev is the active development branch. This assumption must be confirmed before deployment.

Production domains:

- 1 Public website: <https://tisplatform.com>
- 1 Application portal: <https://app.tisplatform.com>

Completed Milestones

M1: Identity and SaaS foundation

- 1 Core SaaS signup/login/account concepts established.
- 1 Platform, tenant, and SaaS account identities separated.
- 1 Identity and SaaS phase tests present.

M2: Onboarding foundation

- 1 SaaS onboarding flow covers organization, contacts, branches, academic setup, and review.
- 1 Pending organization concept supports pre-provisioning state.

M3: Billing and plan foundation

- 1 Plan catalog, checkout, billing status, checkout return, and checkout cancel flows exist.
- 1 Payment and billing code is isolated under saas/ service modules.
- 1 Initial Paddle checkout price IDs are configured through a script-based mapping sync into `subscription_plan_prices.provider_price_id`.
- 1 Sandbox and production Paddle price mappings must remain separate; real mapping files are ignored and credentials stay in environment variables.

M4: Provisioning foundation

- 1 Platform owner provisioning views and actions exist.
- 1 Pending organization review and provisioning queue behavior exist.
- 1 Provisioning retry/run operations are present.

M5: Platform access and owner controls

- 1 Platform owner and platform developer identities exist.
- 1 Platform console and owner/developer management controls exist.
- 1 Permission registry and platform access tests support this boundary.
- 1 Platform Owner pending counts and lists now use one lifecycle-aware query boundary instead of counting every historical `PendingOrganization` row.
- 1 Active/completed tenants are retained in Organization Records, while unresolved completed-provisioning combinations are surfaced conservatively as Lifecycle Review Required.

- 1 Owner-facing organization lifecycle labels reconcile onboarding, payment, subscription, contract, tenant-link, provisioning-job, and active SchoolGroup evidence without mutating historical status fields.

SaaS account setup stabilization:

- 1 Phase 1 TIS Account email verification recovery is accepted.
- 1 Valid verification links now redirect to TIS Account login with a professional success notice.
- 1 Expired or invalid verification links now show a recovery page with a resend option.
- 1 Resend verification safely handles unverified, already verified, and unknown-email cases.
- 1 Unverified password-based accounts are blocked from starting or continuing school workspace setup.
- 1 New verification-flow wording uses "TIS Account" and "school workspace setup".
- 1 Payment, billing, provisioning, database schema, migrations, operational modules, and the landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.

SaaS customer-facing wording and branding:

- 1 Phase 2 TIS Account customer-facing wording cleanup is accepted.
- 1 Customer account/setup pages now use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation.
- 1 The shared customer account shell uses the official full-color horizontal TIS logo on the light account background.
- 1 Transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.
- 1 Customer views label internal statuses through customer-safe wording instead of exposing raw tenant, provisioning, checkout, provider, plan, school group, or attempt identifiers.
- 1 Internal /saas route/module/model names and stored statuses were not renamed.
- 1 Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.

SaaS guided setup framework:

- 1 Phase 3A shared TIS Account guided setup framework is accepted.
- 1 The customer account shell now supports an official-logo guided setup console with an 8-step journey stepper, status banner, one primary next action, main content area, and help/guidance area.
- 1 The account page now uses the framework and removes the old dense dashboard statistics from the customer account landing page.
- 1 Journey state is derived from existing account, onboarding, billing, payment, and activation data without changing stored statuses.
- 1 Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the landing website, internal /saas route names, and Google/Microsoft login were not changed.
- 1 Phase 3B redesigned the five School Workspace Setup onboarding pages on top of the shared setup shell.
- 1 Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use consistent guided wizard sections, one shared-shell primary CTA, and secondary Back/Save Draft actions.

- 1 Phase 3B preserved backend logic, form actions, field names, validation behavior, draft behavior, onboarding progression, payment/billing/provisioning behavior, database schema, migrations, operational modules, the landing website, internal /saas route names, and OAuth behavior.
- 1 Phase 3C redesigned Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages using the same shared setup shell and guided style.
- 1 Phase 3C added customer-safe browser-return/payment confirmation guidance and explicit TIS Platform access-after-activation messaging without changing payment, billing, provisioning, webhook, checkout start/launch, database, migration, operational, landing, route, stored-status, admin, or OAuth behavior.

Documentation/KMS milestones:

- 1 Phase 1 documentation foundation completed and pushed to dev.
- 1 Phase 2A and Phase 2B KMS foundation approved for implementation.
- 1 Phase 2C Platform Owner Knowledge Center completed and pushed to dev.
- 1 KMS v3.0 Phase 3A Engineering Handbook approved for implementation.
- 1 KMS v3.0 Phase 3B approved for implementation.
- 1 KMS v3.0 Phase 3C approved for implementation.
- 1 KMS v3.0 Phase 3D final phase approved for implementation.
- 1 Automatic KMS enforcement implemented with repository instructions, machine-readable KIA, major-change detection, read-only artifact checks, CI validation, and a deployment prerequisite.
- 1 Phase 6 unified KMS command implemented with `scripts/kms.py sync` and `scripts/kms.py check`.
- 1 Phase 7A navigation foundation implemented with task-based reading paths, improved indexes, and normalized supporting-document titles.
- 1 Phase 7B professional PDF navigation implemented with handbook guidance, a page-numbered table of contents, source and major-heading bookmarks, and manifest source-page metadata.
- 1 Phase 7C Platform Knowledge Center navigation implemented with manifest-backed titles and summaries, logical document groups, client-side search and filters, protected booklet page links, and newest-first knowledge activity.
- 1 Phase 7D KMS navigation and catalog enforcement implemented with usable-title checks, approved category/module vocabulary, exact source inventory checks, docs-only navigation links, and generated PDF page-bound validation.

M7 subscription-management milestones:

- 1 Phase 1 entitlement foundation completed.
- 1 Phase 2 customer Subscription Management portal completed.
- 1 Phase 3 active paid branch-quantity management completed.
- 1 Phase 4 upgrades and scheduled downgrades completed with provider-authoritative previews/proration.
- 1 Phase 5 cancellation/reversal and centralized lifecycle/action policy completed.
- 1 Phase 6 provider billing history and protected invoice management completed.
- 1 Webhook and reconciliation safeguards were added across the M7 lifecycle.

M8B-1 workspace-classification foundation:

- 1 Added stable workspace UUID, workspace classification, and workspace lifecycle metadata to SchoolGroup.
- 1 Added onboarding workspace intent, SaaS account purpose, and operational internal-test identity metadata.

- 1 Added constrained enums, indexes, validation helpers, conversion rejection, and a commercial-state skeleton with no commercial behavior.
- 1 Added a read-only relationship diagnostic plus dry-run/default and transactional/idempotent apply backfill commands.
- 1 Confirmed pre-M8B-1 records are backfilled as internal sandbox/test data; no Al-Andalus conversion is included.
- 1 Platform Owners can inspect the metadata read-only in Platform Console; developers and tenant users cannot.
- 1 Existing onboarding, authentication, Paddle, provisioning, permissions, tenant isolation, entitlements, and customer flows remain authoritative and unchanged.

M8B-2 commercial-state and entitlement foundation:

- 1 Added normalized workspace entitlement, workspace entitlement value, and branch entitlement records.
- 1 Added read-only workspace, branch, and effective commercial-state resolvers with conservative manual-review outcomes.
- 1 Paid workspace capability resolution reuses the confirmed M7 subscription entitlement engine; no billing calculations or Paddle calls were added.
- 1 Existing internal sandbox workspaces are seeded with foundation entitlements, while newly created internal test workspaces can use a read-only compatibility entitlement.
- 1 Platform Owners can inspect Commercial State, Workspace Entitlement, and Branch Entitlement Summary; developers and tenants cannot.
- 1 No customer access rule, branch behavior, feature restriction, demo lifecycle, onboarding, provisioning, role, or conversion behavior consumes M8B-2 yet.

M8B-3 demo-request workflow:

- 1 Completed onboarding now ends with an explicit Request Demo or Subscribe Now choice.
- 1 Subscribe Now continues the approved plan-selection, checkout, Paddle, and provisioning lifecycle unchanged.
- 1 Demo requests are validated, snapshot commercial context, prevent duplicate pending requests, and never create or activate a workspace.
- 1 Platform Owners can search, filter, sort, approve, reject, or cancel requests; approval records review only and rejection requires a reason.
- 1 Customers can inspect their request and withdraw it only while Pending Review.
- 1 Submit, approve, reject, cancel, and withdraw transitions create durable audit and internal-notification events. No email is sent.

M8B-4 demo workspace provisioning and activation:

- 1 Only a Platform Owner can provision a coherently approved customer-demo request.
- 1 Demo provisioning reuses the shared operational workspace builder and creates no Paddle, checkout, payment, subscription, or paid-contract record.
- 1 Demo workspaces receive an explicit demo entitlement and a tenant link sourced by the demo request rather than a fabricated subscription contract.
- 1 Workspace creation and activation are atomic; failures preserve the Approved request, roll back workspace records, and retain a retryable failure outcome.
- 1 Successful provisioning activates the SchoolGroup and entitlement, links the request to the workspace, records activation metadata and audit/internal events, and blocks duplicates.

- 1 Customers see safe approval/provisioning/active states; Platform Owners see provisioning result and failure details. No email is sent.

M8B-5 standard customer-demo lifecycle:

- 1 Demo duration is exactly seven days from successful activation; the reminder boundary is exactly Day 6.
- 1 One resolver owns Active, Reminder Due, Expired, Suspended, and Manual Review outcomes using UTC calculations and organization-timezone display.
- 1 The dry-run/default lifecycle command creates idempotent internal reminders and atomically expires due demos only when run with `- - apply`.
- 1 Expiration ends the demo entitlement and suspends the workspace while preserving users, branches, and all tenant data.
- 1 Operational middleware blocks expired or ambiguous demos for web, API, download, and existing-session requests; Platform users, paid tenants, and internal sandboxes are unaffected.
- 1 Customer and Platform Owner pages show safe lifecycle state, expiration timing, reminder state, and processing history. No email is sent.

M8B-6 demo-to-paid conversion:

- 1 Active, coherently provisioned Customer Demo workspaces may proceed through the existing subscription checkout.
- 1 Provider-confirmed subscription payment converts the same SchoolGroup and tenant link; no workspace, organization, branch, user, permission, or academic record is recreated.
- 1 The demo entitlement is ended and replaced by a paid entitlement linked to the confirmed M7 subscription.
- 1 The existing tenant link moves from its demo-request source to the confirmed subscription contract in the same atomic conversion transaction.
- 1 Conversion states and audit/internal events remain durable, failures preserve paid records and demo operation for retry, and completed conversions leave demo lifecycle processing.
- 1 Expired, ambiguous, cross-tenant, internal-sandbox, paid, and already-converted workspaces remain fail-closed.

M8 Landing Integration:

- 1 Final M8 landing integration exposes two clear customer paths from the public Next.js landing website: Request a Demo and Subscribe Now.
- 1 All public conversion links use the shared `NEXT_PUBLIC_TIS_APP_BASE_URL`, with `/saas/signup?intent=demo` and `/saas/signup?intent=subscribe` as the only destinations.
- 1 Signup and School Workspace Setup preserve the valid selected intent and emphasize it on the final commercial-choice step without locking the customer into it.
- 1 A normalized organization-domain eligibility ledger enforces one customer demo opportunity across pending, approved, active, expired, rejected, cancelled, and demo-to-paid history. Internal Sandbox records do not reserve customer eligibility; public email providers require an official organization website or domain before a demo can be requested.
- 1 Migration `20260725_001_demo_domain_eligibility_policy` backfills safe historical reservations and marks ambiguous duplicate history for manual review without merging, deleting, reprovisioning, or changing existing workspace data.

Test workspace reset dependency correction:

- 1 The Platform Owner-only test workspace/account reset now removes `subscription_change_requests` by the selected `school_group_id` before deleting that workspace's operational users.
- 1 It removes selected entitlement values and branch-entitlement children before the selected workspace entitlement and final SchoolGroup, without changing global entitlement definitions, plans, or prices.
- 1 The same guarded reset now clears only the selected organization's linked demo request, domain reservation, review/event history, provisioning/lifecycle history, and demo-to-paid conversion history. This permits a clean internal retest with the same email and organization domain while the production one-demo-per-domain policy remains unchanged.
- 1 Detached same-domain reservations are now cleaned only after the shared demo-domain resolver finds no other organization, demo request, workspace, or customer account using that domain and the reservation has no historical manual-review evidence; conflicts or ambiguous history are surfaced as manual review and preserve all data.
- 1 Safe detached reservation IDs now pass explicitly from reset analysis to deletion. The deletion transaction removes only those IDs, flushes before demo-request and parent deletion, and verifies that no selected row remains before continuing.
- 1 The scoped pre-analysis count, deletion diagnostics, affected-row total, transaction rollback, and preservation of other workspaces remain in place.

Platform Owner demo eligibility maintenance:

- 1 `/saas-admin/demo-eligibility-maintenance` lists domain-eligibility reservations and identifies safely removable historical detached rows.
- 1 Safety analysis blocks deletion when any matching organization, TIS Account, demo request, operational workspace, provisioning record, subscription evidence, Demo-to-Paid conversion, or manual-review evidence remains.
- 1 The destructive action requires explicit owner confirmation, re-analyzes under an exact-row lock, deletes only the selected eligibility ID, flushes, verifies absence, and commits or rolls back atomically.
- 1 Successful deletion records the Platform Owner, eligibility ID, normalized domain, previous status, timestamp, and fixed historical-cleanup reason in the durable audit log.
- 1 Customer demo submission, one-demo-per-domain enforcement, clean-room reset, schema, foreign keys, and customer-facing behavior remain unchanged.

Current Priority

Current priority: validate the final M8 public landing integration before any separately approved M9 work.

Current enforcement scope:

- 1 Codex reads root `AGENTS.md` and authoritative KMS context.
- 1 Every task updates `.kms-impact.yml`.
- 1 Major-change paths are conservatively classified by `scripts/check_kms_impact.py`.
- 1 Local KMS synchronization runs through `scripts/kms.py sync`.
- 1 Generated artifacts and KIA are validated read-only through `scripts/kms.py check`.
- 1 Pull requests and dev pushes run KMS enforcement.
- 1 master deployment waits for the KMS gate.
- 1 Automation validates and blocks; it does not rewrite Markdown.
- 1 Phase 7D also blocks missing or unusable source titles, unapproved catalog values, invalid KMS navigation targets, source-list drift, and missing, non-positive, non-increasing, or out-of-range manifest PDF pages.

Phase 2A and Phase 2B scope:

- 1 Create documentation update policy.
- 1 Create change history.
- 1 Create ADR foundation and initial accepted ADRs.
- 1 Create module history foundation.
- 1 Create AI project context.
- 1 Update master context, project state, and documentation index.
- 1 Update PDF generator to include KMS docs and manifest metadata.
- 1 Regenerate static/docs/TIS_Project_Reference_Booklet.pdf.

Phase 2C completed scope:

- 1 Added read-only knowledge_service.py as the single KMS app access layer.
- 1 Added owner-protected /platform/knowledge page.
- 1 Added owner-protected PDF view/download routes.
- 1 Added an owner-only Platform Console card.
- 1 Added platform knowledge module history.
- 1 Regenerated the PDF and manifest after documentation updates.
- 1 Phase 7C adds client-side source search, category/module/freshness filters, logical document groups, document titles and summaries, protected booklet page links, and improved ADR/module-history ordering without adding routes or write behavior.

Still out of scope:

- 1 Regenerate button.
- 1 Additional app routes beyond the approved read-only Knowledge Center routes.
- 1 ui_shell.py and authorization.py changes unless separately approved.
- 1 SaaS flows.
- 1 Operational logic.
- 1 Database, migrations, or tis.db.
- 1 Landing page implementation.

KMS v3.0 Phase 3A scope:

- 1 Add complete TIS module map.
- 1 Add repository architecture map.
- 1 Add end-to-end user/system workflows.
- 1 Add clear AI/human developer onboarding structure.
- 1 Update generator to include engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3B scope:

- 1 Add database architecture overview.
- 1 Add development standards and non-negotiable rules.
- 1 Add UI/UX and design philosophy.
- 1 Add product roadmap.
- 1 Strengthen AI/human developer onboarding guidance.
- 1 Update generator to include the new engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3C scope:

- 1 Add rejected architectural decisions.
- 1 Add visual documentation framework.
- 1 Add AI optimization guide.
- 1 Add project governance and decision traceability.
- 1 Update generator to include the new engineering docs.
- 1 Regenerate the PDF and manifest.

KMS v3.0 Phase 3D final scope:

- 1 Add knowledge lifecycle documentation.
- 1 Add documentation automation guide.
- 1 Add formal KIA standard.
- 1 Add self-evolving workflow.
- 1 Add documentation dependency map.
- 1 Add AI coding workflow.
- 1 Add future automation roadmap.
- 1 Regenerate the PDF and manifest.

Current Known Issues

M4C existing-workspace paid activation is implemented for verified tenant owners. Professional and Enterprise AI cover all active branches and use branch count as Paddle quantity. Starter is deliberately fail-closed until complete restricted-branch enforcement is proven. Deployment still requires migration `20260806_002_existing_workspace_paid_activation`, environment-specific active Paddle price mappings, and Sandbox validation before production use. Disposable PostgreSQL validation covers migration idempotency and rollback, database constraints, concurrent prepare/launch, duplicate and out-of-order webhooks, drift rollback, and paid-versus-promo source races. Live Paddle validation was not possible without the Sandbox API, client, and webhook credentials.

Organization Account status for an activation-required existing workspace is now derived from its current paid-activation attempt. With no attempt it displays `Activation required`; only a current unexpired checkout-started or payment-processing attempt displays `Payment processing`. Terminal attempts display recovery states.

Existing-workspace plan selection is editable only before real checkout begins. Organization Account reopens the eligible plan-selection surface for `draft` and `checkout_ready` activations, highlights the saved choice, and recalculates a replacement quote without creating a Paddle transaction or `PaymentAttempt`. Checkout-started,

payment-processing, and manual-review/inconsistent activations remain locked against silent replacement.

Known issues and watch points:

1 Student & Academic Placement Foundation and Talent Program & Framework

Foundation have no PostgreSQL validation yet (constraints, concurrent placement/framework writes); only SQLite-backed pytest coverage exists. Neither has a UI, import/merge, or any Rubric/KPI/Assessment/Review/Learner-Profile/analytics/AI code, which remain future milestones.

- 1 KMS policy depends on future developers and AI agents consistently completing the Knowledge Impact Assessment.
- 1 Generated PDF can become stale during local work, but CI now blocks stale artifacts from integration/deployment.
- 1 The owner-only Knowledge Center is implemented as read-only; there is no regenerate button yet.
- 1 Public static storage is not sufficient access control for sensitive docs; Phase 2C should serve docs through protected owner-only routes.
- 1 Render deployment constraints should continue to guide dependency choices.
- 1 Production memory must be treated as a hard constraint. The 2026-06-27 Render restart/502 investigation found two avoidable memory risks: observation diagnostics doing extra production template renders and global location lookup parsing a 47 MB dataset into a complete in-memory index for simple picker requests. Local stabilization changes now gate observation diagnostics and use scoped location loading; future work must follow the Production Memory and Render Stability standards.
- 1 The untracked repository-root directory `tis_scope_test_5i3yf0h5/` has a Windows

security descriptor that denies enumeration and ACL inspection to the normal development process. Repository pytest collection is bounded to `tests/` and explicitly excludes that orphan directory, so root pytest runs do not scan it.

- 1 Google/Microsoft login is still future work; password-based accounts must remain email-verified before school workspace setup.
- 1 GitHub repository settings must mark KMS Enforcement / kms-check as required on protected branches; this cannot be configured by repository file changes alone.

Next Planned Work

Next planned work:

- 1 Review the KMS enforcement rules against real pull requests and tune only demonstrated false positives.
- 1 Keep M7 documentation current as subscription fixes evolve.
- 1 Later consider an explicit owner-only regenerate workflow.
- 1 Review, commit, and deploy the production memory stabilization changes when approved, then monitor Render memory, restart count, and route-level 502s after deployment.

Landing Page Baseline Situation

The public landing page source of truth is:

- 1 `tis-landing-website/`

Marketing docs:

- 1 `docs/marketing/landing_page_source_of_truth.md`
- 1 `docs/marketing/tis_landing_page_master_content.md`

Relevant ADRs:

- 1 docs/adr/0001-separate-nextjs-landing-website.md
- 1 docs/adr/0007-landing-page-visual-system-strategy.md

Legacy FastAPI landing files are not the current public website source of truth:

- 1 templates/landing.html
- 1 static/landing/landing.css

Do not modify landing page design, landing copy, or legacy landing files unless explicitly approved.

Knowledge Update Policy

Every approved implementation must complete the KIA:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

A task is not complete until KIA is assessed and `.kms-impact.yml` matches the actual task diff. If included docs change, regenerate:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf

Then run `.\.venv\Scripts\python.exe scripts\kms.py` check for final read-only validation. When documentation changes, `.\.venv\Scripts\python.exe scripts\kms.py sync` performs generation and post-generation validation together.

Scope Guardrails

- 1 Do not touch SaaS flows unless explicitly approved.
- 1 Do not touch operational logic unless required by the approved task.
- 1 Do not touch database migrations or `tis.db` unless explicitly approved.
- 1 Do not change the landing page unless explicitly approved.
- 1 Do not add a KMS regenerate button until separately approved.
- 1 Do not let automation rewrite authoritative Markdown.
- 1 Do not place customer, personal, production, billing-record, transaction, invoice, webhook payload, credential, secret, environment, or database-row data in KMS docs.
- 1 Do not commit or push unless explicitly requested.

TIS Change History

docs/CHANGE_HISTORY.md

2026-09-04 - Talent & Potential M4 Assessment Cycle And Frozen Population

- 1 Product Owner approved one SchoolGroup-wide Cycle authority with no

branch_id, explicit Draft-only population_effective_at, organization-only Open/Close, Branch-filtered identifiable population visibility, no reopen, no post-Open/late mutation, and deferred assessor assignment.
- 1 Added migration 20260904_004_talent_assessment_cycle_frozen_population

with Cycle, frozen population member, and bounded operational-audit tables. The migration explicitly establishes the scoped Academic Placement unique target needed by frozen-member composite foreign keys.
- 1 Added dynamic Draft preview from effective-dated Academic Placement and

eligible annual-config grades. Open atomically freezes exact Student, Placement, Academic Year, Branch, grade, section, and PlanningSection provenance and stores deterministic full population count/fingerprint.
- 1 Added Administrator-only-by-default

talent_assessment_cycles.view/manage/view_population/govern. Branch Draft authors cannot Open/Close; organization/global scope is additionally required. Branch population readers receive only authorized Branch members and subset counts, based on frozen Branch after Open, with no full integrity disclosure.
- 1 Added /api/talent/assessment-cycles, explicit lifecycle/population service,

append-only TalentAssessmentAudit, and focused security/lifecycle/migration tests. No Student Assessment, candidate instance, AI, entitlement, UI, M5+, deployment, or tis.db change was made.

2026-09-04 - M3 Governance Closure: KPI Approval, Rubric Ordering, And Audit Granularity

- 1 Product Owner approved weighted_level_average as one bounded, optional,

Framework-specific KPI calculation primitive (not a universal/cross-Program score); made the approval explicit in models.py (TalentKpiConfiguration docstring near its calculation_method CHECK) and talent_program_service.configure_kpi (governed closed-enum comment) and in this KMS. Qualitative no-KPI Programs remain first-class; the enum stays closed pending future governed approval of any additional primitive.
- 1 Defined rubric_level_at_or_above candidate-policy semantics as the

configured Rubric Level order (TalentRubricLevel.display_order, lowest proficiency first), independent of numeric_value, so qualitative Programs order and (in a later milestone) evaluate correctly with no numeric levels. Investigated and confirmed display_order safely and unambiguously carries both presentation order and semantic proficiency rank - it is dense, unique per rubric, and fully reindexed on every add/remove/reorder, so no field change was needed; documented this explicitly on the model.
- 1 Extended TalentConfigurationAudit.resource_type (model CHECK constraint

and, since the audit table was created by the separate already-applied M2 migration, a widening step inside the M3 migration 20260904_003_talent_rubric_kpi_candidate_policy_foundation using the repo's existing `_replace_postgres_check/_sqlite_rebuild_from_current_metadata` pattern) to add `rubric`, `rubric_level`, `rubric_descriptor`, `kpi_configuration`, `kpi_component`, `review_candidate_policy`, and `review_candidate_rule`. Every M3 mutation in `talent_program_service.py` now audits its specific child resource (post-flush id for new rows) instead of only `framework_version`, matching the existing M2 `framework_competency` precedent. `reorder_rubric_levels` (the one M3 bulk operation) appends one `rubric_level` audit row per reordered level.

- 1 Added/extended focused regression coverage in

`tests/test_talent_rubric_kpi_candidate_policy.py` for child-resource audit identity (`rubric`, `rubric_level` add/reorder, `rubric_descriptor`, `kpi_configuration`, `kpi_component`, `review_candidate_policy`, `review_candidate_rule`), the CHECK-constraint widening migration, and order-based qualitative rubric semantics. Re-verified the three previously-fixed defects (enabled-KPI numeric-scale guard, duplicate/contradictory candidate-rule rejection, clean `competency_in_use` rejection) remain intact. No M4, AI, or generic rule-engine/expression-language behavior was introduced; no commit, push, merge, deploy, or `tis.db` change was made.

2026-09-04 - Talent & Potential M3 Rubric, KPI, And Review Candidate Policy

- 1 Added migration 20260904_003_talent_rubric_kpi_candidate_policy_foundation

with Framework-versioned rubric, configurable ordered levels, exact competency/level descriptors, optional bounded KPI configuration/components, and deterministic Review Candidate Policy/rules. No data is seeded.

- 1 Added Draft-only, expected-revision-protected service and API mutations;

every change refreshes the complete M2+M3 semantic fingerprint and appends an M3 action through the existing `TalentConfigurationAudit`.

- 1 Kept qualitative Programs first-class: KPI and numeric level values are

optional. When enabled, KPI is limited to a validated weighted level average with basis-point weights totaling 10,000. Candidate policy is limited to ordered `all/any` rubric-threshold and optional KPI-threshold rules; it is configuration only and creates no candidate/result records.

- 1 Extended Framework cloning to deep-copy all M3 configuration while mapping

competency references through durable lineage. Added composite tenant/ Program/Framework integrity and exact membership checks.

- 1 Reused `talent_programs.view`, `.manage`, and `.govern`; organization/global

governance for activation/retirement is unchanged. Added focused M3 and M1/M2/permission/migration regression coverage. No M4+, UI, AI, entitlement, deployment, or `tis.db` change was made.

2026-09-04 - Checkpoint A Governance Closure: Student & Talent Program Permission Reconciliation

- 1 Documented the already-implemented Student & Academic Placement Foundation

(migration 20260904_001_student_academic_placement_foundation: Student as a canonical SchoolGroup-owned identity distinct from User, external identifiers, effective-dated Academic Placement with no

separate Enrollment entity, and append-only audit) and Talent Program & Framework Foundation (migration 20260904_002_talent_program_framework_foundation: durable organization-owned TalentProgram, annual configuration, immutable-once- active Framework Version lifecycle, competency lineage, and append-only audit) as implemented evidence in docs/AI_PROJECT_CONTEXT.md and docs/PROJECT_STATE.md. Neither foundation had prior KMS documentation.

- 1 Reconciled both foundations' temporary permission mapping against the

canonical permission_registry.py architecture. Added dedicated students (.view, .create, .edit, .activate_deactivate, .manage_identifiers, .manage_placements) and talent_programs (.view, .manage, .govern) permission groups, both Administrator-only by default (matching the existing users.* precedent), and updated routers/students.py and routers/talent_programs.py to check them.

- 1 Fixed a real permission-governance defect: Student Academic Placement

mutation and Talent Program/Framework Draft authorship were both gated only by the broad, commonly-granted planning.edit_section permission, so any Branch-scoped Editor/User actor holding it could manage Student placements or author organization-wide Talent configuration with no Student-specific or Branch-relationship constraint. Student creation/edit/identifier management had separately been gated by borrowed users.* keys instead of a dedicated Student permission domain.

- 1 Preserved every already-correct behavior: branch-scope-after-permission-

check ordering, non-enumerating 404-vs-403 discipline on every route, and the M2 organization/global access-scope gate (_organization_authorized()) on Framework activate/retire and Program lifecycle transitions, which was already correct and required no change.

- 1 Added permission-boundary regression tests to

tests/test_student_academic_foundation.py and tests/test_talent_program_framework_foundation.py confirming an Editor-role Branch actor - who still default-holds planning.edit_section

- 1 is now denied Student and Talent Program management.

- 1 No M3 (Rubric/KPI/Assessment/Review/Learner Profile/analytics/AI) code,

tis.db change, migration change, or unrelated route change was made.

2026-09-04 - Planning Subject Requirement Removal (Single And Bulk)

- 1 Corrected Planning section deletion after requirement removal: inactive

zero-period setup suppressions with no Curriculum Adjustment history are now cleaned atomically with an otherwise dependency-free section. This is not a general cascade; active requirements, permanent demand history, teacher assignments, timetable/calendar dependencies, and scheduling rules still block. The removable-demand error now says "Remove Subject Requirement".

- 1 Fixed a real gap where "Remove demand" only appeared for a Planning

requirement with an explicit, untouched PlanningSubjectDemand row - a purely legacy-fallback requirement (resolved only from the Subject catalog, no explicit row at all) had no removal action, so the same teacher's subjects could inconsistently show the action for reasons unrelated to the teacher.

- 1 Added _get_planning_requirement_removal_status, classifying every

requirement as removable, permanent (Curriculum-Adjustment-touched, now shown with an explicit "Protected" explanation instead of a hidden action), fallback, or not_found, reusing the same resolution authority already used to render the page.

- 1 A fallback requirement is now removable by creating an explicit

is_active=False/weekly_periods=0 suppression row, rather than leaving no removal path at all. Unlike a Curriculum Adjustment retirement, this row sets only created_by_user_id (audit trail) and leaves updated_by_user_id NULL, so it stays classified removable/setup-only instead of immediately becoming permanent - corrected same-day after an initial version stamped both columns like a Curriculum Adjustment retirement, which made a plain admin cleanup permanently undeletable and broke the remove requirement -> remove section -> remove subject workflow. See docs/history/workforce-planning/2026-09-04-planning-subject-requirement-removal.md for the full correction.

- 1 Added POST /planning/subject-requirements/remove handling both single

("Remove Subject Requirement") and checkbox-selected bulk ("Bulk Remove Subject Requirements", supported across every section on the page) removal atomically: every target is scope-validated and classified before any mutation, and any protected/invalid/out-of-scope target blocks the whole request with a named reason.

- 1 Confirmation dialogs precede both single and bulk removal. Teacher

assignments, timetable data, and Curriculum Adjustment audit records are never touched. The prior round's demand-id-only delete route remains in place, no longer linked from the UI, and is now the supported way to fully clear a leftover setup-only suppression row.

- 1 Completed the bulk-action UI: each expanded Planning section now has a

section-scoped **Select all** checkbox beside a trash-icon removal action. The label reflects the live selection (singular or exact plural count), partial selection produces the standard indeterminate state, and removal controls remain disabled until at least one requirement is selected.

2026-09-03 - Claude Code KMS Configuration

- 1 Added root CLAUDE.md as the concise Claude Code repository entry point.

- 1 Added .claude/skills/tis-kms/SKILL.md with the reusable TIS investigation,

implementation, KIA, synchronization, validation, and reporting workflow.

- 1 Added .claude/agents/tis-kms-developer.md for bounded delegated TIS work under

the same tenant, worktree, database, deployment, and no-commit safeguards.

- 1 Kept root AGENTS.md and the authoritative KMS as governing sources; the new

configuration adds no application, schema, migration, or deployment behavior.

2026-09-03 - Return-To Reopen-On-Load UI Pattern

- 1 Added redirect_utils.py, a shared safe_redirect_path() open-redirect

guard plus redirect_with_notice()/redirect_with_error(), moved out of main.py (which now imports them under their original private names, so every existing caller keeps working unchanged) so any router can reuse the same validation instead of reimplementing it.

1 Added static/js/reopen-on-load.js, loaded globally from base.html: on page load it resolves a target id from window.location.hash or an optional inline window.__tisReopenTargetId, opens it if it is a <details>, and scrolls it into view; a missing/stale id is a silent no-op.

1 Planning is the first consumer: each section's <details> now has a stable id, "Remove demand" passes return_to back to that section's fragment, and delete_planning_subject_demand/delete_planning_section's in-place blocked renders set the same reopen target - so an admin working inside an expanded section stays there after the action completes.

1 No SPA conversion, no new storage mechanism; Subjects and Teachers were not changed.

2026-09-03 - Planning And Subject Delete Dependency Guards

1 Added a read-only dependency check before Planning section and Subject deletion so every blocking category is named in a customer-safe message instead of raising an unhandled IntegrityError.

1 TeacherSectionAssignment is now a blocker like every other Planning section dependency instead of being silently auto-deleted; no documented product rule required that cascade.

1 Extended Subject delete/Bulk Delete to also check TimetableEntry, CurriculumAdjustmentAudit, and SubjectDistributionRule references, which previously had no application-level check and could be silently orphaned.

1 Bulk Subject delete is atomic: any blocked Subject in the selection stops the whole batch and names every blocked Subject with its reason.

1 Split Planning subject demand into removable and permanent using the existing updated_by_user_id column (set by Curriculum Adjustment, never by the one-time setup backfill) and added a new, narrowly scoped GET /planning/subject-demand/delete/{demand_id} action ("Remove demand" on the Planning page) that hard-deletes an untouched row so the section or subject can then be deleted; a touched row remains a permanent, honestly explained blocker.

1 Added no archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion.

2026-09-02 - Professional Timetable Lifecycle UX

1 Made current publication, editable Draft, validation/approval, and immutable history state explicit in the workspace and version history.

1 Added safe working-Draft creation from current published and historical versions; publication remains unchanged until explicit approval and publication.

- 1 Clarified empty Draft, Generate Draft, Regenerate Draft, and stale-input language.
- 1 Added canonical mutation/publication conflict evidence and visible refresh remediation while preserving legacy response messages.
- 1 Added no schema, migration, solver, or Workflow behavior.

2026-09-01 - Structured Timetable Conflict Evidence

- 1 Added a canonical timetable conflict contract and stable taxonomy across readiness, feasibility, generation terminal outcomes, and validation.
- 1 Added authorization-safe entity serialization and internal requirement/constraint correlation without exposing hashes or solver internals.
- 1 Preserved legacy blocker, diagnostic, validation-error, generation-message, status, and API envelope behavior through additive `conflicts` fields.
- 1 Kept infeasible, timeout, cancellation, stale authority, and execution failure distinct without introducing a conflict table or migration.

2026-09-01 - Timetable Lesson Requirement Projection

- 1 Added one deterministic, read-only projection from explicit-first Planning demand and existing teacher authority into timetable requirements.
- 1 Added internal requirement identities and source fingerprints to schema-v5 snapshots while retaining schema-v3/v4 problem-builder compatibility.
- 1 Routed readiness and snapshot generation through the shared projection and carried its provenance through problem construction and independent validation.
- 1 Added no table, migration, public CRUD API, Room, availability, Resource, or co-teaching behavior.

2026-09-01 - Timetable Solver Adapter Boundary

- 1 Added a solver-neutral worker contract for solve and infeasibility-diagnostic operations while retaining OR-Tools CP-SAT as the only Release 1 binding.
- 1 Kept problem construction, immutable snapshots, independent validation, stale-result protection, and atomic version persistence outside the adapter.
- 1 Added focused contract-forwarding tests; no schema, API, UI, or deployment topology changed.

2026-08-31 - Stale Draft Feasibility And Regeneration Workflow

- 1 Separated stale Draft lifecycle state from current-input structural readiness.
- 1 Enabled a fresh hard-only feasibility check for stale-only scopes and allowed regeneration only after the current authority fingerprint is verified.
- 1 Preserved stale Draft publication protection and stopped older-authority run failures from overriding the current workflow state.
- 1 Classified the current populated mutable unpublished Draft as the regeneration source regardless of manual/generated/regenerated origin, while retaining Generate for empty starters and excluding active or historical versions.

2026-08-30 - Partitioned Subject Sessions

- 1 Replaced overlapping occupied-pair inference with explicit double-block and single-session partitioning in CP-SAT.
- 1 Allowed separate sessions to touch while ensuring each lesson period belongs to exactly one session and physical timeline interruptions still break doubles.
- 1 Strengthened daily-coverage propagation through daily session/load channeling and aligned the independent validator with the same exact partition contract.

2026-08-29 - Reduce-Only Curriculum Adjustment

- 1 Added transfer and reduce-only adjustment action types to the guided preview/apply workflow.
- 1 Reduce-only requires no target subject or teacher decision and preserves all atomic, Draft, rule, lock, publication, permission, and tenant safeguards.
- 1 Subject Details now shows effective Planning weekly periods and links to a prefilled grade-specific reduction flow.
- 1 Replaced customer-facing catalog-default wording with **Original weekly periods**.

2026-08-29 - Curriculum Transfer and Subjects Display Correction

- 1 Made the administrator-entered transfer count authoritative across UI, preview fingerprint, and atomic apply.
- 1 Corrected partial transfer arithmetic and retained source demand unless its reviewed result is zero.
- 1 Updated Subjects to show exact-scope effective Current/New Planning periods, with **Varies** and section detail when demand differs.

2026-08-29 - Guided Curriculum Adjustment UI

- 1 Added a professional five-step administrator workflow for scope, change, preview, teacher decisions, final review, and success.
- 1 Added permission-gated Planning and Subjects entry points and customer-safe stale-review recovery.

- 1 Reused the Stage 3 preview and Stage 4 atomic apply authority; no solver, published-history, or automatic-regeneration behavior changed.

2026-08-29 - Atomic Curriculum Adjustment Apply

- 1 Added dedicated curriculum.adjust and a fingerprint-required apply endpoint.
- 1 Added exact-scope locking, active-generation rejection, qualification/capacity, scheduling-rule and Draft-lock validation, atomic demand/assignment/rule/Draft updates, and complete rollback on failure.
- 1 Added durable applied audits and duplicate-fingerprint protection through migration 20260829_001_curriculum_adjustment_apply_foundation.
- 1 Preserved snapshots, placements, published history, active pointer, and explicit later-regeneration behavior.

2026-08-29 - Read-Only Curriculum Adjustment Preview

- 1 Added exact-scope preview for grade, selected-section, and all-active-use subject demand transfers using PlanningSubjectDemand authority.
- 1 Added current/suggested teacher and capacity projections, Subject Scheduling Rule validation, grouped legacy warnings, Draft stale/regeneration impact, and a deterministic stale-confirmation fingerprint.
- 1 Added a planning.edit_section-protected JSON API without adding any apply, assignment, regeneration, publication, or database mutation behavior.

2026-08-28 - Planning Subject Demand Consumer Transition

- 1 Routed live Planning totals, teacher workload, timetable readiness/workspace/snapshot/generation input, Subject Scheduling Rule arithmetic, and required-hours reports through exact-scope explicit-first section demand resolution.
- 1 Made inactive and zero demand authoritative for one section without affecting other sections; missing rows retain legacy Subject weekly-hours fallback.
- 1 Preserved teacher assignment behavior, solver design, timetable UX, and immutable published timetable history.

2026-08-28 - Explicit Planning Subject Demand Foundation

- 1 Added PlanningSubjectDemand as an additive, normalized future authority for per-section subject weekly periods, including active/retired lifecycle metadata.
- 1 Added composite branch/year integrity and one-active-row-per-section-subject

uniqueness while allowing inactive history.

- 1 Added migration 20260828_004_planning_subject_demands_foundation with an

idempotent, branch/year-isolated Current/New Planning backfill from grade-matched Subject weekly hours that preserves any existing retirement evidence.

- 1 Added explicit-first demand resolution with legacy fallback. Existing Planning,

teacher allocation, reporting, timetable, draft, and publication behavior remains unchanged in this foundation stage.

2026-08-28 - Central Tenant Report Branding

- 1 Added `tenant_report_branding.py` as the reusable tenant-facing report/export

logo authority over existing branch/organization-scoped branding resolution.

- 1 Removed TIS product-logo fallback injection from timetable PDF/XLSX and

academic-calendar PDF exports; an unbranded tenant now receives a clean no-logo report header.

- 1 Routed observation PDF branding through the same authority without changing

report content, permissions, formats, or tenant isolation.

- 1 Kept TIS branding unchanged in the application UI and platform-owned surfaces.

2026-08-28 - Subject Scheduling Rules UI

- 1 Centralized operational timetable grade/section choices in

`planning_scope_service.py`, restricted to Current/New PlanningSection rows for one branch/year. Subject Scheduling, timetable workspace filters, and directly related calendar/grouped section selectors now share this authority.

- 1 Restricted the main and copy-rule grade selectors, plus their subject rows, to

distinct PlanningSection grades in the selected branch and academic year; global and Subject-catalog-only grades no longer appear in operational timetable configuration.

- 1 Corrected the modal visibility regression by removing author CSS that forced a

closed native `<dialog>` to render. The modal now opens only after Edit populates the selected subject and closes through Cancel/Close.

- 1 Polished the modal with aligned two-column double-block/single-session controls,

one full-width arithmetic summary, and a balanced two-column Conditions layout; advanced conditions, Reset To Default, and section overrides remain secondary.

- 1 Added a grade-first Subject Scheduling Rules section to Timetable Settings;

administrators select one grade and can search its compact subject list with read-only Planning totals, current patterns, and Configured/Default status.

- 1 Added a modal editor with double-block and single-session steppers, two-period

Separate Sessions / Consecutive Double Block presets, combinable primary conditions, progressively disclosed advanced options, live client-side arithmetic validation against the Planning weekly total, and authoritative backend re-validation on save (`subject_distribution_rules_ui.py`).

- 1 Added Section Overrides (create/edit/clear per section), Reset To Default

(removes only the grade-level row), and Copy Rules From Grade (subjects matched by name across grades, skipping arithmetic mismatches instead of copying them).

- 1 Added routes under `/system-configuration/timetable-settings/subject-rules`

reusing the existing `timetable.manage_settings` permission and remaining tenant/branch/year-scoped; Planning weekly totals are never mutated.

- 1 Preserved the existing subject-code mappings and raw Swimming/grouped-activity

JSON editor behind a secondary Advanced / Legacy disclosure, Non-Teaching Blocks management, and the unchanged Stage 2 snapshot/solver/validator pipeline; saved rules apply to the next created snapshot only.

2026-08-28 - Subject Distribution Rules Generation Wiring

- 1 Embedded the fully resolved Subject Distribution Rule (section over grade over

branch default, None for legacy fallback) into every Planning demand at snapshot creation time; later rule changes never alter an already-created snapshot, and fingerprint authority is preserved.

- 1 Carried the resolved rule and true physical period adjacency into the problem

builder, which now runs the arithmetic/feasibility validator as a final pre-solve defense and fails cleanly with `distribution_rule_invalid`.

- 1 Added generalized CP-SAT enforcement: exact non-overlapping intentional

blocks using true adjacency, generalized daily-coverage/spread/max-per-day/ min-teaching-days behavior (hard only when explicitly configured hard), and exemption of declared blocks from the consecutive-avoidance penalty.

- 1 Mirrored every new hard rule in the independent validator and added

readiness blockers for genuinely invalid or infeasible normalized configurations.

- 1 Preserved legacy `quality_rules_json` behavior exactly for every demand

without a resolved rule, preserved grouped-activity/teacher/section exclusivity, and preserved the unchanged regeneration diversity default.

2026-08-28 - Subject Distribution Rules Foundation

- 1 Added the normalized `subject_distribution_rules` table (migration

`20260828_003_subject_distribution_rules_foundation`) scoped branch default, grade, or section, alongside the existing `quality_rules_json` authority.

- 1 Added a pure hierarchy resolver (section over grade over branch default, with

field-level inheritance) and a pure arithmetic/feasibility validator confirming configured block/single distributions match the authoritative Planning weekly total.

- 1 Corrected the canonical composed timeline to mark true physical period

adjacency, false across a Break, Prayer, or other non-teaching item, so a later intentional-block feature can rely on genuine continuity.

- 1 No existing CP-SAT, independent validator, readiness, or settings-UI behavior

changed; every existing tenant continues on current `quality_rules_json` behavior until normalized rules are explicitly configured.

2026-08-28 - Added Smart Timetable Academic Scheduling Quality

- 1 Added branch/year settings and migration-backed JSON authority for core, spread,

ICT, double-period, grouped Swimming, shared-resource, and diversity rules.

- 1 Extended immutable snapshots, problem construction, CP-SAT, readiness diagnostics,

and independent validation with daily core coverage, distinct-day spreading, ICT daily limits, and simultaneous grouped activities.

- 1 Preserved the regeneration-diversity default, locks, teacher authority,

publication transactions, and Render Workflow architecture.

2026-08-28 - Primary Draft Workflow And Meaningful Regeneration

- 1 Grouped Regenerate, Approve Draft, and Publish Timetable in the primary Draft area

while keeping deletion, archive, comparison, and version exports in History.

- 1 Replaced the capped regeneration difference with 25 percent of unlocked source

placements, rounded up, while preserving locks and all existing hard constraints.

- 1 Added a controlled insufficient-diversity result instead of silently accepting an

almost-identical timetable.

2026-08-28 - Explicit Draft Approval Before Publication

- 1 Replaced customer-facing Check Timetable with Approve Draft and Draft Approved.

- 1 Added durable approval timestamp and administrator provenance through migration

`20260828_001_smart_timetable_stage52_draft_approval`.

- 1 Kept generated and regenerated drafts unapproved, invalidated approval after every

validity-affecting draft change, and required publication-time revalidation before the unchanged atomic active-pointer transition.

2026-08-28 - Present One Logical Draft Across Generation

- 1 Kept backend generated-result versioning and provenance while presenting the

generated result as the continuation of the selected Draft Timetable.

1 Bound current Draft actions to the selected exact-scope version, refreshed Check
and Publish state coherently, and added Create New submit feedback.

1 Hid obsolete generated-source drafts from normal History cards and separated the
bulk unpublished action visually.

2026-08-28 - Canonical Generation Freshness Authority

1 Excluded generation-only runtime metadata from freshness fingerprints while
retaining it in immutable solve snapshots.

1 Aligned generated-result authority with the generation snapshot contract and added
safe component-level mismatch diagnostics.

2026-08-28 - Simplified Draft Ownership And Unpublished Cleanup

1 Made the newest mutable non-active exact-scope version the current Draft Timetable,
including a fresh Create New Timetable draft, and bound Generate to that context.

1 Changed customer-facing draft deletion to permanent deletion for eligible
never-published versions while retaining History-only archive behavior.

1 Added exact-scope Delete All Unpublished Timetables with dependency-safe cleanup,
preserved snapshots and generation audit records, protected publication history, and visible partial-failure reporting.

1 Reconciled History deletion eligibility with the backend and replaced Create Draft
Copy with Use as New Draft for immutable historical versions.

2026-08-26 - Replaced Always-On Timetable Worker With On-Demand Workflow

1 Added a registered Render Workflow task that receives only a durable generation
run public ID and reuses the Stage 5.1 solve/validate/persist pipeline.

1 Added lightweight post-commit web dispatch, terminal handling for immediate
dispatch failure, delayed-start feedback, and exact-run PostgreSQL claiming.

1 Kept leases, heartbeats, cancellation, staleness, independent validation, atomic
persistence, and duplicate-result guards; solver mathematics and Stage 5.2 UX did not change.

1 Disabled Render automatic task retries and made the prior infinite polling worker
optional/local only, removing the always-running production solver requirement.

2026-08-26 - Implemented Smart Timetable Stage 5.2 Simplified Workflow

- 1 Simplified management around one Working Timetable and secondary Timetable History.
 - 1 Added permissioned Delete Working Timetable archive semantics that preserve history
- and never change the official active pointer.
- 1 Added active-pointer-only official resolution, teacher /my-timetable, a general
- published view, and view-only redirection away from drafts.
- 1 Added scoped visibility, deletion, wording, permission, and Stage 4/5.1 regressions.

2026-08-22 - Implemented Smart Timetable Stage 5.1 Generation

Added worker-isolated OR-Tools CP-SAT generation, schema-v3 immutable problem snapshots, an independent candidate validator, and durable PostgreSQL queue leasing, heartbeat, bounded recovery, cancellation, progress, and active-scope protection. Generate and Regenerate now create separate validated unpublished versions without changing published history; regeneration preserves locks and enforces an explicit minimum difference. Added `timetable.generate`, polling UI, additive migration `20260822_001_smart_timetable_stage51_generator`, and solver/lifecycle coverage.

2026-08-21 - Corrected Timetable Configuration With A Composed Day Timeline

Replaced fixed teaching-period clocks plus overlap classification with one per-day timeline composer. Blocks may be inserted after a period or preserved as fixed time when boundary-aligned; later periods and calculated end times shift automatically. Readiness, assignment validation, snapshots/fingerprints, timetable UI, and exports now share the projection. Added explicit additive placement metadata and expanded the controlled school block catalog. No automatic generation was introduced.

2026-08-21 - Added Smart Timetable Stage 4 Version Publication

Made timetable lifecycle visible through scoped history selection, immutable-version draft copies, draft lesson locks, explicit validation, same-scope comparison, selected-version exports, archive controls, and transactional publication. Publication revalidates authority and hard validity, checks edit/pointer revisions, supersedes the previous active version, and preserves history. No schema, solver, worker, generation endpoint, commit, push, deployment, or production action was included.

2026-08-19 - Added Smart Timetable Stage 3 Readiness And Slot Semantics

Added canonical fixed-period teaching/unavailable/invalid slots, server-side assignment protection, stale-placement preservation, snapshot integration, and an exact-scope read-only readiness service with customer-safe page status. No schema, solver, worker, Generate endpoint, commit, push, deployment, or production action was added.

2026-08-19 - Added Smart Timetable Stage 2 Version Foundation

Accepted ADR 0026 and added durable timetable versions, deterministic authority snapshots and SHA-256 fingerprints, one exact-scope active pointer, persisted placement locks, future generation-run metadata, and per-version section/teacher collision constraints. Existing populated timetables migrate without placement changes into imported compatibility versions; inconsistent rows remain present with safe version-level stale evidence, and settings-only scopes remain empty.

The current timetable page and exports now read one operational version. The existing assignment route uses copy-on-write so active history is immutable while the current editing experience continues through a mutable working draft. No solver, worker, generation route/UI, availability, room/resource, or rule model was introduced.

2026-08-19 - Restricted Compact Descriptions To Intentional UI Components

Changed the shared compact-description enhancer from broad tag and class-name inference to explicit `data-compact-description` opt-in plus the maintained allowlist of approved compact components. Ordinary operational status, helper, note, subtitle, and supporting text now remains visible inline. Existing keyboard/focus tooltip behavior and unrelated native accessible titles remain unchanged.

2026-08-19 - Capacity-Based Packaging And Common Customer Feature Baseline

Accepted ADR 0025 and replaced the temporary plan-feature ladder with one normal customer baseline for coherent active paid, promo, and customer-demo workspaces. Starter, Professional, and Enterprise AI retain their existing branch, staff, and teacher ceilings. Advanced Reporting and enabled AI are source-aware and plan-neutral while permissions, tenant/branch/year scope, commercial lifecycle, branch entitlement, and globally disabled features remain fail-closed.

Added idempotent migration `20260819_001_capacity_based_customer_feature_baseline` for plan values, compatible legacy flags, current active promo operational feature values, and active demo baseline policy. Exact promo capacity and immutable grant/redemption history are unchanged. Added a separate minimal AI consumption-policy boundary and retained reservation, counter, and operation-event accounting. Updated only affected customer copy; report generators and calculations did not change.

2026-08-18 - Enforced SchoolGroup Management Boundary

Direct operational SchoolGroup creation and deletion now require Platform identity, the matching create/delete permission, and `schools.manage_all_schools`. Tenant role defaults no longer include top-level create/delete, stale permission state cannot bypass the handler guard, and tenant updates are limited to the linked SchoolGroup. Unauthorized requests fail before validation, storage, or database mutation.

School Management now uses unified commercial authority for source-aware branch capacity presentation, including accurate promo used of allowed state. Existing locked branch enforcement and atomic promo assignment remain unchanged. The individual last-active-branch safeguard now counts within the target SchoolGroup. Added a PostgreSQL-only, repeatable-read, rollback-on-exit provenance audit that reports suspicious unlinked active internal sandboxes for manual review without repairing them.

2026-08-18 - Completed Promo Expiry Recovery And Paid Continuation

Promotional operational access now ends exactly at `PromoGrant.effective_to.grace_period_days` is a commercial recovery window for Organization Account and paid continuation only; it never extends operational access and does not archive or delete tenant data.

Verified owners of expired or recovery-period promo workspaces can use the existing tenant paid-activation flow. Checkout and pending payment leave promo authority unchanged. Verified provider completion atomically ends the promo entitlement, relinks the existing tenant source to the paid contract, establishes paid workspace and branch entitlements, and preserves tenant identity, data, branding, ownership, and immutable promo history. Active-promo early conversion remains blocked.

Added one authority-driven commercial badge component for Promo, Demo, and Paid sources in Organization Account and the full commercial view. The normal operational header intentionally remains focused on tenant context. Expired

promo access uses promo-specific continuation wording, and protected API responses return a customer-safe generic commercial code while internal reasons remain in server logs.

2026-08-18 - Added Promo-Aware Commercial Access Presentation

Organization Account Billing & Subscription now selects paid, demo, or promo presentation from the selected workspace's authoritative commercial source. Promo customers receive a dedicated read-only page for their grant plan, safe access status and period, masked reference, and centralized branch/system-user/teacher capacity. The page does not invoke Paddle or expose paid-only billing and mutation controls. Existing paid and demo journeys remain unchanged, and promo-to-paid conversion remains deferred.

2026-08-17 - Corrected Promo Branch Entitlement Lifecycle

Promo activation now creates explicit active or inactive entitlement evidence for every preserved branch, including operationally inactive unselected branches that previously fell outside the selectable-branch query. Individual and bulk branch reactivation continue through centralized capacity authority and now update branch status, promo assignment, and entitlement atomically. Capacity exhaustion and contradictory evidence fail before commit.

Added a generic PostgreSQL-only reconciliation command that defaults to dry-run, locks and revalidates on explicit apply, and creates only deterministically missing inactive branch entitlements. It never changes branch status, assignment, grant, capacity, Paddle, or people records. The Al-Andalus audit now distinguishes broad operational-user row inventory from M1 authoritative active staff usage.

2026-08-13 - Corrected Existing Workspace Plan Selection

Organization Account **Choose a Plan** now reopens eligible paid-plan selection for an existing workspace while its activation remains draft or checkout_ready. The saved plan is highlighted and can be replaced with a newly calculated authoritative quote. Browsing or replacing a draft creates no Paddle request or PaymentAttempt and does not mutate branches. Once checkout starts, payment is processing, or evidence requires manual review, replacement fails closed. Promo, PendingOrganization checkout, and verified-payment activation authority are unchanged.

2026-08-07 - Corrected Existing Workspace Payment Status Presentation

Organization Account now displays `Activation required` for a converted existing workspace that has no real unresolved Paddle checkout attempt. Payment processing is shown only for a current, unexpired `PaymentAttempt` in checkout-started or payment-processing state. Failed, cancelled, and expired attempts use recovery states, while completed paid and promo activation continue to show their active commercial state. No payment, Paddle, promo, capacity, or commercial-authority transition changed.

2026-08-06 - Implemented M4C Existing Workspace Paid Activation

Added a SchoolGroup-anchored paid activation workflow for verified owners of activation-required existing customer workspaces. The implementation adds additive schema, strict checkout/payment context constraints, operational capacity and branch quote snapshots, workspace billing/customer mapping, Paddle transaction launch, webhook-only atomic paid authority, Organization Account activation UX, centralized paid entitlement resolution for customer classification, and focused regression coverage. It does not create PendingOrganization or invoke tenant provisioning.

Final PostgreSQL validation hardened reused-transaction verification, persisted workspace-specific provider address/business lineage, refreshed ORM state after workspace row locks for concurrent duplicate webhooks, and proved that paid-versus-promo races create exactly one commercial source with no partial losing records.

Starter remains unavailable until complete restricted-branch enforcement is proven. No production data or real Paddle records were used during implementation. Live Paddle Sandbox checkout remains an environment validation gate.

2026-08-06 - Controlled Existing Workspace Owner Alignment And Conversion

Added a durable, idempotent conversion ledger and dry-run-first PostgreSQL CLI for an existing audited internal sandbox. The process prepares an ownership claim without creating an account, requires normal account verification, aligns or safely reuses one tenant owner, and collects only legal name, IANA timezone, and educational program. A database partial unique index now enforces one active tenant-owner account link per SchoolGroup after duplicate preflight; conversion events are append-only.

Final conversion locks and re-audits the operation and workspace, validates unchanged branch/dependency and sandbox-authority snapshots, retires the active internal entitlement, and sets customer / provisioning atomically. It does not mutate branches or operational records and creates no pending organization, contract, subscription, demo request, promo grant, active entitlement, or tenant link. M1 resolves the result as `activation_required`; M3 promo remains the supported activation path while existing-workspace Paddle activation is deferred. No production data was modified during implementation or validation.

2026-08-05 - Existing Workspace Conversion Read-Only Audit

Added a generic M4A audit service and parameterized PostgreSQL CLI for gathering sanitized evidence before a legacy internal-sandbox conversion is designed. The audit validates the exact workspace identity, resolves the intended owner, inventories tenant/provisioning and paid/demo/promo authority, reflects SchoolGroup-scoped tables, and traverses direct and indirect branch foreign-key dependencies. Unconstrained branch-like references and incomplete traversal fail closed for manual review.

The CLI uses a repeatable-read, read-only transaction, prints one JSON object, or deterministic text, records its PostgreSQL transaction mode, emits a stable SHA-256 snapshot hash, and always rolls back. Coherent, execution-failure, manual-review, and identity-mismatch results use exit codes 0, 1, 2, and 3. Soft-deleted dependencies remain blocking, reflected foreign keys absent from ORM metadata force manual review, and archival recommendations contain only proven dependency-free active branches. This milestone performs no branch archival, owner alignment, workspace or entitlement change, tenant-link creation, write-mode conversion, Paddle call, email, schema change, migration, or production audit.

2026-08-05 - Promo Redemption, Immutable Grants, And Activation

Added customer-side secure promo lookup, resumable capacity review, exact branch selection, and atomic activation for completed onboarding and separately aligned existing organizations. Migration

`20260805_002_promo_redemption_and_grants` adds activation sessions and branch selections, immutable redemptions and grants, grant branch assignments, and redacted redemption events. It also adds promo references to workspace entitlements and tenant links while enforcing exactly one paid, demo, or promo source.

M1 commercial authority, commercial access, branch queries, and future branch creation now resolve active promo grants and explicit branch entitlement. Organization-wide staff or teacher excess blocks activation without changing records; unselected branches remain operationally preserved but commercially inactive. Pending sessions provide no access, expiration fails closed, raw codes remain absent from storage and logs, internal sandboxes are not converted, and Paddle/payment behavior is unchanged.

2026-08-05 - Promo Code Foundation And Platform Console Management

Added secure definition-only Starter, Professional, and Enterprise AI promos with exact capacity bounded by the existing plan catalog, controlled targeting, validity and expiry policy, versioned lifecycle, duplicate/replacement support, optional branch restrictions, and durable redacted audit. Raw codes use at least 100 bits of secure randomness, are shown once in a no-store response, and are represented in storage only by dedicated-secret HMAC lookup authority and safe masked fragments.

Platform Console now exposes permission-controlled promo list, filter, create, detail, edit, activate, pause, revoke, duplicate, and replacement-definition workflows. Platform Developers may manage non-terminal definitions but only a Platform Owner may activate or revoke. Additive migration 20260805_001_promo_code_foundation creates three tables without backfill. No customer redemption, commercial entitlement, M1 authority, Paddle, onboarding, provisioning, payment, or tenant behavior changed.

2026-08-04 - Unified Commercial Access And Capacity Authority

Operational branch, staff-user, and teacher growth now uses one commercial authority facade that composes the existing workspace, entitlement, tenant link, contract, confirmed subscription, commercial-access, demo-lifecycle, and plan-capacity authorities. Missing or contradictory authority fails closed; Customer Demo and Internal Sandbox remain explicitly unmetered in M1.

Paid branch limits use confirmed provider-reconciled quantity capped by the plan branch ceiling. Staff usage counts every distinct active tenant operational user, including operational owners and teacher-position users; platform and account-only identities are excluded. Teacher usage deduplicates normalized known teacher IDs while counting every blank legacy row separately. The same person may consume both one staff and one teacher slot.

Capacity-increasing branch, user, teacher, academic-year, and provisioning paths lock the SchoolGroup, recount, evaluate, mutate, and commit in one transaction. Existing over-capacity tenants retain data and access but cannot increase an exceeded dimension. Paid and demo provisioning validate final authority before activation completes. No schema, migration, Paddle, pricing, webhook, onboarding-transition, permission, or feature-packaging change was made.

2026-08-02 - Billing Contact Editing And Paddle Retry

Subscription Management now presents Billing Contact as read-only organization identity with an explicit Edit, Save Changes, and Cancel interaction. The legal identity label now covers an organization or school name, and the billing country uses the supported country selector.

Valid local billing changes remain committed when Paddle synchronization fails. A dedicated permission- and tenant-scoped retry reuses the saved profile and persisted customer, address, and business mappings, avoids duplicate provider objects, and performs no provider call once synchronized. Customer status copy distinguishes local save, provider synchronization, and retry failure. Safe server diagnostics identify the customer, address, business, or subscription identity step plus provider status/error code. A saved pending or failed billing identity prevents new plan or quantity changes until provider synchronization succeeds; cancellation and legacy active subscriptions without a profile remain unchanged.

2026-08-01 - Explicit Organization Billing Identity

Billing & Subscription now stores one explicit organization billing profile covering billing email, legal/billing name, contact, optional company and tax identifiers, and the supported address structure. This authority is independent from SaaS login email. Initial checkout confirms it, while active-tenant changes require organization ownership or `subscriptions.manage_billing` and revalidate tenant/provider linkage.

TIS reuses the mapped Paddle customer, synchronizes authorized email/name changes, creates or reuses one attributable active Paddle Business, persists the Paddle address/business mappings, updates the active subscription identity, and includes `business_id` on future initial checkout transactions. Ambiguous mapping and provider failures fail closed. Existing financial documents are not silently revised.

Provider transaction presentation now distinguishes paid as Payment received

1 processing from completed as Paid. Subscription change requests record the

payment-received timestamp separately, show that no customer action is required while provider processing finishes, and retain the existing completed reconciliation authority. Additive migration 20260801_001_organization_billing_identity creates organization_billing_profiles, adds Paddle address/business mapping columns, and adds the separate payment-received timestamp.

2026-08-01 - Subscription Capacity Presentation And Billing Navigation

Subscription pages now separate confirmed paid branch quantity from the current plan's maximum branch ceiling. Review Capacity identifies additional required branches as additional billed quantity and retains system-user and teacher counts as non-billed eligibility limits. Unapproved feature claims and empty feature placeholders were removed from customer plan comparison.

System Configuration now exposes Billing & Subscription only to a linked organization owner or user with explicit billing authority. The route revalidates operational tenant scope and the SaaS account-user relationship, then opens the existing Organization Account subscription page. Password and social authentication preserve only the allowlisted internal billing return; external or unrelated destinations fail back to Organization Account. The public landing label is Organization Sign In and still uses centralized SaaS authentication. No pricing, Paddle quantity, lifecycle, webhook, entitlement, schema, or migration behavior changed.

2026-08-01 - Unified Active Subscription Capacity Management

Subscription Management previously presented branch quantity as an isolated commercial action. One authoritative organization-capacity resolver now counts active branches, active tenant operational staff users, and active teachers. The portal exposes one Review Capacity flow and displays usage, plan limits, and remaining capacity for all three dimensions. The highest required dimension selects the minimum eligible plan: Starter supports 1 branch/5 system users/25 teachers, Professional supports 5/20/100, Enterprise AI supports 25/100/500, and Custom is required when any Enterprise AI limit is exceeded. Customers may still select a higher eligible plan for optional feature entitlements. Branch, system-user, teacher, or mixed growth can trigger an upgrade. Required branch growth can combine target plan and branch quantity in one provider preview. Paddle quantity remains active-branch count only; system-user and teacher capacity affect plan eligibility but never become billed quantity units.

Plan-change previews retain their three-dimension capacity evidence. All three dimensions are validated before a downgrade is submitted, and downgrades remain scheduled for the next billing boundary. The same three dimensions are revalidated when provider evidence reaches the effective date. Capacity growth that no longer fits enters manual review, does not activate the lower plan, and preserves the current safe entitlement state. Existing provider-authoritative proration remains unchanged. Higher entitlements require both provider subscription and payment confirmation. Cancellation remains scheduled at paid-period end, preserves access and tenant data through the confirmed date, and remains reversible when supported by Paddle. No schema or migration change was required.

2026-08-01 - Organization Account Sign-In Routing

Public and onboarding SaaS sign-in previously sent an activated, commercially allowed organization customer directly to operational /login. Authentication, already-authenticated restoration, and social sign-in now use one customer journey decision that lands authorized organization account managers on /saas/account. The Organization Account Overview presents only permitted organization, branch, billing, and security sections, and Enter TIS Platform is the sole explicit operational entry action.

Incomplete onboarding and pending demo review still resume their authoritative steps. Restricted or suspended account managers remain in account billing and recovery context; operational users without account-management permission keep their role-based destination. Multiple managed organizations require selection before an overview is rendered. The HTTP-only selected-organization hint is revalidated against current account links and permissions before the existing entitlement resolver accepts its tenant scope. Existing commercial access, permissions, subscription authority, operational authentication, schema, and tenant isolation remain unchanged.

2026-07-31 - Contract-Linked Commercial Access Consistency

Operational access previously selected the newest PaymentSubscription for the onboarding organization, while Subscription Management and entitlements used the tenant link and authoritative SubscriptionContract. A pending plan upgrade or newer stale subscription row could therefore block an otherwise active paid workspace and every blocked paid state was presented as expired.

Operational login, protected requests, returning-customer routing, and access pages now consume one contract-linked commercial access projection over the existing entitlement resolver. The current active/trialing plan remains authoritative while a plan change is pending, failed, incomplete, canceled, or abandoned. Canceled subscriptions remain entitled only through the confirmed paid period end. Restricted states use distinct payment-processing, past-due, paused, expired, suspended, archived, and verification-required guidance.

The specialized plan-change subscription webhook now applies the same provider status normalization as the general and quantity-change paths. It does not complete or activate the target plan without the existing two provider signals. No schema, migration, pricing, quantity, Paddle request, webhook authority, or tenant data changed.

2026-07-30 - Initial Secure Payment Retry And Lineage Hardening

The Secure Payment summary could be correct while launch rejected an otherwise eligible unpaid organization whose persisted billing status or checkout lineage was stale. Plan and interval changes marked the checkout session stale but did not consistently supersede its unfinished payment attempt. Retry could therefore repeat a local readiness failure or retain obsolete transaction authority.

The incident itself occurred before any Paddle API request: `_ensure_checkout_launchable()` rejected a legacy `ready_for_checkout` pre-checkout billing state. That state can now be prepared safely. The Professional Annual mapping was unchanged at USD 790 per active branch, so the verified two-branch quote remains quantity 2 and USD 1,580 annually.

Checkout recovery now treats plan selection, checkout session, payment attempt, quote fingerprint, provider price, interval, and quantity as one lineage. Obsolete sessions and unfinished attempts are superseded, current local authority is cleared, and Retry prepares a fresh session from the authoritative quote. Existing started transactions are reused only after remote billed, automatic-collection, customer, quote, item, quantity, and subtotal validation. Superseded webhooks cannot activate a subscription or workspace.

Paddle address resolution now reuses an active exact-country address for the same customer before creating one. Provider and readiness failures retain status/code and traceback in server logs while the customer sees one safe retry alert. No pricing, capacity, schema, migration, webhook authority, provisioning, or checkout architecture changed.

2026-07-29 - Organization Profile Save And Account Setup Correction

The Organization Profile POST accepted its program values correctly but used MIME/extension-only logo checks and wrote directly to the application filesystem. Its route caught only `ValueError`; an `OSError` while creating or writing `static/uploads/saas/pending_logos` escaped as raw HTTP 500. Corrupt and oversized files could also be accepted. The fresh Account Setup page repeated the same next-step guidance across the status, side action, content, and help panels.

Pending logos now reuse actual-image decoding, the established 4 MB and minimum-dimension checks, UUID filenames, pending-directory confinement, and atomic temporary-file promotion. Empty uploads retain the current reference. Rollback removes a newly written file; successful replacement removes the old file after commit. Validation, storage, and unexpected failures all render a safe page response with preserved entered values, while technical failures receive traceback logging.

The initial account state now shows a smaller official logo, one "Start Your School Workspace Setup" title and sentence, the existing eight-step progress track, and one POST start action. Later setup-console states remain unchanged. No schema, migration, billing, payment, provisioning lifecycle, operational-module, or landing-site behavior changed.

Provisioning's existing logo-copy step was hardened without changing activation authority.

The current storage location remains application-local `static/uploads/saas/pending_logos`. Render durability requires a separately approved persistent-disk or object-storage decision.

Organization branding now appears in the Organization Profile preview and shared School Workspace setup identity without replacing the official TIS logo. Existing paid/demo provisioning continues to promote the pending image into the primary `SchoolGroupLogo` slot; source resolution is now confined to the pending directory, a missing referenced file blocks activation rather than silently losing branding, and the final accessible label includes the organization name. The existing protected organization-asset route and operational shell remain the final display path.

Branch Setup previously asked Jinja to sum `estimated_system_users` and `estimated_teachers` directly. Placeholder and incomplete rows supplied undefined or None, producing `TypeError: unsupported operand type(s) for +: 'int' and 'NoneType'`. Totals now come from a Python helper that normalizes display-only missing values to zero. Required server validation remains active for customer saves, and newly created pending branches receive explicit zero defaults.

2026-07-29 - Post-Verification Sign-In 405 Correction

A fresh verified account had no pending organization or operational account link, so `customer_journey_service.login_destination()` returned `/saas/onboarding/start`. That route accepts POST only. After successful credential POST, the 302 caused the browser to request that destination by GET, and FastAPI returned raw `{"detail": "Method Not Allowed"}` with HTTP 405. Existing tests supplied `/saas/account` explicitly and therefore masked the normal browser journey.

Fresh verified accounts now continue to the GET `/saas/account` dashboard; the existing start-setup action remains the only POST that creates an organization. Login continuation validation accepts only known customer GET destinations and rejects POST-only, malformed, traversal, fragment, and external values. A compatibility GET for `/saas/auth/login` redirects to the normal GET sign-in page. Verification, password authentication, onboarding rules, subscription intent, preferred-plan authority, checkout, Paddle, schema, and landing behavior are unchanged.

2026-07-29 - Safe Public Subscription Pricing Entry

The landing hero Subscribe Now action now scrolls to pricing. Starter, Professional, and Enterprise AI use identical CTA styling and enter the public TIS Account signup route with distinct allowlisted preferred-plan codes. Pricing cards no longer show the small description beneath each plan name. Custom remains contact-only.

The signup GET route previously referenced an undefined `next_path` local, causing every unauthenticated subscription pricing link to return HTTP 500 before account or onboarding state was evaluated. The route now accepts and safely normalizes that optional value. A self-service plan preference is preserved through registration in a secure cookie, creates no plan selection or payment record, and is applied only when the existing organization-wide branch, system-user, and teacher checks confirm eligibility. Invalid, inactive, or undersized preferences are cleared. No price, plan limit, Paddle quantity, payment authority, or AI entitlement changed.

2026-07-29 - Per-Branch System-User And Teacher Subscription Capacity

Branch Setup now stores required non-negative estimates for system users and teachers on every active branch and presents live organization-wide totals. Migration `20260729_001_subscription_capacity_dimensions` preserves legacy organization estimates by assigning them to the primary active branch only when no branch estimates exist. Derived organization totals remain compatibility summaries rather than competing authority.

Eligibility checks branches, non-teacher system users, and teacher records independently. Starter supports 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Before payment, each people dimension uses the greater of branch estimates and actual same-workspace active data; after activation actual data is authoritative. The minimum eligible plan is recommended, higher eligible plans remain available, and exceeding any Enterprise limit selects the

contact-only Custom path.

All quote and checkout fingerprints include both people counts. Capacity changes supersede stale checkout lineage and clear only an undersized plan. Paid non-teacher user creation/reactivation, teacher creation and year-copy preflight, and plan downgrades fail before mutation when capacity would be exceeded.

2026-07-28 - Authoritative Subscription Plan Branch Capacity

Self-service plan eligibility now uses each active plan's persisted `max_branches`: Starter supports one active billable branch, Professional five, and Enterprise AI twenty-five. Pricing remains the selected plan's per-branch price multiplied by the actual authoritative active count.

Plan selection, quote construction, checkout preparation and launch, and payment validation fail closed when capacity is exceeded. Pre-payment branch expansion clears an undersized selection and supersedes its quote and checkout lineage; an eligible higher plan remains selected. Organizations above the Enterprise AI limit receive a customer-safe custom-plan contact state.

2026-07-28 - Pre-Payment Branch Editing And Checkout Supersession

Branch Setup previously rejected any organization with a `TenantProvisioningLink`, incorrectly treating an unpaid demo or prepared workspace as paid provisioning. Editing now closes only on authoritative confirmed-payment or active-paid-subscription evidence. Before that boundary, customers may add, edit, remove, reorder, and reprioritize branches.

Changing branch count or identity states the prepared checkout, supersedes its incomplete payment attempt, clears quote snapshots, and recalculates the next checkout from the final active count. Late provider events from a superseded transaction are recorded for manual review without activating or converting the workspace or disrupting the replacement checkout.

2026-07-28 - Fixed Paddle Checkout Branch Quantity

Paddle checkout exposed quantity controls even when the server-created transaction contained the TIS billable branch count; inline presentation alone does not make transaction items immutable. TIS now supplies the resolved Paddle customer address, requires an automatically collected transaction to reach ready state, validates its quote evidence, and marks it billed before releasing its transaction ID to Paddle.js.

Billed transaction items cannot be changed. The same catalog price remains usable with organization-specific quantities; no price mutation or quantity-specific price is created. Automatically collected payment completion remains the recurring-subscription, webhook, and demo-conversion authority. The public payment launcher now explicitly treats billed as the only valid remote launch state and verifies the local attempt, checkout, customer, and quote context before supplying `transactionId` to Paddle.js. Invalid states fail closed without exposing provider diagnostics.

2026-07-28 - Expired-Demo Checkout Identity Resolution

Expired-demo Paddle checkout could find one active customer by email but reject it when provider custom data retained a deleted SaaS account context and the existing operational tenant owner was treated as an unrelated identity. The checkout guard now accepts or repairs a stale SaaS account-user relationship only when the authenticated account, owned organization, demo source, tenant-provisioning link, SchoolGroup, and sole active operational owner form one coherent relationship. Active previous accounts, unrelated tenants, multiple identities, and live-mode email-only recovery remain blocked.

Customer responses no longer expose identity match counts or internal reason codes, and a failed preparation displays retry/support guidance rather than a ready-payment state. No workspace, branch, tenant, payment authority, pricing, or provider-environment rule changed.

2026-07-28 - Returning Customer Journey And Expired Access

Expired demos previously entered the paid subscription portal, whose model requires paid entitlement evidence, so normal demo values rendered unavailable and no conversion action existed. Some returning sessions could also reach workspace setup before expired paid state was intercepted.

TIS now builds demo subscription choice from the existing organization, workspace, active operational branches, public plans, and intervals, then hands selection to existing Paddle checkout and same-workspace conversion. SaaS and operational login route from authoritative state; a shared commercial guard blocks expected demo or paid expiry before protected page work. Customer communications use TIS-team terminology. Landing navigation adds Sign In and Open TIS App. All M8B9 owner controls remain available under progressive disclosure. No schema, migration, pricing, payment-confirmation, AI-entitlement, workspace replacement, commit, or push change was made.

2026-07-27 - M8B9 Demo Operations, Notifications, And Testing

Added owner-only immediate expiry, same-workspace reactivation, unbounded future custom expiry, rotating final-day reminders, and manual single/global lifecycle execution through existing production rules. Added Standard, Full, and Custom controlled access policies with workspace defaults and isolated branch overrides, durable audits, and customer communications without usage resets, classification changes, or customer-specific rules.

2026-07-27 - Pre-Deploy Database Migration Boundary

Replaced the temporary Render daemon migration worker and HTTP readiness gate with a strict deployment boundary. The FastAPI web process no longer creates tables or runs migrations during import, startup, middleware, or background execution. `python scripts/run_migrations.py` now owns baseline metadata creation and the authoritative ordered migration ledger as Render's required Pre-Deploy Command. A failed migration exits nonzero and prevents activation of the new web version; a successful or already-current run exits zero and logs the applied migration identifiers.

The migration process now configures PostgreSQL `connect_timeout=10s`, `lock_timeout=5s`, and `statement_timeout=30s` at connection creation, before `metadata.create_all()` or any migration-ledger DDL. Flushed progress records bracket connection, metadata, ledger, migration apply, marker, and commit phases. Lock or statement timeout failures therefore identify their boundary and exit nonzero instead of waiting for the deployment timeout. Transactional migration helpers now inspect tables, columns, indexes, and constraints through the active migration connection. They never check out a second engine connection while the first connection owns uncommitted DDL locks, preventing the M8B7 `system_notifications` inspection self-deadlock.

2026-07-27 - M8B8 AI Entitlements And Commercial Foundation

Added one authoritative AI feature registry and entitlement service. Decisions compose tenant scope, role permission, commercial state, feature policy, plan entitlement, and usage without replacing any existing authority. Customer Demo gets two successful uses per feature; internal, demo, and paid usage is persisted separately through locked successful/reserved counters and idempotent operation events. Enterprise AI is the only temporarily mapped paid tier. No AI tool, pricing change, notification expansion, or M8B9 operational control was added.

2026-07-27 - Render Port-Bind Startup Boundary

Render previously executed SQLAlchemy table creation and every pending migration while importing `main:app`. The initial mitigation moved that work to a daemon worker behind an HTTP readiness gate. That temporary boundary has been superseded by the Pre-Deploy Database Migration Boundary above.

2026-07-27 - M8B7 Demo Customer Journey

Platform Owner approval now orchestrates the existing independently retryable provisioning service. Durable branded email intents cover request receipt, activation approval, decline, Day 6, expiry, and same-workspace subscription continuation. Demo events reuse the Platform Owner Notification Center, and active tenant workspaces show a responsive demo indicator through the shared shell. Coherent expired demos may convert after authoritative confirmed payment by reactivating and converting the same SchoolGroup and tenant relationship. M8B8, M8B9, pricing redesign, second-workspace provisioning, and sandbox conversion remain out of scope.

This file is the chronological summary of meaningful TIS changes. It does not replace module history under docs/history/; it gives reviewers, developers, Codex, and ChatGPT a fast timeline of what changed and why.

Newest entries should be added first.

2026-07-26 - Platform Owner Historical Demo Eligibility Maintenance

Area/module: Platform Owner SaaS Admin and Customer Demo domain eligibility

Previous state: The organization-scoped clean-room reset safely removed current test data, but a historical detached eligibility reservation could no longer be reached after its organization and account had already been deleted.

New state: Platform Owners have a separate maintenance page that scans demo-domain eligibility reservations and shows exact blockers. A row is removable only when no matching organization, account, demo request, tenant-profile workspace, provisioning record, subscription evidence, conversion, or manual-review evidence exists. Confirmed deletion rechecks and locks one exact ID, deletes only that ID, flushes, verifies absence, commits atomically, and writes a durable owner audit event.

Reason: Repair verified historical orphaned reservations without weakening production one-demo-per-domain enforcement or changing the existing clean-room reset.

Files changed: New demo eligibility maintenance service and templates, SaaS Admin routes/navigation, focused Phase 5 tests, KMS source documents, and regenerated KMS artifacts.

Documentation updated: AI/master context, project state, change history, SaaS onboarding history, module map, system flows, and database architecture overview.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, migration, demo-request, clean-room reset, customer workflow, commit, or push change.

2026-07-26 - Verified Safe Demo-Domain Reservation Deletion

Area/module: Platform Owner test workspace/account reset and Customer Demo domain eligibility

Previous state: Reset analysis identified safe detached eligibility rows, but deletion depended on a second ORM lookup and reconstructed ID list and did not immediately verify removal before continuing with parent deletion.

New state: Analysis publishes an explicit safe detached-reservation ID list. The owner-only deletion transaction consumes only that list, deletes by primary key before demo requests and parent records, flushes immediately, and verifies that no selected row remains. Structured logs record received IDs, query scope, affected rows, verification count, and the existing transaction commit or rollback.

Reason: Make clean-room reset completion provable and fail closed while preserving conflict/manual-review protections and the production one-demo-per-domain policy.

Files changed: Workspace analysis/deletion services, focused Phase 5 and domain diagnostics tests, read-only diagnostic tooling, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, unique-constraint, production eligibility, customer-message, commit, or push change.

2026-07-26 - Safe Orphaned Demo-Domain Reservation Cleanup

Area/module: Platform Owner test workspace/account reset and Customer Demo domain eligibility

Previous state: The clean-room reset removed eligibility rows only while they still referenced a selected demo request. A prior request deletion could leave a detached same-domain row through `ON DELETE SET NULL`, and that row continued to block a later internal demo request.

New state: The owner-only reset resolves the selected organization's domain through the same Customer Demo domain resolver, counts linked and detached reservations, and checks for another organization, request, workspace, or customer account using that domain. It removes a detached reservation only when that conflict analysis is empty and the row has no historical manual-review evidence; otherwise, reset preflight blocks for manual review and preserves all data. Owner analysis views show the domain-cleanup result.

Reason: Allow repeatable internal M8 testing without broadly deleting domain reservations or weakening the production one-demo-per-domain policy.

Files changed: Workspace analysis/deletion services, owner reset-analysis templates, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: No schema, foreign-key, cascade, customer demo-policy, payment, Paddle, commit, or push change.

2026-07-26 - Internal Test Workspace Commercial Clean-Room Reset

Area/module: Platform Owner test workspace/account reset, M8 demo-commercial history, and customer-facing demo wording

Previous state: The guarded reset removed the selected workspace/account data but retained the selected organization's demo request and normalized-domain reservation. A later internal registration for the same organization domain was therefore blocked by the production one-demo policy.

New state: Within the existing owner-only reset transaction, TIS deletes only the selected pending organization's linked demo request, domain reservation, review/event history, demo provisioning/lifecycle records, and demo-to-paid conversion history before their parent records. The same internal test email and organization domain can complete a new M8 journey. Customer-facing demo review language now refers to the TIS team.

Reason: Support repeatable internal end-to-end M8 testing without weakening the production Customer Demo one-domain restriction.

Files changed: Workspace analysis/deletion services, customer demo wording, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Owner-only test reset exception. No schema, foreign-key, cascade, payment, Paddle, normal customer demo-policy, lifecycle, commit, or push change.

2026-07-26 - Test Workspace Reset Entitlement Dependency Fix

Area/module: Platform Owner test workspace/account reset, workspace entitlements, and scoped cleanup dependencies

Previous state: The controlled reset reached final SchoolGroup deletion while a selected workspace entitlement still referenced that SchoolGroup. Entitlement child values and branch-entitlement rows also required explicit dependency review.

New state: The reset removes selected `workspace_entitlement_values` and `branch_entitlements`, then the selected `workspace_entitlements` rows, before the final SchoolGroup deletion. Every query is scoped to the selected SchoolGroup or its selected entitlement IDs. Global entitlement definitions, subscription plans, prices, and unrelated workspaces remain unchanged.

Reason: Preserve entitlement foreign-key integrity while retaining the existing Platform Owner-only test reset guards, transaction rollback, and customer behavior.

Files changed: Workspace analysis/deletion services, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Minimal scoped dependency fix only. No foreign-key, cascade, entitlement-rule, schema, lifecycle, customer-facing, commit, or push change.

2026-07-26 - Test Workspace Reset Subscription-Change Dependency Fix

Area/module: Platform Owner test workspace/account reset, subscription-change records, and workspace-deletion diagnostics

Previous state: The controlled test workspace reset deleted scoped operational users before deleting scoped subscription-change requests. A request referencing a workspace user could therefore cause the database to reject the user deletion and roll the transaction back.

New state: The reset deletes `subscription_change_requests` for the selected `school_group_id` before any selected workspace user is deleted. Pre-analysis now counts the same scoped records, existing structured diagnostics retain their model/table/row-count events, and all deletion work remains inside the existing transaction.

Reason: Preserve foreign-key integrity while allowing Platform Owners to reset only the selected internal test workspace/account and retain all other workspaces unchanged.

Files changed: Workspace analysis/deletion services, focused Phase 5 regression tests, and KMS sources.

Documentation updated: AI/master context, project state, change history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Minimal dependency-order fix only. No foreign-key, cascade, schema, lifecycle, customer-facing, commit, or push change.

2026-07-25 - Landing CTA Consolidation And Customer Demo Domain Policy

Area/module: Public landing conversion routes, TIS Account signup/onboarding, customer-demo request policy, and Platform Owner demo visibility

Previous state: The landing website exposed overlapping conversion labels. Signup did not retain a selected commercial path, and demo duplicate protection applied only to a pending request for one onboarding record rather than an organization domain.

New state: The landing website exposes Request a Demo and Subscribe Now through environment-configured app URLs with explicit demo or subscribe intent. The selected intent persists through signup and School Workspace Setup, then is emphasized at commercial choice without preventing a change. Customer Demo eligibility now uses a normalized organization-domain reservation with a database unique invariant. Historical duplicate domains are retained as manual-review reservations; no historical request, workspace, or customer data is merged, deleted, migrated, or reprovisioned.

Reason: Schools should have two unambiguous public conversion paths, while each organization receives at most one Customer Demo opportunity and retains the same workspace for Demo-to-Paid.

Files changed: Landing CTA source, SaaS models/services/router/templates, database migration, diagnostic command, focused tests, and KMS sources.

Documentation updated: AI/master context, project state, user/system flows, database architecture, change history, landing history, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: Approved M8 scope only. No M9 work, payment change, workspace conversion change, provisioning redesign, commit, or push.

Entry Template

YYYY-MM-DD - Short Change Title

Area/module:
Previous state:
New state:
Reason:
Files changed:
Documentation updated:
PDF regenerated:
AI project context updated:
Reviewer/approval notes:

2026-07-25 - M8 Landing Integration Open Account Entry Points

Area/module: Next.js public landing website, public-to-account routing, and deployment configuration

Previous state: The public landing website explained TIS and supported demo/early-access conversion, but it did not expose a final M8 Open Account entry point into the deployed TIS Account signup journey.

New state: The Next.js landing website now has Open Account entry points in the navigation, hero CTA area, and final CTA area. All Open Account links use one shared URL derived from `NEXT_PUBLIC_TIS_APP_BASE_URL` and the existing `/saas/signup` account registration path.

Reason: M8A and M8B-1 through M8B-6 are already completed in the SaaS application. The final M8 integration step is to let visitors on `tisplatform.com` enter the deployed customer account setup flow without coupling the landing project to the FastAPI app.

Files changed:

1 `tis-landing-website/src/app/page.tsx`

```
1    tis-landing-website/.env.example
1    tis-landing-website/README.md
1    .kms-impact.yml
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/CHANGE_HISTORY.md
1    docs/history/landing-page/README.md
1    docs/history/landing-page/2026-07-25-m8-landing-integration-open-account.md
```

Documentation updated: AI/master context, project state, change history, and landing-page history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8 landing integration only. No FastAPI SaaS application, authentication, onboarding backend, Paddle, database, API, demo workflow, provisioning, commercial state, commit, push, or M9 work.

2026-07-23 - M8B-6 Demo-To-Paid Workspace Conversion

Area/module: Customer-demo checkout, provider-confirmed subscription reconciliation, workspace classification, commercial entitlements, demo lifecycle, customer status, and Platform Owner inspection

Previous state: An activated Customer Demo could not become Customer Paid without risking a second provisioning path or leaving paid commercial evidence disconnected from the existing operational workspace.

New state: An eligible active Customer Demo can use the existing M7 subscription checkout. After provider-confirmed payment, TIS atomically converts the same SchoolGroup and tenant link to Customer Paid, replaces the demo entitlement with the confirmed subscription-backed paid entitlement, preserves all operational data and history, and exits demo lifecycle processing. Durable conversion state and events support idempotency, audit, and retry after failure.

Reason: Customers must retain one workspace, tenant identity, organization, branches, users, permissions, and academic history when moving from evaluation to a paid subscription.

Files changed: Conversion enums/models/migration/service, classification transition validation, demo checkout and lifecycle integration, payment-webhook reconciliation, customer/owner UI, tests, ADR, and KMS.

Documentation updated: AI/master context, project state, database architecture, module map, flows, roadmap, ADR index/0013, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-6 only. No tenant reprovisioning, Paddle API redesign, pricing change, demo extension, manual conversion override, internal-sandbox conversion, archive/delete, membership, ownership transfer, commit, or push.

2026-07-23 - M8B-5 Standard Customer Demo Lifecycle

Area/module: Customer-demo lifecycle metadata, resolver, scheduled processing, internal notifications, expiration, operational access enforcement, and owner/customer UI

Previous state: Activated customer-demo workspaces had no standard duration, reminder, expiration transition, or request-time access gate.

New state: Customer demos last exactly seven days from activation. Day 6 creates idempotent internal customer and Platform Owner notifications. Day 7 processing atomically ends the demo entitlement and suspends the workspace while preserving tenant data. Operational middleware blocks expired or ambiguous demos across web, API, download, and existing-session requests.

Reason: Customer demos require a predictable, provider-independent lifecycle that cannot be extended by scheduler delay or stale sessions and never destroys evaluation data.

Files changed: Demo lifecycle enums/models/migration/service/command, M8B-4 activation metadata, authorization middleware, operational and SaaS UI, lifecycle tests, ADR, and KMS.

Documentation updated: AI/master context, project state, database architecture, module map, flows, roadmap, ADR index/0012, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-5 only. No demo-to-paid conversion, Paddle change, email, manual extension, archive/delete, read-only expired mode, special migration, membership, ownership transfer, commit, or push.

2026-07-23 - M8B-4 Demo Workspace Provisioning And Activation

Area/module: Approved SaaS demo requests, operational workspace provisioning, commercial entitlement activation, Platform Console, and customer status

Previous state: Platform Owner approval created review evidence only. No supported path could create or activate a customer-demo workspace without entering the paid subscription lifecycle.

New state: Platform Owners can provision an approved, coherent customer-demo request. TIS reuses the shared operational workspace builder, creates an explicit demo entitlement and demo-sourced tenant link, activates the workspace atomically, records provisioning and activation events, and prevents duplicate provisioning. Failures roll back workspace changes while preserving the Approved request and retryable failure details.

Reason: Approved demos require operational access without fabricating Paddle, payment, or subscription-contract evidence.

Files changed: Demo provisioning enums, models, migration, service, shared provisioning builder, owner/customer routes and templates, lifecycle display, tests, ADR, and KMS.

Documentation updated: AI context, master context, project state, database architecture, module map, flows, roadmap, ADR, and SaaS onboarding history.

PDF regenerated: Yes, through `python scripts/kms.py sync`.

AI project context updated: Yes.

Reviewer/approval notes: M8B-4 only. No expiration, reminder, scheduler, login blocking, conversion, suspension workflow, billing/Paddle change, membership, AI-Andalus migration, commit, or push.

2026-07-22 - M8B-3 Demo Request Workflow

Area/module: SaaS onboarding commercial choice, demo request lifecycle, Platform Owner review, audit, and internal notifications

Previous state: Completed onboarding continued directly to subscription plan selection. TIS had no SaaS-owned review lifecycle for a customer demo request.

New state: Customers can choose Request Demo or Subscribe Now after onboarding. Demo submission records validated commercial context and starts in Pending Review. Customers can inspect and withdraw pending requests. Platform Owners can search/filter/sort requests and approve, reject with a reason, or cancel. Every action creates durable audit and internal-notification events. Approval records review only and does not provision or activate a workspace.

Reason: Demo requests need a safe, auditable review boundary before separately approved provisioning work begins.

Files changed:

- 1 SaaS demo enums/models/migration/service, customer and owner routes/templates, Platform Console navigation, and focused regression tests

Documentation updated:

- 1 AI context, master context, project state, database architecture, module map, workflows, roadmap, ADR 0010, and SaaS onboarding history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: M8B-3 only. No demo provisioning, activation, expiration, email delivery, Paddle change, entitlement enforcement, role change, conversion, membership, schema change outside the new review aggregate, commit, or push.

2026-07-22 - M8B-2 Commercial State And Entitlement Foundation

Area/module: Workspace commercial state, entitlement modeling, branch entitlement resolution, database architecture, and Platform Console

Previous state: M8B-1 stored workspace classification and lifecycle metadata but had no commercial decision engine, effective workspace entitlement, or branch-level commercial entitlement model.

New state: Added normalized workspace entitlement envelopes, typed entitlement values tied to the existing catalog, and optional branch inherit/active/inactive records. Added separated read-only validation, workspace entitlement, branch entitlement, and commercial-state services. Paid resolution reuses the confirmed M7 subscription entitlement authority. Platform Owners can inspect effective results without mutation controls.

Reason: Future demo and paid workflows need one conservative commercial decision foundation before any customer access or lifecycle behavior is changed.

Files changed:

- 1 commercial entitlement enums, SaaS models/migration, four resolver/validation services, Platform Console context/template, and focused regression tests

Documentation updated:

- 1 AI context, master context, project state, database architecture, module map, workflows, roadmap, ADR 0009, and workspace-classification history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: No M8B-3 workflow, enforcement, demo expiration, Paddle change, customer onboarding change, conversion, membership, role, or AI-Andalus migration is included.

2026-07-22 - M8B-1 Workspace Classification Foundation

Area/module: Workspace identity, SaaS onboarding metadata, provisioning metadata, Platform Console, and database migration tooling

Previous state: Operational SchoolGroup records had no stable workspace UUID, classification, or dedicated lifecycle metadata. Pending organizations and identities could not express workspace/test intent independently of onboarding, payment, and tenant state.

New state: Added constrained and indexed workspace UUID/classification/lifecycle fields, onboarding workspace intent, SaaS account purpose, and internal-test operational identity metadata. Added centralized validation and conversion rejection, a commercial-state skeleton with no resolution logic, read-only relationship diagnostics, and a dry-run/default transactional idempotent backfill for all confirmed pre-M8B-1 test records. Platform Owners can inspect the metadata read-only. Provisioning only carries intent into metadata and moves the metadata lifecycle from provisioning to active; existing business gates remain unchanged.

Reason: Establish the durable classification boundary required by later M8B packages without changing current customer, billing, entitlement, permission, or tenant behavior.

Files changed:

- 1 models, migration, workspace classification services, diagnostic/backfill scripts, provisioning metadata assignment, Platform Console template/context, and focused tests

Documentation updated:

- 1 AI context, master context, project state, database architecture, module map, user/system flows, roadmap, ADR 0008, and workspace-classification history

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: M8B-2 and all later demo/commercial/conversion workflows remain out of scope.

2026-07-22 - Corrected Platform Owner Pending Organization Lifecycle Views

Area/module: SaaS onboarding, provisioning lifecycle, and Platform Owner administration

Previous state: Platform Console counted every PendingOrganization row, and the Pending Organizations page listed every row regardless of completed checkout, active subscription, tenant link, or completed provisioning. Historical ready_for_checkout values could therefore make active tenants appear pending, while conflicting completed-provisioning records displayed raw internal statuses without a clear review state.

New state: One shared lifecycle-aware query now defines the pending queue as draft/setup, review, checkout/payment, or incomplete/recoverable activation work with no tenant link, completed provisioning job, or final tenant billing state. A shared owner lifecycle projection reconciles payment, subscription, contract, tenant-link, provisioning-job, and active SchoolGroup evidence. Coherent active tenants move to retained Organization Records; conflicting completed evidence is labeled Lifecycle Review Required. Platform Console counts/actions and the owner table use the same rule and readable labels.

Reason: The Platform Owner work queue must represent actionable unfinished onboarding rather than historical database rows, without mutating payment, provisioning, or onboarding evidence.

Files changed:

- 1 centralized SaaS owner lifecycle query and projection

- 1 Platform Console summary and owner admin routes
- 1 pending organization list/detail templates
- 1 focused lifecycle, filtering, count, and access tests
- 1 authoritative KMS context, module map, workflow, project state, and provisioning history

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: No schema, migration, payment, checkout, provisioning transition, permission, production data, commit, or push change. Historical records remain directly accessible.

2026-07-22 - Added Phase 7D Navigation And Catalog Enforcement

Area/module: KMS governance, navigation, catalog, and generated-artifact validation

Previous state: KMS checks enforced KIA, source coverage, deterministic source ordering, normalized hashes, freshness, PDF identity, and positive increasing pdf_page values. They did not require usable titles, constrain catalog taxonomy, validate the KMS navigation guide's targets, or prove that source pages existed within the generated PDF.

New state: scripts/kms.py check now uses a shared approved catalog to validate every included Markdown title, category, and module; requires the normalized manifest inventory to exactly match the generator list; validates every KMS Navigation link as an existing listed Markdown source inside docs/; and rejects missing, non-positive, non-integer, non-increasing, or out-of-range PDF pages. Existing KIA, hashes, freshness, ordering, coverage, and artifact checks are unchanged.

Reason: Phase 7A-7C navigation and catalog conventions must be enforceable through the same local and CI gate that protects KMS synchronization.

Files changed:

- 1 shared KMS catalog vocabulary and path classifier
- 1 ReportLab generator/read-only validation
- 1 focused KMS automation tests
- 1 KMS navigation, policy, automation, repository architecture, project state, and module history
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; first-read product, architecture, workflow, and critical-rule context is unchanged.

Reviewer/approval notes: Phase 7D only. Knowledge Center UI/routes, application behavior, database, dependencies, tis.db, commits, and pushes remain unchanged.

2026-07-22 - Added Phase 7C Knowledge Center Navigation

Area/module: Platform Knowledge Center and KMS navigation

Previous state: The owner-only Knowledge Center showed manifest sources in one path-focused table. It had no document search, category/module/freshness filters, logical source groups, descriptive summaries, or links to the source document's page in the protected booklet. ADRs were listed by ascending filename, and module-history areas

were not ordered by recent activity.

New state: The Knowledge Center enriches manifest-listed sources with Markdown title and summary metadata, groups them into Core, Engineering, Decisions, History, Marketing, and Supporting sections, and provides client-side search plus category, module, and freshness filters. Document and activity links open the existing owner-protected booklet route at the manifest pdf_page. ADRs are newest-first, and module-history areas are ordered by their latest dated entry with entry counts.

Reason: Platform owners need to locate and consume authoritative knowledge quickly without introducing a database, search service, new route, dependency, or public documentation link.

Files changed:

- 1 knowledge_service.py manifest presentation metadata and activity ordering
- 1 templates/platform_knowledge_center.html grouped library, client-side search/filters, and protected deep links
- 1 focused Knowledge Center service tests
- 1 Platform Knowledge KMS source and module-history documents
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 7C only. Owner access checks, existing routes, application data, KMS source authority, and generator enforcement remain unchanged. No regenerate control was added.

2026-07-21 - Added Phase 7B Professional PDF Navigation

Area/module: KMS generated booklet and manifest

Previous state: The booklet was a linear concatenation of source documents with page footers and a source-path list. It had no page-numbered table of contents, source destinations, outline hierarchy, or manifest mapping from Markdown sources to PDF pages.

New state: The ReportLab generator performs a multi-pass build with a "How to Use This Handbook" page, a real table of contents, stable source-document bookmarks, child bookmarks for H2 major headings, and deterministic named destinations. Every manifest source record includes its starting pdf_page, and freshness validation requires positive, strictly increasing page values.

Reason: The engineering handbook must be practical to navigate as a long-form reference while Markdown remains authoritative and generation stays dependency-light.

Files changed:

- 1 ReportLab PDF generator and focused automation tests
- 1 PDF navigation documentation and engineering-handbook history
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; product architecture, current engineering guardrails, and onboarding order are unchanged.

Reviewer/approval notes: No Knowledge Center UI, route, database, dependency, application behavior, or source-document ordering change was introduced.

2026-07-21 - Added Phase 7A KMS Navigation Foundation

Area/module: KMS information architecture and developer onboarding

Previous state: The KMS had complete root and engineering indexes, but they were long manually maintained file lists. Readers had to determine their own document set from 53 Markdown sources, and three supporting documents lacked normalized title metadata.

New state: `docs/KMS_NAVIGATION.md` provides focused reading paths for new humans, new AI conversations, SaaS onboarding, subscriptions, operational modules, database work, Platform Owner tools, landing work, location data, design, decisions, and review/KIA. Root and engineering indexes now use real Markdown links and delegate task selection to the navigation guide. Missing document titles were normalized.

Reason: Readers should reach relevant source material quickly without changing the established Markdown, manifest, PDF, or Knowledge Center architecture.

Files changed:

- 1 KMS navigation guide and documentation indexes
- 1 title metadata for three supporting documents
- 1 fixed booklet source list entry for the new authoritative guide
- 1 project state, change history, module history, PDF, and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; its existing first-read role and product/architecture guidance remain current.

Reviewer/approval notes: Phase 7B catalog/PDF navigation, Phase 7C Knowledge Center changes, and Phase 7D enforcement enhancements were not implemented.

2026-07-21 - Added Unified Phase 6 KMS Commands

Area/module: KMS developer workflow, local automation, CI, and deployment validation

Previous state: Developers separately ran the PDF generator, artifact freshness check, and KIA impact checker. The repository had all required primitives but no single synchronization command or canonical read-only command shared by local work and CI.

New state: `scripts/kms.py sync` validates the task KIA before writing, regenerates the PDF and manifest through the existing generator, runs complete post-generation validation, and prints a concise summary. `scripts/kms.py check` delegates to the existing read-only enforcement logic. GitHub pull-request, dev, and deployment gates use the unified check command.

Reason: One reliable command reduces missed mechanical steps while preserving reviewed Markdown as the source of truth and keeping enforcement strict.

Files changed:

- 1 KMS command orchestrator and reusable checker API
- 1 KMS automation tests
- 1 repository instructions and CI workflows

1 KMS workflow documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: The command never rewrites authoritative Markdown and does not change application behavior or production data.

2026-07-21 - Aligned Push Enforcement With KIA Task Boundaries

Area/module: GitHub Actions and KMS impact validation

Previous state: Pull-request enforcement validated the full feature branch against its base, while push enforcement validated only `github.event.before...github.sha`. A follow-up fix commit therefore evaluated a cumulative `.kms-impact.yml` against only the latest commit and incorrectly reported previously updated KMS files as unchanged.

New state: Pull requests validate the pull-request base SHA against the actual pull-request head SHA. Pushes to dev find the merge base between the repository default branch and the pushed head, then validate that complete task range. Both events apply the unchanged strict declaration and generated-artifact checks to the same logical implementation boundary.

Reason: KIA declarations describe approved implementation tasks, which may contain multiple commits. Event delivery boundaries must not redefine those tasks.

Files changed:

- 1 KMS impact checker
- 1 KMS enforcement workflow
- 1 KMS automation regression tests
- 1 KMS governance documentation and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; product architecture, developer onboarding order, and application behavior are unchanged.

Reviewer/approval notes: Enforcement remains strict across the complete task diff. No application behavior or production data changed.

2026-07-21 - Made KMS Enforcement Cross-Platform Deterministic

Area/module: Repository KMS generation, freshness validation, and CI enforcement

Previous state: Markdown source hashes used raw checkout bytes, and dynamically discovered ADR and history sources used native `Path` ordering. A Windows checkout with CRLF line endings could generate a manifest that passed locally but failed on GitHub Linux, where the same committed text used LF and path ordering differed.

New state: Markdown is decoded as UTF-8, normalized to LF, and then hashed. Source paths are normalized to repository-relative POSIX paths, dynamic sources use a stable case-insensitive ordering with a deterministic tie-breaker, and source comparison still rejects missing, unexpected, duplicate, or reordered entries. Git diff inspection now includes deleted files.

Reason: KMS enforcement must evaluate committed content consistently across developer workstations and GitHub Actions without weakening freshness or source-coverage checks.

Files changed:

- 1 KMS generator and impact checker
- 1 Knowledge Center freshness hashing helper
- 1 KMS automation tests and line-ending attributes
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: No; onboarding, architecture, product behavior, and current priorities are unchanged.

Reviewer/approval notes: Repository-governance correction only. No application behavior, production data, SaaS flows, database, or migrations changed.

2026-07-21 - Added Automatic KMS Synchronization Enforcement

Area/module: Repository governance, KMS automation, CI, deployment gate, and AI workflow

Previous state: KMS updates depended on developers and AI assistants remembering the written KIA policy.

PDF/manifest generation was manually triggered, the Knowledge Center detected stale hashes only when viewed, and no test, commit, pull-request, or deployment gate blocked stale or missing documentation.

New state: Root `AGENTS.md` makes KMS onboarding mandatory. `.kms-impact.yml` records task-level impact. `scripts/check_kms_impact.py` compares declarations with Git changes, conservatively classifies major paths, validates declared Markdown, and invokes generated-artifact checks. The PDF generator has read-only `--check` mode and manifest PDF hashes. GitHub Actions enforce checks on pull requests and dev, and master deployment depends on the same gate. Automation never rewrites Markdown.

Reason: Major TIS work must not be mergeable or deployable while engineering knowledge is stale, while reviewed Markdown must remain authoritative and free from runtime/customer data.

Files changed:

- 1 `AGENTS.md`
- 1 `.kms-impact.yml`
- 1 `.github/pull_request_template.md`
- 1 `.github/workflows/kms-enforcement.yml`
- 1 `.github/workflows/deploy-on-master.yml`
- 1 `scripts/check_kms_impact.py`
- 1 `scripts/generate_docs_pdf.py`
- 1 `tests/test_kms_automation.py`
- 1 relevant KMS Markdown and generated artifacts

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Repository-governance automation only. No application behavior, production/customer/tenant/billing data, runtime records, database, migrations, or business logic changed.

2026-07-20 - Backfilled Completed M7 Subscription Management

Area/module: SaaS entitlements, subscription portal, quantity/plan changes, cancellation, billing history, invoices, Paddle webhooks, and reconciliation

Previous state: Module history covered M7 Phases 1, 2, 3, and 6 only. Central project state, architecture maps, workflows, roadmap, change history, PDF, and manifest still described the pre-M7 billing foundation; Phases 4 and 5 and reconciliation protections were absent.

New state: KMS records the completed M7 entitlement foundation, read/write customer portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized lifecycle and allowed-action policy, provider billing history, protected invoice downloads, and fail-closed webhook/reconciliation safeguards.

Reason: The engineering handbook must describe current implemented subscription behavior before automatic enforcement becomes authoritative.

Files changed:

- 1 central KMS context/state files
- 1 subscription and payment ADRs
- 1 engineering module, repository, database, flow, and roadmap docs
- 1 subscription module history
- 1 generated PDF and manifest

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Documentation backfill only; it records already committed M7 behavior and introduces no SaaS behavior change.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Area/module: SaaS subscriptions, Paddle initial checkout configuration, tests, and KMS documentation

Previous state: The checkout launch flow safely blocked when the selected subscription plan price did not have `subscription_plan_prices.provider_price_id` configured, but there was no structured mapping sync process and the customer-facing error could expose provider configuration wording.

New state: Added a script-based Paddle price ID sync process using structured sandbox/production mapping examples. The database remains the source of truth through `subscription_plan_prices.provider_price_id`, real mapping files are ignored, and missing provider price IDs now surface a customer-safe Secure Payment support message while internal diagnostics retain plan code, billing interval, and currency details.

Reason: Initial subscription checkout needs environment-specific Paddle provider price IDs without hardcoding live IDs in source or changing payment state behavior.

Files changed:

- 1 .gitignore
- 1 scripts/sync_paddle_price_ids.py


```
1    config/paddle/paddle_prices.sandbox.example.json
1    config/paddle/paddle_prices.production.example.json
1    saas/payment_service.py
1    saas/router.py
1    tests/test_paddle_price_sync.py
1    tests/test_saas_phase1.py
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/CHANGE_HISTORY.md
1    docs/history/subscriptions/README.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Initial checkout configuration only. No proration, upgrade, downgrade, cancellation, payment state transition, webhook, provisioning behavior, database schema, migration, operational module, landing website, OAuth, internal route rename, live Paddle ID hardcoding, commit, or push was performed.

2026-06-27 - Accepted Subscription And Workspace Activation Guided Journey Phase 3C

Area/module: Subscription Selection, Secure Payment summary, Payment Return/Cancel, Subscription Status, Workspace Activation status, tests, and KMS documentation

Previous state: The subscription, secure payment, billing status, payment return/cancel, and workspace activation pages were functionally correct but still used page-local CTAs, dense status blocks, and inconsistent guidance. Browser return messaging existed but was not part of the shared guided setup experience.

New state: Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared setup shell and Phase 3B guided page style. Each page has one shared-shell primary CTA, customer-safe status labels, concise supporting cards, clear browser-return guidance, and explicit messaging that TIS Platform access becomes available after Workspace Activation.

Reason: The accepted Phase 3C scope required making subscription/payment/activation pages feel like one guided customer journey while preserving payment, billing, provisioning, webhook, checkout start/launch, database, and operational behavior.

Files changed:

```
1    saas/router.py
1    templates/saas/plan_selection.html
1    templates/saas/checkout_summary.html
```

```
1 templates/saas/checkout_return.html
1 templates/saas/checkout_cancel.html
1 templates/saas/account_billing.html
1 templates/saas/billing_status.html
1 tests/test_saas_phase1.py
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/CHANGE_HISTORY.md
1 docs/history/saas-onboarding/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3C customer-facing subscription/payment/status redesign only. No payment behavior change, billing behavior change, provisioning behavior change, webhook logic change, checkout start/launch behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, internal /saas route rename, stored-status change, admin view change, commit, or push was performed.

2026-06-27 - Accepted School Workspace Setup Guided Wizard Phase 3B

Area/module: School Workspace Setup onboarding templates, shared SaaS customer shell, onboarding setup context, tests, and KMS documentation

Previous state: The five onboarding pages used functional but dense form layouts with repeated progress notices, mixed card styles, and page-local primary actions. Branch setup felt like repeated blank blocks, organization logo upload was plain, and the review page felt like a basic summary rather than a confident handoff to Subscription Selection.

New state: Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided enterprise setup wizard style on top of the Phase 3A shared shell. Each page has one shared-shell primary CTA, secondary Back/Save Draft actions, grouped form sections, concise guidance, cleaner spacing, a more premium logo upload area, compact branch panels, and a stronger review summary.

Reason: The accepted Phase 3B scope required redesigning only the School Workspace Setup onboarding pages while preserving all business logic, routes, field names, validation, draft behavior, payment, billing, provisioning, database, and operational boundaries.

Files changed:

```
1 saas/router.py
1 templates/saas/base.html
1 templates/saas/onboarding_organization.html
1 templates/saas/onboarding_branches.html
```

```
1 templates/saas/onboarding_academic_setup.html
1 templates/saas/onboarding_contacts.html
1 templates/saas/onboarding_review.html
1 tests/test_saas_phase1.py
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/CHANGE_HISTORY.md
1 docs/history/saas-onboarding/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3B onboarding page redesign only. No backend business logic change, route rename, form field rename, validation change, onboarding progression change, draft behavior change, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration, operational module change, Next.js landing website change, OAuth change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Guided Setup Framework Phase 3A

Area/module: TIS Account customer dashboard, shared SaaS customer shell, setup journey display helper, and KMS documentation

Previous state: The customer account page still behaved like a dense dashboard with statistics, session details, multiple competing actions, and page-specific journey fragments. The shared customer shell had a logo and onboarding-specific progress UI, but it did not yet provide a reusable 8-step guided setup framework.

New state: The shared customer shell supports a guided setup console for pages that pass setup context. The TIS Account page now presents an official-logo guided console with an 8-step journey stepper, current-step/status area, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation. Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Reason: The accepted Phase 3A scope required only the shared framework and account page foundation for a professional TIS Account / School Workspace Setup experience, while leaving full onboarding and payment page redesigns for later phases.

Files changed:

```
1 saas/router.py
1 saas/service.py
1 templates/saas/base.html
1 templates/saas/account.html
1 tests/test_saas_phase1.py
```

```
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/CHANGE_HISTORY.md
1 docs/history/saas-onboarding/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 3A shared framework only. Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, Google/Microsoft login, internal /saas route names, admin views, commits, and pushes were not changed.

2026-06-27 - Accepted TIS Account Customer-Facing Wording And Logo Cleanup

Area/module: SaaS customer account pages, school workspace setup pages, billing/subscription status views, transactional account emails, and KMS documentation

Previous state: Customer-facing TIS Account and school workspace setup pages could display internal or technical language such as SaaS-oriented product copy, raw status labels, checkout/payment internals, provider identifiers, tenant/provisioning terms, or account setup labels that were less polished. The shared customer account shell did not consistently present an official TIS logo image across inherited customer forms/pages.

New state: Customer-facing account/setup pages use professional labels such as TIS Account, Account Dashboard, School Workspace Setup, Organization Profile, Branch Setup, Academic Setup, Subscription Setup, Secure Payment, and Workspace Activation. Customer views use display labels for internal statuses and hide customer-irrelevant provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs. The shared customer account shell uses the official full-color horizontal TIS logo, and transactional TIS Account emails use an existing official dark-blue TIS wordmark asset.

Reason: The accepted Phase 2 plan required a focused customer-facing wording cleanup and official logo usage pass before any larger account setup UI redesign.

Files changed:

```
1 saas/router.py
1 saas/service.py
1 saas/provisioning_service.py
1 email_templates.py
1 templates/saas/base.html
1 templates/saas/signup.html
1 templates/saas/login.html
1 templates/saas/account.html
```

1 templates/saas/account_billing.html
1 templates/saas/billing_status.html
1 templates/saas/onboarding_organization.html
1 templates/saas/onboarding_branches.html
1 templates/saas/onboarding_academic_setup.html
1 templates/saas/onboarding_contacts.html
1 templates/saas/onboarding_review.html
1 templates/saas/plan_selection.html
1 templates/saas/checkout_summary.html
1 templates/saas/checkout_return.html
1 templates/saas/checkout_cancel.html
1 templates/saas/profile.html
1 templates/saas/security.html
1 templates/saas/sessions.html
1 tests/test_saas_phase1.py
1 tests/test_saas_phase5.py
1 tests/test_email_templates.py
1 docs/AI_PROJECT_CONTEXT.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/CHANGE_HISTORY.md
1 docs/history/saas-onboarding/README.md
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 2 implementation only. No Phase 3 UI redesign, Google/Microsoft login, internal route/module rename, payment behavior change, billing behavior change, provisioning behavior change, database schema change, migration change, operational module change, Next.js landing website change, commit, or push was performed.

2026-06-27 - Accepted TIS Account Email Verification Recovery

Area/module: SaaS onboarding, TIS Account email verification, verification resend recovery, and school workspace setup gate

Previous state: Valid verification links rendered a static verification page instead of continuing the customer toward account setup. Expired or invalid verification links could feel like a dead end. Resend verification existed but did not provide a fully professional recovery path for expired links, already verified accounts, and unknown emails.

Password-based accounts that were still pending verification could sign in and reach account/setup routes.

New state: Valid verification links mark the account email verified/active and redirect to the TIS Account login page with a professional success notice. Expired or invalid links show a recovery page with a resend option. Resend verification safely handles unverified accounts, already verified accounts, and unknown-email cases without revealing account existence. Unverified password-based accounts are blocked from starting or continuing school workspace setup. New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Reason: Testing showed that the customer account setup journey could be blocked after email verification, especially when a verification link expired or the customer needed to recover/resend the link.

Files changed:

```
1    saas/router.py
1    saas/service.py
1    templates/saas/login.html
1    templates/saas/verify_email.html
1    templates/saas/verification_sent.html
1    email_templates.py
1    tests/test_saas_phase1.py
1    tests/test_saas_phase5.py
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/CHANGE_HISTORY.md
1    docs/history/saas-onboarding/README.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Phase 1 verification recovery implementation accepted. Phase 2 wording cleanup, Phase 3 setup UI redesign, Google/Microsoft login, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed. No commit or push was performed.

2026-06-27 - Added Production Memory Stability Guardrails

Area/module: Operational app stability, observations, global location lookup, Render deployment constraints, and engineering standards

Previous state: Production traffic could hit avoidable memory pressure. The observations page included diagnostic stage logging and extra template rendering in the normal request path, and the global location picker could parse a 47 MB reference dataset into a complete in-memory index for simple picker requests. KMS standards did not yet explicitly forbid unbounded caches, duplicate production template renders, or normal-request diagnostic warning spam.

New state: The local stabilization patch gates observation diagnostics behind TIS_OBSERVATION_DIAGNOSTICS, removes duplicate observation template pre-renders, and changes location lookup behavior to use streaming/scoped country loading for normal country, region, city, and validation requests. KMS now documents strict production memory and Render stability rules.

Reason: Render logs showed app restarts and user-facing 502s around normal app navigation after deployment. A 512 MB service can be enough for TIS only if the app avoids unnecessary full-dataset memory loads, duplicate rendering, and production debug noise.

Files changed:

```
1    location_service.py
1    routers/observations.py
1    docs/engineering/DEVELOPMENT_STANDARDS.md
1    docs/PROJECT_STATE.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/TIS_MASTER_CONTEXT.md
1    docs/CHANGE_HISTORY.md
1    docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Local changes only. No commit, push, migration, database change, SaaS route change, billing change, tenant logic change, or deployment was performed.

2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3C documented rejected decisions, visual documentation framework, AI optimization, governance, and traceability. It still needed a complete self-evolving lifecycle standard that ties implementation, validation, KIA, documentation updates, generated artifacts, review, commit, push, and deployment together.

New state: TIS now has final Phase 3D docs for knowledge lifecycle, documentation automation, KIA standard, self-evolving workflow, documentation dependency map, AI coding workflow, and future automation roadmap. This completes the KMS v1.0 lifecycle foundation.

Reason: Ensure every future approved implementation naturally keeps KMS synchronized without relying on uncontrolled app-side rewriting of source docs.

Files changed:

```
1    docs/engineering/KNOWLEDGE_LIFECYCLE.md
1    docs/engineering/DOCUMENTATION_AUTOMATION.md
1    docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
1    docs/engineering/SELF_EVOLVING_WORKFLOW.md
```



```
1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
1 docs/engineering/AI_CODING_WORKFLOW.md
1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
1 docs/engineering/README.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3D final phase only. SaaS flows, landing page code, database models, migrations, tis.db, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3B documented database architecture, development standards, UI/UX design philosophy, roadmap, and stronger onboarding guidance. It did not yet preserve rejected decisions, visual documentation standards, a definitive AI optimization guide, project governance, or explicit decision traceability.

New state: TIS now has Phase 3C engineering docs for rejected decisions, visual documentation framework, AI optimization, project governance, and decision traceability. The PDF generator includes these docs.

Reason: Future developers and AI assistants need to understand why TIS became what it is, not only what currently exists.

Files changed:

```
1 docs/engineering/REJECTED_DECISIONS.md
1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
1 docs/engineering/AI_OPTIMIZATION_GUIDE.md
1 docs/engineering/PROJECT_GOVERNANCE.md
1 docs/engineering/README.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
```

```
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceabilit
y.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Area/module: Knowledge Management System and engineering handbook

Previous state: KMS v3.0 Phase 3A added module map, repository architecture, user/system flows, and onboarding structure. The handbook still needed database architecture, development standards, UI/UX philosophy, roadmap, and stronger human/AI guidance.

New state: TIS now has Phase 3B engineering docs for database architecture, development standards, UI/UX design philosophy, and product roadmap. Core KMS docs and AI onboarding guidance reference these layers, and the PDF generator includes them.

Reason: Make the generated booklet more useful for new senior developers, Codex conversations, ChatGPT conversations, and future technical reviewers.

Files changed:

```
1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
1 docs/engineering/DEVELOPMENT_STANDARDS.md
1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md
1 docs/engineering/PRODUCT_ROADMAP.md
1 docs/engineering/README.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added KMS v3.0 Engineering Handbook

Area/module: Knowledge Management System and engineering onboarding

Previous state: The generated booklet included KMS source documents, ADRs, module history, AI context, and the Knowledge Center foundation, but it did not fully onboard a new human developer or future Codex/ChatGPT conversation into TIS modules, repository architecture, and end-to-end flows.

New state: TIS now has an engineering handbook layer with a complete module map, repository architecture guide, user/system flow guide, and engineering onboarding index. The PDF generator includes these docs and emits documentation version 3.0.

Reason: Make the generated booklet a true TIS Engineering Handbook rather than only a documentation bundle.

Files changed:

```
1 docs/engineering/README.md
1 docs/engineering/TIS_MODULE_MAP.md
1 docs/engineering/REPOSITORY_ARCHITECTURE.md
1 docs/engineering/USER_AND_SYSTEM_FLOWS.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/AI_PROJECT_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 docs/CHANGE_HISTORY.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only. App behavior, SaaS flows, landing page code, database, migrations, routes, commits, and pushes remain out of scope.

2026-06-26 - Added Platform Owner Knowledge Center

Area/module: Platform Knowledge Center and KMS access

Previous state: TIS had KMS source docs, ADRs, module history, a generated PDF booklet, and a manifest, but no protected in-app owner page for KMS status or booklet access.

New state: TIS now has a read-only Platform Owner Knowledge Center with KMS health score, manifest metadata, freshness detection, source document status, coverage checks, latest change-history entries, ADR list, module history

areas, KIA checklist, and protected PDF view/download routes.

Reason: Platform owners need an internal utility for verifying KMS health and accessing the generated PDF without exposing direct public static links.

Files changed:

```
1    knowledge_service.py
1    main.py
1    templates/platform_knowledge_center.html
1    templates/platform_console.html
1    scripts/generate_docs_pdf.py
1    docs/TIS_MASTER_CONTEXT.md
1    docs/PROJECT_STATE.md
1    docs/README.md
1    docs/CHANGE_HISTORY.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/history/platform-knowledge/README.md
1    docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md
1    static/docs/TIS_Project_Reference_Booklet.pdf
1    static/docs/docs_manifest.json
```

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2C only. Regenerate button, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

2026-06-26 - Established Knowledge Management System Foundation

Area/module: Documentation and project knowledge management

Previous state: TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal change history, ADR system, module history foundation, KMS policy, manifest, or compact AI onboarding file.

New state: TIS now has a Knowledge Management System foundation with chronological change history, documentation update policy, ADR structure and initial accepted ADRs, module history folders, AI project context, updated source docs, and an expanded PDF generator.

Reason: Preserve project knowledge for future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and technical reviewers.

Files changed:

```
1    docs/CHANGE_HISTORY.md
1    docs/DOCUMENTATION_UPDATE_POLICY.md
1    docs/AI_PROJECT_CONTEXT.md
1    docs/adr/README.md
```

1 docs/adr/0001-separate-nextjs-landing-website.md
1 docs/adr/0002-separate-saas-identity-and-operational-users.md
1 docs/adr/0003-paddle-payment-architecture.md
1 docs/adr/0004-webhook-only-payment-confirmation.md
1 docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md
1 docs/adr/0006-documentation-as-source-knowledge-management-system.md
1 docs/adr/0007-landing-page-visual-system-strategy.md
1 docs/history/README.md
1 docs/history/*/README.md
1 docs/history/provisioning/2026-06-26-kms-foundation.md
1 docs/TIS_MASTER_CONTEXT.md
1 docs/PROJECT_STATE.md
1 docs/README.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
1 static/docs/docs_manifest.json

Documentation updated: Yes

PDF regenerated: Yes

AI project context updated: Yes

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only. Platform Owner Knowledge Center, app routes, SaaS flows, database, migrations, landing page implementation, commits, and pushes remain out of scope.

TIS Architecture Decision Records

docs/adr/README.md

Architecture Decision Records

ADRs record major TIS architectural and product decisions. They explain why the system is shaped the way it is.

ADR Rules

- 1 Create an ADR for major decisions affecting architecture, identity, SaaS, payment, provisioning, deployment, documentation governance, or long-term product boundaries.
- 1 Do not use ADRs for ordinary bug fixes.
- 1 If a decision is replaced, mark the older ADR as Superseded and link the new ADR.
- 1 ADRs complement docs/TIS_MASTER_CONTEXT.md; they do not replace it.

Status Values

- 1 Proposed
- 1 Accepted
- 1 Superseded
- 1 Deprecated

ADR Index

- 1 0001-separate-nextjs-landing-website.md
- 1 0002-separate-saas-identity-and-operational-users.md
- 1 0003-paddle-payment-architecture.md
- 1 0004-webhook-only-payment-confirmation.md
- 1 0005-delayed-tenant-provisioning-after-verified-payment.md
- 1 0006-documentation-as-source-knowledge-management-system.md
- 1 0007-landing-page-visual-system-strategy.md
- 1 0008-workspace-classification-foundation.md
- 1 0009-commercial-state-and-entitlement-resolution.md
- 1 0010-review-only-saas-demo-requests.md
- 1 0011-demo-workspace-provisioning-and-commercial-source-links.md
- 1 0012-seven-day-demo-lifecycle-and-access-enforcement.md
- 1 0013-demo-to-paid-workspace-conversion.md
- 1 0014-demo-customer-communications-and-approval-orchestration.md
- 1 0015-centralized-ai-entitlements-and-usage-accounting.md
- 1 0016-pre-deploy-database-migration-boundary.md

1 0017-platform-owner-demo-operations-and-access-profiles.md
1 0018-unified-commercial-access-and-capacity-authority.md
1 0019-secure-promo-code-definition-and-governance.md
1 0020-promo-redemption-and-commercial-grants.md
1 0021-existing-workspace-conversion-read-only-audit.md
1 0022-controlled-existing-workspace-conversion.md
1 0023-existing-workspace-paid-activation.md
1 0024-promo-expiry-recovery-and-paid-continuation.md
1 0025-capacity-based-packaging-and-common-customer-feature-baseline.md
1 0026-versioned-constraint-based-smart-timetable-generation.md

TIS Engineering Handbook

docs/engineering/README.md

This folder turns the TIS KMS into a practical engineering handbook. It is intended for new human developers, future Codex conversations, future ChatGPT conversations, project owners, platform owners, and reviewers.

Choose A Reading Path

Use the [KMS Navigation Guide](../KMS_NAVIGATION.md) to select the smallest relevant document set for a specific task.

For a broad engineering onboarding, read:

Recommended onboarding order:

- 1 [AI Project Context](../AI_PROJECT_CONTEXT.md)
- 2 [TIS Master Context](../TIS_MASTER_CONTEXT.md)
- 3 [Project State](../PROJECT_STATE.md)
- 4 [TIS Module Map](TIS_MODULE_MAP.md)
- 5 [Repository Architecture](REPOSITORY_ARCHITECTURE.md)
- 6 [User and System Flows](USER_AND_SYSTEM_FLOWS.md)
- 7 [Database Architecture Overview](DATABASE_ARCHITECTURE_OVERVIEW.md)
- 8 Relevant [ADRs](../adr/README.md) and [module history](../history/README.md)

Engineering Documents

- 1 [TIS Module Map](TIS_MODULE_MAP.md): product and system modules, maturity, ownership, risks, and guardrails.
- 1 [Repository Architecture](REPOSITORY_ARCHITECTURE.md): repository structure and responsibilities.
- 1 [User and System Flows](USER_AND_SYSTEM_FLOWS.md): end-to-end public, SaaS, payment, provisioning, operational, owner, and KMS flows.
- 1 [Database Architecture Overview](DATABASE_ARCHITECTURE_OVERVIEW.md): conceptual data relationships and isolation boundaries.
- 1 [Development Standards](DEVELOPMENT_STANDARDS.md): non-negotiable engineering rules for humans and AI assistants.
- 1 [UI/UX Design Philosophy](UI_UX_DESIGN_PHILOSOPHY.md): design principles by product surface.
- 1 [Product Roadmap](PRODUCT_ROADMAP.md): completed, current, next, and future roadmap.
- 1 [Rejected Decisions](REJECTED_DECISIONS.md): significant declined alternatives and their consequences.
- 1 [Visual Documentation Guide](VISUAL_DOCUMENTATION_GUIDE.md): screenshot and diagram standards.
- 1 [AI Optimization Guide](AI_OPTIMIZATION_GUIDE.md): definitive AI onboarding and operating guide.
- 1 [Project Governance](PROJECT_GOVERNANCE.md): ownership, approvals, quality gates, and traceability.
- 1 [Knowledge Lifecycle](KNOWLEDGE_LIFECYCLE.md): documentation and engineering lifecycle.
- 1 [Documentation Automation](DOCUMENTATION_AUTOMATION.md): current and future KMS automation rules.
- 1 [KIA Standard](KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md): formal KIA examples, decision tree, and failure cases.

- 1 [Self-Evolving Workflow](SELF_EVOLVING_WORKFLOW.md): official task-to-deployment workflow.
- 1 [Documentation Dependency Map](DOCUMENTATION_DEPENDENCY_MAP.md): propagation rules across KMS documents.
- 1 [AI Coding Workflow](AI_CODING_WORKFLOW.md): required workflow for future AI coding assistants.
- 1 [Future Automation Roadmap](FUTURE_AUTOMATION_ROADMAP.md): possible future automation improvements.

Before Coding

Before changing code:

- 1 confirm the approved scope,
- 1 inspect affected files and tests,
- 1 read relevant ADRs,
- 1 read relevant module history,
- 1 preserve tenant isolation,
- 1 preserve identity boundaries,
- 1 avoid touching SaaS, landing, database, or migrations unless explicitly approved.

After Coding

Every implementation must complete the Knowledge Impact Assessment:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

If included source docs changed, synchronize:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Run `.\.venv\Scripts\python.exe scripts\kms.py check` for final read-only validation.

TIS Module Map

docs/engineering/TIS_MODULE_MAP.md

Talent Assessment Cycles

- 1 Models: `TalentAssessmentCycle`, `TalentAssessmentCyclePopulationMember`, `TalentAssessmentAudit` in `models.py`.
- 1 Authority: `talent_assessment_cycle_service.py` derives Draft eligibility, performs atomic Open/Close, freezes historical context, and computes the canonical population fingerprint.
- 1 API: `routers/talent_assessment_cycles.py` exposes bounded Cycle and authorization-filtered population operations under `/api/talent/assessment-cycles`.
- 1 Permissions: dedicated `talent_assessment_cycles.*`; population visibility and lifecycle governance are distinct from Talent Program configuration.

Teacher Scheduling Rules Components

- 1 `teacher_scheduling_rules.py`: exact-scope normalized CRUD, rule-shape and deterministic conflict validation, canonical slot/target resolution, UI data, and unpublished Draft invalidation.
- 1 `models.py` and migration `20260830_001_teacher_scheduling_rules`: rule headers, normalized slots and grade/section targets, composite teacher/section scope integrity, and migration-order-safe deferred metadata creation.
- 1 `templates/system_configuration_timetable.html` and `main.py`: permissioned Teacher Scheduling Rules configuration under Timetable Settings.
- 1 `timetable_snapshot_service.py`: schema-v4 immutable teacher-rule authority and constraint fingerprinting.
- 1 `timetable_readiness_service.py` and `timetable_problem_builder.py`: deterministic target, workload, availability, hard-conflict, and lock defenses.
- 1 `timetable_cp_sat_solver.py` and `timetable_solution_validator.py`: hard timing enforcement, soft preference scoring, and solver-independent parity.

Guided Curriculum Adjustment UI

- 1 `routers/planning.py`: serves the exact-scope wizard page and continues to own preview/apply HTTP adapters.
- 1 `templates/curriculum_adjustment.html`: five-step selection, impact, teacher-decision, review, and success experience.
- 1 `templates/planning.html` and `templates/subjects.html`: permission-gated entry points.

- 1 `authorization.py`: protects the page and apply operation with `curriculum.adjust`; preview accepts either the established Planning edit permission or the dedicated adjustment permission.
- 1 `curriculum_adjustment_preview_service.py`: exposes target eligibility on teacher suggestions for safe selector filtering.

The preview request carries `requested_transfer_periods`; the preview derives section-specific after values from Planning authority, and apply consumes those fingerprinted values unchanged. `routers/subjects.py` decorates exact-scope catalog rows with effective Current/New Planning demand using `planning_subject_demand_service.py`; `templates/subjects.html` renders uniform values or a **Varies** section breakdown.

`adjustment_type="reduce_only"` uses the same preview/apply services with no target demand or target teacher mutation. `templates/edit_subject.html` and the exact-scope subject route expose effective Planning demand plus a prefilled reduction link when `curriculum.adjust` is allowed.

Atomic Curriculum Adjustment Apply

- 1 `curriculum_adjustment_apply_service.py`: transaction owner for locking, stale revalidation, demand/assignment/rule reconciliation, Draft invalidation, audit, concurrency rejection, rollback, and duplicate protection.
- 1 `models.CurriculumAdjustmentAudit` and migration
- `20260829_001_curriculum_adjustment_apply_foundation`: durable applied outcome and unique reviewed-fingerprint authority.
- 1 `routers/planning.py`: apply JSON endpoint protected by `curriculum.adjust`.
- 1 `permission_registry.py` and `authorization.py`: dedicated permission registration and route enforcement.

Curriculum Adjustment Preview

- 1 `curriculum_adjustment_preview_service.py`: read-only scope selection, demand transfer projection, teacher/capacity suggestions, rule/grouped warnings, Draft impact, and deterministic stale-confirmation fingerprint.
- 1 `routers/planning.py`: permissioned JSON preview endpoint; no apply endpoint.
- 1 `authorization.py`: `planning.edit_section` protects the preview route.
- 1 Existing demand, assignment, timetable version, active pointer, and published placement models remain read-only inputs in Stage 3.

Explicit Planning Subject Demand Foundation

- 1 `models.PlanningSubjectDemand`: future section-subject weekly-period authority, with scoped integrity, active uniqueness, and retained retirement state.
- 1 `planning_subject_demand_service.py`: single-section and bulk exact-scope explicit-first demand resolution with transitional legacy fallback.

1 `db_migrations.py`: idempotent Current/New section backfill from grade-matched
Subjects through migration `20260828_004_planning_subject_demands_foundation`.

1 `routers/planning.py`, `routers/teachers.py`, `main.py`, `timetable_logic.py`,
`timetable_readiness_service.py`, `timetable_snapshot_service.py`, and
`subject_distribution_rules_ui.py` consume resolved section demand for live required-period arithmetic.
Snapshot-fed generation therefore receives the same authority; publication history remains unchanged.

Smart Timetable Stage 5.2 Components

1 `timetable_visibility_service.py`: active-pointer-only reads and scoped teacher identity.

1 `routers/timetable.py`: simplified workflow, working deletion, published routes,
and backward-compatible canonical lifecycle/mutation conflict responses.

1 `timetable_logic.py`: workspace/history lifecycle facts, source linkage,
mutability, active identity, and validation/approval/publication timestamps.

1 `templates/timetable.html`: visible lifecycle steps, immutable history actions,
Draft-only generation language, and stale concurrency refresh UX.

1 `templates/published_timetable.html`: control-free official lesson view.

1 `timetable_version_service.py`: working resolution and safe archive/discard.

Smart Timetable Stage 5.1 Components

1 `timetable_conflicts.py`: stable conflict taxonomy, evidence classification,
safe entity references, public redaction, and legacy adapters.

1 `timetable_requirement_projection.py`: deterministic read-only Planning-to-
timetable requirement contract, exact-scope authority, identity, and provenance.

1 `timetable_generation_service.py`: exact-scope enqueue/idempotency, generation
state, claims, leases, heartbeat, recovery, cancellation, staleness recheck, and atomic version persistence; contains no
OR-Tools import.

1 `timetable_problem_builder.py`: schema-v3 immutable snapshot normalization,
deterministic lesson IDs, canonical slots, Planning/HRT demand, locks, and regeneration source contract.

1 `timetable_cp_sat_solver.py`: task-runtime-only CP-SAT model and solve result.

1 `timetable_solver.py`: solver-neutral worker contract and the Release 1 CP-SAT
adapter; it owns no scheduling rules or persistence.

1 `timetable_solution_validator.py`: solver-independent hard validation.

1 `timetable_workflow_dispatch.py`: lightweight server-side Render client and

environment-driven task slug.

1 `timetable_generation_workflow.py`: registered on-demand task accepting only the durable generation run public ID.

1 `timetable_generation_worker.py`: reusable single-run executor and optional local polling fallback; it is not an always-on production requirement.

1 `routers/timetable.py` and `templates/timetable.html`: permissioned enqueue, scoped status/cancel, polling, generated review selection, and regeneration action.

1 `requirements-workflow.txt`: web/runtime requirements plus pinned OR-Tools 9.15.6755.

1 `requirements-worker.txt`: compatibility dependency set for the optional local worker.

Commercial Feature Packaging Components

1 `saas/customer_feature_policy.py`: stable normal-customer and internal-feature classification. It does not decide permission, scope, lifecycle, or capacity.

1 `saas/entitlement_service.py`: centralized source-aware customer feature decision, including permission, commercial state, optional branch/year scope, catalog state, and BranchEntitlement.

1 `saas/ai_consumption_policy.py`: consumption decision separate from commercial AI availability; current demo allowances are preserved without creating a provider billing engine.

1 `saas/demo_feature_registry.py` and `saas/demo_access_service.py`: Standard, Full, and Custom retain the normal baseline; Custom continues to control AI allowances, scope, safety, and experimental configuration.

1 `saas/promo_redemption_service.py`: new promo activation snapshots the common baseline and exact grant capacity.

1 `db_migrations.py`: migration

`20260819_001_capacity_based_customer_feature_base_line` reconciles plan, active promo, and active demo operational values idempotently.

The boundary excludes Platform Console, Developer/System Owner controls, global audit/support export, promo administration, demo administration, and payment administration. Capacity continues through `commercial_authority_service`.

SchoolGroup Management Boundary Components

1 `main.py`: Platform identity/global capability checks for SchoolGroup create and delete, tenant-linked update scope, unified branch-capacity presentation, and the SchoolGroup-scoped final-active-branch safeguard.

1 `permission_registry.py`: top-level `schools.create` and `schools.delete` are

developer-only; `schools.manage_all_schools` is the global manual-management capability.

1 `templates/system_configuration_schools.html`: hides global create/delete actions

from tenants and presents source-neutral authoritative branch usage.

1 `saas/commercial_authority_service.py` and `saas/promo_grant_service.py`: unchanged

authority and atomic capacity/assignment boundary used by the page and mutations.

1 `scripts/audit_schoolgroup_provenance.py`: sanitized read-only detection of active,

unlinked internal sandboxes without explicit workspace entitlement.

M3 Promo Redemption Components

1 `saas/promo_redemption_service.py`: secure lookup, owner/context validation,

resumable review and selection, locking, immutable snapshots, activation, idempotency, and redacted durable events.

1 `saas/promo_grant_service.py`: read-time grant/expiry resolution and safe

assignment of a newly created branch from remaining promo capacity.

1 `saas/models.py` and `db_migrations.py`: six M3 tables plus promo entitlement

and tenant-source references under migration `20260805_002_promo_redemption_and_grants`.

1 `saas/commercial_authority_service.py`, commercial access/state/workspace and

branch entitlement resolvers, and `auth.py`: M1 promo source composition and fail-closed branch enforcement.

1 `saas/router.py`, `templates/saas/promo_activation.html`,

`templates/saas/commercial_choice.html`, and `templates/saas/account.html`: owner-only onboarding and existing-organization activation journey.

Existing Workspace Paid Activation

Primary files

1 `saas/existing_workspace_paid_activation_service.py`

1 `saas/billing_identity_service.py`

1 `saas/payment_service.py`

1 `templates/saas/existing_workspace_paid_activation.html`

Responsibility

Activate a previously converted existing customer workspace through an authoritative Paddle subscription while preserving the existing SchoolGroup and all operational data. The module owns direct activation eligibility, branch/capacity quote snapshots, checkout lineage, workspace billing/customer attribution, and the atomic webhook-confirmed commercial transition. It does not own tenant provisioning, normal onboarding checkout, promo redemption, or demo conversion.

M3 never calls Paddle, creates payment evidence, converts internal sandboxes, or mutates staff/teacher activity. Exactly one paid/demo/promo tenant source is required.

M2 Promo Code Components

- 1 `saas/promo_code_service.py`: secure generation and HMAC lookup authority,

shared plan-capacity validation, controlled scope validation, lifecycle row locking, definition duplication/replacement, and redacted durable audit.
- 1 `saas/models.py`: `PromoCode`, `PromoCodeBranchRestriction`, and

`PromoCodeAuditEvent` remain the M2 definition records; M3 redemption/grant records are separate.
- 1 `saas/router.py` and `templates/saas/admin_promo_code*.html`: Platform

Console list/filter, generated-code one-time response, detail, edit, and lifecycle actions under `/saas-admin/promo-codes`.
- 1 `permission_registry.py`: separate `promo_codes.view` and

`promo_codes.manage` platform permissions. Owner identity is additionally required for activation and revocation.
- 1 `db_migrations.py`: additive migration

`20260805_001_promo_code_foundation` with no data backfill.

M2 definition management does not itself integrate with M1 authority, `WorkspaceEntitlement`, `TenantProvisioningLink`, or `Paddle`. M3 performs that integration only after final customer activation.

M1 Commercial Access And Capacity Components

- 1 `saas/commercial_authority_service.py`: single commercial-access composition,

authoritative branch/staff/teacher counters, structured decisions, and `SchoolGroup` row locking.
- 1 `saas/commercial_access_service.py`, `saas/commercial_state_service.py`,

`saas/workspace_entitlement_service.py`, and `saas/entitlement_service.py`: existing authorities composed by the facade; they are not duplicated.
- 1 `main.py`, `routers/users.py`, and `routers/teachers.py`: permission- and

tenant-scoped operational mutation entry points that enforce the shared decision before writing.
- 1 `saas/provisioning_service.py` and `saas/demo_provisioning_service.py`: paid

and demo final-authority invariants inside their existing atomic provisioning transactions.

Guardrails: every capacity-increasing mutation locks then recounts in the same transaction; platform identity never counts as tenant staff; account-only SaaS identity never counts; demo/internal authority is unmetered only while its existing lifecycle/access resolver is coherent; contradictory authority fails closed.

M8B9 Demo Operations Components

- 1 `saas/demo_operations_service.py`: owner authorization, lifecycle

orchestration, communication references, summaries, and durable audits.
- 1 `saas/demo_access_service.py` and `saas/demo_feature_registry.py`: Standard,

Full, and Custom resolution with workspace defaults and branch overrides.

1 DemoAccessPolicy and DemoOperationAudit: policy and attempt history;

M8B8 usage counters/events remain unchanged.

1 Existing owner demo detail and queue surfaces host the controls.

M8B8 AI Entitlement Components

1 saas/ai_feature_registry.py: stable feature identifiers, display labels,

required permission/entitlement, eligible plans, demo allowance, and enabled state.

1 saas/ai_entitlement_service.py: tenant-safe decisions, consistent denial

metadata, and the only reservation/finalization/consumption API.

1 AIFeatureUsageCounter and AIFeatureUsageEvent: separated durable

internal/demo/paid accounting.

1 permission_registry.py: assignable ai.use authorization boundary.

M8B7 Demo Customer Journey Components

1 saas/demo_email_service.py: durable demo email intents, branded rendering inputs, provider dispatch, and retries.

1 saas/demo_notification_service.py: deduplicated Platform Owner Notification Center events.

1 Existing demo request, provisioning, lifecycle, and conversion services remain authoritative.

1 ui_shell.py and templates/base.html own the active tenant demo indicator.

1 scripts/process_demo_lifecycle.py remains dry-run by default and is the production apply/dispatch entry point.

This map describes the known TIS modules, where they live, their maturity, related docs/ADRs, and the guardrails developers must respect.

Platform Owner

Purpose: Own global TIS platform operations, owner/co-owner controls, platform developer accounts, cross-organization oversight, and protected KMS access.

Main files/folders:

1 auth.py

1 main.py

1 templates/platform_console.html

1 templates/platform_knowledge_center.html

1 knowledge_service.py

1 permission_registry.py

Maturity/status: Implemented for owner identity, platform console, developer/co-owner controls, and read-only Knowledge Center.

Related docs/ADRs:

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/history/platform-knowledge/

Risks/guardrails:

- 1 Do not treat platform developers as owners.
- 1 Do not expose owner utilities to tenant users.
- 1 Reuse `auth.is_platform_owner(...)` and existing owner access helpers.

Platform Console

Purpose: Allow platform identities to inspect organizations, switch context, and manage platform owner/developer controls.

Main files/folders:

- 1 main.py
- 1 templates/platform_console.html
- 1 ui_shell.py

Maturity/status: Implemented and active.

Related docs/ADRs:

- 1 docs/PROJECT_STATE.md

Risks/guardrails:

- 1 Context switching must not weaken tenant isolation.
- 1 Owner-only management controls must remain owner-only.

Knowledge Center

Purpose: Show KMS health, source coverage, PDF freshness, searchable document metadata, ADRs, module history, change history, and KIA policy to platform owners.

Main files/folders:

- 1 knowledge_service.py
- 1 main.py
- 1 templates/platform_knowledge_center.html
- 1 static/docs/docs_manifest.json
- 1 static/docs/TIS_Project_Reference_Booklet.pdf

Maturity/status: Read-only Phase 2C implementation complete and Phase 7C navigation enhanced. The manifest-backed library groups documents by knowledge area, filters locally by category/module/freshness, searches titles/summaries/paths, and links to protected booklet pages. Repository-level impact and freshness enforcement is

active; no app regenerate button exists.

Related docs/ADRs:

- 1 docs/adr/0006-documentation-as-source-knowledge-management-system.md
- 1 docs/history/platform-knowledge/

Risks/guardrails:

- 1 Do not link directly to `/static/docs/...` from UI.
- 1 Treat the generated manifest as the source inventory and `pdf_page` authority; do not create a parallel catalog.
- 1 Do not let the app rewrite Markdown source docs.
- 1 Do not add regenerate behavior without approval.

SaaS Account System

Purpose: Handle public SaaS signup/login/account profile, sessions, security, billing visibility, and onboarding status.

Main files/folders:

- 1 saas/router.py
- 1 saas/service.py
- 1 saas/models.py
- 1 templates/saas/

Maturity/status: M1-M5 foundation implemented.

Related docs/ADRs:

- 1 docs/adr/0002-separate-saas-identity-and-operational-users.md
- 1 docs/history/saas-onboarding/

Risks/guardrails:

- 1 Do not merge SaaS account identity with operational tenant users.
- 1 Do not bypass verification/security/session boundaries.

SaaS Onboarding

Purpose: Collect organization, contact, branch, academic setup, and review data before provisioning.

Main files/folders:

- 1 saas/router.py
- 1 saas/service.py
- 1 templates/saas/onboarding_*.html

Maturity/status: Implemented foundation.

Related docs/ADRs:

- 1 docs/history/saas-onboarding/
- 1 docs/adr/0002-separate-saas-identity-and-operational-users.md

Risks/guardrails:

- 1 Pending onboarding data is not the same as operational tenant data.
- 1 Do not create live tenant records directly from public forms.

SaaS Demo Request Workflow

Purpose: Offer verified customers a post-onboarding choice between the unchanged subscription workflow and a reviewed customer-demo workspace.

Main files/folders:

- 1 demo_workflow.py
- 1 saas/demo_request_service.py
- 1 saas/demo_eligibility_maintenance_service.py
- 1 saas/demo_provisioning_service.py
- 1 saas/demo_conversion_service.py
- 1 saas/provisioning_service.py
- 1 saas/router.py
- 1 templates/saas/commercial_choice.html
- 1 templates/saas/demo_request_status.html
- 1 templates/saas/admin_demo_requests.html
- 1 templates/saas/admin_demo_request_detail.html
- 1 templates/saas/admin_demo_eligibility_maintenance.html
- 1 templates/saas/admin_delete_demo_eligibility.html

Maturity/status: M8B-6 implemented. Submission, review, atomic provisioning, activation, seven-day lifecycle resolution, Day 6 internal reminders, Day 7 expiration, server-side expired-access enforcement, and provider-confirmed conversion of the existing demo tenant to Customer Paid are available. Manual extension, internal-sandbox conversion, archive/delete, and email delivery are not implemented.

Risks/guardrails:

- 1 Do not confuse SaaS demo requests with legacy public marketing demo leads.
- 1 Approval remains review evidence only; a separate Platform Owner provisioning action creates the workspace.
- 1 Provisioning requires coherent approval, customer-demo intent, commercial state, entitlement snapshot, and organization data.
- 1 Demo provisioning must not create or infer paid subscription evidence.
- 1 Failed provisioning must roll back workspace records and preserve the Approved request for a controlled retry.
- 1 activated_at is the only lifecycle clock authority; approval and submission timestamps never determine expiration.
- 1 Expiration preserves tenant data and blocks all normal operational access rather than creating a read-only mode.
- 1 Paid workspaces, internal sandboxes, and Platform Console access must never pass through demo expiration enforcement.

- 1 Demo-to-paid conversion requires an active coherent demo and a confirmed M7 subscription for the same organization.
- 1 Conversion preserves the SchoolGroup and tenant link row; it must never invoke operational reprovisioning.
- 1 A failed workspace conversion preserves confirmed payment evidence, rolls back workspace mutations, records failure, and remains retryable.
- 1 Completed conversions must not re-enter demo reminder or expiration processing.
- 1 Historical eligibility maintenance is Platform Owner-only, deletes by exact eligibility ID only, and must fail closed on any organization, account, request, workspace, provisioning, subscription, conversion, or manual-review evidence.
- 1 Eligibility maintenance must not replace, bypass, or weaken customer demo eligibility or organization-scoped clean-room reset.

Main M8B-5 and M8B-6 files:

- 1 `saas/demo_lifecycle_service.py`
- 1 `saas/demo_conversion_service.py`
- 1 `scripts/process_demo_lifecycle.py`
- 1 `authorization.py`
- 1 `templates/demo_access_blocked.html`
- 1 Subscribe Now must continue through the existing provider-authoritative billing path.
- 1 Only Platform Owners may review, reject, approve, or cancel requests.
- 1 Customer withdrawal is valid only while Pending Review.

Workspace Classification Foundation

Purpose: Provide stable, constrained metadata for distinguishing internal sandbox, customer demo, and customer-paid workspaces without changing commercial or customer behavior.

Main files/folders:

- 1 `workspace_classification.py`
- 1 `saas/workspace_classification_service.py`
- 1 `saas/workspace_classification_admin_service.py`
- 1 `saas/commercial_state_service.py`
- 1 `scripts/diagnose_workspace_classification.py`
- 1 `scripts/backfill_workspace_classification.py`

Maturity/status: M8B-6 commercial transition implemented only for the dedicated active Customer Demo to Customer Paid workflow. Classification, lifecycle, workspace entitlement, branch entitlement, and effective commercial state remain centrally resolved. Other classification conversions remain prohibited.

Related docs/ADRs:

- 1 `docs/adr/0008-workspace-classification-foundation.md`
- 1 `docs/history/workspace-classification/README.md`
- 1 `docs/adr/0009-commercial-state-and-entitlement-resolution.md`

Risks/guardrails:

- 1 Do not use classification as a payment, entitlement, permission, tenant-isolation, or reset gate in M8B-1.
- 1 Permit classification conversion only through the dedicated M8B-6 demo-to-paid service.
- 1 Do not expose workspace metadata outside Platform Owner views.
- 1 Do not infer customer-paid classification from incomplete onboarding or provider records.
- 1 Keep paid plan capabilities authoritative in the existing M7 subscription entitlement resolver.
- 1 Fail closed on missing, ambiguous, orphaned, stale, or cross-tenant entitlement relationships.
- 1 Do not use M8B-2 results for tenant authorization or feature enforcement yet.

Commercial Entitlement Resolution

Purpose: Resolve effective workspace entitlement, branch commercial activity, and commercial state from persisted local evidence without modifying rows or calling Paddle.

Main files/folders:

- 1 `commercial_entitlements.py`
- 1 `saas/commercial_validation_service.py`
- 1 `saas/workspace_entitlement_service.py`
- 1 `saas/branch_entitlement_service.py`
- 1 `saas/commercial_state_service.py`
- 1 `saas/models.py`

Maturity/status: M8B-2 read-only foundation implemented. No customer-facing enforcement, demo expiration, billing mutation, or conversion orchestration.

Pending Organizations

Purpose: Represent the Platform Owner work queue for organizations still waiting on setup, review, payment, or incomplete/recoverable provisioning, while preserving completed and historical records separately.

Main files/folders:

- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `templates/saas/admin_pending_organizations.html`
- 1 `templates/saas/admin_pending_organization_detail.html`

Maturity/status: Implemented with lifecycle-aware pending filtering and retained Organization Records.

Related docs/ADRs:

- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

Risks/guardrails:

- 1 Keep pending state separate from provisioned tenant state.

- 1 Do not count a row as pending when a tenant link, completed provisioning job, or final tenant billing state exists.
- 1 Resolve active tenant display from coherent payment, active subscription, contract, tenant-link, and active SchoolGroup evidence; conflicting completed evidence must fail closed to Lifecycle Review Required.
- 1 Preserve raw onboarding fields as history rather than rewriting them solely for owner presentation.
- 1 Preserve platform owner review and auditability.

Plans And Pricing

Purpose: Define SaaS plan catalog and pricing behavior.

Main files/folders:

- 1 `saas/pricing_service.py`
- 1 `saas/router.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/subscription_portal_service.py`
- 1 `saas/subscription_change_service.py`
- 1 `saas/subscription_plan_change_service.py`
- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `templates/saas/plan_catalog.html`
- 1 `templates/saas/plan_selection.html`
- 1 `templates/saas/subscription.html`

Maturity/status: M7 implemented: entitlement catalog, customer portal, quantity and plan management, scheduled changes, cancellation/reversal, and centralized lifecycle/action policy.

Related docs/ADRs:

- 1 `docs/history/subscriptions/`
- 1 `docs/adr/0003-paddle-payment-architecture.md`

Risks/guardrails:

- 1 Plan changes must update docs and change history.
- 1 Do not hard-code provider behavior outside service boundaries.

Paddle / Payment

Purpose: Integrate subscription checkout, payment state, billing status, and provider events.

Main files/folders:

- 1 `saas/paddle_client.py`
- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`
- 1 `saas/currency_service.py`

- 1 `saas/router.py`
- 1 `saas/billing_history_service.py`
- 1 `saas/payment_lifecycle_reconciliation_service.py`

Maturity/status: Implemented through M7, including provider-authoritative previews/proration, billing history, invoice access, webhook idempotency, fail-closed reconciliation, diagnostics, and guarded repair tooling.

Related docs/ADRs:

- 1 `docs/adr/0003-paddle-payment-architecture.md`
- 1 `docs/adr/0004-webhook-only-payment-confirmation.md`
- 1 `docs/history/subscriptions/`

Risks/guardrails:

- 1 Webhook-confirmed payment state is authoritative.
- 1 Checkout return alone must not trigger verified payment/provisioning.
- 1 TIS must not calculate replacement financial outcomes when Paddle preview/transaction data is authoritative.
- 1 Scheduled provider changes must not become local entitlement truth before verified effective evidence.
- 1 Invoice URLs must be freshly resolved and must not be stored.

Provisioning

Purpose: Convert verified/approved pending organizations into operational tenant structures.

Main files/folders:

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `templates/saas/admin_provisioning.html`

Maturity/status: Implemented foundation.

Related docs/ADRs:

- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- 1 `docs/history/provisioning/`

Risks/guardrails:

- 1 Keep provisioning recoverable and reviewable.
- 1 Do not merge tenants or create records in the wrong school group.

Operational Login

Purpose: Authenticate operational users and route them into platform or tenant context.

Main files/folders:

- 1 `main.py`
- 1 `auth.py`

1 templates/login.html if present through root templates

Maturity/status: Implemented.

Related docs/ADRs:

1 docs/adr/0002-separate-saas-identity-and-operational-users.md

Risks/guardrails:

1 Platform users and tenant users follow different context behavior.

1 Preserve permission and active-user checks.

Dashboard

Purpose: Give tenant users operational visibility into staffing, planning, reports, and current school context.

Main files/folders:

1 main.py

1 templates/dashboard.html

1 related report/export helpers

Maturity/status: Implemented and evolving.

Related docs/ADRs:

1 docs/history/workforce-planning/

Risks/guardrails:

1 Dashboard data must remain scoped to tenant/branch/year context.

Organizations / School Groups

Purpose: Represent tenant organizations and top-level school group context.

Main files/folders:

1 models.py

1 main.py

1 templates/system_configuration_schools.html

Maturity/status: Implemented.

Related docs/ADRs:

1 docs/TIS_MASTER_CONTEXT.md

Risks/guardrails:

1 Tenant isolation starts at school group boundaries.

Branches

Purpose: Represent campuses/branches inside a school group.

Main files/folders:

- 1 `models.py`
- 1 `main.py`
- 1 `templates/system_configuration_branches.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Branch scope affects teachers, users, planning, timetable, calendar, branding, and reports.

Academic Years

Purpose: Scope operational data by academic year and active-year context.

Main files/folders:

- 1 `models.py`
- 1 `main.py`
- 1 `templates/system_configuration_years.html`

Maturity/status: Implemented.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Do not mix data across academic years.

Users And Roles

Purpose: Manage tenant users, platform users, roles, profile data, active status, and branch/year context.

Main files/folders:

- 1 `auth.py`
- 1 `routers/users.py`
- 1 `models.py`
- 1 `templates/users.html`
- 1 `templates/edit_user.html`

Maturity/status: Implemented with platform identity refinements.

Related docs/ADRs:

- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Risks/guardrails:

- 1 Do not assign owner controls to developers or tenant users.
- 1 Preserve role normalization.

Permissions

Purpose: Control route/module actions by permission key and role package.

Main files/folders:

- 1 `authorization.py`
- 1 `permission_registry.py`
- 1 `role_permission_service.py`
- 1 `auth.py`
- 1 `templates/system_configuration_role_permissions.html`

Maturity/status: Implemented and critical.

Related docs/ADRs:

- 1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

- 1 Do not bypass `authorization.enforce_route_permission`.
- 1 Do not rely only on UI hiding for protected actions.

Teachers

Purpose: Manage teacher records, qualifications, capacity, workloads, and profile-related academic staffing data.

Main files/folders:

- 1 `routers/teachers.py`
- 1 `teacher_qualifications.py`
- 1 `teacher_capacity.py`
- 1 `models.py`
- 1 `templates/teachers.html`
- 1 `templates/edit_teacher.html`

Maturity/status: Implemented and central to operational value.

Related docs/ADRs:

- 1 `docs/history/workforce-planning/`

Risks/guardrails:

- 1 Teacher data must remain tenant/branch/year scoped.

Subjects

Purpose: Manage subject catalog, colors, qualifications, and planning/timetable relationships.

Main files/folders:

- 1 routers/subjects.py
- 1 subject_colors.py
- 1 models.py
- 1 templates/subjects.html
- 1 templates/edit_subject.html

Maturity/status: Implemented.

Related docs/ADRs:

- 1 docs/history/workforce-planning/

Risks/guardrails:

- 1 Subject changes can affect planning, timetable, teacher qualifications, and reports.

Sections

Purpose: Represent class/section structures used by planning and timetabling.

Main files/folders:

- 1 routers/planning.py
- 1 models.py
- 1 templates/planning.html

Maturity/status: Implemented through planning workflows.

Related docs/ADRs:

- 1 docs/history/workforce-planning/

Risks/guardrails:

- 1 Section changes can affect timetable allocation and workload reporting.

Workforce Planning

Purpose: Plan assignments, homeroom ownership, workloads, subject coverage, and staffing needs.

Main files/folders:

- 1 routers/planning.py
- 1 teacher_capacity.py
- 1 homeroom_defaults.py
- 1 templates/planning.html

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- 1 docs/history/workforce-planning/

Risks/guardrails:

- 1 Planning logic must preserve branch/year scope and avoid accidental cross-year copies.

Timetable

Purpose: Place planned lessons into weekly timetable grids and exports.

Main files/folders:

- 1 routers/timetable.py
- 1 timetable_logic.py
- 1 timetable_version_service.py
- 1 timetable_snapshot_service.py
- 1 timetable_slot_service.py
- 1 timetable_readiness_service.py
- 1 timetable_publication_service.py
- 1 templates/timetable.html
- 1 templates/system_configuration_timetable.html

Maturity/status: Implemented and evolving.

Related docs/ADRs:

- 1 docs/history/workforce-planning/
- 1 docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md

Risks/guardrails:

- 1 Timetable rules interact with planning, sections, subjects, and teacher capacity.
- 1 Planning remains demand/assignment/HRT authority. Placements are version-owned;

the exact-scope active pointer is separate, active/superseded history is immutable, and legacy editing uses a copy-on-write working draft.

- 1 Stage 3.5 owns the canonical composed per-day timeline and exact-scope structural readiness.

The composer outputs ordered teaching/block items and valid teaching slots; after-period blocks shift later clocks while invalid fixed-time placement fails closed. It has no solver, worker, availability, room/resource, or Generate UI.

- 1 Stage 4 owns draft validation, publication transactions, same-scope comparison,

lifecycle presentation, explicit version exports, archive, and draft lock actions.

Academic Calendar

Purpose: Manage academic events, responsibilities, dates, exports, and branch/year scoped calendar views.

Main files/folders:

- 1 routers/academic_calendar.py
- 1 templates/academic_calendar.html

1 `templates/system_configuration_calendar.html`

Maturity/status: Implemented.

Related docs/ADRs:

1 `docs/history/academic-calendar/`

Risks/guardrails:

1 Calendar permissions and branch/year scoping must remain intact.

Observations

Purpose: Support teacher observations, feedback, evidence, scoring, history, and supervision records.

Main files/folders:

1 `routers/observations.py`

1 `templates/observations.html`

1 `templates/observation_form.html`

1 `templates/observation_detail.html`

1 `templates/observation_history.html`

Maturity/status: Implemented.

Related docs/ADRs:

1 `docs/TIS_MASTER_CONTEXT.md`

Risks/guardrails:

1 Observation records are sensitive and must remain tenant scoped.

Reports / Dashboards

Purpose: Expose operational summaries, exports, allocation reports, and decision visibility.

Main files/folders:

1 `main.py`

1 `tenant_report_branding.py`

1 `report/export helper code`

1 `templates/dashboard.html`

Maturity/status: Implemented and evolving.

Related docs/ADRs:

1 `docs/history/workforce-planning/`

Risks/guardrails:

1 Report data must be permission-checked and tenant scoped.

1 Tenant-facing exports may use configured branch/organization logos only. Missing

tenant branding produces no logo and must never fall back to the TIS product mark.

Branding / Design Settings

Purpose: Manage organization logos, design settings, visual shell behavior, and platform visual controls.

Main files/folders:

```
1 branding_storage.py
1 design_tokens.py
1 tenant_report_branding.py
1 visual_design.py
1 ui_shell.py
1 static/css/branding.css
1 static/css/design-studio.css
1 templates/system_configuration_logos.html
1 templates/system_configuration_design.html
```

Maturity/status: Implemented.

Related docs/ADRs:

```
1 docs/adr/0007-landing-page-visual-system-strategy.md
```

Risks/guardrails:

```
1 Do not confuse operational app branding with public landing website design.
1 Protect uploaded/owned assets.
1 Keep product-surface branding separate from tenant-facing report branding.
```

Landing Website

Purpose: Public marketing and conversion surface for TIS.

Main files/folders:

```
1 tis-landing-website/
1 docs/marketing/
```

Maturity/status: Implemented as separate Next.js app.

Related docs/ADRs:

```
1 docs/adr/0001-separate-nextjs-landing-website.md
1 docs/adr/0007-landing-page-visual-system-strategy.md
1 docs/marketing/landing_page_source_of_truth.md
```

Risks/guardrails:

```
1 Do not modify landing code during operational/backend tasks unless explicitly approved.
```

- 1 Legacy FastAPI landing files are not the public source of truth.

AI Future Roadmap

Purpose: Future subscription-gated AI-assisted planning, analytics, assessment generation, recommendations, and decision support.

Main files/folders:

- 1 No dedicated AI production module yet.
- 1 Future work should be documented before implementation.

Maturity/status: Roadmap / future.

Related docs/ADRs:

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 future ADRs required before major AI architecture decisions.

Risks/guardrails:

- 1 AI features must use verified tenant data and preserve privacy, permissions, and subscription boundaries.

TIS Repository Architecture

docs/engineering/REPOSITORY_ARCHITECTURE.md

Published Timetable Consumption Boundary

`timetable_visibility_service.py` validates exact scope and resolves official data only through `TimetableActiveVersion`; it never uses operational draft resolution. `routers/timetable.py` retains management workflow, while separately mounted `/my-timetable` and `/published-timetable` routes expose filtered published payloads. `timetable_version_service.py` owns locked, history-preserving working discard.

Smart Timetable Generation Boundary

`routers/timetable.py`, `timetable_workflow_dispatch.py`, and `timetable_generation_service.py` are the web-safe enqueue/dispatch/status/cancel boundary and never import OR-Tools. `timetable_problem_builder.py` builds solely from captured snapshot JSON. `timetable_generation_workflow.py` registers the on-demand Render task and imports the reusable single-run executor from `timetable_generation_worker.py`; its dependencies are in `requirements-workflow.txt`. The polling worker command remains local/fallback only.

The web commits a durable queued run before requesting one Render task and marks an unclaimed run terminal if dispatch fails. The task claims only that public ID, maintains the existing lease and heartbeat, solves, and asks the independent validator to verify it. Persistence rechecks current input/source revision, then creates the version, entries, linkage, and success in one transaction. The browser polls only TIS state. Render retries are explicitly zero; duplicate invocation is a safe no-op after the PostgreSQL claim or terminal result.

Customer Feature Policy Boundary

`saas/customer_feature_policy.py` classifies the normal organization product baseline and explicitly excludes internal audit capability. `saas.entitlement_service.evaluate_feature_access` composes existing identity, permission, commercial-state, workspace-entitlement, branch-entitlement, and scope authorities. Routes and templates consume this one decision rather than branching on paid, promo, demo, or plan names.

`saas/ai_entitlement_service.py` consumes that availability decision. `saas/ai_consumption_policy.py` separately returns metric context and usage limit, preserving reservations and usage accounting. This split is intentionally small while no provider-backed AI route exists.

Migration and activation writes remain in `db_migrations.py` and `saas/promo_redemption_service.py`; feature policy never owns capacity, payment, permission assignment, or immutable promo evidence.

SchoolGroup Configuration Boundary

`main.py` owns the operational School Management route boundary. Direct top-level SchoolGroup creation and deletion require Platform identity plus `schools.manage_all_schools` and the matching action permission; tenant updates are limited to the linked SchoolGroup. `permission_registry.py` keeps organization create/delete out of tenant role assignments. Templates consume explicit server-side capabilities and never substitute UI hiding for handler authorization.

School Management branch capacity is a presentation adapter over `saas/commercial_authority_service.py`; it must not call a paid-only resolver or recalculate promo usage. Mutation routes remain the authority for locked capacity enforcement and atomic promo evidence. `scripts/audit_schoolgroup_provenance.py` is an operator-only PostgreSQL read-only diagnostic. It emits no names or contact data, rolls back on exit, and owns no remediation behavior.

Existing Workspace Controlled Conversion Layer

Existing Workspace Paid Activation Boundary

`saas/existing_workspace_paid_activation_service.py` owns eligibility, operational capacity recount, plan/branch quote snapshots, idempotent preparation, workspace Paddle-customer association, transaction launch, and webhook-only activation for an existing `SchoolGroup`. It reuses `saas/billing_identity_service.py`, `saas/paddle_client.py`, and the centralized commercial resolvers. It never calls the tenant provisioning engine and never creates a `PendingOrganization`.

`ExistingWorkspacePaidActivation` is the durable authority context. `CheckoutSession` and `PaymentAttempt` accept exactly one of the established `PendingOrganization` context or this context. `SchoolGroup`-anchored billing profiles, contracts, and subscriptions support the existing workspace without weakening onboarding constraints. Payment-customer workspace associations retain the exact provider address and business lineage used by each `SchoolGroup`.

The shared Paddle webhook dispatcher recognizes this context before onboarding and subscription-mutation handlers. Only matching `transaction.completed` evidence can create paid `WorkspaceEntitlement`, `BranchEntitlement`, `PaymentSubscription`, and the paid `TenantProvisioningLink`. Workspace classification remains customer and the lifecycle becomes active. A transaction is released only after complete provider billed-state and quote-lineage validation. Locking queries refresh mapped state before concurrent idempotency checks.

`saas/existing_workspace_conversion_service.py` owns M4B preparation, verified owner alignment, setup validation, fresh-audit comparison, row locking, idempotency, and the final activation-required transition. It composes M4A, normal SaaS verification, provisioning owner identity conventions, M1, and M3; it does not duplicate those responsibilities. `ExistingWorkspaceConversionOperation` is the durable claim/state ledger and `ExistingWorkspaceConversionEvent` is an append-only redacted event stream.

`scripts/convert_existing_workspace.py` is dry-run by default. PostgreSQL write mode requires Platform Owner actor IDs plus an exact `PREPARE <operation UUID>` or `CONVERT <operation UUID>` phrase. Preparation requires the current complete M4A hash. Final execution performs another M4A audit in the locked transaction and compares stable branch/dependency and sandbox-entitlement evidence to the operation baseline; intended owner and setup changes are the only expected differences. The CLI never creates passwords, sends email, calls Paddle, or redeems promo codes.

Customer routes under `/saas/existing-workspace/` expose only verified claim and three-field setup review. `commercial_state_service`, `commercial_access_service`, and `commercial_authority_service` recognize the specific customer / provisioning / no source result as fail-closed `activation_required`. The ordinary subscription portal redirects this state back to Organization Account because existing-workspace Paddle activation is not yet implemented.

Existing Workspace Conversion Audit Layer

`saas/existing_workspace_conversion_audit_service.py` is a query-only evidence collector. It accepts an exact `SchoolGroup` ID, workspace UUID, organization name, and owner email; reflects the connected schema; follows branch foreign keys recursively; identifies unconstrained branch-like columns; and returns allowlisted identity, lifecycle, ownership, provisioning, and commercial metadata. It does not own or commit a transaction and exposes provider references only as presence flags.

`scripts/audit_existing_workspace_conversion.py` is the manual production entry point. It requires PostgreSQL and `DATABASE_URL`, establishes `REPEATABLE READ ONLY`, invokes the service, emits one sanitized JSON or text report, then rolls back and closes. The service canonicalizes ordered evidence into a stable SHA-256 snapshot, classifies soft-deleted rows without ignoring them, compares reflected branch foreign keys with ORM metadata, and emits archival candidates only when each referenced count is zero. The CLI reports transaction

settings and uses exit 0 for coherent evidence, 1 for execution/configuration failure, 2 for manual review, and 3 for workspace identity mismatch. The script is not imported by application runtime. Audit readiness means only that later conversion design may proceed; hard deletion and write conversion remain explicitly unapproved.

Promo Redemption Layer

`saas/promo_redemption_service.py` is the write boundary between secure M2 definitions and operational commercial authority. It accepts either an owned completed `PendingOrganization` or an existing aligned `SchoolGroup` with a tenant-owner account link. The latter path deliberately creates no synthetic onboarding, contract, subscription, payment, or demo row.

Final activation locks session, definition, and workspace in stable order and commits immutable `PromoRedemption/PromoGrant`, branch assignments and entitlements, `promo WorkspaceEntitlement`, `promo TenantProvisioningLink`, lifecycle updates, and audit together. `saas/promo_grant_service.py` resolves effective grants and expiration and owns atomic assignment/entitlement activation when a branch is created or reactivated; M1 composes that result. `auth.py` restricts customer-classified operational branch queries to explicit active promo branch entitlements. Pending or inconsistent evidence fails closed.

Existing-branch selection does not alter `Branch.status`. Staff and teachers are organization-wide validation inputs and are never selected, disabled, or deleted. Normal future branch creation may consume only an unused grant branch slot. Paddle remains outside this layer.

`saas/promo_branch_entitlement_reconciliation_service.py` is the conservative repair boundary for legacy incomplete promo evidence. It requires one resolved active promo authority, tenant source, owner, and workspace entitlement; inventories all branches, assignments, and entitlement rows; and can create only a missing inactive entitlement for an unassigned branch. The PostgreSQL CLI defaults to dry-run, locks and revalidates on apply, and commits or rolls back as one unit.

`saas/commercial_portal_service.py` is the read-only customer presentation adapter for promo authority. It consumes `commercial_authority_service` rather than recounting capacity, then resolves the attributable `PromoGrant` and `PromoRedemption` only for immutable dates and the masked promo reference. The SaaS subscription route selects this adapter by authoritative source before invoking the paid portal. Promo, paid, and demo view models remain separate; Jinja templates do not decide commercial authority. The promo view exposes the existing-workspace paid continuation entry only when `existing_workspace_paid_activation_service` proves an expired or recovery-period promo source and verified tenant owner.

`promo_grant_service` derives active, recovery, and expired time states. Recovery is presentation/conversion eligibility only; commercial access remains blocked from the exact expiry timestamp.

`existing_workspace_paid_activation_service` owns the locked, provider-confirmed promo-to-paid transition. It keeps promo authority during checkout and atomically moves the existing `TenantProvisioningLink`, workspace entitlement, and branch entitlements to the confirmed paid contract/subscription at completion. No replacement tenant or synthetic onboarding organization is created.

`saas/commercial_badge_service.py` is the sole commercial identity presentation adapter. It accepts centralized commercial access, rejects unresolved placeholder sources, and returns semantic source, status, icon, and plan tokens for the reusable templates/`_commercial_badge.html` component. Organization Account and commercial access use the shared presentation; the operational shell deliberately performs no commercial-badge lookup or rendering.

Promo Definition Layer

Promo administration is an isolated SaaS/Platform layer. The router requires a Platform identity plus granular promo permission; the service owns validation, secure generation, lifecycle locking, and safe audit; models and migration own only definition/history persistence. Templates receive raw code only in the immediate create/duplicate/replace response and all later views use a masked representation.

The M3 commercial adapter contract is implemented as a grant identity, tenant and organization identity, source=promo, status, plan, effective window, three capacity limits, selected branch IDs, immutable snapshot, resolution status, and reason code. M2 definition operations still provide no access; only a completed M3 grant is authoritative.

Three-Dimension Subscription Capacity Authority

saas/branch_pricing_quote_service.py owns pre-payment plan eligibility and quote capacity across active billable branches, system-user estimates, and teacher estimates. PendingOrganizationBranch owns onboarding estimated_system_users and estimated_teachers; organization-level legacy values are migration inputs and derived compatibility summaries, not an independent source of truth.

Before payment, each people count is the greater of the sum across active onboarding branches and actual active data in the uniquely linked SchoolGroup. After activation, saas/commercial_authority_service.py is the single facade. Staff usage is every distinct active tenant operational User, including operational owners, teacher-position users, and internal-test-attributed tenant users. Platform users, inactive users, account-only SaaS identities, and other tenants do not count. Teachers are deduplicated by normalized Teacher.teacher_id across active branches and active academic years; every blank legacy teacher identity counts separately.

The decision is consumed by selection, quote construction, checkout preparation/launch, Paddle creation, payment reconciliation, and plan changes. Both people counts are part of quote fingerprint authority. Branch estimate changes clear an undersized selected plan, invalidate its quote, mark ready or started checkout sessions stale, and supersede incomplete payment attempts. Operational branch, staff-user, teacher, and academic-year growth locks the SchoolGroup, composes the existing commercial authorities, recounts, evaluates the proposed final state, and mutates in the same transaction. Paid and demo provisioning perform the same final invariant. Downgrade impact still reports branch, system-user, and teacher conflicts independently. No capacity failure silently deletes or deactivates operational data.

Paid limits are provider-confirmed subscription quantity capped by the plan branch ceiling plus the plan's staff and teacher limits. Customer Demo and Internal Sandbox are explicitly unmetered in M1 when their existing authority is coherent. Missing or contradictory authority fails closed. Existing over-capacity tenants preserve data and access but cannot further increase an exceeded dimension.

The current Teacher model has no inactive/reactivation lifecycle, and the repository has no teacher import endpoint. Capacity enforcement therefore covers the supported teacher-growth paths: individual creation and atomic academic-year copying. Adding teacher deactivation/reactivation or import workflows remains a separate lifecycle design task.

Starter persists 1 branch/5 system users/25 teachers, Professional 5/20/100, and Enterprise AI 25/100/500. The lowest plan passing all dimensions is recommended while any higher eligible plan remains selectable. The in-app Custom card is informational and mail-based; exceeding any Enterprise limit emphasizes it, but it creates no catalog plan, Paddle transaction, or enterprise-request workflow.

Subscription Plan Branch-Capacity Authority

saas/branch_pricing_quote_service.py evaluates the authoritative active billable branch count against SubscriptionPlan.max_branches. Starter, Professional, and Enterprise AI retain persisted capacities of one, five, and twenty-five. The evaluator supplies both server-side rejection and plan-page eligibility metadata; pricing remains the unit price multiplied by the actual active count.

Plan selection and every checkout/payment phase consume a fail-closed quote, so an undersized plan cannot produce a checkout or Paddle transaction. A pre-payment branch change clears an undersized selection while preserving an eligible higher plan, and the existing checkout-supersession boundary rejects stale transaction completion. Counts are resolved only inside the current pending organization or its same-workspace demo provisioning.

Paddle Checkout Quantity Authority

`saas/branch_pricing_quote_service.py` supplies the authoritative billable branch quantity and total to `saas/payment_service.py`. The payment service sends that exact quantity with the selected catalog price and resolved customer address when creating an automatically collected Paddle transaction. The transaction must reach ready; returned item quantity, provider price, subtotal, and quote fingerprint are validated before TIS marks it billed. Only the resulting immutable transaction is released to checkout.

The public `/saas/payment` launcher opens the pre-created transaction with Paddle's inline one-page checkout settings and `transactionId`; it never passes an editable items array. The billed transaction is a fixed financial record, so item and quantity changes are rejected by Paddle. Automatic `transaction.completed` webhook evidence remains the recurring-subscription and conversion authority.

Pre-payment Branch Setup remains authoritative even when an unpaid demo workspace or tenant link already exists. Only confirmed payment evidence or an active paid subscription closes onboarding mutation. A branch identity or count change marks ready/started checkout sessions stale, marks their incomplete payment attempts superseded, clears quote snapshots, and returns the organization to plan selection. Late events for superseded transactions are retained for manual review but cannot change the current organization, subscription, provisioning, or workspace state.

Before rendering Paddle.js, the launcher requires exactly one local payment attempt and matching checkout session, organization, customer, quote, and provider transaction. Remote status must be billed with automatic collection; draft, ready, canceled, past-due, unrelated, or mismatched transactions fail closed with a customer-safe message.

Checkout Tenant-Identity Recovery Boundary

`saas/payment_service.py` owns Paddle customer identity resolution. For an existing demo workspace, a stale `SaaSAccountUserLink` may be reassigned to the authenticated account only when organization ownership, demo request and provisioning source, the unique `TenantProvisioningLink`, `SchoolGroup`, exact operational owner, and email identity all agree. The current account must have no conflicting workspace link, and any previous linked account must be absent or inactive. Missing, unrelated, active-previous-owner, cross-workspace, or multiple identity evidence fails closed.

This recovery does not select a Paddle customer by email in live mode, bypass provider confirmation, provision a tenant, or create another `SchoolGroup`. Internal match diagnostics are logged; customer routes receive only retry and support guidance.

Returning Customer And Commercial-Access Boundary

`saas/customer_journey_service.py` owns customer login destinations and the expired-demo subscription projection. It reads onboarding/request records, the durable SaaS-account operational link, demo lifecycle, `SchoolGroup` classification, actual active Branch rows, public plans, and subscription evidence. It also resolves linked organization-account management scope from the existing `SaaSAccountUserLink`, operational user, ownership, and permission records. Activated account managers land on `/saas/account`; ambiguous multi-organization identities select an organization there. The selected UUID is an HTTP-only request hint that is revalidated against `SaaSAccountUserLink`, the operational user, ownership, and permissions before `saas.entitlement_service` may resolve that `SchoolGroup`. Linked users with no account-management permission retain the existing role-based operational destination. `saas/commercial_access_service.py` is the operational access projection used after credential validation, by customer journey routing, and by request authorization. For Customer Paid workspaces it consumes the existing `saas.entitlement_service` tenant-link and `SubscriptionContract`-linked subscription resolution; it never chooses the newest organization-level row. It adds workspace lifecycle, pending-change context, state-specific messaging, and the allow/deny decision without replacing payment, entitlement, plan-change, or demo-lifecycle authorities. Terminal or unresolved plan-change requests do not replace the current paid entitlement.

`saas/router.py` applies that resolver after password and social login, for an already-authenticated sign-in page, and from the SaaS root. The Organization Account Overview is an account-management boundary, not an operational bypass: it filters sections by existing permissions, suppresses operational entry when commercial access is restricted, and exposes `/login` only through Enter TIS Platform. Operational authentication and request authorization remain separate authorities.

`/saas/subscription/demo/select` records conversion intent and reuses existing plan selection and checkout routes; it never provisions or copies a second workspace. Public navigation remains in `tis-landing-website/` and builds app routes exclusively from `NEXT_PUBLIC_TIS_APP_BASE_URL`.

M8B9 Demo Operations Boundary

The owner HTTP surface delegates mutations to `saas/demo_operations_service.py`, which validates and locks the Customer Demo context, calls existing lifecycle and communication services, and writes audit state. Access decisions resolve through `saas/demo_access_service.py` before the M8B8 counter. Workspace policy is the default; branch policy requires same-tenant validation. M8B9 DDL runs only through `python scripts/run_migrations.py` during pre-deploy.

M8B8 AI Entitlement Boundary

AI route code must never inspect classifications, plans, or usage rows directly. It requests a decision from `saas.ai_entitlement_service`, performs the real AI operation only when allowed, and records consumption only after a usable result. The registry is code-controlled configuration. Commercial state and permissions remain upstream authorities; accounting is downstream and tenant-scoped.

M8B7 Communication Boundaries

Demo email intent creation and dispatch belong to `saas/demo_email_service.py`; Platform Owner Notification Center creation belongs to `saas/demo_notification_service.py`. Routes orchestrate existing services but do not own lifecycle calculations. Provider calls occur outside lifecycle locks. The shared shell consumes the read-only lifecycle resolver and never becomes lifecycle authority.

This document explains the main repository areas, what each owns, and what must not be changed casually.

Root FastAPI Application

`main.py`

Responsibility: Primary FastAPI app, route registration, many app-level workflows, middleware, startup checks, platform console routes, dashboards, exports, system configuration, scope switching, and operational pages.

Render boot boundary: The ASGI web process never performs SQLAlchemy metadata creation or pending migrations. `scripts/run_migrations.py` is the sole deployment command for baseline schema creation plus `db_migrations.run_pending_migrations`; Render runs it as the Pre-Deploy Command and activates the new version only after exit code zero. `main:app` startup retains only security configuration validation and process-local upload-directory creation, so Uvicorn binds without PostgreSQL DDL. There is no migration daemon or schema-readiness middleware.

Do not change casually:

- 1 authentication/session flow,
- 1 platform owner access checks,
- 1 tenant scope handling,
- 1 route behavior shared by many modules,

- 1 startup/schema repair behavior.

auth.py

Responsibility: Password hashing/verification, session cookies, current-user lookup, role normalization, platform identity helpers, permission helpers, email verification helpers, and tenant/platform identity boundaries.

Do not change casually:

- 1 is_platform_owner, is_platform_user, is_platform_developer,
- 1 role normalization,
- 1 session cookie behavior,
- 1 permission helper semantics.

authorization.py

Responsibility: Protected route rules, permission enforcement middleware integration, access-denied responses.

Do not change casually:

- 1 route permission mapping,
- 1 public path patterns,
- 1 permission matching semantics.

database.py

Responsibility: Database engine/session setup.

Do not change casually:

- 1 database URL handling,
- 1 session creation behavior,
- 1 engine configuration.

models.py

Responsibility: Primary operational SQLAlchemy models for users, school groups, branches, teachers, planning, timetable, observations, configuration, and related app data.

Do not change casually:

- 1 tenant ownership fields,
- 1 platform user fields,
- 1 relationships used by existing workflows,
- 1 schema without matching migration/repair strategy.

db_migrations.py

Responsibility: Local schema migration and repair logic used by the app.

Do not change casually:

- 1 migration ordering,

- 1 destructive schema operations,
- 1 production-sensitive repair behavior.

Timetable Version And Snapshot Services

`timetable_version_service.py` owns exact-scope version resolution, draft creation/copying, mutable-state enforcement, placement mutation, lock metadata, archival foundation, next-number allocation, and imported active-pointer setup. It does not own Planning demand rules.

`timetable_snapshot_service.py` converts current Planning/settings/lock authority into deterministic schema-versioned canonical JSON and SHA-256 component/full fingerprints without teacher display names or other unnecessary personal data. It is a reusable foundation, not a readiness checker or solver.

`timetable_slot_service.py` composes the canonical per-day teaching/block timeline used by snapshots and operational consumers. `timetable_readiness_service.py` performs a read-only exact-scope completeness evaluation using Planning, HRT, teacher capacity, slots, locks, and existing fingerprints. Neither service executes a solver.

`timetable_publication_service.py` owns version-specific hard validation, same-scope comparison, and atomic active-pointer publication. Routes enforce permissions and explicit scope; the service enforces lifecycle, revisions, freshness, hard validity, and transaction invariants. It never generates placements.

`permission_registry.py`

Responsibility: Permission groups, labels, default role permissions, system owner permissions, developer-assignable permission boundaries.

Do not change casually:

- 1 owner/developer permissions,
- 1 defaults for managed roles,
- 1 permission keys referenced by routes/templates/tests.

`role_permission_service.py`

Responsibility: Persistence and service helpers for role permission rows.

Do not change casually:

- 1 global vs school-scoped permission behavior,
- 1 role permission constraints.

`ui_shell.py`

Responsibility: Shared application shell, navigation, page metadata, scoped organization/year context display, logos, visual design CSS, and permission-based nav generation.

Do not change casually:

- 1 navigation visibility,
- 1 platform-vs-tenant shell behavior,
- 1 design-studio gating,
- 1 scope display.

knowledge_service.py And kms_catalog.py

Responsibility: knowledge_service.py reads the generated KMS manifest and approved Markdown metadata for the owner-only Knowledge Center. kms_catalog.py provides the dependency-free approved category/module vocabulary and deterministic source-path classification shared by the service and KMS validation tooling.

Do not change casually:

- 1 owner-only read behavior,
- 1 manifest freshness semantics,
- 1 approved category or module slugs,
- 1 path classification without matching KMS tests and documentation.

Feature Routers: routers/

Responsibility: Modular operational route handlers:

- 1 routers/users.py
- 1 routers/teachers.py
- 1 routers/subjects.py
- 1 routers/planning.py
- 1 routers/timetable.py
- 1 routers/academic_calendar.py
- 1 routers/observations.py

Do not change casually:

- 1 tenant/branch/year scoping,
- 1 permission checks,
- 1 import/export behavior,
- 1 bulk operations.

SaaS Package: saas/

Responsibility: Public SaaS account flows, onboarding, plans, billing, Paddle integration, pending organizations, and provisioning.

Key files:

- 1 saas/router.py
- 1 saas/service.py
- 1 saas/models.py
- 1 saas/pricing_service.py
- 1 saas/payment_service.py
- 1 saas/paddle_client.py
- 1 saas/billing_service.py
- 1 saas/provisioning_service.py

- 1 saas/currency_service.py
- 1 saas/oauth.py
- 1 saas/entitlement_service.py
- 1 saas/subscription_portal_service.py
- 1 saas/subscription_change_service.py
- 1 saas/subscription_plan_change_service.py
- 1 saas/subscription_cancellation_service.py
- 1 saas/subscription_lifecycle_service.py
- 1 saas/billing_history_service.py
- 1 saas/payment_lifecycle_reconciliation_service.py

Do not change casually:

- 1 identity separation,
- 1 payment confirmation rules,
- 1 provisioning readiness,
- 1 Paddle/webhook boundaries,
- 1 public signup/onboarding state transitions.

Templates: templates/

Responsibility: Jinja templates for operational app, platform console, Knowledge Center, SaaS pages, system configuration, and feature workflows.

Do not change casually:

- 1 form action routes,
- 1 hidden scope fields,
- 1 permission-dependent controls,
- 1 app shell extension patterns.

Static Assets: static/

Responsibility: Operational CSS/JS, images, branding assets, generated documentation PDF and manifest.

Important:

- 1 static/docs/TIS_Project_Reference_Booklet.pdf is a generated snapshot.
- 1 static/docs/docs_manifest.json is generated metadata.

Do not change casually:

- 1 generated docs manually,
- 1 shared CSS without checking all templates,
- 1 protected document access assumptions.

Tests: tests/

Responsibility: Regression coverage for SaaS phases, tenant isolation, platform access, permissions, email, branding, and critical workflows.

Do not change casually:

- 1 tests should follow behavior, not hide regressions.
- 1 update tests when behavior intentionally changes.

Docs: docs/

Responsibility: KMS source of truth, engineering handbook, ADRs, change history, module history, AI context, marketing docs.

Do not change casually:

- 1 historical records should preserve old/new state.
- 1 update docs through the KIA process.

Scripts: scripts/

Responsibility: Maintenance, diagnostics, and governance scripts. Important examples:

- 1 `scripts/generate_docs_pdf.py`: generate or read-only validate KMS PDF/manifest artifacts.
- 1 `scripts/check_kms_impact.py`: compare KIA declarations, Git changes, major-path classification, and artifact freshness.
- 1 `scripts/kms.py`: supported Phase 6 command surface for one-step synchronization and complete read-only validation; delegates to the generator and checker.
- 1 `scripts/sync_paddle_price_ids.py`: environment-specific initial checkout price mapping.
- 1 `scripts/diagnose_paddle_plan_preview.py` and `scripts/diagnose_payment_lifecycle.py`: safe subscription/payment diagnostics.
- 1 `scripts/reconcile_finalized_payment_lifecycle.py`: guarded sandbox reconciliation from attributable provider evidence.

Do not change casually:

- 1 PDF generator dependency assumptions,
- 1 source list behavior,
- 1 manifest metadata.
- 1 title, catalog, navigation-link, source-inventory, and PDF page-bound validation.

Repository Governance

Responsibility: Make KMS impact visible and enforceable without rewriting documentation.

Key files:

- 1 `AGENTS.md`
- 1 `.kms-impact.yml`

- 1 .github/pull_request_template.md
- 1 .github/workflows/kms-enforcement.yml
- 1 .github/workflows/deploy-on-master.yml

Do not change casually:

- 1 major-path classification,
- 1 explicit no-impact override requirements,
- 1 authoritative Markdown/path rules,
- 1 deployment dependency on KMS validation,
- 1 prohibition on customer/runtime/secrets data in documentation.

Next.js Landing Website: `tis-landing-website/`

Responsibility: Public marketing website at <https://tisplatform.com>.

Do not change casually:

- 1 landing implementation during backend tasks,
- 1 visual system without approval,
- 1 public assets/copy without checking marketing docs and ADRs.

TIS User And System Flows

docs/engineering/USER_AND_SYSTEM_FLOWS.md

Talent Assessment Cycle Population Flow

- 1 An authorized Draft author creates a SchoolGroup-wide Cycle and explicitly selects Program, Academic Year, exact Framework Version, and `population_effective_at`.
- 1 Draft preview resolves effective Academic Placements and annual eligible grades dynamically. Organization population readers see the whole preview; Branch readers see only authorized Placement branches and a subset count.
- 1 An organization/global governor opens the Draft. The server locks and revalidates it, derives the complete SchoolGroup population, inserts exact frozen historical member snapshots, stores full count/fingerprint, changes status to Open, and audits the event in one transaction.
- 1 Open/Closed reads use frozen Branch context. Branch readers never receive the full count/fingerprint; later Student transfers do not change their historical visibility. Close is final. Reopen and late population mutation are not available.

Configure And Generate With Teacher Scheduling Rules

- 1 An administrator with `timetable.manage_teacher_rules` opens System Configuration ? Timetable Settings ? Teacher Scheduling Rules in one exact SchoolGroup, branch, and academic-year scope.
- 1 The administrator selects a teacher, Schedule within/Must teach/Unavailable, all or selected working days, numbered periods, and Select All or selected Current/New Planning sections. Grade and first/last controls are not part of the normal form.
- 1 The service validates scope and rule shape, rejects deterministic hard contradictions, persists normalized rule/slot/target rows, and marks only unpublished Drafts stale while clearing approval. Published history is not modified.
- 1 Generate or Regenerate captures canonical rules in a schema-v4 immutable snapshot. Readiness and the problem builder reject invalid targets, impossible workload/availability, conflicting hard rules, and unavailable-period locks.
- 1 CP-SAT confines existing eligible demand to Schedule-within windows, enforces Must teach occupancy and Unavailable slots, and continues scoring saved teaching/free preferences. No rule creates demand and hard rules never relax.
- 1 The independent validator repeats window, required-occupancy, and unavailability checks and reports preference satisfaction. Only a valid, current-fingerprint candidate may become a new unpublished Draft.

- 1 Select All mirrors every visible Current/New section and saved-rule summaries

show either exact grade-section names, All assigned sections, or All classes for teacher-wide Unavailable rules. Editing restores the same literal scope.

- 1 Manual Draft placement rejects hard Unavailable and Schedule-within violations

before writing. Complete Draft validation also detects missing Must-teach occupancy and wrong selected-section occupancy; these hard blockers prevent approval and publication, while preferences never block lifecycle actions.

- 1 Multiple Schedule-within rows that apply to the same scoped demand contribute

slots to one combined allowed window. Before solving, TIS matches every teacher's effective workload to those combined windows and reports provable capacity or Subject Distribution Rule conflicts, including unavailable slots, Must-teach slots, daily coverage, minimum/maximum days, and double blocks.

- 1 When the full solver alone proves infeasibility, the Workflow runs bounded

family-isolation solves over base collisions, locks, grouped activities, subject rules, teacher rules, and their interaction. It persists the proven customer-safe category and lock/group counts on the same Generation Run; it never saves a relaxed candidate or changes the original hard constraints. The Workflow uses `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` (default 60) independently from the primary solve and skips absent lock/group profiles.

Guided Curriculum Adjustment UI

- 1 A user with `curriculum.adjust` opens **Adjust Curriculum** from Planning or Subjects.
- 2 The user chooses grade, selected Current/New sections, or all active source uses, then chooses source, target, and the proposed source reduction.
- 3 The browser requests the Stage 3 preview and presents section demand changes, teacher/capacity impact, rule and grouped warnings, blockers, and Draft regeneration impact.
- 4 The user explicitly chooses an eligible, within-capacity teacher or **Leave unassigned** for every affected section. Nothing is preconfirmed.
- 5 Final review requires confirmation and submits the unchanged preview fingerprint to Stage 4 apply.
- 6 If Planning changed, the preview is refreshed and the user reviews again; compatible teacher selections are preserved where possible.
- 7 Success reports affected sections and Draft status and offers Timetable/regeneration links. Regeneration is never automatic.

The period count entered in step 2 is the transfer amount for every selected section. Preview rejects zero/negative transfers and any section whose current source demand is smaller than the request. A partial transfer leaves its remaining source demand active. After apply, Subjects shows the effective Planning value when uniform; differing section values show **Varies** and can be expanded by section. Catalog weekly hours remain secondary defaults.

For **Reduce periods only**, the user selects no target subject. Preview shows the source before/after demand, source teacher load reduction, rule/grouped warnings, and Draft impact. No teacher reassignment is requested. Apply changes only the selected source demands, retires only zero results, and otherwise follows the same fingerprinted atomic transaction. Subject Details can open this flow prefilled for its one grade-specific subject. Multi-grade selection remains outside this single-subject workflow.

Atomic Curriculum Adjustment Apply

- 1 A user with curriculum.adjust submits the exact reviewed request, Stage 3 fingerprint, and one explicit target-teacher decision per section.
- 1 The service owns one transaction, locks exact SchoolGroup/branch/year authority, and rejects an active generation run.
- 1 It rebuilds the preview and rejects stale fingerprints, preview blockers, incomplete decisions, invalid qualification/capacity, rule conflicts, or Draft locks that cannot survive the proposed demand/teacher state.
- 1 It updates only selected Current/New source/target demands and confirmed teacher assignments. Source zero becomes inactive retirement evidence and any active source section rule is inactivated.
- 1 It marks only the current unpublished Draft stale, clears approval, preserves its snapshot/placements/history, and records one durable applied audit.
- 1 All changes commit together. Any failure rolls back the entire adjustment.
Regeneration remains a later explicit action; published history is untouched.

Read-Only Curriculum Adjustment Preview

- 1 An authorized Planning editor selects grade, selected sections, or all active source uses and identifies source/target subjects plus proposed source periods.
- 1 The service resolves only Current/New sections in the selected SchoolGroup, branch, and academic year and reads explicit-first section demand.
- 1 It projects released/target periods, current teachers, uncommitted teacher options and capacity, scheduling-rule arithmetic, grouped warnings, and Draft stale/regeneration impact.
- 1 It returns blockers/warnings and a deterministic preview fingerprint. No row is changed and no teacher is automatically selected.
- 1 A future apply stage must rebuild and compare the fingerprint before any atomic mutation. Published timetable history remains outside the write path.

Simplified Timetable And Official Visibility

- 1 A manager creates or opens the exact-scope Draft Timetable and configures inputs until Configuration Complete.
- 2 The manager runs Verify Feasibility. The durable Workflow solves the exact immutable snapshot with every hard constraint and no soft objective, then independently validates one complete solution.
- 3 Only a validated result produces Feasibility Verified and enables Generate Timetable. Infeasibility exposes the isolated hard-rule family; timeout remains inconclusive and cannot enable generation.
- 4 The verified placements are retained by full input fingerprint. Any Planning, allocation, rule, slot, lock, grouped-resource, or other snapshot-authority change requires verification again.

- 5 Full generation optimizes quality from the verified placement hint. If optimization times out, the worker independently validates and persists the verified placement as the Draft fallback instead of returning no timetable.
- 6 When authority changes after a Draft exists, the Draft remains stale,

versioned, and unpublishable. If current inputs have no genuine structural blocker, the state is Draft Needs Regeneration and still permits Verify Feasibility against a fresh immutable snapshot. A verified current fingerprint then permits Regenerate; the stale source is never rewritten.
- 1 The regeneration source is the current populated, mutable, unpublished Draft,

whether its origin is manual, generated, or regenerated. An empty starter Draft has no prior arrangement and therefore follows Generate. Active and historical versions cannot be selected directly as regeneration sources.
- 1 Generate and Regenerate operate against that current draft context; Regenerate retains Stage 5.1 locks and concurrency.
- 2 Approve Draft validates that exact draft and records the reviewing administrator;

generation alone never grants approval. Regenerate is presented beside Approve Draft and requires at least 25 percent of unlocked source placements to change, rounded up.
- 1 Any placement, drag/drop, swap, lock, regeneration, or stale authority change

invalidates approval and requires review again.
- 1 Publish Timetable requires approval, revalidates the draft, atomically makes it

official, and retains the prior publication in history.
- 1 /my-timetable resolves exact-scope teacher identity, reads only the active pointer,

and filters to that teacher. View-only non-teachers use the general published page. Destructive and technical version actions remain in Timetable History.

Generate And Regenerate Timetable Flow

Academic Scheduling Quality is configured using explicit subject codes and optional Swimming section groups. Saved branch/year rules enter the immutable run snapshot. CP-SAT applies hard daily core coverage when demand permits, optional ICT one-per-day, simultaneous group/resource constraints, and soft distinct-day/non-consecutive preferences. Independent validation repeats hard rules before persistence.

- 1 Require `timetable.generate`, exact tenant branch/year scope,

`generation_ready`, a current mutable draft context, and no active run.
- 1 Select Generate for a fresh/manual current draft; select Regenerate for a current

generated candidate and freeze the source revision, arrangement, and locks.
- 1 Capture schema-v3 canonical input and fingerprint, commit one durable run, then

dispatch its public ID to the configured Render Workflow task. Immediate dispatch failure ends the still-unclaimed run safely and permits Generate Again.
- 1 The on-demand task claims that exact run, leases it, builds from only the snapshot,

solves with CP-SAT, and records Building, Solving, Checking, and Saving phases.

1 The independent validator checks exact demand, scope, Planning/HRT authority, canonical slots, collisions, locks, fingerprint/source revision, and diversity.

1 Rebuild current authoritative input. Changed inputs end as `stale_input` and create no version.

1 Atomically create one unpublished generated/regenerated publication-ready version, its entries, links, and succeeded run state. Do not change the active pointer.

1 Polling selects the result for review. Validation and Publish remain separate.

Cancellation ends queued work immediately or requests cooperative cancellation of leased work. Reload resolves the active durable run. Infeasibility and timeout create no version. A queued task delayed beyond the configured threshold reports that it is waiting for compute. The browser never polls Render. Regeneration is unavailable when all source lessons are locked.

Common Customer Feature Decision Flow

1 Resolve the authenticated operational user and authorized SchoolGroup.

2 Require the registered user permission; denial ends the decision.

3 For branch-scoped actions, require accessible tenant branch scope and a matching academic year when supplied.

1 Resolve one coherent commercial state and active workspace entitlement.

Paid, promo, demo, and internal sources remain distinct; inactive, expired, suspended, missing, or ambiguous evidence fails closed.

1 Require an active catalog definition. Developer-only `feature.audit_log` is explicitly outside the customer baseline.

1 Resolve normal customer availability from the common policy. Optional, beta, disabled, or custom features continue through their explicit policy.

1 For branch-scoped work, resolve an active coherent BranchEntitlement.

2 Apply capacity or consumption separately. Feature availability never grants additional branches, staff users, teachers, or AI provider usage.

Allocation XLSX/PDF use this flow with `feature.advanced_reporting` and `reports.export`. Starter, Professional, Enterprise AI, active promo, and active demo therefore share availability while permission, scope, branch entitlement, and lifecycle continue to win.

AI Availability And Consumption Flow

1 Resolve enabled AI definition, tenant workspace, `ai.use`, commercial source, and the shared `module.ai` baseline.

1 Return availability independently of consumption. A globally disabled AI

definition remains unavailable.

- 1 Resolve consumption policy by entitlement source. Demo Standard retains two

successful uses per feature; Full is unrestricted; Custom may set legitimate allowances. Paid and promo currently have no configured provider ceiling because no executable provider-backed AI route exists.

- 1 Reserve by operation key, execute outside the entitlement service, and

complete successful or failed accounting exactly as before.

SchoolGroup Management And Branch Capacity Flow

- 1 School Management resolves the authenticated identity and registered permission.

- 2 Direct SchoolGroup create/delete additionally requires a Platform identity and

`schools.manage_all_schools`; tenant users fail before validation or side effects.

- 1 A tenant with `schools.edit` may update only the SchoolGroup resolved from the

tenant link/scope. Platform users require global management capability for an arbitrary SchoolGroup.

- 1 The page resolves branch usage and limits through unified commercial authority.

It shows customer-safe used/allowed state and only shows branch creation when one more active branch is currently permitted.

- 1 Branch POST requests independently re-resolve tenant scope, lock the SchoolGroup, recount capacity, and reject stale or over-limit requests before insertion.

- 1 For promo authority with remaining capacity, the same transaction creates the branch, promo assignment, and active branch entitlement. At capacity it changes nothing.

- 1 The last-active-branch check counts only active branches in the target SchoolGroup.

Existing Workspace Owner Alignment And Conversion

Existing Workspace Paid Activation

- 1 A verified tenant owner opens Organization Account and selects **Choose a Plan**.

TIS displays all currently eligible paid plans. If the open activation is draft or checkout_ready, its plan is highlighted but the owner may select another eligible plan. Opening or changing this selection calls no Paddle API and creates no PaymentAttempt.

- 1 TIS locks and validates the customer/provisioning SchoolGroup, exact owner link, absence of commercial authority, and absence of conflicting paid, promo, or demo activity.

- 1 TIS recounts active operational branches and organization-wide staff users and

teachers. Professional and Enterprise AI require all active branches. Starter remains unavailable until complete restricted-branch enforcement is proven.

- 1 The owner confirms the workspace billing profile. TIS builds an authoritative

USD quote from the active plan price and stores branch-selection and quote hashes. Changing an editable selection recalculates this quote and safely supersedes the unfinished local draft without changing any branch.

- 1 Preparation creates a SchoolGroup-anchored contract, paid-activation aggregate,

immutable branch snapshot, and checkout session. It creates no pending organization, provisioning job, workspace, tenant, or entitlement. Until launch creates a real PaymentAttempt, Organization Account continues to show Activation required rather than Payment processing.

- 1 A checkout_started, payment_processing, manual-review, or inconsistent

activation cannot be silently replaced; plan changes fail closed until that payment path is resolved. Launch revalidates the current quote and either reuses a still-billed matching Paddle transaction or creates one automatic-collection transaction with branch quantity and stable authority metadata. Customer, address, and business mappings are reused only through explicit account/workspace attribution. The workspace association records the selected address and business, and every returned or reused transaction must match the complete current billed quote and custom lineage.

- 1 Browser return never activates access. transaction.paid displays processing.

Failed, cancelled, or expired attempts instead display a recovery state.

- 1 A signature-verified transaction.completed locks the SchoolGroup and activation,

revalidates all local and provider evidence, and atomically creates the paid subscription authority, active branch entitlements, and paid tenant link. The existing workspace becomes active without tenant provisioning.

- 1 Duplicate webhooks are idempotent. Quote drift, capacity drift, branch drift,

stale transactions, provider mismatch, or a competing commercial source fail closed without partial authority. PostgreSQL lock acquisition refreshes the activation state before duplicate-event decisions.

Guardrails:

- 1 workspace_classification remains customer; paid versus promo is commercial-source evidence.

- 1 Paddle quantity is branch count only. Staff and teacher totals affect eligibility only.

- 1 Existing operational and academic rows are never recreated or mutated by activation.

- 1 Promo activation and PendingOrganization/new-organization checkout retain their

existing independent selection and payment behavior.

- 1 Organization Account remains available while operational access is blocked for recovery.

Controlled Existing Workspace Conversion

- 1 A Platform Owner runs the M4B CLI in dry-run mode with the exact workspace

tuple, intended-owner email, current M4A hash, operation UUID, and idempotency key. No row is changed.

- 1 Write preparation requires PostgreSQL, Platform Owner approval/execution

identities, and PREPARE <operation UUID>. It locks the workspace, reruns M4A, rejects stale or conflicting evidence, and records an ownership claim. It does not create an account or send email.

1 The intended owner registers normally, establishes a password or approved

external identity, and verifies the exact email. Login continuation opens the existing-workspace setup review.

1 The verified owner explicitly claims the workspace. TIS rejects unverified,

suspended, duplicate, cross-tenant, or platform identities, safely reuses or creates the operational owner identity, and creates a tenant_owner account link with no pending organization. A different current owner requires prior transfer approval.

1 The owner confirms existing organization identity and supplies only legal

name, official IANA timezone, and educational program. Branches and all existing operational records remain unchanged.

1 Final CLI execution requires CONVERT <operation UUID>. It locks the

operation and SchoolGroup, performs a fresh M4A audit, and requires unchanged identity, branch/dependency evidence, one active internal entitlement, no commercial source, complete setup, and unique verified ownership.

1 One commit ends the internal entitlement and sets customer / provisioning.

It creates no replacement entitlement or tenant link. Any failure rolls back all conversion changes and records only redacted failure evidence.

1 M1 resolves activation_required. Organization Account remains accessible,

normal operations remain blocked, and M3 promo activation is offered. The existing-workspace Paddle path remains hidden until its separate milestone; new onboarding checkout is unaffected.

Existing Workspace Conversion Audit

1 An operator supplies the exact SchoolGroup ID, workspace UUID, expected

organization name, and intended owner email to the standalone M4A CLI.

1 The CLI requires deployed PostgreSQL and begins a repeatable-read, read-only

transaction before the first application query.

1 The service validates the exact workspace tuple, resolves normalized owner

identities and links, and inventories tenant, provisioning, entitlement, paid, demo, and promo evidence.

1 For every target branch, the service follows reflected direct and indirect

foreign-key descendants and separately reports branch-like columns without a foreign key. Soft-deleted rows remain blocking. Foreign keys absent from ORM metadata force manual review. It returns counts and paths, not private row payloads.

1 Identity mismatch, duplicate ownership, existing non-sandbox commercial

authority, unavailable schema evidence, or uncertain traversal produces Manual Review Required. An active internal-sandbox entitlement is expected and is not treated as customer authority.

1 The CLI writes deterministic sanitized JSON or text with the observed transaction mode and a stable SHA-256 evidence hash. Exit 0 is coherent, 1 is execution/configuration failure, 2 is manual review, and 3 is identity mismatch. Archival candidates include only active dependency-free branches; hard deletion and write conversion remain unapproved.

1 The CLI rolls back and closes. It never archives, deletes, aligns, converts, calls Paddle, or sends email.

Customer Promo Activation

1 A verified organization owner chooses Use Promo Code after setup review or from an eligible existing Organization Account.

1 TIS HMAC-normalizes the submitted code, validates approval, lifecycle, dates, replacement, target scope, owner relationship, redemption policy, source compatibility, and setup readiness, then creates a short-lived resumable activation session. The raw code is never stored.

1 TIS shows authoritative branch, system-user, and teacher usage. If eligible branches exceed the grant, the owner selects exactly the grant count. If fewer exist, all are selected and the remainder stays available for future branch creation.

1 Excess staff or teachers blocks activation and identifies each exceeded dimension. TIS does not disable, select, delete, or change those records.

1 Final activation locks session, promo, and workspace, then repeats every validation and recount. It rejects internal sandboxes and every conflicting commercial source.

1 One transaction creates immutable redemption/grant snapshots, explicit branch assignments and entitlement states for the complete preserved branch inventory, a promo workspace entitlement, the promo-sourced tenant link, customer/active lifecycle, and redacted audit. Selected branches are assigned and active; every unselected branch is explicitly inactive without changing its operational status or consuming grant capacity.

1 A pending session grants nothing. Active access resolves through M1 and only selected branches are queryable. Expiry or inconsistent evidence fails closed. No Paddle call or payment object is involved.

1 Reactivating an unassigned branch locks the SchoolGroup, recounts M1 usage, and either rejects at capacity or commits operational status, one grant assignment, and active branch entitlement together. Bulk reactivation applies the same rule to the complete batch and rolls back all changes on any conflict.

1 `scripts/reconcile_promo_branch_entitlements.py` targets one SchoolGroup and workspace UUID. Dry-run reports only safe missing inactive evidence. Explicit apply revalidates under lock and inserts only those inactive entitlements; ambiguous authority, ownership, assignments, or entitlement lineage requires manual review.

- 1 An authorized owner opening Organization Account Billing & Subscription is routed by the selected workspace's authoritative source. Promo authority renders Commercial Access with the grant plan, safe status/window, masked reference, and M1 branch/system-user/teacher usage and remaining capacity. It never invokes the paid subscription resolver, Paddle billing history, invoices, plan/quantity changes, or cancellation while promo access is active.
- 1 Operational promo access ends exactly at `effective_to`. Before that instant the grant is active; at that instant and through `grace_period_days` it is expired in a recovery period; after grace it remains expired. Both expired states block all normal tenant operations while preserving Organization Account and tenant data.
- 1 A verified owner may begin paid continuation only for recovery-period or expired promo authority. Plan selection and Paddle checkout reuse the existing SchoolGroup and M4C quote/identity lineage. Pending, failed, canceled, expired, or abandoned checkout does not end promo authority or restore expired operational access.
- 1 `transaction.completed` locks the SchoolGroup and activation, revalidates current capacity, quote, provider identity, promo source, and paid evidence, then atomically ends the promo workspace entitlement, relinks the sole tenant source to the paid contract, establishes paid workspace and branch entitlements, and records immutable conversion audit. Duplicate confirmation is idempotent; conflicts roll back the authority transition and enter manual review.
- 1 Active promo early conversion is blocked. Promotional value is not a paid credit, and no proration or unused-promo-value calculation is invented.

Platform Promo Definition

- 1 A Platform Owner or permission-authorized Developer opens Promo Codes.
- 2 Create validates one active Starter, Professional, or Enterprise AI tier, exact positive capacity through the shared plan-capacity service, coherent scope anchors, dates, expiry XOR, and redemption policy.
- 1 TIS generates at least 100 bits of secure random code material, normalizes with NFKC/uppercase/separator removal, and derives an HMAC-SHA256 lookup hash with `TIS_PROMO_CODE_HMAC_SECRET`. Missing configuration fails closed.
- 1 The draft and allowlisted audit commit. The raw code appears once in a no-store/no-referrer response; subsequent pages show only its mask.
- 1 A Developer may edit Draft, pause Active, duplicate, or create a linked replacement definition when authorized. Active material edits first require pause, clear approval, increment version, and return to Draft.
- 1 Only a Platform Owner may approve/activate or terminally revoke with a reason. Lifecycle transitions lock the row through validation, mutation, audit insertion, and commit.

1 No Platform definition step grants access or calls Paddle. Customer M3

activation is a separate owner-authorized transaction.

Organization Profile Save

1 The customer opens the owned pending organization's Organization Profile.

2 The browser submits the existing multipart POST with a required controlled

educational-program value and an optional logo.

1 TIS normalizes National, International, or Both and validates all profile

fields. Customer-correctable input returns the same page with preserved text/select values.

1 When a logo is supplied, TIS reads no more than the 4 MB boundary, decodes

the actual image as PNG/JPG/WEBP, checks dimensions, and ignores the customer filename when creating the opaque stored filename.

1 TIS writes the new image to a temporary sibling and atomically promotes it.

2 Organization changes, progress, activity, and events commit together.

Only after commit is an obsolete prior pending logo removed.

1 If logo storage or a later database step fails, TIS rolls back the

transaction, removes the newly written file, retains the prior logo reference, logs the traceback, and renders a customer-safe error.

1 The successful response redirects to Branch Setup. No duplicate pending

organization or operational workspace is created.

Pending logo files currently live on the application-local filesystem. Durable Render operation requires a separately approved persistent disk or object storage architecture. The owner-only workspace deletion workflow currently removes database records but does not yet perform transactional cleanup of pending or promoted organization-logo files on the filesystem.

Organization Logo Activation

1 Successful Organization Profile save stores a safe relative pending-logo

path; setup and checkout pages render only its public static URL.

1 Organization Profile and the shared setup identity header show the customer

logo and organization name while the official TIS mark remains platform branding.

1 Checkout does not move or remove the pending image.

2 Paid and demo activation both call `create_workspace_records()`.

3 Provisioning resolves the pending path only inside the pending-logo

directory, requires the file to exist, decodes it again, and writes a new organization-owned branding file.

1 A primary `SchoolGroupLogo` row stores the final relative path and an

organization-name label. SchoolGroup has no direct logo field.

- 1 The operational shell resolves that row through the protected

organization-asset route, renders it with contain sizing, and keeps the official TIS logo separately visible.

- 1 Missing pending source data blocks activation for correction instead of producing an active workspace with silently lost branding.

Branch Estimate Rendering And Save

- 1 The service loads active branch rows and computes capacity totals in Python.

- 2 Null, missing, or malformed display values contribute zero; mixed valid integer values continue to total normally.

- 1 The template receives normalized totals and renders null row fields as zero, avoiding Jinja `int + None` failures.

- 1 Every customer save still requires explicit non-negative whole-number values for estimated system users and teachers.

- 1 Newly created pending branches receive explicit zero defaults before their submitted values are applied.

Combined Subscription Capacity And Custom Contact

- 1 Each active onboarding branch supplies required non-negative system-user and teacher estimates; TIS sums them across the organization.

- 1 Before payment, system-user and teacher authority is independently the greater of the estimate total and actual same-workspace active data. After activation, actual active data is authoritative. Every active tenant operational User counts as staff regardless of position, so an operational teacher-user with a Teacher record consumes one staff slot and one teacher slot.

- 1 A self-service plan is eligible only when all three counts fit: Starter is 1 branch/5 system users/25 teachers, Professional 5/20/100, and Enterprise AI 25/100/500.

- 1 The lowest eligible plan is recommended; higher eligible plans remain selectable. Ineligible cards explain every failed dimension. Exceeding twenty-five branches, 100 system users, or 500 teachers emphasizes the always-visible Custom contact action.

- 1 Selection and every quote, checkout, Paddle, and payment-validation boundary repeats the same server-side decision.

- 1 A pre-payment branch estimate change retains an eligible higher

plan, but clears an undersized selection and supersedes its checkout and attempt so stale payment cannot activate the workspace.

- 1 On a paid workspace, every tenant operational-user creation/reactivation and

teacher creation/year-copy preflight requires remaining capacity. Blocked operations create no partial user or teacher data.

- 1 A downgrade is blocked separately by current branches, system users, or teachers. Existing data is preserved.

M1 Operational Commercial-Capacity Enforcement

- 1 The route first validates the existing permission and tenant scope.

- 2 TIS locks the owning SchoolGroup with SELECT . . . FOR UPDATE; internal

multi-tenant operations acquire locks in ascending SchoolGroup ID order.

- 1 The commercial authority facade composes classification/lifecycle,

workspace entitlement, tenant link, contract, confirmed subscription, commercial access, demo lifecycle, and plan capacity.

- 1 The service recounts active branches, distinct active tenant operational

users, and active teacher people. Known teacher IDs are normalized and deduplicated; blank legacy identities each count independently.

- 1 Paid limits use confirmed branch quantity capped by plan maximum plus the

plan staff and teacher limits. Pending subscription changes do not increase capacity. Demo and Internal Sandbox are unmetered only when their existing authority resolves coherently.

- 1 TIS evaluates deltas or proposed absolute totals across one or several

dimensions and returns a structured safe decision.

- 1 Allowed branch, user, teacher, year-switch, or provisioning mutation occurs

before the same transaction commits. A denial rolls back and exposes no provider or internal identifiers.

- 1 Existing over-capacity data remains accessible and may be reduced; only a

further increase in an already exceeded dimension is blocked.

The separate Next.js landing page presents the same four-plan structure and organization-wide limits. The hero Subscribe Now action scrolls to pricing. Starter, Professional, and Enterprise AI retain their published monthly and annual per-active-branch prices and enter the configured public signup route with an allowlisted preferred-plan code. That code is a presentation preference only: signup creates no plan selection, checkout, payment attempt, or Paddle object. After organization setup, TIS applies it only when the live plan remains active and eligible across branches, system users, and teachers; otherwise TIS clears it and asks the customer to review eligible plans. Custom has no fixed public price, uses only the Contact the TIS Team mail action, and never starts signup or Paddle checkout.

Initial Subscription Plan Capacity

- 1 TIS counts the current organization's authoritative active billable

branches.

- 1 Starter is eligible for one branch, Professional for up to five, and

Enterprise AI for up to twenty-five. Higher-capacity plans remain available below their limits.

- 1 Plan cards retain per-branch pricing and explain their branch and staff

limits. Ineligible plans are disabled; above twenty-five branches the customer is directed to contact the TIS team for a custom plan.

- 1 The server repeats the capacity decision during selection, quote creation, checkout preparation and launch, and payment validation.

- 1 If pre-payment editing makes the selected plan too small, TIS clears that

selection, supersedes its quote and checkout lineage, and requires a new eligible plan. An eligible higher plan remains selected.

- 1 After activation, branch-capacity changes continue through Subscription

Management.

Fixed-Quantity Initial Checkout

- 1 Before confirmed payment, customers may add, edit, remove, reorder, or

reprioritize onboarding branches while keeping at least one active branch. A demo SchoolGroup, tenant link, customer, or prepared checkout does not close editing.

- 1 A legacy `ready_for_checkout` pre-checkout billing state is eligible for

safe preparation. The original incident was a local rejection in `_ensure_checkout_launchable()` before TIS called Paddle.

- 1 If the plan, interval, branches, capacity estimates, or authoritative quote

change after checkout preparation, TIS supersedes the local checkout and unfinished attempt, clears the old quote lineage, and recalculates from the current selection and active count. A late old transaction event cannot activate that quote.

- 1 TIS resolves the selected plan, billing interval, active billable branches, unit price, and total for the current organization.

- 1 The customer reviews those values in TIS. For Professional Annual, the

active mapping is USD 790 per branch, so two branches produce quantity 2 and USD 1,580 annually.

- 1 TIS creates one Paddle transaction item using the mapped provider price and

the exact authoritative branch quantity; quantity is never defaulted to one.

- 1 TIS reuses an active same-country Paddle address for the same customer, or

creates one when no compatible address exists. The resolved address makes the automatically collected transaction ready. Returned item quantity, price, subtotal, and quote fingerprint must agree with TIS before the transaction is marked billed.

- 1 The payment launcher passes only the billed transaction ID to Paddle inline

checkout after verifying the local attempt, organization, customer, quote, and remote billed status. Billed transaction items and quantities are immutable; draft, ready, canceled, past-due, unrelated, or mismatched transactions are not launched.

- 1 Paddle completion remains the recurring-subscription authority. Later branch

changes use the established TIS subscription-quantity workflow.

- 1 Retry revalidates unpaid checkout eligibility and may replace an incomplete

or non-launchable session. A reusable started transaction must still be billed, automatic, customer-matched, and current-quote matched at Paddle.

Returning Customer Login And Organization Account

- 1 SaaS login authenticates and resolves authoritative onboarding, demo,

workspace-link, lifecycle, and subscription state.

- 1 Incomplete setup resumes; pending demos open request status; and unpaid

completed setup opens subscription selection.

- 1 An activated owner or linked user with account-management permission lands

on /saas/account, the Organization Account Overview. Multiple managed organizations require selection before the overview is shown. The HTTP-only selected UUID is revalidated against the account's current tenant links and permissions on every request before entitlement or billing resolution.

- 1 The overview exposes Organization Profile, Branches, Billing & Subscription,

and Account & Security only when ownership or existing permissions allow. Restricted or suspended account managers retain permitted billing/recovery access, but no active Enter TIS Platform action.

- 1 Enter TIS Platform is the only Organization Account action into /login.

Linked users without account-management permission retain their approved role-based destination and cannot see owner or billing controls.

- 1 Operational login runs the commercial guard before branch and academic-year

setup. Protected requests run it before page services. Authentication, sign-out, SaaS account, subscription/checkout/payment-return, support, and expiry routes remain safe and cannot recursively redirect.

- 1 Expired-demo Subscribe Now resolves the existing organization, SchoolGroup,

and active operational branches, offers public plans and monthly/annual intervals, then launches existing Paddle checkout.

- 1 Only confirmed provider payment converts the same workspace UUID, tenant

link, users, permissions, branches, and data.

Password login, social login, already-authenticated sign-in, post-verification sign-in, and SaaS root/session restoration use the same destination resolver.

Platform Owner Demo Operations

- 1 A Platform Owner opens an existing Customer Demo operational detail.
- 2 The owner supplies required reason/date/configuration and an operation key.
- 3 The service validates and locks the same workspace and tenant scope.
- 4 Lifecycle work reuses M8B7 rules; feature policy uses controlled registries.
- 5 Material changes create durable customer email/Notification Center records,
and every attempt records before/after/result references.
- 1 Profile changes retain usage and never convert the Customer Demo to paid.

M8B8 AI Entitlement And Consumption Flow

- 1 Resolve the operational SchoolGroup and reject cross-tenant scope.
- 2 Require the registered feature's underlying ai . use permission.
- 3 Resolve existing commercial state; demo expiry/restriction takes precedence.
- 4 Resolve the controlled feature definition.
- 5 Internal Sandbox is unlimited; Customer Demo checks two successful uses per
feature; Customer Paid checks eligible plan plus module . ai .
- 1 Reserve an operation key under the locked successful-plus-reserved limit.
- 2 If reserved, execute the real AI operation outside the entitlement service.
- 3 Finalize a usable result as successful consumption; finalize failure without
consumption and release the reservation.

There are currently no executable AI routes, so steps 6-8 are a reusable contract rather than a fabricated customer feature.

M8B7 Demo Customer Journey

Submission creates pending status, a request-received email intent, and owner Notification Center events. Normal approval records review evidence then invokes the retryable provisioning service; activation communication exists only after success. Day 6 and expiry create lifecycle events, customer notices, owner notifications, and email intents atomically. Provider delivery occurs outside locks. A coherent expired demo may subscribe and, after authoritative confirmed payment, reactivate and convert the same workspace.

This document describes the major end-to-end flows a developer must understand before changing TIS.

Public Customer Flow

Flow:

- 1 Public visitor opens <https://tisplatform.com>.

- 2 Visitor chooses Request a Demo or Subscribe Now.
- 3 Visitor reaches SaaS signup at /saas/signup?intent=demo or /saas/signup?intent=subscribe.
- 4 SaaS account is created.
- 5 Email verification is completed when required.
- 6 Email verification redirects by HTTP 302 to the GET /saas/login page.

Its form submits credentials by POST to /saas/auth/login.

- 1 A fresh verified account with no organization enters the GET

/saas/account dashboard. A supplied continuation is used only when it is a known customer GET destination; POST-only, malformed, traversal, and external values fall back to Account Setup.

- 1 The user invokes the existing POST start action and completes organization onboarding:

- 1 organization details,
- 1 contacts,
- 1 branches,
- 1 academic setup,
- 1 review.

- 1 The final commercial-choice page emphasizes the saved intent, while the user may still choose Request Demo or Subscribe Now.
- 2 Subscribe Now continues to plan selection and the existing Paddle checkout path.
- 3 Paddle handles payment.
- 4 Return/cancel page informs the user of checkout navigation result.
- 5 Paddle webhook confirms payment.
- 6 Local payment/billing state is updated.
- 7 Pending organization becomes ready for provisioning.
- 8 Platform owner reviews/runs provisioning.
- 9 Operational tenant structures are created.
- 10 Operational login becomes available through /login.

Guardrails:

- 1 Checkout return is not authoritative payment confirmation.
- 1 Public signup must not directly create operational tenant data.
- 1 Provisioning should occur only through the approved flow.

Demo Request Flow

- 1 A verified customer completes organization, contact, branch, academic, and review onboarding.
- 2 TIS presents Request Demo and Subscribe Now.

- 3 Request Demo revalidates account ownership, verification, onboarding completeness, branch configuration, absence of conflicting payment/provisioning state, and normalized organization-domain eligibility.
- 4 TIS creates one Pending Review SaaS demo request plus a transactionally unique customer-demo eligibility reservation for the normalized organization domain.
- 5 The customer can view status and withdraw only while Pending Review.
- 6 A Platform Owner searches, filters, and sorts the review queue.
- 7 Approval creates a review record only; rejection requires a reason; owner cancellation is allowed only while pending.
- 8 A Platform Owner separately starts provisioning for an Approved request.
- 9 TIS revalidates review evidence, customer-demo intent, commercial/entitlement snapshots, organization completeness, and duplicate absence.
- 10 One atomic transaction creates the operational workspace through the shared provisioning builder, creates the demo entitlement and demo-sourced tenant link, activates both, and links the request.
- 11 Failure rolls back workspace records, leaves the request Approved and unprovisioned, and records a retryable failure outcome.
- 12 Each review, provisioning, and activation action creates durable audit and internal-notification events.

Guardrails:

- 1 Request and approval alone create no SchoolGroup or entitlement.
- 1 Demo provisioning creates no checkout, payment, paid subscription, subscription contract, or Paddle record.
- 1 A prior customer demo request, activation, expiry, rejection, cancellation, or demo-to-paid conversion for the same normalized organization domain blocks a new customer demo. Platform Owners extend/reactivate where separately allowed, or the customer subscribes using the existing workspace.
- 1 Public email providers require an official organization website or domain before a demo request. Internal Sandbox workspaces do not consume customer demo eligibility.
- 1 Non-owner platform users and tenant/customer identities cannot access review actions.
- 1 Duplicate, rejected, cancelled, incoherent, or already provisioned requests fail closed.
- 1 M8B-4 sends no email and does not implement expiration, scheduling, login blocking, or conversion.

Historical Demo Eligibility Maintenance Flow

- 1 A Platform Owner opens /saas-admin/demo-eligibility-maintenance.
- 2 TIS scans the demo-domain eligibility ledger and resolves matching organization, account, request, tenant-profile workspace, provisioning, subscription, conversion, and manual-review evidence.
- 3 Linked or ambiguous reservations remain protected and display their exact blockers.
- 4 A safely detached row exposes a review action for its exact eligibility ID.
- 5 The owner types the same ID and confirms permanent removal.
- 6 TIS locks and re-analyzes that exact row before deletion.
- 7 Any new blocker aborts and rolls back the action.
- 8 TIS deletes only the selected primary key, flushes, verifies no row remains for that ID, and commits.
- 9 A durable audit event records the Platform Owner, eligibility ID, normalized domain, previous status, timestamp, and historical-cleanup reason.

Guardrails:

- 1 Never delete by normalized domain.
- 1 Never expose the workflow to customers, tenant users, or Platform Developers.
- 1 Never use this workflow to override a linked, historical, converted, subscribed, provisioned, or manual-review Customer Demo reservation.
- 1 Do not change the normal one-demo-per-domain rule or the organization-scoped clean-room reset.

Customer Demo Lifecycle Flow

- 1 Successful M8B-4 activation records the authoritative demo start.
- 2 TIS derives reminder due at start plus six days and expiration at start plus seven days.
- 3 Before Day 6, the resolver returns Active and operational access continues.
- 4 At Day 6, the resolver returns Reminder Due; the processor creates one customer notification and one per active Platform Owner.
- 5 At Day 7, access fails closed immediately even if scheduled processing has not run.
- 6 Apply-mode processing atomically ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, and records lifecycle events.
- 7 Web requests redirect to the preserved-data and subscription page; API/download requests receive a safe 403.
- 8 Existing sessions are rechecked on every protected request. Platform users remain able to inspect the workspace.

Guardrails:

- 1 Use `activated_at`, never request submission or approval, for lifecycle calculations.
- 1 Store and calculate in UTC; convert only customer/owner display values to the organization timezone.
- 1 Expiration never deletes, archives, deactivates branches, or mutates operational tenant data.
- 1 Reminder and expiration actions are idempotent and failure-audited.
- 1 No email, Paddle change, conversion, extension, or read-only expired mode is included.

Demo-To-Paid Conversion Flow

- 1 An active, coherently provisioned Customer Demo chooses Subscribe Now.
- 2 TIS records a requested conversion and continues through the existing M7 plan selection and Paddle checkout.
- 3 No new operational workspace, tenant link, branch, user, permission, or academic record is created.
- 4 `transaction.completed` remains the payment authority and establishes the confirmed `SubscriptionContract` and active `PaymentSubscription`.
- 5 If the existing tenant link is demo-sourced, webhook reconciliation invokes the dedicated conversion service instead of paid provisioning.
- 6 The service locks and revalidates the request, provisioning aggregate, SchoolGroup, demo entitlement, tenant link, contract, subscription, price, interval, quantity, and tenant ownership.
- 7 One atomic workspace transaction ends the demo entitlement, creates the subscription-backed paid entitlement, relinks branch entitlements, changes the SchoolGroup to Customer Paid Active, and moves the same tenant link to the confirmed contract.

- 8 Existing M7 entitlement, workspace-entitlement, and commercial-state resolvers validate the resulting Customer Paid Active state before commit.
- 9 Success records completion and lets the customer continue into the same operational workspace. Completed conversions no longer enter demo lifecycle processing.
- 10 Failure rolls back workspace mutations, preserves provider-confirmed payment records, records a safe retryable failure, and may be retried from a later subscription webhook.

Guardrails:

- 1 Never infer conversion from checkout return navigation, onboarding selection, or an unconfirmed payment attempt.
- 1 Never run paid tenant provisioning for a valid existing demo tenant.
- 1 Expired, suspended, ambiguous, cross-tenant, internal-sandbox, already-paid, and incoherent workspaces fail closed.
- 1 Conversion does not change Paddle pricing, subscription mutation, webhook authority, tenant isolation, or operational permissions.

SaaS Identity Flow

Flow:

- 1 User signs up through /saas/signup.
- 2 SaaS account/session records are created.
- 3 User signs in through /saas/login.
- 4 Account dashboard is available at /saas/account.
- 5 User can access profile, sessions, security, billing status, and onboarding state.

Important distinction:

- 1 SaaS account identity is not the same as operational tenant user identity.
- 1 Platform identity is also separate.

Guardrails:

- 1 Do not merge SaaS accounts with operational users unless an approved provisioning flow creates the needed operational records.
- 1 Keep SaaS authentication and operational authentication boundaries clear.

Payment Flow

Flow:

- 1 User selects a plan.
- 2 App creates or references a checkout session.
- 3 User goes to Paddle checkout.
- 4 User returns through checkout return or cancel pages.
- 5 Paddle sends webhook events.
- 6 Webhook-confirmed payment updates local payment/billing state.
- 7 Verified payment can make a pending organization ready for provisioning.

Guardrails:

- 1 Webhook confirmation is authoritative.
- 1 Do not use return-page navigation as proof of payment.
- 1 Keep Paddle-specific details inside `saas/` payment/client service boundaries.

Active Subscription Management Flow

- 1 Authorized billing administrator opens `/saas/subscription`.
- 2 TIS resolves one confirmed active subscription, entitlements, lifecycle state, allowed actions, and one operational capacity snapshot covering active branches, tenant operational staff users, and teachers. Paid branch quantity is displayed separately from the current plan's maximum branch ceiling; unused ceiling is not prepaid branch capacity.
- 3 Review Capacity accepts proposed totals for all three dimensions. It shows confirmed paid branches, required active branches, additional billed branches, the plan ceiling, and the resulting minimum eligible plan. Branch growth may create a combined plan-and-quantity upgrade, while system-user and teacher growth affects eligibility without changing Paddle quantity.
- 4 Quantity or plan changes are previewed through Paddle; TIS displays provider-returned totals and never recalculates proration.
- 5 Immediate increases/upgrades use provider payment-failure prevention and remain locally pending until authoritative confirmation. The current confirmed plan and workspace access remain authoritative while the plan change is pending, failed, incomplete, expired, canceled, or abandoned. Thus an active Professional subscription remains accessible while an Enterprise AI upgrade is `payment_pending`.
- 6 Reductions/downgrades are scheduled for the next billing boundary and retain current local access until verified effective evidence. Live branch, system-user, and teacher counts are revalidated at that boundary; a mismatch enters manual review without applying the lower local entitlement.
- 7 Scheduled plan or quantity changes may be canceled or replaced before their effective boundary when provider state agrees.
- 8 Cancellation is scheduled at period end; reversal removes the provider-scheduled cancellation after reauthorization and validation.
- 9 The centralized lifecycle resolver exposes only actions valid for current provider/local state.
- 10 Billing Contact is explicit organization-owned state. Initial checkout requires a confirmed billing email, legal/billing organization or school name, and supported country/address data. The active portal view is read-only until an authorized owner or `subscriptions.manage_billing` user selects Edit; validation remains in edit mode, Cancel discards unsaved form values, and login identity is unaffected.
- 11 TIS saves valid local billing changes before attempting provider synchronization. It reuses the mapped Paddle customer, synchronizes its explicit billing email, creates or reuses one attributable active Paddle Business, persists address/business mappings, updates the active subscription identity, and includes `business_id` on future initial transactions. Provider failure leaves the local profile saved with `retry-needed` status. The dedicated retry revalidates permission and tenant scope, uses the stored profile and mappings, avoids duplicates, and makes no provider call after successful synchronization. Historical billing documents are not silently revised.
- 12 Billing history is read from Paddle transactions. `paid` displays Payment received - processing; only `completed` displays Paid and satisfies the existing final processing signal. Invoice download reauthorizes the user and requests a fresh provider URL.
- 13 Operational login, protected requests, and returning-customer routing use the same contract-linked commercial access projection. The specialized plan-change webhook synchronizes provider subscription status, but the target plan becomes authoritative only after the existing required provider signals are both confirmed.

- 14 If production provider state is confirmed but local commercial status is stale, an operator replays the attributable stored, signature-verified Paddle webhook through the existing reconciliation path. Operators do not edit subscription or lifecycle status fields manually. Replay may synchronize the current `PaymentSubscription.status`, but it cannot bypass the separate provider payment signal required to activate a target plan.

Guardrails:

- 1 provider and local ownership must match,
- 1 active branch usage cannot exceed a requested reduced capacity,
- 1 target-plan eligibility is evaluated across branches, system users, and teachers before submission and again when a scheduled downgrade becomes effective,
- 1 Paddle quantity contains branches only; system-user and teacher counts never become provider quantity units,
- 1 webhook processing is idempotent,
- 1 ambiguous outcomes enter manual review,
- 1 return pages and local requests are not payment confirmation.
- 1 organization billing identity is tenant-scoped and provider mappings are revalidated before synchronization,
- 1 a saved billing profile in pending or failed provider synchronization blocks new plan and quantity mutations until retry succeeds; cancellation and legacy active subscriptions with no saved profile retain their established behavior,
- 1 provider synchronization logs the safe failed step, provider error code, and HTTP status server-side; provider identifiers and raw diagnostics are never rendered to the customer,
- 1 user login email is never an implicit permanent billing-email authority,
- 1 stale or unrelated organization-level subscription rows cannot supersede the `TenantProvisioningLink` and `SubscriptionContract`-linked subscription,
- 1 canceled subscriptions retain access only until the confirmed paid period ends; ambiguous evidence fails closed without being mislabeled as expired,
- 1 renewal guidance is shown only for genuine expiration, not payment processing, past due, paused, suspended, archived, or inconsistent commercial evidence.

Operational Billing Entry Flow

- 1 System Configuration includes Billing & Subscription only for a linked operational organization owner or user with `subscriptions.manage_billing` in the selected tenant.
- 1 The bridge revalidates the operational session, `SchoolGroup` scope, `SaaSAccountUserLink`, organization/workspace UUID, and billing authority.
- 1 The bridge opens `/saas/subscription` for that organization; it never duplicates billing UI inside operations.
- 1 If SaaS authentication is missing, password and social sign-in preserve the allowlisted internal subscription continuation and revalidate it against the signed-in account before rendering billing.
- 1 Unknown organizations, cross-account links, unauthorized users, duplicate

continuation parameters, and external return URLs fail closed. A user with multiple managed organizations must resolve an organization before billing.

- 1 The landing-page Organization Sign In action enters `/saas/login` without an

operational destination. Activated account managers therefore land in Organization Account unless they arrived through the validated billing continuation. Enter TIS Platform remains the only explicit operational entry.

Provisioning Flow

Flow:

- 1 Pending organization has completed required onboarding.
- 2 Payment is verified or owner-approved readiness is satisfied.
- 3 Provisioning job is queued or run.
- 4 Operational records are created or connected:
 - 1 school group,
 - 1 branch,
 - 1 academic year,
 - 1 initial operational user,
 - 1 permissions/role context,
 - 1 required setup defaults.
- 1 Provisioning status is updated.
- 2 Activation/access email may be sent.
- 3 User enters operational portal through `/login`.

Guardrails:

- 1 Keep provisioning idempotent where possible.
- 1 Do not mix school groups.
- 1 Do not skip platform owner visibility.

Operational Login Flow

Flow:

- 1 User opens `/login`.
- 2 Credentials are verified.
- 3 Active-user status is checked.
- 4 Platform users route toward platform context.
- 5 Tenant users receive branch/year scope.
- 6 Session and scope cookies are set.
- 7 User lands on `/platform` or `/dashboard`.
- 8 Middleware enforces route permissions.

Guardrails:

- 1 Preserve platform vs tenant branching.
- 1 Preserve idle timeout behavior for tenant users.
- 1 Do not bypass permission middleware.

Platform Owner Flow

Flow:

- 1 Platform owner logs in through `/login`.
- 2 Owner lands in platform context.
- 3 Owner uses `/platform` for organization context and owner/developer controls.
- 4 Platform Console pending counts include only organizations still requiring setup, review, payment, or incomplete/recoverable activation work.
- 5 Owner opens Pending Queue for current work or Organization Records for active, completed, rejected, and lifecycle-review history.
- 6 Owner uses SaaS admin pages for payments and provisioning.
- 7 Owner can inspect Workspace UUID, Classification, and Lifecycle as read-only metadata on `/platform`.
- 8 Owner uses `/platform/knowledge` to review KMS health.
- 9 Owner views/downloads the PDF through protected routes:
 - 1 `/platform/knowledge/booklet`
 - 1 `/platform/knowledge/booklet/download`

Guardrails:

- 1 Platform developers are not owners.
- 1 Owner-only pages must use existing owner access helpers.
- 1 Active tenant evidence takes precedence over stale onboarding status; conflicting completed evidence is labeled Lifecycle Review Required and excluded from the normal pending queue.
- 1 Do not expose KMS PDF through direct static links.
- 1 Workspace classification metadata does not authorize access or change commercial state in M8B-1.

Workspace Classification Diagnostic And Backfill Flow

- 1 Operator runs `scripts/diagnose_workspace_classification.py` to inspect every SchoolGroup and its tenant, onboarding, and Paddle relationship presence.
- 2 Operator runs `scripts/backfill_workspace_classification.py` without `--apply` for a read-only plan.
- 3 After review, operator reruns with `--apply`.
- 4 One transaction classifies all pre-M8B-1 records as internal sandbox/test data and records an idempotency marker.
- 5 A repeated apply reports `already_applied` and changes nothing.

Guardrails:

- 1 The diagnostic and dry run do not change rows.
- 1 Apply does not call Paddle, migrate AI-Andalus, convert workspaces, or change payment/provisioning state.
- 1 Failures roll back the full backfill transaction.

Commercial State Resolution Flow

- 1 Resolve the SchoolGroup workspace classification and lifecycle.
- 2 Resolve exactly one effective workspace entitlement, or use the compatibility-only implicit entitlement for an internal sandbox created outside migration.
- 3 Validate entitlement type against workspace classification and parse explicit values through the shared entitlement catalog.
- 4 For customer-paid workspaces, resolve the existing M7 confirmed subscription entitlement and require the linked local PaymentSubscription to match.
- 5 Resolve each branch as inherited, explicitly active, or commercially inactive while independently respecting operational branch status.
- 6 Return a read-only effective commercial state or fail closed to Manual Review Required.

Guardrails:

- 1 No resolver writes rows or calls Paddle.
- 1 No resolver changes current tenant access, feature checks, branch mutations, onboarding, or provisioning.
- 1 Demo expiration and commercial-state mutation remain later work.
- 1 Cross-tenant and orphan branch entitlement relationships fail closed.

Knowledge Management Flow

Flow:

- 1 Developer or Codex reads docs/AI_PROJECT_CONTEXT.md.
- 2 Developer reads master context, project state, engineering docs, relevant ADRs, and module history.
- 3 Approved change is implemented.
- 4 .kms-impact.yml and the human-readable Knowledge Impact Assessment are completed.
- 5 Relevant docs are updated:
 - 1 master context,
 - 1 project state,
 - 1 change history,
 - 1 ADRs if needed,
 - 1 module history if needed,
 - 1 AI project context if needed,
 - 1 engineering docs if architecture/module/flow understanding changed.
- 1 PDF generator runs.
- 2 PDF snapshot and manifest are regenerated.

- 3 Knowledge Center checks manifest freshness and health.
- 4 CI compares the declaration with changed files and blocks stale pull requests, dev integration, or master deployment.

Guardrails:

- 1 Markdown remains source of truth.
- 1 PDF is generated and must not be edited manually.
- 1 App must not silently rewrite source docs.
- 1 Regenerate button is not implemented yet.

Timetable Version Compatibility Flow

- 1 A scoped timetable read resolves SchoolGroup, Branch, and Academic Year.
- 2 It loads the compatibility operational version: a working draft sourced from

the active version when one exists after an edit, otherwise the active version, otherwise the newest scoped draft.
- 1 Section and teacher views, XLSX, and PDF use only that version's placements.
- 2 On the first legacy assignment edit of an active timetable, the service copies

every placement and intended lock into a new mutable draft.
- 1 The requested edit changes the draft, increments `edit_revision`, and records

manual-change evidence. The active pointer and imported history do not change.
- 1 Later legacy edits reuse the same working draft until explicit version and

publication workflows are introduced.

Migration flow:

- 1 Find only branch/year scopes with existing placements.
- 2 verify Branch and Academic Year resolve to one SchoolGroup or fail the whole

migration transaction;
- 1 capture current Planning/settings authority in a deterministic snapshot;
- 2 preserve and attach every placement to one imported compatibility version;
- 3 record safe stale reason codes without changing a placement; and
- 4 create one exact-scope active pointer without fabricating publication approval.

Guardrails:

- 1 Active, superseded, and archived versions are not mutated in place.
- 1 No timetable mutation is allowed without explicit valid tenant scope.
- 1 Planning remains authority for real section-subject demand and HRT fallback.
- 1 Stage 2 executes no generation and treats no display-only block as a solver rule.

Stage 3.5 configuration flow:

- 1 The administrator selects working days, teaching-period count/duration, and shift start.
 - 2 A block is placed after a teaching period with a duration, or retained as fixed time.
 - 3 One composer walks each day, inserts blocks at deterministic boundaries, shifts later teaching periods, and calculates day/end metrics.
- 1 Settings preview, timetable grid, readiness, assignments, snapshots, and exports consume that projection. A valid inserted block is not a readiness blocker; an ambiguous fixed time, invalid duration/day/boundary, or conflicting block is.
 - 1 Configuration fingerprint changes make existing versions stale without deleting their placements or mutating published history.

Stage 3 derives canonical slots, resolves positive Planning demand (including HRT), checks allocation/capacity/locks/staleness, and returns stable readiness blockers and warnings for one exact scope. Ready to Generate means only that inputs can be submitted to a future solver; no generation action runs.

Stage 4 version/publication flow:

- 1 Default read resolves an active-derived mutable draft, otherwise active, otherwise the newest scoped mutable draft; explicit history selection changes only the view.
- 1 Immutable history may be copied to a new draft with placements, valid locks, and source provenance preserved.
- 1 Draft placements and locks use exact permissions plus `edit_revision`; locks update snapshot authority and any edit returns publication-ready state to draft.
- 1 Validate Draft checks freshness, demand completion, collisions, canonical slots, Planning teacher authority, stale placements, and locks without a solver.
- 1 Publish rechecks scope, permission, revisions, fingerprint, and validation; locks the pointer; supersedes prior active history; and activates atomically.
- 1 Historical versions remain reviewable, comparable, safely archivable, and explicitly exportable. Viewing or exporting never publishes.

Human / AI Developer Onboarding Flow

Flow:

- 1 Read docs/AI_PROJECT_CONTEXT.md.
- 2 Read docs/README.md.
- 3 Read docs/TIS_MASTER_CONTEXT.md.
- 4 Read docs/PROJECT_STATE.md.
- 5 Read docs/engineering/TIS_MODULE_MAP.md.

- 6 Read docs/engineering/REPOSITORY_ARCHITECTURE.md.
- 7 Read docs/engineering/USER_AND_SYSTEM_FLOWS.md.
- 8 Read relevant ADRs and module history.
- 9 Inspect code with rg before editing.
- 10 Make scoped changes only.
- 11 Update KMS docs and regenerate the PDF if needed.
- 12 Report the KIA.

Before coding, inspect:

- 1 affected routes,
- 1 models and scope fields,
- 1 permission rules,
- 1 templates/forms,
- 1 tests for the touched module,
- 1 related docs/ADRs/history.

After coding, update:

- 1 docs/CHANGE_HISTORY.md for meaningful changes,
- 1 module history for area-specific changes,
- 1 ADRs for major decisions,
- 1 engineering docs when module maps, architecture, or flows change,
- 1 AI context when onboarding truth changes,
- 1 project state when priority/status changes.

TIS Database Architecture Overview

docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

Talent Assessment Cycle Frozen Population

M4 adds `talent_assessment_cycles` as the SchoolGroup-wide Program + Academic Year + exact Framework authority, with explicit effective time, one-way Draft/Open/Closed lifecycle, revision, and organization-level population count/fingerprint. `talent_assessment_cycle_population_members` freezes exact Student + Academic Placement + Academic Year + Branch + grade + section context through composite scoped foreign keys; current Placement and nullable PlanningSection provenance are never historical rendering authority.

`talent_assessment_audits` is the bounded append-only operational lifecycle audit. Migration

`20260904_004_talent_assessment_cycle_frozen_population` creates all three tables and the exact Academic Placement composite unique target required by the member FK.

Teacher Scheduling Rules

Migration `20260830_001_teacher_scheduling_rules` adds `teacher_scheduling_rules`, `teacher_scheduling_rule_slots`, and `teacher_scheduling_rule_targets`. The header carries exact SchoolGroup, branch, academic-year, and teacher scope plus rule type, hard/soft semantics, target scope, active state, actor metadata, and timestamps. Slot children normalize all/selected days with numbered, first, or last period selectors; target children normalize selected grades or composite-scoped Planning sections.

The migration adds or verifies (`teachers.id`, `branch_id`, `academic_year_id`) as a composite parent key before creating the dependent rule table. All three new tables are excluded from baseline `metadata.create_all()` and created by the ordered, idempotent migration, preserving the production migration sequence on PostgreSQL and SQLite. No backfill occurs: absence of rules preserves existing generation behavior.

Migration `20260830_002_teacher_scheduling_window_semantics` adds the additive `restrict_to_window` discriminator. Existing rows default false and retain their original Must-teach occupancy meaning. New Schedule-within rules store the same validated hard-rule family with the discriminator true and serialize canonically as `schedule_within`; this avoids rebuilding SQLite check constraints and prevents silent reinterpretation of deployed rows.

Curriculum Adjustment Audit And Transaction

Migration `20260829_001_curriculum_adjustment_apply_foundation` adds `curriculum_adjustment_audits`. Each applied record carries UUID identity, exact SchoolGroup/branch/year, actor/time, scope and source/target subjects, reviewed fingerprint and request, per-section before/after periods and teacher decision, warnings, Draft linkage/stale state, regeneration requirement, and applied status. A unique scope+fingerprint constraint prevents duplicate apply.

The apply service locks and revalidates scoped Planning sections/demands, assignments, rules, timetable versions, and active generation state. Demand, assignment, section-rule retirement, Draft stale/approval state, and audit insert commit together or roll back together. Snapshots, published versions, placements, and `TimetableActiveVersion` are not rewritten.

Curriculum Adjustment Preview Read Boundary

Stage 3 adds no table or migration. The preview reads exact-scope Current/New PlanningSection, resolved PlanningSubjectDemand, scoped Subjects and Teachers, section assignments, teacher subject allocations, Subject Distribution Rules, Timetable Settings grouped configuration, and the newest mutable Draft Timetable. Its SHA-256 fingerprint binds the canonical analysis and Draft revision/authority; it is returned to the caller but is not persisted. Published versions, active pointers, demands, and assignments are never mutated by preview.

Planning Subject Demand Foundation

`planning_subject_demands` normalizes future weekly-period authority at the Planning-section and subject level. Each row carries branch/year scope, `planning_section_id`, scoped `subject_code`, weekly periods, active/retired state, timestamps, and optional creating/updating actors. Composite foreign keys require the section and Subject to belong to the row's branch/year. A partial unique index allows at most one active row per section and subject while retaining inactive rows.

Migration `20260828_004_planning_subject_demands_foundation` inserts missing active rows for Current/New Planning sections joined to Subjects by branch, academic year, and normalized grade. NOT EXISTS for any prior demand makes the backfill repeat-safe without reactivating retired rows. The demand service bulk-resolves Current/New sections inside one exact branch/year: any explicit row, including inactive or zero, is authoritative; only a missing row falls back to the grade-matched Subject weekly value. Live Planning, workload, timetable inputs, scheduling-rule arithmetic, and required-hours reporting consume this resolved demand. Historical published timetable data is not recomputed.

Subject Scheduling Rules UI

No schema change was required. `subject_distribution_rules_ui.py` lists, creates, updates, resets, and copies `subject_distribution_rules` rows through the existing Stage 1 table and Stage 1/2 resolver/validator; Timetable Settings routes under `/system-configuration/timetable-settings/subject-rules` remain tenant/branch/year-scoped and reuse `timetable.manage_settings`. The grade-first modal, search, two-period presets, and progressive disclosure are presentation only; Planning totals, normalized rule persistence, legacy fallback, and grouped Swimming authority are unchanged. Operational timetable grade and section selectors use `planning_scope_service.py`, which reads only Current/New PlanningSection rows for one exact branch and academic year; global grade catalogs remain creation/edit choices and are not operational selector authority.

Subject Distribution Rules Generation Wiring

`timetable_snapshot_service.py` resolves and embeds the effective Subject Distribution Rule per Planning demand at snapshot creation time using `subject_distribution_rules.resolve_subject_distribution_rule`. The problem builder, CP-SAT solver, independent validator, and `TimetableReadinessService` all consume that resolved, per-demand authority; no schema change was required beyond the Stage 1 table. `TimetableActiveVersion` and the published/draft/history lifecycle are unaffected.

Subject Distribution Rules Foundation

`subject_distribution_rules`, added by migration `20260828_003_subject_distribution_rules_foundation`, is a normalized table scoped branch_default, grade, or section per branch/academic year, with partial unique indexes enforcing exactly one row per scope tier. It coexists with `TimetableSetting.quality_rules_json`; an absent normalized row means the existing JSON authority applies unchanged for that scope.

Timetable Academic Quality Configuration

`TimetableSetting.quality_rules_json`, added by migration `20260828_002_smart_timetable_academic_quality_rules`, stores normalized exact-scope academic distribution and grouped-activity authority. Generation copies it into an immutable snapshot; timetable versions and published-pointer relationships are unchanged.

On-Demand Generation Execution

No schema migration is required for Render Workflows. The existing generation run public ID identifies task input; status, exact scope, snapshot, attempt, lease, heartbeat, cancellation, safe failure, and result columns remain authoritative. An on-demand task locks and claims exactly its queued public ID. Duplicate task invocations see an active or terminal row and do not solve or persist again. Immediate web dispatch failure atomically marks only a still-queued run `internal_error` with `workflow_dispatch_failed`. The partial unique active-scope index and atomic candidate transaction remain unchanged.

Smart Timetable Stage 5.1 Queue Migration

Migration `20260822_001_smart_timetable_stage51_generator` is additive and idempotent. It extends the existing `timetable_generation_runs` table with `progress_phase`, non-negative `attempt_count`, `cancel_requested_at`, and a nullable cancellation actor foreign key. It adds a (`status`, `lease_expires_at`, `queued_at`) claim index and a partial unique exact-scope index for active statuses. Duplicate active scopes fail migration preflight instead of being guessed or rewritten. PostgreSQL receives named progress/attempt checks and the actor FK.

No parallel queue table, lesson-instance column, room/resource table, quality score, or active-pointer mutation is introduced. Successful persistence inserts one `TimetableVersion`, all `TimetableEntry` rows, snapshot/run linkage, and terminal run state in one transaction; failure rolls the candidate back.

Capacity-Based Packaging Reconciliation

Migration `20260819_001_capacity_based_customer_feature_baseline` is additive and idempotent. It upserts the nine normal customer `PlanEntitlement` booleans to active/true for Starter, Professional, and Enterprise AI, synchronizes the legacy AI, Advanced Reporting, and multi-branch availability flags, and leaves `priority_support` separate. It does not update any branch/staff/teacher ceiling or Paddle quantity.

For a promo grant that is active at migration time, missing or false normal customer `WorkspaceEntitlementValue` rows are inserted or corrected. The derived branch quota, staff/teacher grant capacity, `PromoGrant`, `PromoRedemption`, branch assignments, tenant source, and payment tables are untouched. Expired or incoherent promo authority is not reactivated or reconciled.

Active customer-demo policy rows receive the normal baseline while retaining stored experimental keys and all AI allowance/unrestricted configuration. Paid workspaces need no value backfill because their workspace entitlement resolves the current `PlanEntitlement` matrix. `feature.audit_log` remains `owner_approval_required`; the implemented audit export is Developer-only.

AI counters now permit the existing promo metric context in addition to paid, demo, and internal contexts through the already text-based field. No new AI table or provider billing record is introduced.

SchoolGroup Creation Provenance Audit

No schema or migration is added for the Stage 5 authorization correction. Direct operational create/delete is restricted to Platform identities with global `SchoolGroup` capability, while tenant updates resolve the linked `SchoolGroup` before a row is loaded for mutation. Branch capacity continues to lock and recount the owning `SchoolGroup` in the mutation transaction.

`scripts/audit_schoolgroup_provenance.py` queries active `internal_sandbox` `SchoolGroups` that lack both `TenantProvisioningLink` and explicit `WorkspaceEntitlement` evidence. It reports only a numeric review key, hashed workspace reference, timestamp, aggregate operational counts, and review guidance. The PostgreSQL transaction is repeatable-read and read-only and is always rolled back. Findings are never automatically linked, reclassified, suspended, or deleted.

Promo Redemption And Grant Persistence

Migration 20260805_002_promo_redemption_and_grants creates:

- 1 `promo_activation_sessions` and `promo_activation_branch_selections` for short-lived, resumable, non-authoritative customer review state;
- 1 `promo_redemptions` for completed immutable redemption evidence;
- 1 `promo_grants` for immutable plan, capacity, scope, and effective-window authority;
- 1 `promo_grant_branch_assignments` for selected covered operational branches;
- 1 `promo_redemption_events` for redacted, idempotent durable outcomes.

Raw promo material is absent from every table. Snapshot hashes bind the definition, scope, tier, limits, and effective dates used at activation. Unique and partial indexes protect session/idempotency identity, one redemption per session, one grant per redemption, one active grant per workspace, branch assignment uniqueness, and durable operation/event deduplication.

`workspace_entitlements.promo_grant_id` links only promo entitlements.

`tenant_provisioning_links.pending_organization_id` is nullable for an existing aligned tenant and `promo_grant_id` identifies promo ownership. A check requires exactly one of subscription contract, demo request, or promo grant. The migration preserves existing rows, rebuilds equivalent SQLite constraints where required, installs PostgreSQL foreign keys/checks/indexes, and is idempotent.

Promo Code Foundation Persistence

`promo_codes` owns UUID identity, HMAC-SHA256 lookup hash and key ID, safe display fragments, controlled lifecycle, exact plan-bounded capacity, target anchors and deletion-safe snapshot, redemption-policy definition, one expiry policy, replacement predecessor, and actor/approval timestamps. Unique indexes protect UUID, lookup hash, and one replacement per predecessor. CHECK constraints enforce lifecycle, benefit, scope, positive capacities, version, redemption limits, date ordering, expiry XOR, and required primary targets.

`promo_code_branch_restrictions` preserves the selected existing branch ID and name even if the live branch reference is later removed. It grants no access. `promo_code_audit_events` stores actor, allowlisted before/after JSON, result, reason, operation/correlation identity, and failure code without raw code, lookup hash, HMAC key ID, or secret. Organization and actor references use ON DELETE SET NULL while snapshots preserve history. Migration 20260805_001_promo_code_foundation is additive, idempotent, transaction-safe, SQLite/PostgreSQL compatible, and has no data backfill.

M8B9 Demo Operations Persistence

`demo_access_policies` stores a workspace default and optional branch override with controlled profile/registry values. `demo_operation_audits` stores actor, scope, action, reason, before/after JSON, result, communication references, failure code, and operation key. Provisioning adds `expiry_policy`; email intents add variant payload JSON. Idempotent migration 20260727_003_m8b9_demo_operations uses the active DDL connection and existing pre-deploy PostgreSQL lock and statement timeout protections.

Deployment Migration Boundary

Database schema work belongs to `python scripts/run_migrations.py`, never the FastAPI web process. The command registers core and SaaS metadata, creates the repository's baseline metadata schema, and then calls the ordered, authoritative `db_migrations.run_pending_migrations` ledger. It logs each newly-applied identifier, is idempotent when the ledger is current, and exits nonzero on any schema or migration failure. Render must use it as a

Pre-Deploy Command so a failed migration prevents the new web version from becoming active. Migration-owned PostgreSQL lock and statement timeout protections remain in force. The migration process additionally applies a 10-second connection timeout, 5-second lock timeout, and 30-second statement timeout at PostgreSQL connection creation so baseline metadata.create_all(), schema_migrations initialization, every ordered migration, and commit are bounded. Flushed progress logging brackets each phase and migration identifier. Within a migration transaction, catalog inspection and DDL must use the same SQLAlchemy Connection; helper code must not inspect the Engine, because a second PostgreSQL session can wait on uncommitted DDL locks held by the first session and self-deadlock the migration process.

M8B8 AI Usage Persistence

ai_feature_usage_counters is unique by SchoolGroup, registered feature key, and metric context (internal_sandbox, demo, or paid). Its locked successful and reserved counts provide the concurrency-safe allowance boundary. ai_feature_usage_events is the durable operation ledger and uniquely deduplicates an operation within the same scope. Pending reservations become successful or failed; failure releases capacity and does not increment usage. Classification and plan snapshots preserve historical metric separation without changing workspace classification authority.

M8B7 Demo Communication Persistence

saas_demo_email_deliveries is the durable email outbox. It links each logical message to a demo request and optionally its provisioning aggregate, constrains email type and delivery state, and uniquely deduplicates delivery. system_notifications carries destination, deduplication, category, and severity metadata for owner demo events. Existing request, provisioning, lifecycle, entitlement, and conversion rows remain authoritative.

This document explains the conceptual TIS data model. It is intentionally high level and does not list every field. Use models.py, saas/models.py, and tests for exact implementation details.

Core Boundary Principle

TIS has three identity/data worlds that must never be casually mixed:

- 1 Platform data: platform owners, co-owners, developers, platform permissions, and cross-organization oversight.
- 1 SaaS account data: public signup, onboarding, billing, pending organizations, and payment/provisioning readiness.
- 1 Operational tenant data: provisioned school groups, branches, academic years, users, teachers, subjects, planning, timetable, calendar, and observations.

The safest mental model:

Platform Owner oversees many organizations.
SaaS account prepares an organization for subscription/provisioning.
Operational tenant data belongs to one provisioned school group/branch/year context.

Platform Identities

Represents: Platform Owner, Co-Owner, and Platform Developer identities.

Ownership boundary: Platform identities can operate outside ordinary tenant scope only through approved platform workflows.

Must never be mixed:

- 1 Platform Developer must not become Platform Owner through permission drift.
- 1 Platform users must not be treated as normal tenant users unless an explicit context workflow establishes scope.

Related files:

```
1 auth.py
1 models.py
1 permission_registry.py
1 main.py
```

SaaS Accounts

Represents: Public SaaS signup/login/account identities used before or alongside operational tenant access.

Ownership boundary: SaaS accounts own onboarding and billing/account state, not operational school records directly.

Must never be mixed:

```
1 SaaS account identity is not operational user identity.
1 SaaS account creation must not directly create live tenant data.
```

Related files:

```
1 saas/models.py
1 saas/service.py
1 saas/router.py
1 docs/adr/0002-separate-saas-identity-and-operational-users.md
```

Pending Organizations

Represents: An organization moving through SaaS onboarding before operational provisioning.

Ownership boundary: Pending organization data belongs to the SaaS onboarding/provisioning pipeline.

Must never be mixed:

```
1 Pending organization data is not the live school group until provisioning creates or connects operational records.
```

Related files:

```
1 saas/models.py
1 saas/service.py
1 saas/provisioning_service.py
```

Payment Records

Represents: Payment, billing, checkout, and provider-confirmed subscription state.

Ownership boundary: Payment state belongs to SaaS billing/subscription logic and should not be embedded directly into academic modules.

Subscription capacity uses three separate persisted dimensions.

`pending_organization_branches.estimated_system_users` and `estimated_teachers` store required non-negative onboarding estimates. `subscription_plans.max_branches`, `max_system_users`, and `max_teachers` store plan limits. Migration `20260729_001_subscription_capacity_dimensions` adds the new columns and assigns legacy organization-level system-user and teacher totals to the primary active branch only when all branch estimates are zero. Subsequent organization totals are derived compatibility summaries.

Must never be mixed:

- 1 Checkout return navigation must not be treated as verified payment.
- 1 Payment/provider details must not leak into operational teacher/planning/calendar logic.

Related files:

- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`
- 1 `saas/paddle_client.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/subscription_change_service.py`
- 1 `saas/subscription_plan_change_service.py`
- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `saas/billing_history_service.py`
- 1 `docs/adr/0003-paddle-payment-architecture.md`
- 1 `docs/adr/0004-webhook-only-payment-confirmation.md`

Entitlement And Subscription-Change Records

`EntitlementDefinition` defines commercial capability keys and value types. `PlanEntitlement` associates reviewed values with subscription plans. Runtime entitlement resolution starts from the provisioned school group, paid operational contract, and one confirmed active `PaymentSubscription`; it does not trust onboarding selections, page values, or pending checkout attempts.

`SubscriptionChangeRequest` is durable workflow/audit state for branch quantity, plan transition, and cancellation actions. It records requested/provider-observed state and lifecycle outcomes, but Paddle remains authoritative for monetary previews, proration, scheduled changes, transactions, and invoice documents.

Existing Workspace Paid Activation

Migration `20260806_002_existing_workspace_paid_activation` adds a direct paid activation context for an existing `SchoolGroup`. `ExistingWorkspacePaidActivation` stores workspace/account/owner identity, selected plan and interval, provider price, branch quantity and immutable selection hash, quote fingerprint and amounts, idempotency, checkout/payment/contract lineage, provider transaction/subscription identity, lifecycle, and safe failure state. Versioned branch rows retain stable branch snapshots, and append-only redacted events retain lifecycle evidence.

`CheckoutSession` and `PaymentAttempt` enforce an XOR between `PendingOrganization` and existing-workspace activation contexts. `SubscriptionContract` and `PaymentSubscription` permit a null `PendingOrganization` only when commercial authority is anchored through `SchoolGroup/contract`. `OrganizationBillingProfile` supports an exclusive `PendingOrganization` or `SchoolGroup` context. Explicit `PaymentCustomerWorkspaceAssociation` rows prevent implicit provider-customer sharing. They persist the provider address and business selected for each `SchoolGroup` so workspace billing identity remains stable even when the underlying customer is shared. Unique indexes enforce one unresolved activation per `SchoolGroup` and unique provider transaction/subscription mappings. No tenant-owned operational table is copied.

Guardrails:

- 1 `PaymentSubscription.quantity` is paid branch-capacity authority.

- 1 unresolved ownership, duplicate active relationships, provider mismatches, or incomplete evidence fail closed.
- 1 scheduled changes do not update effective local entitlements before verified provider/webhook evidence.
- 1 billing history is retrieved from Paddle and is not copied into a new local financial ledger.
- 1 invoice URLs are requested fresh and are not persisted.

Provisioning Jobs

Represents: Controlled work that turns a ready pending organization into operational school structures.

Ownership boundary: Provisioning bridges SaaS data and operational tenant data, but only through explicit platform-owner-visible workflows.

Must never be mixed:

- 1 Do not directly create operational tenant records from public signup or checkout return pages.
- 1 Do not create records under the wrong school group.

Related files:

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`

School Groups / Organizations

Represents: The top-level operational tenant boundary.

Ownership boundary: Most tenant operational data belongs to a school group directly or through branch/year relationships.

Must never be mixed:

- 1 Data from one school group must not appear in another school group's operational workflows.

Related files:

- 1 `models.py`
- 1 `main.py`

Workspace Classification Metadata

SchoolGroup is the canonical operational workspace record. M8B-1 adds:

- 1 globally unique, non-null `workspace_uuid`,
- 1 constrained/indexed `workspace_classification`,
- 1 constrained/indexed `workspace_lifecycle_status`.

Pre-provisioning intent remains on `PendingOrganization.workspace_intent`; identity intent remains on `SaaSAccount.account_purpose` and `User.is_internal_test_identity`. These fields are metadata only in M8B-1 and are not joined into payment, entitlement, authorization, or reset decisions.

Allowed workspace classifications are `internal_sandbox`, `customer_demo`, and `customer_paid`. Allowed lifecycle values are `provisioning`, `active`, `suspended`, and `archived`. New-schema tables use named check

constraints and non-null columns. The compatibility migration fills legacy values, adds indexes and PostgreSQL constraints, and installs equivalent SQLite value guards without rebuilding existing tables.

The read-only diagnostic resolves relationship presence across `TenantProvisioningLink`, `PendingOrganization`, `SubscriptionContract`, `PaymentSubscription`, and `PaymentCustomer`. It reports no Paddle identifiers. The controlled backfill is one transaction, defaults to dry-run, records a durable marker, and never performs a workspace conversion.

Commercial Entitlement Records

M8B-2 adds three normalized tables:

- 1 `workspace_entitlements`: one effective entitlement envelope per `SchoolGroup`, with type, lifecycle status, source, optional confirmed payment-subscription link, and a validity window reserved for later workflows.
- 1 `workspace_entitlement_values`: typed feature/limit values linked to the existing `EntitlementDefinition` catalog.
- 1 `branch_entitlements`: optional branch-level inherit/active/inactive intent linked to the branch, `SchoolGroup`, and effective workspace entitlement.

A partial unique index permits only one active workspace entitlement per `SchoolGroup`. Branch entitlement is unique per branch. Check constraints protect entitlement type, status, source, mode, and validity-window ordering. Service validation additionally rejects cross-tenant branch links, stale workspace-entitlement references, invalid typed values, and classification/entitlement mismatches.

Migration `20260722_003_commercial_entitlement_foundation` seeds one foundation entitlement for each existing classified workspace without changing classification. It does not create branch overrides, convert AI-Andalus, or infer demo/paid policy. Paid rows are linked only when exactly one persisted active/trialing subscription can be identified; ambiguity remains unresolved and fails closed.

SaaS Demo Request Records

M8B-3 adds three review-only tables:

- 1 `saas_demo_requests`: one customer submission with requester, pending organization, optional future workspace reference, immutable classification/commercial/entitlement snapshots, status, and transition timestamps.
- 1 `saas_demo_request_reviews`: one Platform Owner approval or rejection decision per request; rejected reviews require a reason.
- 1 `saas_demo_request_events`: append-only audit and internal-notification events for submission and every status transition.
- 1 `saas_demo_domain_eligibilities`: one unique normalized organization-domain reservation for a Customer Demo opportunity, optionally linked to historical request evidence or marked for manual review when legacy records are ambiguous.

A partial unique index permits only one Pending Review request per pending organization. Check constraints protect request status, review decision, event category/type/actor, classification snapshot, commercial-state snapshot, and rejection-reason requirements. Migration `20260722_004_saas_demo_request_workflow` creates the records without backfill and without changing existing onboarding, payment, or workspace data.

Migration `20260725_001_demo_domain_eligibility_policy` adds customer journey-intent fields, a nullable normalized domain snapshot, and the eligibility table with a database unique invariant. It backfills unambiguous Customer Demo records, reserves ambiguous duplicate domains for manual review, and never merges, deletes, reprovisions, or replaces customer workspaces. `scripts/diagnose_demo_domain_eligibility.py` is dry-run by default and can apply the same safe reservation backfill after review.

Historical detached eligibility rows are handled without schema changes through a Platform Owner-only maintenance service. It treats the eligibility primary key as the only deletion scope and checks domain-bearing organization, account, request, tenant-profile workspace, provisioning, subscription, and conversion relationships plus manual-review evidence before removal. The row is locked and re-analyzed, then deleted by exact ID, flushed, verified absent, and committed atomically with a durable external audit record. Normal customer eligibility rows remain durable.

Demo Workspace Provisioning Records

M8B-4 adds:

- 1 `saas_demo_workspace_provisioning`: one durable provisioning aggregate per demo request, with optional resulting SchoolGroup, workspace entitlement, and tenant link references; attempt count; status; activation time; and safe result/failure fields.
- 1 `saas_demo_provisioning_events`: append-only provisioning audit/internal events for started, completed, failed, and activation-completed outcomes.

`tenant_provisioning_links` now permits either `subscription_contract_id` or `demo_request_id`, with a check constraint requiring exactly one source. Existing paid links retain their subscription contract. Demo links cannot carry a contract, and request/source uniqueness prevents one approved request from identifying multiple operational tenants.

Migration 20260723_001_demo_workspace_provisioning generalizes the existing link without changing paid rows and creates the demo provisioning/event tables. Demo workspace, entitlement, link, request association, and activation updates are performed in one savepoint-backed transaction; failed workspace changes roll back while the outer provisioning aggregate retains the failure for audit and retry.

Demo Workspace Lifecycle Records

M8B-5 extends `saas_demo_workspace_provisioning` with reminder due/sent, demo expiration, expired, processing-status, last-processed, and failure-code metadata. `activated_at` remains the authority; persisted reminder and expiration timestamps are derived indexes and must validate as activation plus six and seven days.

Two normalized tables support lifecycle history and in-app delivery:

- 1 `saas_demo_lifecycle_events`: deduplicated audit records for reminder due/delivery, expiration start/completion, workspace suspension, access blocking, and processing failure.
- 1 `saas_demo_lifecycle_notifications`: recipient-scoped internal reminder notifications for the requesting SaaS account and active Platform Owners.

Migration 20260723_002_demo_workspace_lifecycle adds constrained lifecycle metadata, backfills activation-derived timestamps for existing active demos, aligns demo entitlement `effective_to`, and creates the event/notification tables. It does not expire data during migration. The separately scheduled processor performs expiration after a dry run.

Demo-To-Paid Conversion Records

M8B-6 adds:

- 1 `saas_demo_to_paid_conversions`: one durable conversion aggregate per demo request, provisioning record, and SchoolGroup. It links the existing demo workspace to the confirmed SubscriptionContract and PaymentSubscription, records the previous demo entitlement and resulting paid entitlement, and tracks requested, processing, completed, or failed status.
- 1 `saas_demo_conversion_events`: append-only audit and internal-notification history for conversion requested, started, completed, and failed outcomes.

Migration 20260723_003_demo_to_paid_conversion also permits the terminal converted demo lifecycle processing status. Conversion never creates another SchoolGroup or tenant link. One atomic workspace transaction ends the demo entitlement, creates the paid entitlement, relinks existing branch entitlements, changes the SchoolGroup classification, and moves the existing tenant link from the preserved demo-request source to the confirmed subscription contract. Provider payment records commit independently and remain intact if workspace conversion rolls back.

Related files:

```
1 workspace_classification.py
1 saas/workspace_classification_service.py
1 saas/workspace_classification_admin_service.py
1 scripts/diagnose_workspace_classification.py
1 scripts/backfill_workspace_classification.py
1 docs/adr/0008-workspace-classification-foundation.md
1 commercial_entitlements.py
1 saas/commercial_validation_service.py
1 saas/workspace_entitlement_service.py
1 saas/branch_entitlement_service.py
1 saas/commercial_state_service.py
1 docs/adr/0009-commercial-state-and-entitlement-resolution.md
1 demo_workflow.py
1 saas/demo_provisioning_service.py
1 saas/demo_lifecycle_service.py
1 saas/demo_conversion_service.py
1 scripts/process_demo_lifecycle.py
1 authorization.py
1 saas/provisioning_service.py
1 docs/adr/0011-demo-workspace-provisioning-and-commercial-source-links.md
1 docs/adr/0012-seven-day-demo-lifecycle-and-access-enforcement.md
1 docs/adr/0013-demo-to-paid-workspace-conversion.md
1 saas/demo_request_service.py
1 docs/adr/0010-review-only-saas-demo-requests.md
```

Branches

Represents: Campuses or branches inside a school group.

Ownership boundary: Branches scope users, teachers, planning, timetable, academic calendar, branding, and reports.

Must never be mixed:

- 1 Branch-scoped data must not silently cross campuses.
- 1 Platform context switching must remain explicit.

Related files:

- 1 `models.py`
- 1 `main.py`
- 1 `ui_shell.py`

Academic Years

Represents: The academic period used to scope operational records.

Ownership boundary: Planning, timetable, subjects, calendar, and related data may depend on active academic year.

Must never be mixed:

- 1 Do not copy or read current-year data into another year unless the workflow explicitly does so.

Related files:

- 1 `models.py`
- 1 `main.py`
- 1 `year_copy.py`

Operational Users

Represents: Users who work inside an operational school group/branch/year context.

Ownership boundary: Operational users belong to tenant structures and are governed by roles, permissions, branch scope, and active status.

Must never be mixed:

- 1 Operational users are not SaaS accounts by default.
- 1 Tenant users should not gain platform owner access.

Related files:

- 1 `models.py`
- 1 `auth.py`
- 1 `routers/users.py`

Roles And Permissions

Represents: Role packages, permission keys, platform developer permissions, and route/action authorization.

Ownership boundary: Permissions decide what a user can view or change in the current scope.

Must never be mixed:

- 1 UI hiding is not enough; protected actions need route/service checks.
- 1 Owner controls must remain outside developer-assignable permission drift.

Related files:

- 1 `permission_registry.py`

```
1    role_permission_service.py
1    authorization.py
1    auth.py
```

Teachers

Represents: Teacher records, qualifications, capacity, workload, and staffing-relevant academic data.

Ownership boundary: Teacher records are tenant/branch/year-sensitive operational data.

Must never be mixed:

```
1    Teacher data from one branch or school group must not appear in another tenant's planning/reporting.
```

Related files:

```
1    routers/teachers.py
1    teacher_qualifications.py
1    teacher_capacity.py
1    models.py
```

Subjects

Represents: Subject catalog, colors, requirements, qualification relationships, planning and timetable dependencies.

Ownership boundary: Subjects are academic configuration data inside tenant/year context.

Must never be mixed:

```
1    Subject changes can affect planning, timetable, teacher matching, and reports; update all affected docs/tests.
```

Related files:

```
1    routers/subjects.py
1    subject_colors.py
1    models.py
```

Sections / Classes

Represents: Class sections used by planning and timetabling.

Ownership boundary: Sections belong to operational branch/year structures.

Must never be mixed:

```
1    Section structures should not cross branch/year boundaries accidentally.
```

Related files:

```
1    routers/planning.py
1    models.py
```

Workforce Planning

Represents: Teacher assignments, homeroom ownership, capacity, workload, subject coverage, and staffing needs.

Ownership boundary: Planning is operational data tied to school group, branch, academic year, teachers, subjects, and sections.

Must never be mixed:

- 1 Planning changes can affect dashboards, reports, timetable, and staffing decisions.

Related files:

- 1 routers/planning.py
- 1 teacher_capacity.py
- 1 homeroom_defaults.py

Timetable

Represents: Weekly lesson placement, timetable settings, durable versions, immutable input snapshots, one active-version pointer per scope, placement locks, and future generation-run evidence.

`TimetableNonTeachingBlock` retains legacy start/end and period columns and adds `placement_mode`, `insert_after_period`, and `duration_minutes`. The additive migration defaults existing rows to `fixed_time`; compatibility composition does not rewrite their stored clock values. After-period metadata is authoritative for newly inserted blocks.

Ownership boundary: Timetable data depends on planning, teacher, subject, section, branch, and year context.

Must never be mixed:

- 1 Timetable edits must preserve scheduling constraints and scope.
- 1 `TimetableEntry` belongs to exactly one version and preserves section/teacher

collision guarantees inside that version. Its branch/year must match the pointed version.

- 1 `TimetableActiveVersion` is the only active-selection authority and its

`SchoolGroup/Branch/Academic-Year` tuple must match the target version exactly.

- 1 Active, superseded, and archived versions are immutable. Mutable compatibility

edits occur on a draft copy.

Related files:

- 1 routers/timetable.py
- 1 timetable_logic.py
- 1 timetable_version_service.py
- 1 timetable_snapshot_service.py
- 1 docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md

Academic Calendar

Represents: Events, academic dates, responsibilities, exports, and calendar settings.

Ownership boundary: Calendar records are branch/year/tenant scoped.

Must never be mixed:

- 1 Calendar events for one tenant or branch must not appear in another.

Related files:

- 1 routers/academic_calendar.py
- 1 templates/academic_calendar.html

Observations

Represents: Teacher observation records, feedback, scoring, evidence, and history.

Ownership boundary: Observation data is sensitive tenant operational data.

Must never be mixed:

- 1 Observation records must remain tenant-scoped and permission-protected.

Related files:

- 1 routers/observations.py
- 1 templates/observation_*.html

Knowledge Management System Artifacts

Represents: Markdown docs, ADRs, module history, generated PDF, manifest, and Knowledge Center read-only status.

Ownership boundary: Markdown under docs/ is source of truth. Generated artifacts under static/docs/ are snapshots.

Must never be mixed:

- 1 The app must not silently rewrite Markdown source docs.
- 1 The PDF must not be edited manually.
- 1 Protected documentation access should go through owner-only routes.

Related files:

- 1 docs/
- 1 scripts/generate_docs_pdf.py
- 1 static/docs/
- 1 knowledge_service.py

Tenant Isolation Rules

- 1 Always know the current school group, branch, and academic year.
- 1 Do not assume platform users have tenant scope.
- 1 Do not assume SaaS accounts have operational records.
- 1 Keep pending SaaS organization state separate from provisioned operational data.
- 1 Preserve route permissions and service-level checks.

- 1 Tests touching cross-tenant data should be treated as high value.

TIS Development Standards

docs/engineering/DEVELOPMENT_STANDARDS.md

These standards are mandatory for future human developers, Codex conversations, ChatGPT conversations, and technical reviewers working on TIS.

Inspect Before Editing

Before changing files:

- 1 inspect the current code with `rg` and targeted file reads,
- 1 understand the affected route/service/template/model,
- 1 read relevant docs, ADRs, and module history,
- 1 check tests for the touched module,
- 1 check the current branch and working tree.

Do not code from memory when local context is available.

Plan Before Implementation

For non-trivial work:

- 1 identify the affected modules,
- 1 identify tenant/permission/identity boundaries,
- 1 identify documentation updates,
- 1 identify tests or smoke checks,
- 1 keep the implementation path small.

Planning does not need to be long, but the risk boundaries must be clear.

Keep Changes Small And Scoped

- 1 Implement only the approved request.
- 1 Avoid unrelated refactors.
- 1 Avoid formatting churn in unrelated files.
- 1 Preserve existing patterns unless there is a clear reason to change them.
- 1 Do not move code across modules as a side effect.

Preserve Tenant Isolation

Tenant isolation is non-negotiable.

Always protect:

- 1 school group boundaries,
- 1 branch boundaries,
- 1 academic year scope,

- 1 user role/permission scope,
- 1 observation and teacher data sensitivity.

Any change that touches tenant scope should be treated as high risk.

Preserve Login And Platform Owner Flows

Do not casually change:

- 1 /login,
- 1 session cookies,
- 1 platform owner detection,
- 1 platform developer permissions,
- 1 /platform,
- 1 /platform/knowledge,
- 1 scope switching,
- 1 access denied handling.

Platform developers are not platform owners unless existing owner logic explicitly treats them that way.

Protect SaaS, Payment, And Provisioning

Do not casually change:

- 1 /saas/signup,
- 1 /saas/login,
- 1 /saas/account,
- 1 SaaS onboarding steps,
- 1 Paddle checkout/client code,
- 1 payment confirmation logic,
- 1 billing state,
- 1 provisioning readiness,
- 1 provisioning jobs.

Webhook-confirmed payment is authoritative. Checkout return pages are not proof of payment.

Database And Migration Rules

- 1 Never modify `tis.db` unless explicitly approved.
- 1 Never modify migrations casually.
- 1 Do not change `models.py` without understanding migration/repair implications.
- 1 Do not add schema changes without tests and documentation.
- 1 Never use destructive database operations without explicit approval.

Landing Website Rule

The public landing website is separate:

- 1 implementation: `tis-landing-website/`,
- 1 marketing docs: `docs/marketing/`,
- 1 ADRs: 0001, 0007.

Do not modify landing page code during backend or operational app tasks unless explicitly approved.

Protected Documentation Rule

- 1 Do not expose generated PDFs through direct public UI links.
- 1 Use owner-protected routes for PDF access.
- 1 Markdown source docs are reviewed development artifacts.
- 1 The app must not silently rewrite Markdown source docs.

Production Memory And Render Stability

Render memory is a hard production budget. Treat a 512 MB service as constrained and design routes, services, and assets accordingly.

Strict rules:

- 1 Do not load, parse, or cache complete large datasets at startup or on a normal first request when a scoped lookup, streaming parser, pagination, or bounded cache can be used.
- 1 Do not keep both raw and transformed copies of large JSON, CSV, Excel, PDF, image, or uploaded payloads in memory unless explicitly justified and validated.
- 1 Do not run duplicate template renders in production request paths. Diagnostic pre-renders must be removed or gated behind an explicit environment flag.
- 1 Do not emit stage-by-stage debug logs at warning level during normal traffic. Production diagnostics must be opt-in and should use appropriate log levels.
- 1 Filter by tenant, school group, branch, academic year, and permission scope before materializing query results.
- 1 Keep caches bounded by size or scope. Global caches for user-facing lookup data must be justified by measured memory impact.
- 1 Large exports should stream or build bounded artifacts; avoid repeatedly constructing large in-memory workbooks, PDFs, or payloads inside loops.
- 1 New static assets, videos, generated files, and location/reference datasets must be reviewed for repository size, runtime memory, and deployment impact.
- 1 Any feature that adds broad data aggregation, new startup work, large reference files, or route-level diagnostics must include a memory/performance smoke check before deployment.

Minimum validation for memory-sensitive changes:

- 1 run targeted tests for the touched module,
- 1 run a compile/import check for changed Python modules,
- 1 measure peak memory locally when a route or service parses large data,

- 1 review production logs after deployment for restarts, OOM symptoms, and unexpected warning spam.

Commit And Push Rule

- 1 Do not commit unless explicitly requested.
- 1 Do not push unless explicitly requested.
- 1 Do not merge unless explicitly requested.
- 1 Final reports should describe changes and validation clearly.

KMS Update Rule

Every meaningful implementation must update KMS if affected:

- 1 docs/TIS_MASTER_CONTEXT.md,
- 1 docs/PROJECT_STATE.md,
- 1 docs/CHANGE_HISTORY.md,
- 1 docs/AI_PROJECT_CONTEXT.md,
- 1 ADRs if major decisions changed,
- 1 module history for module-specific before/after state,
- 1 engineering docs if module maps, architecture, data model, standards, design philosophy, roadmap, or flows changed.

Regenerate the PDF if included docs changed.

Knowledge Impact Assessment

Every final implementation report must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

A task is not complete until KIA is assessed.

Validation Standards

Run the narrowest meaningful validation:

- 1 compile checks for Python scripts/modules,
- 1 targeted pytest tests when behavior changes,
- 1 route/template smoke checks when pages are touched,
- 1 PDF generation when docs change,
- 1 frontend checks when landing/frontend code changes.

If validation cannot be run, say why.

TIS UI UX Design Philosophy

docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

TIS UI/UX Design Philosophy

Restoring Expanded State After A Server Redirect

TIS is server-rendered FastAPI/Jinja, not an SPA, so expand/collapse state (native <details>, dialogs) lives only in the DOM and is normally lost the moment an action redirects to a fresh render. The sanctioned fix reuses two things that already existed rather than introducing a new mechanism: the existing `return_to` form-field/query-param convention, and its `#fragment` preservation already built into `redirect_utils.redirect_with_notice()` / `redirect_with_error()` (moved out of `main.py` so any router can share the same open-redirect guard, `redirect_utils.safe_redirect_path()`).

A route that wants the user to land back on a specific element accepts a `return_to` value shaped `<path>[?query]#<element-id>`, validates it with `redirect_utils.safe_redirect_path(return_to, default=<page path>)`, and redirects there. The shared client script `static/js/reopen-on-load.js`, loaded globally from `base.html`, runs on `DOMContentLoaded`, resolves a target id from `window.location.hash` (or an optional inline `window.__tisReopenTargetId` a page can render for a non-redirect, in-place response), and if the matching element is a `<details>`, opens it and scrolls it into view. A missing, stale, or renamed id is a silent no-op by design - `document.getElementById` never throws, so this can never break a page load.

Do not invent a page-local variant of this pattern. Give the element to reopen a stable, predictable id, pass `return_to` through the existing convention, and let the shared script do the reopening. Planning's Remove Demand action (`routers/planning.py:delete_planning_subject_demand`) is the reference implementation.

Progressive Disclosure And Commercial States

Expected commercial states use branded, actionable pages rather than generic errors or internal state labels. Expired customers see data-preservation, plan or renewal, support, and sign-out guidance. Dense Platform Owner pages show only the decision summary and common actions initially; lifecycle processing, branch/feature overrides, audit evidence, and technical metadata belong under More Actions, Advanced Settings, and Activity History. Presentation simplification must not remove operational controls.

TIS should feel like a clean, premium academic SaaS platform. It should not feel like a generic admin dashboard, spreadsheet wrapper, or decorative marketing shell disconnected from real product value.

Overall Product Identity

Design principles:

- 1 professional,
- 1 light,
- 1 trustworthy,
- 1 academic,
- 1 calm,
- 1 data-aware,
- 1 premium without being flashy.

The product serves serious academic operations. UI should reduce confusion and support confident decisions.

Operational FastAPI App

The operational app should prioritize:

- 1 clarity,
- 1 role-based navigation,
- 1 data density,
- 1 fast scanning,
- 1 predictable controls,
- 1 strong permission boundaries,
- 1 branch/year context visibility.

Avoid:

- 1 generic admin clutter,
- 1 decorative cards that do not help workflows,
- 1 large marketing-style hero sections inside operational tools,
- 1 hiding important state behind vague labels,
- 1 layouts that make repeated operational use slow.

Operational pages should help users answer:

- 1 What am I looking at?
- 1 Which school/branch/year is active?
- 1 What needs action?
- 1 What can my role do here?
- 1 What changed after I saved?

Platform Owner Console

The Platform Owner Console should feel like a controlled operations console.

Priorities:

- 1 global visibility,
- 1 owner/developer separation,
- 1 clear organization switching,
- 1 safe access to sensitive tools,
- 1 explicit status labels,
- 1 no accidental tenant context confusion.

Avoid:

- 1 broad controls without owner-only checks,
- 1 exposing platform tools to developers by appearance alone,
- 1 unclear organization/branch state.

Knowledge Center

The Knowledge Center is an internal owner utility, not a marketing page.

Priorities:

- 1 KMS health,
- 1 source coverage,
- 1 freshness status,
- 1 protected PDF actions,
- 1 ADR/change/module visibility,
- 1 KIA policy reminder.

Avoid:

- 1 direct public static PDF links,
- 1 regenerate actions until approved,
- 1 app-side Markdown rewriting,
- 1 decorative presentation that hides status.

SaaS Onboarding Pages

SaaS onboarding should feel guided, calm, and customer-facing.

Priorities:

- 1 clear next step,
- 1 progress visibility,
- 1 plain customer language,
- 1 reassurance around setup and billing,
- 1 no internal engineering terms.

Customer-facing language should avoid:

- 1 tenant,
- 1 provisioning,
- 1 M1/M2/M3/M4/M5,
- 1 schema,
- 1 migration,
- 1 internal role mechanics.

Use customer language such as:

- 1 organization,
- 1 school,
- 1 branch or campus,

- 1 setup,
- 1 plan,
- 1 billing,
- 1 account,
- 1 getting your workspace ready.

Next.js Landing Website

The landing website should use premium storytelling and strong visual assets.

Priorities:

- 1 clear problem/solution narrative,
- 1 actual product credibility,
- 1 school operations language,
- 1 strong visual hierarchy,
- 1 polished screenshots or generated assets when appropriate,
- 1 conversion path into demo/signup.

Avoid:

- 1 weak fake 3D visuals,
- 1 generic SaaS gradients without product specificity,
- 1 vague claims unsupported by product reality,
- 1 internal terms like tenant/provisioning/milestones,
- 1 cluttered feature walls.

The landing page source of truth is `tis-landing-website/`, not legacy FastAPI landing files.

Visual System Direction

Future design system should support:

- 1 consistent cards, tables, filters, status badges, and action buttons,
- 1 clear empty/loading/error states,
- 1 accessible contrast,
- 1 stable layouts across modules,
- 1 consistent icon and label patterns,
- 1 role-aware navigation,
- 1 responsive behavior without losing operational density.

Compact Descriptions And Tooltips

Compact-description tooltips are an explicit progressive-disclosure treatment, not a default for ordinary text. Shared JavaScript may enhance elements marked with `data-compact-description` and the maintained allowlist of approved compact components. It must not infer tooltip behavior from generic `p` or `small` tags, or from broad class

fragments such as description, subtitle, note, lede, helper, or supporting.

Operational status, helper, validation, and record text remains visible inline. Intentional tooltips must retain keyboard focus behavior, Escape dismissal, and an accessible relationship between their trigger surface and tooltip content. Native titles and accessible names on unrelated controls and visualizations are not removed by compact-description processing.

Tone And Copy

Internal app copy:

- 1 concise,
- 1 action-oriented,
- 1 operationally clear.

Customer-facing copy:

- 1 plain,
- 1 confident,
- 1 benefit-oriented,
- 1 free of internal implementation terms.

Platform owner copy:

- 1 precise,
- 1 status-driven,
- 1 explicit about access and risk.

TIS Product Roadmap

docs/engineering/PRODUCT_ROADMAP.md

Talent & Potential M4 Status

Implemented: SchoolGroup-wide Assessment Cycle Draft/Open/Closed lifecycle, explicit-time dynamic eligibility preview, atomic frozen Student population, historical placement snapshots, full population integrity fingerprint, dedicated permission boundaries, Branch-filtered identifiable reads, and operational audit. Deferred to later milestones: assessor assignment, Student Assessment, results, Review Candidate instances/evaluation, Official Identification, analytics, AI, development plans, and late-population exception workflow.

Simplified Timetable Workflow And Published Visibility

Stage 5.2 is implemented with one Working Timetable, safe archive-based deletion, customer validation/publication language, secondary history, strict active-pointer official reads, and scoped My Timetable. Stage 5.1 solver constraints and concurrency are unchanged. Availability, rooms/resources, preferences, and student identities remain future separately approved work.

Production generation now dispatches one on-demand Render Workflow task per durable run. The always-running solver worker is no longer required; solver rules and Stage 5.2 publication/visibility behavior remain unchanged.

M3 Promo Redemption And Organization Activation

Completed secure customer lookup, resume-safe activation sessions, capacity review, exact branch selection, immutable redemption/grant authority, promo workspace entitlement, promo tenant-source ownership, existing-aligned and onboarding activation, M1 adapter/enforcement, expiry resolution, and redacted audit. Staff/teacher overage blocks without mutation; internal sandbox conversion, Paddle billing, promo renewal/conversion, and customer transfer remain outside M3.

M2 Promo Code Foundation And Platform Management

Completed secure promo definition persistence, exact plan-bounded capacity, scope and validity policy, lifecycle governance, branch-restriction snapshots, durable redacted audit, granular Platform permissions, and Platform Console administration. M3 now consumes approved definitions through its separate customer redemption and grant lifecycle. Communication, billing treatment, promo renewal, and broader conversion remain future work.

M8B9 Demo Operations, Notifications, And Testing

Completed generic Platform Owner lifecycle operations, manual production lifecycle execution, Standard/Full/Custom policies, isolated branch overrides, durable audit, and M8B7-based customer communications while preserving M8B8 usage history and Customer Demo classification.

M8B8 AI Entitlements And Commercial Foundation

Completed the commercial and usage boundary required before executable AI features: controlled registry, central decisions, two-use-per-feature Customer Demo allowance, Enterprise AI mapping, durable tenant-safe accounting, and upgrade guidance. Actual AI tools and M8B9 owner operations remain future work.

M8B7 Demo Customer Journey

Completed approval-orchestrated activation, durable lifecycle communications, Notification Center integration, shared-shell demo status, data-preserving expiry communication, and verified-payment continuation of the same active or expired workspace. M8B8 AI usage limits and M8B9 owner testing controls remain separate future work.

This roadmap summarizes completed, current, next, and future product directions. It is not a release commitment; it is the current planning map for developers, owners, and reviewers.

Completed

SaaS Identity Foundation

Established SaaS account signup, login, account area, and the boundary between SaaS accounts, platform identities, and operational tenant users.

Pending Organizations Zone

Created pending organization structures and owner-facing review concepts for organizations moving through SaaS onboarding.

Plans And Billing Foundation

Added plan catalog, plan selection, billing status, checkout summary, checkout return/cancel, and payment/billing service boundaries.

Paddle Payment Collection

Added Paddle-oriented payment architecture with provider logic isolated behind service/client modules.

Tenant Provisioning Engine

Added provisioning workflows and jobs to create or connect operational school group/branch/year/user records from ready pending organizations.

KMS Foundation

Created source-of-truth Markdown docs, change history, ADRs, module history, AI project context, generated PDF booklet, and manifest.

Platform Owner Knowledge Center

Added protected owner-only Knowledge Center for KMS health, source freshness, manifest metadata, ADRs, module history, and protected PDF view/download.

KMS v3.0 Phase 3A

Added engineering handbook foundation with module map, repository architecture, user/system flows, and onboarding structure.

M7 Subscription Management

Completed entitlement foundations, customer Subscription Management portal, paid branch quantity management, upgrades and scheduled downgrades, provider-authoritative proration, cancellation/reversal, centralized lifecycle/action policy, Paddle billing history, protected invoice downloads, and webhook/reconciliation safeguards.

Automatic KMS Enforcement

Added root AI instructions, machine-readable task KIA, major-change detection, read-only PDF/manifest checks, pull-request and dev CI validation, and a required KMS gate before master deployment.

M8B-1 Workspace Classification Foundation

Added constrained workspace identity/classification/lifecycle metadata, pre-provisioning and identity intent fields, validation-only services, read-only owner visibility, and safe diagnostic/backfill tooling. No commercial behavior or

conversion workflow consumes the metadata yet.

M8B-2 Commercial State And Entitlement Foundation

Added normalized workspace and branch entitlement records, typed feature/limit values, read-only effective entitlement/commercial-state resolvers, conservative validation, and Platform Owner-only visibility. No enforcement or customer workflow changed.

M8B-3 Demo Request Workflow

Added the post-onboarding Request Demo or Subscribe Now choice, validated review-only demo requests, customer status/withdrawal, a Platform Owner review queue, constrained review decisions, and durable audit/internal-notification events. Approval does not provision or activate a demo.

M8B-4 Demo Workspace Provisioning And Activation

Added Platform Owner-triggered provisioning for approved customer-demo requests. The flow reuses the operational provisioning engine, creates an explicit demo entitlement and demo-sourced tenant link, activates atomically, records retryable failure and activation events, and creates no paid billing evidence.

M8B-5 Standard Customer Demo Lifecycle

Added the activation-based seven-day demo clock, Day 6 internal reminders, Day 7 atomic expiration, data-preserving workspace suspension, server-side web/API access enforcement, customer subscription guidance, owner lifecycle visibility, and a dry-run/default scheduled processor.

M8B-6 Demo-To-Paid Workspace Conversion

Added provider-confirmed conversion of an eligible active Customer Demo into Customer Paid on the same SchoolGroup and tenant link. The flow preserves operational identity and data, replaces the demo entitlement with the confirmed subscription-backed entitlement atomically, records durable conversion history, supports safe retry after failure, and removes completed conversions from demo lifecycle processing.

M8B8 AI Entitlements And Commercial Foundation

Added the centralized AI feature registry, permission/commercial/plan decision service, per-feature Customer Demo allowance, and separated durable usage accounting. No AI business feature or M8B9 operational control is included.

Current

Smart Timetable Version And Publication Foundation

Stages 2, 3, 3.5, 4, and 5.1 are complete: durable timetable versions, snapshots/fingerprints, exact-scope active selection, locks, generation-run schema, imported-current migration, and copy-on-write compatibility, canonical composed day timelines, structural readiness, visible history, locks, draft validation, same-scope comparison, explicit version export, archive, atomic publication, and worker-isolated CP-SAT generation/regeneration. Generated results remain unpublished until the separate publication flow. Stage 5.2 may simplify customer terminology/history and add Delete Working Timetable and published-only teacher My Timetable visibility.

KMS Enforcement Review

Review enforcement against real development tasks, keep classification conservative, and tune only demonstrated false positives without permitting silent no-impact declarations.

Landing / Customer Experience Preparation

The product is preparing for stronger public customer journey work from landing page storytelling through SaaS signup, onboarding, billing, and operational access.

Next

M8B-7 Next Approved Commercial Package

Proceed only from a separately approved M8B-7 specification. Manual demo extension/override, internal-sandbox conversion, special migrations, memberships, ownership transfer, and expanded role architecture remain unimplemented.

Premium Landing Page Redesign

Improve public website quality, product storytelling, visuals, and conversion clarity while preserving the separate Next.js source-of-truth.

Customer Journey From Landing To Signup

Clarify the path from public website to SaaS signup, plan selection, onboarding, checkout, and workspace readiness.

SaaS Onboarding Refinement

Improve onboarding language, progress, validation, customer reassurance, and platform owner visibility.

Paddle Live Configuration

Configure live Paddle behavior when the payment account and production readiness are approved.

Existing Workspace Paid Activation

Implemented the SchoolGroup-anchored M4C Paddle activation path for verified owners of preserved existing customer workspaces. Sandbox validation and migration rollout remain deployment gates. Starter stays unavailable until complete restricted-branch enforcement is separately proven.

Expand Entitlement Enforcement

Extend plan entitlement checks beyond the current approved pilot only when commercial rules for each module are reviewed.

Subscription Operations Hardening

Continue production validation, renewal/payment-failure handling, reconciliation observability, and owner support workflows without weakening Paddle authority or tenant isolation.

Future

Constraint-Based Smart Timetable Generation

Stage 5.1 delivered structural-readiness-gated CP-SAT generation, independent candidate validation, durable background execution, lock preservation, explicit regeneration diversity, and unpublished version persistence. Future solver work may add only separately approved availability, rooms/resources, cross-campus coordination, additional normalized slot semantics, preferences, or quality objectives.

AI Academic Assistant

Support academic leaders with intelligent recommendations based on verified tenant data.

AI Assessment Generation

Generate curriculum-aware assessments or question banks when curriculum/planning data is reliable enough.

AI Observation Improvement Plans

Assist supervisors with improvement plans, follow-up suggestions, and structured feedback from observation records.

AI Calendar Activity Planning

Suggest academic calendar activities, reminders, and coordination plans.

AI Dashboards And Academic Analytics

Provide richer academic insight, trends, risks, and multi-branch intelligence.

Multi-Branch Executive Dashboards

Give leadership cross-branch visibility into staffing, workload, observations, calendar, and planning health.

Curriculum And Planning Workflows

Expand from operational staffing/planning toward curriculum-aware academic planning if approved.

Reporting Improvements

Improve exports, dashboards, executive summaries, and decision-support reports.

Mobile Or Portal Expansion

Explore teacher/mobile/parent-style portals only if later approved and aligned with product strategy.

Roadmap Guardrails

- 1 Do not build AI features before data quality, permissions, and plan boundaries are clear.
- 1 Do not extend subscription lifecycle actions without verified Paddle behavior, fail-closed reconciliation, and updated KMS records.
- 1 Do not redesign landing/customer journey without preserving the Next.js boundary.
- 1 Do not expose internal terms to customers.
- 1 Update KMS docs and roadmap after meaningful roadmap changes.

TIS Rejected Architectural Decisions

docs/engineering/REJECTED_DECISIONS.md

This document records significant ideas that were considered and rejected. It helps future developers and AI assistants avoid reopening old decisions without new evidence.

Rejected decisions are not failures. They are part of the design record.

Single Application For Landing And Portal

Date: 2026-06-26

Context: TIS needs both a public marketing site and a protected operational app. The public site needs premium storytelling and conversion design; the app needs authenticated academic operations.

Proposed solution: Use the FastAPI/Jinja app for both the public landing website and the operational portal.

Why rejected: The two surfaces have different runtimes, audiences, design needs, deployment concerns, and risk profiles. Mixing them would make backend/operational tasks more likely to accidentally change the public website.

Chosen solution: Keep `tis-landing-website/` as a separate Next.js public landing website and keep the FastAPI app as the operational portal.

Long-term consequences: Developers must respect the landing/portal boundary. Landing changes require explicit approval and must use the Next.js source of truth.

Stripe Instead Of Paddle

Date: 2026-06-26

Context: TIS needs subscription payment collection, billing state, and future subscription lifecycle workflows.

Proposed solution: Use Stripe as the payment provider.

Why rejected: The current approved architecture is Paddle-oriented. Changing providers would reopen checkout, webhook, billing, tax/compliance, customer portal, and subscription lifecycle assumptions.

Chosen solution: Use Paddle behind service/client boundaries in `saas/`.

Long-term consequences: Provider-specific behavior must remain isolated. A future provider change would require a new ADR and careful migration plan.

Immediate Tenant Provisioning

Date: 2026-06-26

Context: Public signup and SaaS onboarding collect organization data before a school is ready for live operational access.

Proposed solution: Create operational school group, branch, user, and academic records immediately after signup or checkout return.

Why rejected: Immediate provisioning risks creating live tenant data before payment is verified, before onboarding is complete, or before platform owner review. Checkout return pages are not authoritative payment confirmation.

Chosen solution: Delay tenant provisioning until payment/readiness is verified and use platform-owner-visible provisioning jobs.

Long-term consequences: There are more states between signup and operational login, but tenant data is safer and provisioning can be reviewed/retried.

Public Documentation Access

Date: 2026-06-26

Context: The KMS generates a PDF under `static/docs/`, but the content includes internal architecture, roadmap, and operational guidance.

Proposed solution: Link directly to the generated PDF through public static paths.

Why rejected: Internal documentation should not be exposed as a public asset. Static file serving is not authorization-aware.

Chosen solution: Expose the PDF through protected Platform Owner routes only.

Long-term consequences: The UI must not link directly to `/static/docs/...` Future storage may move generated docs outside public static storage.

Documentation Without ADRs

Date: 2026-06-26

Context: TIS decisions span SaaS identity, payments, provisioning, landing architecture, and KMS governance.

Proposed solution: Keep only a master context and change history.

Why rejected: Chronological history does not explain major decision tradeoffs deeply enough. Future developers would repeatedly reopen settled architecture questions.

Chosen solution: Maintain ADRs under `docs/adr/` for major accepted decisions and this rejected-decision record for important alternatives.

Long-term consequences: Major decisions require explicit traceability. ADRs and rejected decisions must be updated when architecture changes.

Weak Generic Landing Page Design

Date: 2026-06-26

Context: The public landing website is the first impression for schools evaluating TIS.

Proposed solution: Use a generic SaaS layout with vague claims, weak fake 3D visuals, or internal implementation language.

Why rejected: TIS needs credibility with school leaders. Generic visuals and internal language reduce trust and do not communicate academic operations value.

Chosen solution: Use premium storytelling, strong visual assets, clear customer language, and the separate Next.js landing visual system.

Long-term consequences: Landing work should be deliberate, visually strong, and customer-facing. Avoid terms like tenant, provisioning, and internal milestone labels.

Mixing SaaS Identities With Operational Users

Date: 2026-06-26

Context: TIS has public SaaS accounts, operational tenant users, and platform identities.

Proposed solution: Use one identity concept for signup, billing, tenant access, and platform administration.

Why rejected: This would weaken security and make it easy to confuse pending accounts with live operational users or platform owners.

Chosen solution: Keep SaaS account identity, operational tenant identity, and platform identity distinct.

Long-term consequences: Developers must always ask which identity world a change belongs to. Provisioning bridges worlds only through approved flows.

App-Side Automatic Documentation Rewriting

Date: 2026-06-26

Context: The KMS requires docs to remain current after meaningful implementation work.

Proposed solution: Let the running app automatically rewrite Markdown docs when it detects staleness.

Why rejected: Source-of-truth docs must be reviewed development artifacts. Silent rewriting would damage reviewability and could introduce incorrect historical records.

Chosen solution: Developers or AI assistants update Markdown docs as part of approved implementation work. The app may detect freshness and expose status, but not rewrite source docs.

Long-term consequences: Knowledge Impact Assessment remains a required human/AI workflow step. Future regenerate actions may rebuild PDFs only from reviewed Markdown.

Uncontrolled Regenerate Button

Date: 2026-06-26

Context: The Knowledge Center can show whether the PDF snapshot is current or stale.

Proposed solution: Add an immediate button that regenerates docs and/or source files from the app.

Why rejected: Phase 2C is read-only. Runtime regeneration raises deployment persistence, audit, concurrency, and source-control questions.

Chosen solution: Do not add regenerate behavior until separately approved. Keep source docs updated through reviewed development work.

Long-term consequences: A future regenerate action must be explicit, owner-only, audited where appropriate, and must not rewrite Markdown source docs.

TIS Visual Documentation Guide

docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md

This guide defines how visual documentation should evolve. No screenshots are required yet, but future screenshots and diagrams should follow this structure.

Purpose

Visual documentation should help developers, reviewers, owners, and future AI assistants understand TIS screens, workflows, data relationships, and design expectations.

Visual docs should not replace Markdown source docs. They should support them.

Future Storage Location

Recommended future location:

```
docs/visuals/  
  screenshots/  
  diagrams/  
  flows/
```

Recommended generated/reference output:

```
static/docs/visuals/
```

Do not store sensitive production data in screenshots.

Screenshot Standards

Screenshots should:

- 1 use clean test/demo data,
- 1 avoid private student/teacher/customer information,
- 1 show the full relevant viewport or focused workflow area,
- 1 include enough context to identify branch/year/page state,
- 1 avoid browser chrome unless needed,
- 1 be updated when UI behavior or layout meaningfully changes.

Preferred formats:

- 1 PNG for UI screenshots,
- 1 SVG or PNG for diagrams,
- 1 PDF only for exported documentation snapshots.

Naming Convention

Use stable, descriptive names:

```
YYYY-MM-DD_module_screen_state.png  
YYYY-MM-DD_flow_name_step_number.png  
YYYY-MM-DD_architecture_diagram_name.svg
```

Examples:

```
2026-06-26_platform-console_owner-view.png
```

Update Policy

Update visual docs when:

- 1 a screen layout meaningfully changes,
- 1 workflow steps change,
- 1 a module gains a new major view,
- 1 navigation or role visibility changes,
- 1 a diagram no longer reflects architecture.

Visual doc updates should be mentioned in the Knowledge Impact Assessment.

Future Diagram Strategy

Future diagrams should cover:

- 1 identity separation,
- 1 SaaS onboarding to provisioning,
- 1 payment webhook confirmation,
- 1 tenant isolation,
- 1 operational data model,
- 1 KMS source-to-PDF-to-Knowledge-Center flow,
- 1 landing-to-signup customer journey.

Diagrams should be simple and versioned through Markdown references.

Planned Visual Areas

Platform Console

Future visuals:

- 1 owner account controls,
- 1 developer management,
- 1 organization/branch context switching.

Knowledge Center

Future visuals:

- 1 healthy status,
- 1 stale status,
- 1 source document table,
- 1 KIA panel.

Dashboard

Future visuals:

- 1 operational dashboard overview,
- 1 staffing/report widgets,
- 1 scope context display.

Teachers

Future visuals:

- 1 teacher list,
- 1 teacher edit/profile,
- 1 qualification/capacity views.

Workforce Planning

Future visuals:

- 1 planning table,
- 1 section ownership,
- 1 workload/capacity states.

Academic Calendar

Future visuals:

- 1 calendar view,
- 1 event editing,
- 1 exports or responsibility states.

SaaS Onboarding

Future visuals:

- 1 signup,
- 1 organization step,
- 1 contacts,
- 1 branches,
- 1 academic setup,
- 1 review.

Billing

Future visuals:

- 1 plan selection,
- 1 checkout summary,
- 1 billing status,
- 1 payment pending/verified states.

Landing Page

Future visuals:

- 1 hero,
- 1 problem/solution section,
- 1 product capability section,
- 1 signup/demo conversion paths.

Reports

Future visuals:

- 1 allocation report,
- 1 dashboards,
- 1 export states.

Guardrails

- 1 Do not include sensitive real data.
- 1 Do not let visual docs become the only source of truth.
- 1 Do not update landing visuals during backend tasks unless explicitly approved.
- 1 Keep visual docs aligned with KMS change history.

TIS AI Optimization Guide

docs/engineering/AI_OPTIMIZATION_GUIDE.md

This is the definitive onboarding guide for future AI assistants working on TIS.

Preferred Onboarding Order

For any new AI coding conversation:

- 1 Read docs/AI_PROJECT_CONTEXT.md.
- 2 Read docs/README.md.
- 3 Read docs/PROJECT_STATE.md.
- 4 Read docs/TIS_MASTER_CONTEXT.md.
- 5 Read docs/engineering/README.md.
- 6 Read task-specific engineering docs.
- 7 Read relevant ADRs.
- 8 Read relevant module history.
- 9 Inspect code with rg before editing.

Choosing Task-Specific Docs

Use this map:

- 1 SaaS signup/login/account: docs/history/saas-onboarding/, ADR 0002.
- 1 Billing/payment/Paddle: docs/history/subscriptions/, ADR 0003, ADR 0004.
- 1 Provisioning: docs/history/provisioning/, ADR 0005.
- 1 Landing page: docs/marketing/, ADR 0001, ADR 0007.
- 1 Platform owner/Knowledge Center: docs/history/platform-knowledge/.
- 1 Workforce planning/timetable/teachers/subjects: docs/history/workforce-planning/.
- 1 Calendar: docs/history/academic-calendar/.
- 1 Architecture/data model: docs/engineering/REPOSITORY_ARCHITECTURE.md, docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md.
- 1 Design/UI: docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md.
- 1 Standards: docs/engineering/DEVELOPMENT_STANDARDS.md.
- 1 Roadmap: docs/engineering/PRODUCT_ROADMAP.md.

How To Interpret ADRs

ADRs explain accepted long-term decisions.

When an ADR applies:

- 1 treat it as design authority,
- 1 do not reverse it casually,

- 1 if a change conflicts with it, propose a new ADR or mark it superseded only with approval,
- 1 link related docs/history.

How To Use Module History

Module history preserves deeper before/after context.

Before changing a module:

- 1 read the module history folder,
- 1 identify the previous documented state,
- 1 update it if the module's meaning changes,
- 1 do not replace chronological CHANGE_HISTORY.md with module history.

How To Complete KIA

Every final response must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

If meaningful behavior, architecture, workflow, roadmap, design, data model, or product state changes, documentation probably changed too.

Avoiding Architectural Regressions

Do not:

- 1 merge SaaS account identity with operational users,
- 1 treat checkout return as verified payment,
- 1 provision tenants immediately from public signup,
- 1 expose internal docs publicly,
- 1 let platform developers act as platform owners,
- 1 modify landing code during backend tasks,
- 1 rewrite source docs from the running app,
- 1 change database/migrations without explicit approval.

Preserving Tenant Isolation

Always identify:

- 1 school group,
- 1 branch,
- 1 academic year,
- 1 current user,

- 1 user type,
- 1 role/permissions,
- 1 whether the user is platform, tenant, or SaaS account.

If scope is unclear, inspect code and tests before editing.

Distinguishing Operational vs SaaS Code

Operational app:

- 1 root FastAPI files,
- 1 routers/,
- 1 templates/ operational templates,
- 1 models.py,
- 1 branch/year/teacher/planning/calendar/observation workflows.

SaaS:

- 1 saas/,
- 1 templates/saas/,
- 1 signup/login/account/onboarding/billing/provisioning readiness.

Landing:

- 1 tis-landing-website/,
- 1 docs/marketing/.

KMS:

- 1 docs/,
- 1 scripts/generate_docs_pdf.py,
- 1 knowledge_service.py,
- 1 templates/platform_knowledge_center.html,
- 1 static/docs/.

Prompt Engineering Recommendations

Future prompts should specify:

- 1 implementation vs planning,
- 1 allowed files,
- 1 forbidden files,
- 1 whether docs must be updated,
- 1 whether PDF regeneration is required,
- 1 whether routes/app behavior can change,
- 1 whether database/migrations are in scope,

- 1 whether landing is in scope.

Good prompt pattern:

Implement [specific goal].
Allowed files: [...]
Do not modify: [...]
Update KMS docs and regenerate PDF if docs change.
Run: [...]
Report KIA.

AI Readiness Review

Current strengths:

- 1 AI context exists.
- 1 Engineering docs cover modules, repository, flows, database, standards, UI/UX, roadmap, rejected decisions, governance, and visual docs.
- 1 ADRs and module history preserve decision context.
- 1 PDF/manifest provide generated snapshot and freshness metadata.

Remaining gaps for future KMS improvements:

- 1 More screenshots and diagrams are needed.
- 1 More module-specific deep docs may be useful for timetable, observations, permissions, and dashboard/reporting.
- 1 Test strategy documentation could be expanded.
- 1 Deployment/runbook documentation could be expanded.
- 1 API/route inventory could be generated or manually documented later.
- 1 Data migration history could be summarized more explicitly.

Do not implement these gaps unless a future phase approves them.

TIS Project Governance

docs/engineering/PROJECT_GOVERNANCE.md

This document defines how TIS engineering decisions, approvals, documentation, and quality gates should be governed.

Ownership Model

Product/project ownership: Defines business direction, roadmap priorities, customer experience, and approval for major changes.

Platform Owner: Owns platform-level operations, platform accounts, provisioning oversight, and KMS visibility.

Engineering implementer: Human developer, Codex, or ChatGPT-assisted coding session responsible for scoped implementation and validation.

Reviewer: Reviews implementation, KMS updates, risk boundaries, and validation results.

Architectural Authority

Architectural authority comes from:

- 1 approved user direction,
- 1 ADRs,
- 1 docs/TIS_MASTER_CONTEXT.md,
- 1 engineering handbook docs,
- 1 module history,
- 1 existing code patterns.

If these conflict, pause and clarify before making irreversible changes.

Approval Workflow

Major work should follow:

- 1 clarify objective and scope,
- 2 identify allowed/forbidden files,
- 3 inspect code/docs,
- 4 implement scoped change,
- 5 update KMS,
- 6 run validation,
- 7 report KIA,
- 8 wait for review before broader phases.

Documentation Authority

Markdown under docs/ is the source of truth.

Generated artifacts:

- 1 PDF booklet,
- 1 manifest,
- 1 future screenshots/diagrams.

Generated artifacts support the docs; they do not replace them.

Coding Workflow

Default workflow:

- 1 inspect before editing,
- 1 prefer existing patterns,
- 1 keep changes small,
- 1 preserve tenant isolation,
- 1 preserve identity boundaries,
- 1 avoid unrelated rewrites,
- 1 run targeted validation,
- 1 update KMS when meaningful.

Review Expectations

Review should check:

- 1 behavior matches scope,
- 1 no forbidden files changed,
- 1 tenant isolation preserved,
- 1 permissions preserved,
- 1 SaaS/payment/provisioning untouched unless approved,
- 1 landing untouched unless approved,
- 1 docs/KIA complete,
- 1 validation credible.

Branch Strategy

Current assumption:

- 1 active development branch: dev,
- 1 production/live branch is separate and must be confirmed before deployment.

Rules:

- 1 do not push unless requested,
- 1 do not commit unless requested,
- 1 do not merge unless requested,
- 1 preserve unrelated local changes.

Release Philosophy

Prefer:

- 1 small reviewable increments,
- 1 documentation updated with implementation,
- 1 explicit milestone boundaries,
- 1 staged rollout for risky systems,
- 1 no hidden operational changes.

High-risk areas:

- 1 authentication,
- 1 platform owner access,
- 1 tenant isolation,
- 1 payment,
- 1 provisioning,
- 1 database/migrations,
- 1 landing conversion path.

Quality Gates

Before final report:

- 1 run relevant compile/tests/smoke checks,
- 1 verify generated docs if docs changed,
- 1 inspect working-tree scope,
- 1 mention validation gaps if any.

Documentation Gates

Every meaningful implementation must answer:

- 1 Did master context change?
- 1 Did project state change?
- 1 Did change history change?
- 1 Is an ADR needed?
- 1 Is module history needed?
- 1 Did AI project context need updating?
- 1 Did engineering docs need updating?
- 1 Was PDF regenerated?

KIA Requirement

Final reports must include:

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

Engineering Decision Traceability

Use this traceability model:

- 1 ADR: why a major decision was accepted.
- 1 Rejected decisions: why significant alternatives were not chosen.
- 1 CHANGE_HISTORY: chronological summary of meaningful changes.
- 1 Module history: deeper before/after context for one area.
- 1 Master context: durable current truth.
- 1 Project state: current status, priority, known issues, next work.
- 1 AI project context: compact first-read AI onboarding.
- 1 Engineering handbook: module, architecture, flow, standards, data, UI, roadmap, governance knowledge.
- 1 Generated PDF: snapshot for review/reference.
- 1 Manifest: generated metadata and source freshness basis.

When to update each:

- 1 Update ADRs for major decisions.
- 1 Update rejected decisions when a significant alternative is intentionally declined.
- 1 Update change history for meaningful changes.
- 1 Update module history for area-specific evolution.
- 1 Update master context for durable current truth.
- 1 Update project state for current status and next work.
- 1 Update AI context when first-read onboarding truth changes.
- 1 Update engineering docs when developer understanding changes.
- 1 Regenerate PDF/manifest when included docs change.

TIS Knowledge Lifecycle

docs/engineering/KNOWLEDGE_LIFECYCLE.md

This document defines how the TIS Knowledge Management System evolves with the software from planning through deployment and maintenance.

Core Principle

Documentation evolves with implementation. Markdown is the source of truth. The PDF and manifest are generated snapshots. The Knowledge Center is the owner-facing status surface.

No meaningful implementation is complete until the Knowledge Impact Assessment is complete.

Documentation Lifecycle

- 1 A task is proposed.
- 2 Relevant docs, ADRs, and module history are read.
- 3 The likely documentation impact is identified.
- 4 Implementation happens within approved scope.
- 5 KIA is completed.
- 6 Relevant Markdown docs are updated.
- 7 ADRs are added or updated if decisions changed.
- 8 Module history is updated if a module's documented state changed.
- 9 PDF and manifest are regenerated if included docs changed.
- 10 Final report describes the knowledge impact.

Engineering Lifecycle

- 1 Inspect before editing.
- 2 Plan the smallest safe implementation.
- 3 Preserve tenant, identity, permission, SaaS, payment, provisioning, and landing boundaries.
- 4 Implement scoped changes.
- 5 Validate with focused checks.
- 6 Update KMS.
- 7 Report KIA.
- 8 Wait for review, commit, push, or deployment approval.

Approval Lifecycle

- 1 Owner defines goal and constraints.
- 2 Implementer confirms scope.
- 3 Implementer makes only approved changes.
- 4 Reviewer checks behavior, scope, docs, validation, and KIA.

- 5 Commit/push/deployment happens only after explicit approval.

Review Lifecycle

Review should verify:

- 1 no forbidden files changed,
- 1 tenant isolation remains intact,
- 1 SaaS/payment/provisioning boundaries are preserved,
- 1 landing code remains untouched unless approved,
- 1 database/migrations are untouched unless approved,
- 1 KMS docs and generated artifacts are current,
- 1 KIA is complete.

Release Lifecycle

Before release:

- 1 Confirm branch and deployment target.
- 2 Confirm tests/checks.
- 3 Confirm docs/PDF/manifest are current.
- 4 Confirm change history and project state.
- 5 Confirm release notes or owner communication if needed.
- 6 Deploy only through approved process.

Maintenance Lifecycle

Maintenance includes:

- 1 periodic KMS freshness review,
- 1 periodic ADR/rejected-decision review,
- 1 module history cleanup for discoverability,
- 1 PDF readability review,
- 1 Knowledge Center health checks,
- 1 updating roadmap/current state after milestones.

Planning To Production

Planning creates the first knowledge obligation. Production closes it.

Expected path:

Idea / task

- > scope and constraints
- > docs and code inspection
- > implementation
- > validation
- > KIA
- > docs/history/ADR updates

- > PDF/manifest regeneration
- > review
- > commit/push
- > deployment
- > post-deployment state update if needed

Guardrails

- 1 The app must not silently rewrite Markdown docs.
- 1 Generated PDFs must not be edited manually.
- 1 Regeneration is artifact generation, not source editing.
- 1 KMS update work must remain reviewable.

TIS Documentation Automation

docs/engineering/DOCUMENTATION_AUTOMATION.md

This document defines current and future automation expectations for the TIS KMS.

Current Automation

Current approved automation:

- 1 `scripts/generate_docs_pdf.py` reads approved Markdown sources.
- 1 It generates `static/docs/TIS_Project_Reference_Booklet.pdf`.
- 1 It generates `static/docs/docs_manifest.json`.
- 1 The manifest records documentation version, branch, commit SHA, source paths, mtimes, sizes, and hashes.
- 1 The manifest records each included source document's starting PDF page.
- 1 The manifest records the generated PDF hash and size.
- 1 Markdown source hashes are computed from UTF-8 text normalized to LF, so equivalent CRLF and LF checkouts produce the same hash.
- 1 Manifest source paths are repository-relative POSIX paths; dynamically discovered sources use a stable case-insensitive sort with an explicit tie-breaker.
- 1 `scripts/generate_docs_pdf.py --check` validates source coverage, titles, approved catalog taxonomy, navigation links, source hashes, PDF identity, manifest metadata, PDF page bounds, and documentation version without writing files.
- 1 `kms_catalog.py` is the dependency-free shared vocabulary and path classifier for Knowledge Center presentation and KMS enforcement.
- 1 `.kms-impact.yml` records task-level Knowledge Impact in a small machine-readable schema.
- 1 `scripts/check_kms_impact.py` classifies likely major changes, validates the declaration against the Git diff, and runs generated-artifact freshness checks.
- 1 `scripts/kms.py sync` runs KIA preflight validation, regenerates the PDF and manifest, runs complete freshness validation, and prints a completion summary.
- 1 `scripts/kms.py check` is the single read-only validation entry point for local work and CI.
- 1 The PDF generator uses ReportLab multi-pass layout to produce a stable table of contents, named source destinations, and document/major-heading outlines without external dependencies.
- 1 GitHub Actions enforce KMS checks on pull requests, dev pushes, and before master deployment.
- 1 The Knowledge Center reads the manifest and checks freshness.

Repository owners must configure the KMS Enforcement / `kms-check` status as a required branch-protection check for protected integration branches. Workflow code makes production deployment depend on KMS validation directly; GitHub branch protection is the external setting that makes failed pull requests unmergeable.

Current automation does not:

- 1 rewrite Markdown docs,
- 1 create ADRs,
- 1 create module history entries,
- 1 decide KIA outcomes,

- 1 generate documentation prose,
- 1 commit or push changes.

Automation blocks stale or undeclared work; humans and approved AI assistants remain responsible for reviewed Markdown updates.

Cross-platform normalization does not relax enforcement. Changed text still changes the source hash, and missing, unexpected, duplicate, or reordered source entries still fail validation.

Phase 7D Navigation And Catalog Validation

The complete `scripts/kms.py` check path enforces:

- 1 a usable front-matter title or H1 for every included Markdown source,
- 1 categories and modules from the approved values in `kms_catalog.py`,
- 1 exact normalized manifest/source-list membership and deterministic order,
- 1 a positive integer `pdf_page` for every source, strict source-page ordering, and a page no greater than the generated PDF page count,
- 1 relative links in `docs/KMS_NAVIGATION.md` that remain inside `docs/`, resolve to existing Markdown files, and target listed booklet sources.

The checker reports genuine missing, unexpected, duplicate, reordered, stale, invalid-taxonomy, invalid-title, invalid-page, and invalid-navigation conditions. It does not repair them or weaken existing KIA and freshness gates.

Git Event Task Boundaries

KIA declarations apply to an implementation task, not to an individual GitHub delivery event. A task may contain an implementation commit followed by one or more review or CI correction commits.

- 1 Pull-request checks compare the pull-request base SHA with the actual pull-request head SHA.
- 1 Push checks on dev calculate the merge base between the repository default branch and the pushed head, then compare that merge base with the pushed head.
- 1 Push checks must not use `github.event.before` as the KIA base because it represents only the latest delivery and can split a multi-commit task.
- 1 Both event types use three-dot comparison and validate every changed KMS Markdown file against the cumulative declaration.

This preserves strict checks for undeclared, missing, or falsely declared documentation while allowing a follow-up commit to modify only the subset relevant to that correction.

Machine-Readable KIA Declaration

Every task updates `.kms-impact.yml`:

```
knowledge_impact: yes
summary: Describe the implemented engineering change.
affected_areas:
  - subscriptions
kms_files_updated:
  - docs/CHANGE_HISTORY.md
no_impact_reason:
major_change_override: no
```

Rules:

- 1 yes requires changed authoritative docs/* .md files.
- 1 no requires a specific reason and no declared KMS Markdown changes.
- 1 a detected major path with no also requires `major_change_override: yes`.
- 1 every changed authoritative Markdown file must be listed.
- 1 generated PDF and manifest must be current in both cases.

When Documentation Must Be Updated

Mandatory updates are required when a task changes:

- 1 architecture,
- 1 data model,
- 1 tenant isolation,
- 1 identity boundaries,
- 1 SaaS/account flows,
- 1 billing/payment/provisioning,
- 1 operational modules,
- 1 platform owner behavior,
- 1 UI/UX philosophy,
- 1 landing strategy,
- 1 roadmap,
- 1 governance,
- 1 development standards,
- 1 KMS behavior.

Optional updates may be skipped for:

- 1 typo-only code comments,
- 1 internal refactors with no behavior, architecture, flow, or ownership change,
- 1 generated files that do not affect source truth.

Even when docs are skipped, KIA must explain why.

Phase 6 Commands

Synchronize generated artifacts after reviewing and updating authoritative Markdown:

```
.\.env\Scripts\python.exe scripts\kms.py sync
```

Run complete read-only validation:

```
.\.env\Scripts\python.exe scripts\kms.py check
```

`sync` aborts before writing when KIA or authoritative Markdown validation fails. It writes only the generated PDF and manifest, then applies the same complete validation used by `check`. The lower-level generator and impact checker remain reusable implementation modules and diagnostic entry points.

Manifest Lifecycle

The manifest is generated every time the PDF is regenerated.

It should be treated as:

- 1 generated metadata,
- 1 source freshness basis,
- 1 Knowledge Center input,
- 1 not the source of truth.

If included Markdown docs change and the manifest is not regenerated, the Knowledge Center should show stale status.

PDF Lifecycle

The PDF is:

- 1 a generated snapshot,
- 1 owner reference material,
- 1 a reviewable artifact,
- 1 navigable through a page-numbered table of contents and PDF outlines,
- 1 not manually edited.

Regenerate after included Markdown docs change.

Every manifest source record must contain a positive integer pdf_page, pages must increase in fixed source order, and each page must be within the generated PDF's actual page count. Missing, inconsistent, or out-of-range page metadata fails freshness validation.

AI Context Lifecycle

Update docs/AI_PROJECT_CONTEXT.md when:

- 1 first-read onboarding changes,
- 1 current priority changes,
- 1 critical rules change,
- 1 new major docs are added,
- 1 architecture/identity/flow truth changes.

Do not overload AI context with every detail. It is a compact entry point.

Handbook Lifecycle

Update engineering handbook docs when:

- 1 module boundaries change,
- 1 repository ownership changes,
- 1 user/system flows change,
- 1 database concepts change,

- 1 standards change,
- 1 design philosophy changes,
- 1 roadmap changes,
- 1 governance changes.

Module History Lifecycle

Update module history when:

- 1 a specific module's previous documented state should be preserved,
- 1 a feature area changes meaningfully,
- 1 before/after context matters for future implementers.

ADR Lifecycle

Create or update ADRs when:

- 1 major accepted decisions are made,
- 1 accepted decisions are superseded,
- 1 architecture or product boundaries change.

Use rejected decisions when important alternatives are intentionally declined.

Mandatory vs Optional Summary

Mandatory:

- 1 KIA in final report,
- 1 docs update for meaningful changes,
- 1 change history for meaningful changes,
- 1 PDF/manifest regeneration when included docs change.

Conditional:

- 1 ADR for major decisions,
- 1 module history for area-specific state changes,
- 1 AI context for first-read truth changes,
- 1 engineering docs for handbook-level knowledge changes.

Optional:

- 1 visual docs until screenshots/diagrams are approved,
- 1 local Git hooks; CI is authoritative because hooks are optional and bypassable.
- 1 future search, semantic indexing, and documentation analytics.

TIS Knowledge Impact Assessment Standard

docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md

The Knowledge Impact Assessment (KIA) is a required engineering standard. It confirms whether a task changed project knowledge and whether the KMS was kept current.

Required KIA Fields

Knowledge impact: Yes/No
Docs updated:
Change history updated: Yes/No
ADR needed: Yes/No
Module history updated: Yes/No
PDF regenerated: Yes/No
AI project context updated: Yes/No
Reason if not updated:

The same decision must be recorded in `.kms-impact.yml` for machine validation. The repository declaration adds `summary`, `affected_areas`, `kms_files_updated`, `no_impact_reason`, and `major_change_override`.

Decision Tree

Ask:

- 1 Did behavior change?
- 2 Did architecture change?
- 3 Did a workflow change?
- 4 Did a module boundary change?
- 5 Did data ownership or tenant scope change?
- 6 Did SaaS/payment/provisioning change?
- 7 Did UI/UX or customer language change?
- 8 Did roadmap or current state change?
- 9 Did development standards/governance change?
- 10 Would a future developer or AI assistant need to know this?

If any answer is yes, knowledge impact is probably yes.

Checklist

For knowledge-impacting work:

- 1 update relevant source docs,
- 1 update docs/CHANGE_HISTORY.md,
- 1 update module history if module state changed,
- 1 update ADRs if major decisions changed,
- 1 update rejected decisions if an important alternative was declined,
- 1 update AI project context if first-read truth changed,
- 1 regenerate PDF/manifest,

- 1 report validation.
- 1 update `.kms-impact.yml` and run `scripts/check_kms_impact.py`.

Examples

Example: Docs Updated

```
Knowledge impact: Yes
Docs updated:
- docs/TIS_MASTER_CONTEXT.md
- docs/CHANGE_HISTORY.md
- docs/history/provisioning/...
Change history updated: Yes
ADR needed: No
Module history updated: Yes
PDF regenerated: Yes
AI project context updated: No
Reason if not updated: AI onboarding truth did not change.
```

Example: No Docs Needed

```
Knowledge impact: No
Docs updated: None
Change history updated: No
ADR needed: No
Module history updated: No
PDF regenerated: No
AI project context updated: No
Reason if not updated: Change was limited to a typo in an internal variable name and
did not affect behavior, architecture, workflow, roadmap, or developer knowledge.
```

Approval Expectations

Reviewers should reject final reports that omit KIA.

If docs were skipped, the reason must be specific. "Not needed" is not enough.

Failure Cases

KIA fails when:

- 1 final report omits it,
- 1 meaningful behavior changed but docs did not,
- 1 docs changed but PDF/manifest were not regenerated,
- 1 ADR-worthy decisions were hidden in code changes,
- 1 module state changed without module history,
- 1 AI context became stale after major onboarding truth changed.
- 1 `.kms-impact.yml` is missing, stale, inconsistent with the Git diff, or uses an unexplained override.
- 1 generated-artifact validation fails.

Developer Responsibilities

Developers must:

- 1 assess KIA honestly,
- 1 update docs in the same task,

- 1 keep generated artifacts current,
- 1 avoid unrelated documentation churn,
- 1 preserve historical truth.

AI Responsibilities

AI assistants must:

- 1 read relevant docs before coding,
- 1 identify likely KMS impact,
- 1 not invent history,
- 1 not silently skip docs,
- 1 not commit or push unless requested,
- 1 include KIA in every implementation final report.

TIS Self-Evolving Workflow

docs/engineering/SELF_EVOLVING_WORKFLOW.md

This is the official engineering workflow for keeping TIS software and knowledge synchronized.

Workflow

Task

- > Implementation
- > Validation
- > Knowledge Impact Assessment
- > Update .kms-impact.yml
- > Documentation Updates
- > ADR if required
- > Module History if required
- > KMS Sync (PDF + Manifest + Freshness)
- > KMS Read-Only Check
- > Review
- > Commit
- > Push
- > Deployment

Task

Clarify:

- 1 objective,
- 1 allowed files,
- 1 forbidden files,
- 1 validation requirements,
- 1 documentation requirements,
- 1 commit/push/deployment instructions.

Implementation

Implement only the approved scope.

Protect:

- 1 tenant isolation,
- 1 identity boundaries,
- 1 payment/provisioning correctness,
- 1 owner-only controls,
- 1 landing boundaries,
- 1 database/migration safety.

Validation

Run focused checks:

- 1 compile checks,
- 1 relevant tests,

- 1 route/template smoke checks,
- 1 PDF generation for docs,
- 1 frontend checks if frontend is in scope.

Knowledge Impact Assessment

Complete KIA before final report.

Do not treat documentation as optional when knowledge changed.

Record the same assessment in `.kms-impact.yml`. The declaration must change with the task and match the actual Git diff.

Documentation Updates

Update relevant docs:

- 1 master context,
- 1 project state,
- 1 AI context,
- 1 engineering docs,
- 1 change history,
- 1 module history,
- 1 ADRs/rejected decisions.

ADR If Required

Create/update ADR when architecture or product boundaries change.

Module History If Required

Update module history when area-specific before/after context matters.

Synchronize And Check KMS

If included Markdown docs changed:

```
.\.venv\Scripts\python.exe scripts\kms.py sync
```

Final read-only confirmation:

```
.\.venv\Scripts\python.exe scripts\kms.py check
```

sync validates KIA before generating derived artifacts and validates freshness afterward. It never edits authoritative Markdown.

Review

Reviewers check:

- 1 scope,
- 1 behavior,
- 1 tests,

- 1 KMS updates,
- 1 generated artifacts,
- 1 KIA.

CI repeats these checks and blocks pull-request completion, dev integration, or master deployment when declaration or freshness validation fails.

Commit / Push / Deployment

These are explicit actions, not assumptions.

- 1 Commit only when requested.
- 1 Push only when requested.
- 1 Deploy only through approved process.

Why This Is Self-Evolving

The workflow makes knowledge updates part of engineering completion. The system evolves because every meaningful change carries its context, history, decisions, and generated reference forward.

TIS Documentation Dependency Map

docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md

This document explains how KMS documents relate to each other and how changes propagate.

Dependency Model

Master Context
-> Project State
-> Engineering Handbook
-> AI Project Context
-> ADR
-> History
-> Knowledge Center
-> Manifest
-> PDF

The arrows show common reading and propagation flow. They are not strict ownership hierarchy.

Master Context

Purpose: Durable product, architecture, workflow, roadmap, and critical-rule truth.

Update when:

- 1 long-term architecture changes,
- 1 product vision changes,
- 1 major workflow changes,
- 1 critical rules change,
- 1 KMS structure changes.

Project State

Purpose: Current branch, priority, milestone, known issue, and next-work truth.

Update when:

- 1 active priority changes,
- 1 phase status changes,
- 1 known issues change,
- 1 next planned work changes.

Engineering Handbook

Purpose: Developer-facing deep knowledge: modules, repository, flows, database, standards, design, roadmap, governance, AI guidance, rejected decisions.

Update when:

- 1 developer understanding changes,
- 1 module maps change,
- 1 architecture or data ownership changes,
- 1 standards/governance changes,

- 1 future onboarding would be incomplete.

AI Project Context

Purpose: Compact first-read file for future AI coding conversations.

Update when:

- 1 first-read onboarding changes,
- 1 major docs are added,
- 1 current priority changes,
- 1 critical boundaries change.

ADR

Purpose: Accepted major decision record.

Update when:

- 1 a major architectural/product decision is made,
- 1 a prior decision is superseded,
- 1 a decision needs formal traceability.

Rejected Decisions

Purpose: Record important alternatives that were declined.

Update when:

- 1 future teams are likely to reconsider a rejected path,
- 1 a rejected alternative explains the chosen architecture.

History

Purpose: Chronological and module-specific evolution.

Update when:

- 1 meaningful change occurs,
- 1 module before/after state matters,
- 1 project knowledge changes.

Knowledge Center

Purpose: Owner-facing app view of KMS health, freshness, and protected PDF access.

Update when:

- 1 KMS metadata behavior changes,
- 1 Knowledge Center UI/status behavior changes,
- 1 protected documentation access changes.

Manifest

Purpose: Generated metadata for PDF and source freshness.

Update when:

- 1 PDF is regenerated.

Do not edit manually.

PDF

Purpose: Generated handbook snapshot.

Update when:

- 1 included Markdown docs change.

Do not edit manually.

Propagation Examples

Feature changes workflow:

Implementation -> CHANGE_HISTORY -> module history -> master/project state if needed
-> PDF/manifest

Architecture decision workflow:

Decision -> ADR -> rejected decision if relevant -> master context -> engineering
handbook -> change history -> PDF/manifest

Roadmap change workflow:

Roadmap -> PRODUCT_ROADMAP -> PROJECT_STATE -> AI_PROJECT_CONTEXT if first-read truth
changed -> PDF/manifest

KMS structure change workflow:

KMS docs -> README -> MASTER_CONTEXT -> AI_PROJECT_CONTEXT -> CHANGE_HISTORY ->
PDF/manifest

TIS AI Coding Workflow

docs/engineering/AI_CODING_WORKFLOW.md

This guide defines how future AI assistants should work inside TIS.

Repository-Native Assistant Configuration

Root AGENTS.md remains the repository-wide governance authority. Claude Code also reads root CLAUDE.md; its reusable project workflow lives at .claude/skills/tis-kms/SKILL.md, and bounded delegated TIS work may use .claude/agents/tis-kms-developer.md. These files point assistants back to this KMS rather than embedding a separate architectural source of truth. Configuration for another assistant must preserve the same KIA, safety, validation, and completion-report requirements.

Planning Before Coding

AI assistants must:

- 1 follow root AGENTS.md,
- 1 read docs/AI_PROJECT_CONTEXT.md,
- 1 read relevant engineering docs,
- 1 read relevant ADRs/module history,
- 1 inspect the codebase with rg,
- 1 identify allowed and forbidden files,
- 1 identify likely KMS impact,
- 1 update .kms-impact.yml for the current task,
- 1 avoid starting from assumptions.

Implementation Review Before Editing

Before editing:

- 1 confirm the module boundary,
- 1 confirm tenant/identity/payment/provisioning risk,
- 1 confirm tests or validation,
- 1 confirm docs to update,
- 1 confirm that app, SaaS, landing, database, or routes are in scope before touching them.

Implementation

During implementation:

- 1 keep changes narrow,
- 1 follow existing patterns,
- 1 avoid unrelated rewrites,
- 1 use apply_patch for manual edits,
- 1 do not commit or push,

- 1 do not modify `tis.db` unless explicitly approved.

Validation Expectations

Run appropriate validation:

- 1 compile checks for Python scripts/modules,
- 1 targeted tests for behavior,
- 1 PDF generation for docs,
- 1 template/route smoke checks for UI routes,
- 1 frontend checks for landing/frontend tasks.

If validation cannot be run, state why.

Documentation Updates

AI assistants must update KMS when knowledge changes:

- 1 change history,
- 1 project state,
- 1 master context,
- 1 AI context,
- 1 engineering docs,
- 1 ADRs/rejected decisions,
- 1 module history.

KIA

Every final response must include KIA.

Do not omit KIA because the task "felt small." Assess it.

The machine-readable declaration must agree with the final KIA and actual changed Markdown. Use the explicit major-change override only for a genuinely non-behavioral change and provide a specific explanation.

Commit Strategy

AI assistants must not commit unless the user explicitly asks.

When committing is approved:

- 1 summarize scope,
- 1 ensure generated docs are current,
- 1 avoid unrelated changes,
- 1 use clear commit messages.

Push Strategy

AI assistants must not push unless explicitly asked.

Before pushing:

- 1 confirm branch,
- 1 confirm tests/checks,
- 1 confirm KMS artifacts,
- 1 run `scripts/kms.py sync` when documentation changed and `scripts/kms.py` check for final read-only enforcement,
- 1 confirm no forbidden files changed.

Deployment Strategy

AI assistants must not deploy unless explicitly asked.

Before deployment:

- 1 confirm production branch/environment,
- 1 confirm database/migration implications,
- 1 confirm SaaS/payment/provisioning risks,
- 1 confirm KMS state,
- 1 confirm rollback or recovery expectations.

Final Response Pattern

Include:

- 1 files changed,
- 1 behavior changed or not,
- 1 validation,
- 1 known issues,
- 1 KIA,
- 1 no vague claims about tests that were not run.

TIS Future Automation Roadmap

docs/engineering/FUTURE_AUTOMATION_ROADMAP.md

This document lists possible future automation improvements. These are not implemented in Phase 3D.

Git Hooks

Potential:

- 1 pre-commit check for changed docs without regenerated PDF/manifest,
- 1 warning when KIA-related docs are stale,
- 1 formatting checks for Markdown.

Considerations:

- 1 hooks must not block emergency work without clear override,
- 1 hooks must be documented for Windows/dev environments.

CI Documentation Validation

Potential:

- 1 run PDF generator,
- 1 verify manifest is current,
- 1 verify required docs exist,
- 1 verify source hashes match,
- 1 fail pull requests when docs/PDF are stale.

Automatic PDF Regeneration

Potential:

- 1 CI-generated PDF artifacts,
- 1 deployment-time regeneration,
- 1 explicit owner-only regenerate action.

Constraint:

- 1 automatic regeneration must not rewrite Markdown source docs.

Automatic Manifest Generation

Potential:

- 1 generate manifest in CI,
- 1 compare manifest against committed PDF,
- 1 expose freshness status in Knowledge Center.

Documentation Diff Reports

Potential:

- 1 summarize doc changes between commits,
- 1 show which docs changed after a feature,
- 1 include KIA summary in release notes.

Stale Documentation Alerts

Potential:

- 1 Knowledge Center warning when manifest/source hashes differ,
- 1 CI alerts,
- 1 dashboard badge for owner users.

Knowledge Center Search

Potential:

- 1 search docs, ADRs, module history, and roadmap from owner page,
- 1 filter by module,
- 1 link to protected document views.

AI Semantic Search

Potential:

- 1 indexed semantic retrieval over KMS docs,
- 1 AI assistant prompt context packs,
- 1 module-specific context bundles.

Constraints:

- 1 preserve privacy,
- 1 do not expose internal docs publicly,
- 1 avoid unreviewed AI-generated source edits.

Documentation Analytics

Potential:

- 1 track stale areas,
- 1 track frequently referenced docs,
- 1 identify docs missing module ownership.

Release Documentation

Potential:

- 1 release notes generated from change history,
- 1 deployment checklist,
- 1 customer-facing change summaries,
- 1 internal technical release notes.

Architecture Dashboards

Potential:

- 1 owner-facing architecture/decision dashboard,
- 1 ADR status summaries,
- 1 module health and documentation freshness.

Priority Recommendation

Likely future order:

- 1 CI documentation validation.
- 2 Stale documentation alert in Knowledge Center.
- 3 Documentation diff report.
- 4 Owner-only explicit PDF regenerate action.
- 5 Search.
- 6 Semantic AI retrieval.

Separate Next.js Landing Website

docs/adr/0001-separate-nextjs-landing-website.md

ADR 0001: Separate Next.js Landing Website

Status: Accepted

Date: 2026-06-26

Context

TIS needs a public marketing website and a protected operational application. The public site has different performance, design, routing, and deployment needs than the FastAPI operational portal.

Decision

Keep the public landing website in `tis-landing-website/` as a separate Next.js app. Keep the FastAPI app as the operational portal at `https://app.tisplatform.com`.

Alternatives Considered

- 1 Use FastAPI/Jinja for both marketing and app pages.
- 1 Put marketing pages inside the operational app templates.
- 1 Use a static site generated outside the repository.

Consequences

Positive:

- 1 Clear separation between marketing and operational concerns.
- 1 Landing page can evolve with a modern frontend stack.
- 1 Operational FastAPI app remains focused on authenticated workflows.

Tradeoffs:

- 1 Two runtimes must be understood.
- 1 Deployment and routing boundaries must be respected.

Related Docs / Files

- 1 `tis-landing-website/`
- 1 `docs/marketing/landing_page_source_of_truth.md`
- 1 `docs/TIS_MASTER_CONTEXT.md`
- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

Separate SaaS Identity And Operational Users

docs/adr/0002-separate-saas-identity-and-operational-users.md

ADR 0002: Separate SaaS Identity And Operational Users

Status: Accepted

Date: 2026-06-26

Context

TIS has SaaS account users who sign up, select plans, onboard organizations, and manage billing. It also has operational tenant users who work inside a provisioned school context. Platform owners and developers are separate platform identities.

Decision

Keep SaaS account identity, platform identity, and operational tenant identity distinct.

Alternatives Considered

- 1 Use one user table concept for all public signup, platform, and tenant users.
- 1 Automatically convert every SaaS signup into an operational tenant user.
- 1 Treat platform users as ordinary tenant administrators.

Consequences

Positive:

- 1 Clearer security boundaries.
- 1 Safer tenant isolation.
- 1 Billing/onboarding can proceed before operational provisioning.

Tradeoffs:

- 1 Identity flows require careful documentation.
- 1 Developers must avoid mixing account, platform, and tenant assumptions.

Related Docs / Files

- 1 `auth.py`
- 1 `models.py`
- 1 `saas/models.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `docs/TIS_MASTER_CONTEXT.md`

Paddle Payment Architecture

docs/adr/0003-paddle-payment-architecture.md

ADR 0003: Paddle Payment Architecture

Status: Accepted

Date: 2026-06-26

Context

TIS requires subscription payments and billing visibility while preserving a clean boundary between academic operations and payment provider behavior.

Decision

Use Paddle behind service/client boundaries in the `saas/` package. Keep payment, pricing, billing, and currency logic separate from academic modules.

Active branch-quantity changes use `saas.subscription_change_service` and durable `SubscriptionChangeRequest` records. TIS requests Paddle previews and never calculates monetary proration. Immediate increases use `prorated_immediately` and prevent applying the provider change when payment fails. Reductions are billed for the next period without an automatic refund; TIS preserves current local capacity until the renewal boundary is webhook-confirmed. Every update sends Paddle the complete retained recurring item list.

Plan upgrades and downgrades use the same provider-authoritative principles through `saas.subscription_plan_change_service`. Upgrades are immediate only after provider-confirmed payment lifecycle evidence; downgrades are scheduled and preserve current entitlements until effective confirmation. Scheduled changes may be replaced or canceled only while Paddle and local request state agree.

Cancellation and reversal use `saas.subscription_cancellation_service`. Cancellation is scheduled at period end, does not revoke current paid access early, and may be reversed before the effective boundary after provider validation. `saas.subscription_lifecycle_service` centralizes displayed lifecycle state and allowed customer actions.

Billing history and invoices remain provider-owned. `saas.billing_history_service` reads subscription-scoped Paddle transactions and requests fresh expiring invoice URLs without persisting a duplicate financial ledger or provider URL.

Paddle does not support arbitrary client-supplied idempotency keys. TIS therefore serializes unresolved requests per subscription, uses deterministic local request keys, locks request/subscription rows during submission, and reconciles webhook retries by provider event and request state.

Alternatives Considered

- 1 Inline Paddle calls directly inside route handlers.
- 1 Store provider-specific behavior across operational modules.
- 1 Delay payment architecture until after provisioning.

Consequences

Positive:

- 1 Provider integration is isolated.
- 1 Academic workflows stay independent of payment details.
- 1 Billing status can be reasoned about separately from onboarding and provisioning.

Tradeoffs:

- 1 Service boundaries must be maintained.
- 1 Webhook/payment state must be carefully tested.

Related Docs / Files

- 1 `saas/paddle_client.py`
- 1 `saas/payment_service.py`
- 1 `saas/billing_service.py`
- 1 `saas/pricing_service.py`
- 1 `saas/currency_service.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/subscription_portal_service.py`
- 1 `saas/subscription_change_service.py`
- 1 `saas/subscription_plan_change_service.py`
- 1 `saas/subscription_cancellation_service.py`
- 1 `saas/subscription_lifecycle_service.py`
- 1 `saas/billing_history_service.py`
- 1 `saas/router.py`
- 1 `docs/TIS_MASTER_CONTEXT.md`

Webhook-Only Payment Confirmation

docs/adr/0004-webhook-only-payment-confirmation.md

ADR 0004: Webhook-Only Payment Confirmation

Status: Accepted

Date: 2026-06-26

Context

Checkout return pages can be reached by browser redirects, refreshes, or user navigation. They should not be treated as authoritative proof that payment succeeded.

Decision

Treat provider webhook confirmation as the authoritative source for successful payment state. Checkout return pages may inform the user but should not independently confirm payment or trigger provisioning as if payment is verified.

The same authority boundary applies to M7 subscription mutations. Local requests and browser submissions are intent, not final financial truth. Webhook processing validates signatures, provider event identity, subscription/customer ownership, expected items/quantity, attributable transactions, and idempotent event state before confirming local outcomes. Ambiguous or mismatched evidence fails closed to manual review.

Initial checkout and post-activation subscription-change lifecycles remain distinct so a completed change transaction cannot accidentally replay initial provisioning/payment transitions. Guarded reconciliation may repair finalized local lifecycle fields only from attributable stored webhook evidence plus authoritative Paddle transaction data.

Alternatives Considered

- 1 Confirm payment based on checkout return route.
- 1 Allow manual UI confirmation from the SaaS account page.
- 1 Provision tenant records immediately after checkout redirect.

Consequences

Positive:

- 1 Reduces risk of false-positive payment confirmation.
- 1 Aligns billing state with provider-confirmed events.
- 1 Supports delayed provisioning after verified payment.

Tradeoffs:

- 1 Users may need clear pending-payment status while webhook processing completes.
- 1 Webhook handling must be reliable and observable.

Related Docs / Files

- 1 `saas/paddle_client.py`
- 1 `saas/payment_service.py`

```
1    saas/billing_service.py
1    saas/subscription_change_service.py
1    saas/subscription_plan_change_service.py
1    saas/subscription_cancellation_service.py
1    saas/payment_lifecycle_reconciliation_service.py
1    scripts/reconcile_finalized_payment_lifecycle.py
1    saas/router.py
1    docs/TIS_MASTER_CONTEXT.md
```

Delayed Tenant Provisioning After Verified Payment

docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md

ADR 0005: Delayed Tenant Provisioning After Verified Payment

Status: Accepted

Date: 2026-06-26

Context

Public SaaS onboarding collects organization and branch information before an operational tenant exists. Provisioning creates operational school records and must be safe, reviewable, and recoverable.

Decision

Delay tenant provisioning until payment is verified and the organization is ready for provisioning. Use platform owner provisioning workflows and jobs to create or update operational tenant structures.

Alternatives Considered

- 1 Provision immediately after signup.
- 1 Provision immediately after checkout return.
- 1 Let public users create operational tenants directly.

Consequences

Positive:

- 1 Protects operational tenant data.
- 1 Keeps pending SaaS onboarding separate from live school records.
- 1 Allows platform owner oversight and retry behavior.

Tradeoffs:

- 1 More state transitions exist between signup and operational access.
- 1 Provisioning status must be communicated clearly.

Related Docs / Files

- 1 `saas/provisioning_service.py`
- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/admin_provisioning.html`
- 1 `templates/saas/admin_pending_organizations.html`
- 1 `docs/TIS_MASTER_CONTEXT.md`

Documentation As Source / Knowledge Management System

docs/adr/0006-documentation-as-source-knowledge-management-system.md

ADR 0006: Documentation As Source / Knowledge Management System

Status: Accepted

Date: 2026-06-26

Context

TIS is growing across product, SaaS, architecture, payment, provisioning, landing, and operational modules. Future humans and AI coding assistants need reliable context and change history.

Decision

Use Markdown files under docs/ as the source of truth. Generate the PDF booklet as a snapshot. Maintain change history, ADRs, module history, project state, master context, and AI project context as part of every meaningful development task.

Alternatives Considered

- 1 Keep knowledge only in chat history.
- 1 Keep only a generated PDF.
- 1 Let the app rewrite docs automatically.
- 1 Store decisions only in commits.

Consequences

Positive:

- 1 Project knowledge is reviewable and version-controlled.
- 1 Future Codex and ChatGPT conversations can onboard quickly.
- 1 Major decisions are preserved.

Tradeoffs:

- 1 Developers must maintain the KIA workflow.
- 1 Docs and PDF can become stale if the policy is ignored.

Related Docs / Files

- 1 docs/README.md
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md
- 1 docs/CHANGE_HISTORY.md

```
1 docs/AI_PROJECT_CONTEXT.md
1 scripts/generate_docs_pdf.py
1 static/docs/TIS_Project_Reference_Booklet.pdf
```

Landing Page Visual System Strategy

docs/adr/0007-landing-page-visual-system-strategy.md

ADR 0007: Landing Page Visual System Strategy

Status: Accepted

Date: 2026-06-26

Context

The public landing page must communicate trust, product maturity, and academic operations value. It should not be accidentally changed during backend or operational app work.

Decision

Treat the Next.js landing website and its visual system as a separate product surface. Keep marketing content references under docs/marketing/. Do not modify landing design, copy, or legacy FastAPI landing files unless explicitly approved.

Alternatives Considered

- 1 Let backend tasks freely alter landing page presentation.
- 1 Continue using legacy FastAPI landing files as the public source.
- 1 Keep marketing strategy only in informal notes.

Consequences

Positive:

- 1 Protects brand and public conversion strategy.
- 1 Keeps landing work deliberate and reviewable.
- 1 Reduces accidental coupling with operational app changes.

Tradeoffs:

- 1 Landing page changes need explicit planning.
- 1 Developers must check the source-of-truth docs before editing public website files.

Related Docs / Files

- 1 tis-landing-website/
- 1 docs/marketing/landing_page_source_of_truth.md
- 1 docs/marketing/tis_landing_page_master_content.md
- 1 docs/history/landing-page/README.md

Workspace Classification Foundation

docs/adr/0008-workspace-classification-foundation.md

ADR 0008: Workspace Classification Foundation

Status: Accepted

Date: 2026-07-22

Context

TIS needs a permanent distinction between internal sandbox, customer demo, and customer-paid workspaces. Existing onboarding, payment, and provisioning statuses describe workflow evidence, but none is a stable workspace classification authority. All records created before M8B-1 are confirmed test data.

Decision

Make SchoolGroup the canonical owner of stable workspace identity, classification, and lifecycle metadata. Carry pre-provisioning intent on PendingOrganization, account purpose on SaaSAccount, and internal-test attribution on operational User.

M8B-1 is metadata-only. Classification conversion is unavailable. No payment, entitlement, permission, tenant-isolation, reset, or customer workflow consumes the new fields. Existing records are classified through an explicit dry-run/default, transactional, idempotent backfill as internal sandbox/test data.

Consequences

Positive:

- 1 Later workspace policies have one constrained metadata foundation.
- 1 Stable UUIDs separate workspace identity from numeric database IDs.
- 1 Existing test data is explicitly classified without a legacy/unknown state.
- 1 Platform Owners can inspect metadata without receiving mutation controls.

Tradeoffs:

- 1 M8B-1 does not yet enforce commercial separation.
- 1 Deployment requires running and reviewing the controlled data backfill after schema migration.
- 1 Workspace conversions require a later approved package.

Related Docs / Files

- 1 workspace_classification.py
- 1 models.py
- 1 saas/models.py
- 1 saas/workspace_classification_service.py
- 1 saas/workspace_classification_admin_service.py
- 1 saas/commercial_state_service.py

```
1    scripts/diagnose_workspace_classification.py
1    scripts/backfill_workspace_classification.py
1    docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
```


Commercial State And Entitlement Resolution

docs/adr/0009-commercial-state-and-entitlement-resolution.md

ADR 0009: Commercial State And Entitlement Resolution

Status: Accepted

Date: 2026-07-22

Context

Workspace classification identifies what a workspace is, but it must not be conflated with lifecycle, subscription evidence, feature values, or branch commercial activity. Future demo and paid workflows require one conservative resolution boundary before access enforcement is introduced.

Decision

Represent the effective commercial grant with `WorkspaceEntitlement`, typed values through `WorkspaceEntitlementValue` and the existing `EntitlementDefinition` catalog, and optional per-branch intent through `BranchEntitlement`.

Resolve commercial state read-only from `SchoolGroup` classification/lifecycle and one coherent workspace entitlement. Customer-paid resolution delegates plan features, quantity, and confirmed subscription ownership to the existing M7 entitlement resolver. Branches inherit by default; an explicit branch record may state active or inactive but must match the branch tenant and effective workspace entitlement.

M8B-2 does not enforce the result. Missing, invalid, ambiguous, stale, orphaned, or cross-tenant evidence returns manual review.

Consequences

Positive:

- 1 One extensible commercial decision contract can support later demo and paid workflows.
- 1 Existing M7 subscription authority is preserved.
- 1 Branch commercial intent remains separate from operational branch status.
- 1 Read-only resolution can be inspected safely before enforcement.

Tradeoffs:

- 1 Existing internal development workspaces need a compatibility entitlement until all creation paths persist grants.
- 1 Demo expiration and customer access behavior remain deliberately incomplete.
- 1 Platform review is required for inconsistent persisted relationships.

Related Docs / Files

- 1 `commercial_entitlements.py`
- 1 `saas/models.py`
- 1 `saas/commercial_validation_service.py`
- 1 `saas/workspace_entitlement_service.py`

- 1 saas/branch_entitlement_service.py
- 1 saas/commercial_state_service.py
- 1 saas/entitlement_service.py
- 1 docs/adr/0008-workspace-classification-foundation.md

Review-Only SaaS Demo Requests Before Provisioning

docs/adr/0010-review-only-saas-demo-requests.md

ADR 0010: Review-Only SaaS Demo Requests Before Provisioning

Context

M8B-3 introduces a customer demo-request workflow after onboarding, while demo provisioning and activation remain explicitly deferred to M8B-4. Existing public marketing demo leads, SaaS onboarding organizations, operational workspaces, and paid subscriptions have different ownership and lifecycle boundaries.

Decision

Use a dedicated `SaaS Demo Request` aggregate tied to the verified SaaS requester and `Pending Organization`. Capture classification, commercial-state, and entitlement context at submission. Store Platform Owner approval/rejection in a separate review record and every transition in append-only audit/internal-notification events.

Approval is review evidence only. It must not create a `School Group`, workspace entitlement, checkout, payment record, Paddle object, or provisioning job. `Subscribe Now` continues the existing payment workflow unchanged. The legacy public marketing `Demo Request` remains independent.

Consequences

- 1 Customer and Platform Owner views have a durable, permission-scoped request lifecycle.
- 1 Duplicate pending requests and invalid transitions fail closed.
- 1 M8B-4 can consume an approved request without redefining review history.
- 1 Approval does not imply activation, entitlement, or operational access.
- 1 Email delivery, expiration, schedulers, conversion, and provisioning remain future work.

M8B7 Amendment

M8B7 preserves approval as durable review evidence but makes the normal owner approval route orchestrate the separately callable provisioning service. Failed activation leaves the request `Approved` and retryable. Request-received, approval, and decline communications are now durable; approval communication is created only after activation succeeds. ADR 0014 owns the orchestration and communication decision.

Demo Workspace Provisioning And Commercial Source Links

docs/adr/0011-demo-workspace-provisioning-and-commercial-source-links.md

ADR 0011: Demo Workspace Provisioning And Commercial Source Links

Context

M8B-4 must create an operational customer-demo workspace after Platform Owner approval. The existing tenant-provisioning link required a paid subscription contract, but a demo has no Paddle or billing obligation. Fabricating a contract would weaken commercial-state resolution and confuse demo access with paid ownership.

Provisioning must also reuse the proven operational tenant builder, remain retryable after failure, and preserve M8B-3 approval as review evidence rather than making approval itself activate a workspace.

Decision

Use a dedicated `SaaS Demo Workspace Provisioning` aggregate for attempts, result, activation metadata, and resulting resource references. Store append-only provisioning events separately.

Generalize `Tenant Provisioning Link` so exactly one commercial source is required: either `subscription_contract_id` for paid provisioning or `demo_request_id` for customer-demo provisioning. Existing paid rows remain unchanged.

Extract the workspace-record creation sequence from the paid provisioning service into a shared builder. The demo service invokes that builder inside a nested transaction, creates and validates a pending demo entitlement, creates the demo-sourced tenant link, then activates the `School Group` and entitlement. A failure rolls back all workspace changes while the outer transaction records a failed provisioning attempt and leaves the demo request `Approved`.

Consequences

- 1 Paid and demo tenants share one operational workspace creation engine.
- 1 Demo access has explicit commercial provenance without fabricated payment evidence.
- 1 Duplicate provisioning is prevented by request uniqueness, tenant-link uniqueness, and service validation.
- 1 Customers see safe lifecycle states; Platform Owners retain actionable failure details and audit history.
- 1 Logo filesystem writes remain governed by the existing provisioning engine; database provisioning is atomic.
- 1 Expiration, reminders, schedulers, login restrictions, conversion, email delivery, and billing changes remain future work.

M8B7 Amendment

The provisioning service remains independently callable, but the normal approval route now invokes it after recording approval. Provisioning failure preserves the `Approved` request and retry surface; successful activation creates the approval communication intent. No second workspace is created.

Seven-Day Demo Lifecycle And Access Enforcement

docs/adr/0012-seven-day-demo-lifecycle-and-access-enforcement.md

ADR 0012: Seven-Day Demo Lifecycle And Access Enforcement

Context

M8B-4 activates customer-demo workspaces but intentionally defines no expiration. M8B-5 requires a standard seven-day lifecycle, a Day 6 reminder, data-preserving expiration, and enforcement that cannot be bypassed by a stale authenticated session or a delayed scheduler.

Decision

Use `SaaS Demo Workspace Provisioning.activated_at` as the only demo clock authority. Persist derived reminder and expiration timestamps for querying, but require the lifecycle resolver to verify that they equal activation plus six and seven days. Perform calculations as timezone-aware UTC values and convert only display values to the onboarding organization's validated IANA timezone.

Use one read-only resolver for Active, Reminder Due, Expired, Suspended, and Manual Review. Scheduled processing is independently callable, dry-run by default, row-locked, idempotent, and failure-audited. Day 6 creates internal SaaS-account and Platform Owner notifications. Day 7 atomically ends the demo entitlement, suspends the SchoolGroup, marks the demo tenant link expired, and preserves all tenant data.

Enforce the same resolver in the existing operational authentication middleware. Customer-demo web requests that are expired or ambiguous are redirected to a subscription activation page; APIs and downloads receive a safe 403. Access-block audit is deduplicated. Platform identities, customer-paid workspaces, internal sandboxes, public/authentication routes, secure logout, and SaaS commercial routes bypass the demo-only gate.

Consequences

- 1 Scheduler delay cannot extend demo access because request-time resolution uses the authoritative timestamp.
- 1 Existing sessions cannot bypass expiration.
- 1 Expiration is reversible by a future approved commercial transition because no tenant data is deleted.
- 1 Processing failure rolls back expiration changes and remains safely retryable.
- 1 External email, conversion, extension, archive/delete, and read-only expired access remain out of scope.

M8B7 Amendment

Day 6 now atomically records the customer notice, Platform Owner Notification Center events, and reminder email intent. Expiry atomically records customer/owner notifications plus expiry and subscription-continuation email intents. Provider delivery occurs outside lifecycle row locks through the durable outbox. The CLI remains dry-run by default and is the Render Cron entry point.

Demo-To-Paid Workspace Conversion

docs/adr/0013-demo-to-paid-workspace-conversion.md

ADR 0013: Demo-To-Paid Workspace Conversion

Context

A provisioned Customer Demo already owns the customer's operational SchoolGroup, branches, users, permissions, academic data, and audit history. Running the paid provisioning engine after subscription payment would either skip the existing demo tenant without establishing paid commercial ownership or risk creating a duplicate tenant.

Decision

Convert only an active, coherently provisioned Customer Demo after TIS receives a provider-confirmed active M7 subscription for the same pending organization. A durable conversion ledger records Requested, Processing, Completed, and Failed states plus audit and internal-notification events.

The conversion runs in one nested database transaction. It preserves the SchoolGroup, workspace UUID, tenant link row, organization, branches, users, permissions, academic records, and history. It ends the demo entitlement, creates a subscription-backed paid entitlement, moves existing branch entitlement rows to that envelope, changes the workspace classification to Customer Paid, and relinks the existing tenant link to the confirmed subscription contract.

The transaction validates its result through the existing M7 subscription entitlement resolver, workspace entitlement resolver, and commercial-state resolver. It succeeds only when the same workspace resolves as Customer Paid Active with the confirmed subscription and sufficient purchased branch capacity. A failure rolls back every workspace conversion mutation while retaining provider-confirmed payment records and a retryable failed conversion audit.

Consequences

Expired-demo plan selection resolves the existing operational workspace and its authoritative active branch quantity. It enters the existing checkout and conversion ledger, never onboarding or a second workspace. Missing plan or branch configuration fails safely with support guidance while preserving all tenant data.

- 1 No tenant, workspace, organization, user, branch, permission, or academic record is recreated.
- 1 Demo request, provisioning, lifecycle, reminder, and expiration history remains available.
- 1 Completed conversions are excluded from demo reminder and expiration processing.
- 1 Existing paid provisioning remains unchanged for organizations without a demo tenant link.
- 1 Ambiguous, cross-tenant, internal-sandbox, or already-paid workspaces fail closed.
- 1 Manual conversion overrides, demo extensions, workspace deletion, and unrelated billing changes remain out of scope.

M8B7 Amendment

A coherent expired/suspended Customer Demo may now convert after authoritative confirmed payment. Conversion reactivates the same SchoolGroup, replaces the ended demo entitlement with the paid entitlement, and relinks the same tenant relationship. Checkout return, sandbox evidence, and unverified payment remain non-authoritative.

Demo Customer Communications And Approval Orchestration

docs/adr/0014-demo-customer-communications-and-approval-orchestration.md

ADR 0014: Demo Customer Communications And Approval Orchestration

Context

M8B-3 through M8B-6 established review, provisioning, lifecycle enforcement, and same-workspace paid conversion, but deliberately omitted customer lifecycle email, shared Notification Center delivery, a workspace-wide demo indicator, approval-driven activation, and expired-demo continuation.

Decision

M8B7 makes the normal Platform Owner approval route orchestrate the existing approval and provisioning services. Approval evidence remains separate from provisioning, and a failed activation preserves an Approved request plus the existing owner-only retry action. An approval email intent is created only after activation succeeds.

Demo communications use the existing branded email renderer and provider through a durable `SaaS Demo Email Delivery` outbox. Logical events are uniquely deduplicated and provider calls occur after lifecycle transactions. Platform Owner events use the existing Notification Center with explicit destinations and deduplication keys.

The shared authenticated shell resolves active Customer Demo state through the existing lifecycle resolver and shows a persistent demo indicator only to tenant identities. Expired access remains blocked.

A coherent expired Customer Demo may enter the existing Paddle checkout flow. Only authoritative confirmed payment converts and reactivates the same SchoolGroup, workspace UUID, tenant link, branches, users, permissions, and tenant data.

Consequences

- 1 Request, approval, decline, reminder, expiry, and continuation communications are durable and retryable.
- 1 Normal approval creates exactly one operational workspace; failed provisioning remains independently retryable.
- 1 Scheduler delay never extends access.
- 1 Expired customers can subscribe without tenant reprovisioning.
- 1 Sandbox, checkout-return, and unverified payment evidence cannot establish Customer Paid state.
- 1 M8B8 AI demo limits and M8B9 lifecycle testing controls remain out of scope.

Centralized AI Entitlements And Usage Accounting

docs/adr/0015-centralized-ai-entitlements-and-usage-accounting.md

ADR 0015: Centralized AI Entitlements And Usage Accounting

Context

TIS has workspace classification, commercial-state resolution, normalized paid plan entitlements, and role permissions, but no single authority for future AI features or demo usage. Route-local feature names and counters would risk cross-tenant leakage, inconsistent plan rules, and double consumption.

Decision

`saas/ai_entitlement_service.py` is the sole AI entitlement and consumption authority. Its decision order is tenant/workspace scope, underlying permission, commercial state, registered AI policy, then successful-use consumption. Commercial and demo-expiry denial remains authoritative before allowance.

`saas/ai_feature_registry.py` defines stable feature keys, display names, permission and entitlement keys, eligible plan codes, demo allowance, and enabled state. The temporary reviewed tier mapping grants registered AI features to `enterprise_ai`; Starter and Professional receive upgrade guidance. No price or AI business feature is introduced.

Each active Customer Demo receives two successful uses per registered feature. Evaluation never consumes. Before provider execution, callers reserve an operation key; locked successful-plus-reserved capacity prevents concurrent attempts from exceeding the allowance. Completion converts the reservation to a successful use only for a usable result, or marks it failed and releases the capacity. Durable counters and operation events are unique by SchoolGroup, feature, metric context, and operation key as applicable. Internal Sandbox, demo, and paid metrics use separate contexts.

Consequences

1 Permissions, commercial access, and AI entitlement remain separate checks.

1 Unknown, disabled, expired, restricted, non-entitled, and unauthorized cases

fail closed with explicit reason codes.

1 Demo exhaustion returns the approved message and Subscribe Now destination.

1 Internal Sandbox is unlimited but separately measured.

1 No failed/no-result operation consumes usage.

1 Future M8B9 controls may inspect or extend the persisted model, but M8B8 adds

no reset, override, expiry, reminder, or lifecycle-control interface.

1 No executable AI feature currently exists, so no customer AI route is added.

ADR 0016 - Pre-Deploy Database Migration Boundary

docs/adr/0016-pre-deploy-database-migration-boundary.md

Decision

The TIS FastAPI web process does not create database tables or execute pending migrations during module import, FastAPI startup, middleware, request handling, or background threads. Render executes `python scripts/run_migrations.py` as the service Pre-Deploy Command. The command creates the repository's baseline SQLAlchemy metadata schema, executes `db_migrations.run_pending_migrations(engine)`, logs newly applied migration identifiers, and exits nonzero on any failure.

Render activates the new web version only after this command exits zero. The web Start Command remains `uvicorn main:app --host 0.0.0.0 --port $PORT`.

Rationale

PostgreSQL DDL can wait on table locks for longer than Render's port-detection window. Running DDL inside the web process couples schema lock duration to service availability and previously required a process-local readiness gate. A platform pre-deploy step provides the required ordering without exposing a partially migrated schema or delaying Uvicorn's bind.

Consequences

- 1 Every deployment must configure the migration command before the Start Command is allowed to activate.
- 1 Migration failure blocks deployment with a nonzero exit code.
- 1 Repeated execution is safe because metadata creation is check-first and the ordered migration ledger is idempotent.
- 1 PostgreSQL connections use a 10-second connection timeout, 5-second lock timeout, and 30-second statement timeout before baseline metadata or ledger DDL begins. Migration-local protections remain effective.
- 1 Flushed logs identify connection, metadata, ledger, per-migration apply, marker, and commit progress. Timeout failures exit nonzero.
- 1 Schema inspection performed during a migration transaction uses that transaction's active SQLAlchemy Connection, never a secondary connection checked out through the Engine.
- 1 The former daemon migration worker and readiness middleware are removed.

Platform Owner Demo Operations And Access Profiles

docs/adr/0017-platform-owner-demo-operations-and-access-profiles.md

ADR 0017: Platform Owner Demo Operations And Access Profiles

Context

M8B7 established Customer Demo provisioning, expiry, durable communications, and same-workspace conversion. M8B8 established centralized AI entitlement decisions and durable per-feature usage. Platform Owners need generic testing controls without customer-specific rules, destructive reprovisioning, usage resets, or a second lifecycle implementation.

Decision

`saas/demo_operations_service.py` is the owner-only orchestration boundary. It may expire an active demo immediately, reactivate an expired demo with a mandatory future expiry, set an unbounded future custom expiry, send distinct idempotent final-calendar-day reminders, and synchronously invoke the existing single-demo or batch lifecycle processor. The same SchoolGroup, workspace UUID, branches, users, permissions, classification, and data are preserved.

`saas/demo_access_service.py` resolves a workspace default and an optional tenant-validated branch override. Standard retains M8B8's two successful uses per enabled AI feature. Full enables every currently enabled controlled demo feature and unrestricted AI while the demo is active. Custom uses registry identifiers, selected features, per-feature allowances, and explicit unlimited selections. Unknown features fail closed. Moving among profiles never deletes or rewrites usage history.

Every attempted state-changing operation records actor, time, workspace and optional branch, action, reason, before/after state, result, operation key, and communication references. Material customer-visible changes use the M8B7 durable email outbox and Notification Center. Required reasons apply to expiry, reactivation, custom expiry, and access changes.

Permissions, commercial/demo lifecycle access, and feature entitlement remain separate checks in that order. Full access cannot convert a Customer Demo to Customer Paid, cannot bypass normal permissions, and cannot bypass expiry.

Consequences

- 1 Platform Developer status alone never grants these controls.
- 1 Branch overrides cannot affect another branch or tenant.
- 1 Custom expiry has no maximum duration but must be future-dated.
- 1 Manual lifecycle processing reuses production rules and reports bounded checked/reminder/expiry/no-action/failure/skipped counts.
- 1 M8B9 adds no usage reset, pricing change, paid-conversion redesign, or customer-specific exception.
- 1 The owner detail page uses progressive disclosure: the initial view is a

seven-field demo summary plus common operations; lifecycle processing, branch/feature configuration, audit evidence, and technical history remain present but collapsed until explicitly opened.

Unified Commercial Access And Capacity Authority

docs/adr/0018-unified-commercial-access-and-capacity-authority.md

ADR 0018: Unified Commercial Access And Capacity Authority

Context

TIS already had authoritative workspace classification, commercial-state, workspace-entitlement, paid-subscription, plan-capacity, and demo-lifecycle resolvers. Capacity checks were nevertheless split among branch, user, teacher, subscription, and provisioning code. Some paths counted people differently or could check capacity outside the transaction that performed the mutation.

Decision

`saas/commercial_authority_service.py` is the single read-only facade for effective commercial access and operational capacity. It composes the existing authorities rather than replacing or persisting them. Paid workspaces require one contract-linked confirmed subscription and active paid workspace entitlement. Customer Demo and Internal Sandbox workspaces remain explicitly unmetered in M1. Missing, contradictory, or unsupported authority fails closed.

Paid branch capacity is the confirmed `PaymentSubscription.quantity` capped by the plan branch ceiling. Staff capacity counts every distinct active tenant operational `User` in the `SchoolGroup`, regardless of role, title, position, or internal-test attribution. Platform identities and account-only SaaS records do not count. Teacher capacity counts distinct normalized `Teacher.teacher_id` values in active branches and active academic years; each blank legacy identity counts separately and is never silently merged.

Every capacity-increasing mutation locks its owning `SchoolGroup` with PostgreSQL `SELECT . . . FOR UPDATE`, resolves and recounts authority, evaluates the proposed final totals, writes, and commits in one transaction. Multi-tenant internal operations lock `SchoolGroups` in ascending ID order. Existing over-capacity tenants keep their records and access, may reduce usage, and may change unaffected dimensions, but cannot further increase an exceeded dimension.

Consequences

- 1 Branch create/reactivation/bulk status updates, operational user

create/reactivation/cross-tenant activation, teacher create/year-copy, and academic-year activation use the same structured capacity decision.

- 1 Paid and demo provisioning validate the final commercial authority before

their nested transaction may complete.

- 1 Capacity checks supplement existing permissions and tenant isolation.

- 1 Pending plan or quantity changes grant no capacity before provider-confirmed

subscription reconciliation.

- 1 Paddle quantity remains active branches only; staff and teacher counts affect

plan eligibility and limits but are never Paddle quantity units.

- 1 M1 adds no schema, migration, pricing, packaging, Paddle, webhook, onboarding, authentication, or permission-definition change.

Secure Promo Code Definition And Governance

docs/adr/0019-secure-promo-code-definition-and-governance.md

ADR 0019: Secure Promo Code Definition And Governance

Context

TIS needs owner-governed commercial promo definitions before customer-side redemption or promo-based tenant grants can be introduced. Promo capacity must reuse the existing Starter, Professional, and Enterprise AI plan ceilings, while raw codes, lifecycle transitions, target restrictions, and historical governance require dedicated security and audit boundaries.

Decision

M2 adds definition-only `PromoCode`, `PromoCodeBranchRestriction`, and `PromoCodeAuditEvent` records. Every promo selects one active public TIS tier and exact positive branch, system-user, and teacher capacities that pass the existing plan-capacity validator. Scope may be global, organization, pending organization, account email, or email domain; reinforcing restrictions are allowed only when coherent. Branch restrictions identify existing eligible branches but grant no access.

Raw codes contain at least 100 bits of cryptographically secure randomness. TIS normalizes them with Unicode NFKC, uppercase, and approved separator removal, then persists only a deterministic HMAC-SHA256 lookup hash, key ID, and safe display prefix/suffix. `TIS_PROMO_CODE_HMAC_SECRET` is dedicated and mandatory when generation or lookup is invoked; there is no insecure fallback. The raw value appears once in a no-store response and is never persisted, logged, audited, or placed in a URL.

Persisted lifecycle states are Draft, Active, Paused, and Revoked; expiration is derived from the redemption deadline. Active material edits require pause, clear approval, increment definition version, and return to Draft. Platform Developers with explicit `promo_codes.view` and `promo_codes.manage` may manage non-terminal definitions, but only Platform Owners may activate or revoke. Lifecycle mutations lock the promo row through validation, mutation, durable audit insertion, and commit.

Consequences

- 1 Migration `20260805_001_promo_code_foundation` is additive, idempotent, and contains no data backfill.
- 1 Platform Console owns list, create, one-time display, detail, edit, activate, pause, revoke, duplicate, and replacement-definition routes.
- 1 Durable audit uses an allowlist and the application audit channel; neither contains raw codes, lookup hashes, key IDs, or secrets.
- 1 M1 commercial authority, `WorkspaceEntitlement`, `TenantProvisioningLink`, onboarding, Paddle, payment, provisioning, and tenant capacity behavior do not recognize promo as a commercial source in M2.
- 1 Promo redemption and immutable `PromoGrant` adaptation remain M3 work.

Promo Redemption And Commercial Grants

docs/adr/0020-promo-redemption-and-commercial-grants.md

ADR 0020: Promo Redemption And Commercial Grants

Context

M2 created secure promo definitions but deliberately granted no customer access. M3 needs to activate either a newly onboarded organization or a separately aligned existing tenant without fabricating payment, subscription, demo, or onboarding evidence. Promo capacity must participate in M1 while remaining distinct from Paddle-paid authority.

Decision

Completed activation creates immutable `PromoRedemption` and `PromoGrant` records. The grant snapshots tier, exact branch/system-user/teacher limits, scope, and effective window. A `promo WorkspaceEntitlement`, explicit branch assignments/entitlements, and `TenantProvisioningLink.promo_grant_id` make the grant operationally attributable. `TenantProvisioningLink` requires exactly one source: paid contract, approved demo request, or promo grant; its pending organization reference is optional for already aligned tenants.

Only a verified tenant owner may activate. The final transaction locks the activation session, promo definition, and target workspace, then revalidates all scope, lifecycle, redemption, source, capacity, and branch-selection invariants before one commit. Pending sessions grant no access. M1 and branch authorization resolve only an effective active grant and coherent entitlement chain.

Branch selection is explicit and reversible without changing branch status or deleting data. Organization-wide system-user and teacher counts are validation inputs only; overage blocks activation and never modifies people or roles. Active internal sandboxes cannot be converted by this workflow. Paddle and all payment records remain outside promo activation.

The selectable set and entitlement inventory are distinct. Selection considers eligible operationally active branches. Final activation inventories every preserved workspace branch: selected branches receive one assignment and active entitlement, while unselected branches receive an inactive entitlement without an assignment or operational-status change. Branch reactivation uses the existing M1 `SchoolGroup` lock and capacity decision, then updates operational status, assignment, and entitlement in the same transaction.

Legacy missing inactive evidence may be reconciled only when one active promo grant, one promo tenant source, one owner, one matching workspace entitlement, and coherent assignments are proven. The generic PostgreSQL CLI is dry-run by default and explicit `apply` creates only missing inactive entitlements. It never changes branch status, assignments, grant capacity, or people records; uncertainty remains manual review.

Customer commercial presentation follows the same source boundary. After organization and billing permission validation, the subscription route resolves M1 authority first. Paid and demo sources keep their existing dedicated portals; promo source uses a separate read-only view model. That adapter accepts M1 plan, status, usage, limits, and remaining capacity and reads the attributable grant/redemption only for effective dates and the masked reference. It must not query Paddle, expose paid-only controls, or move commercial decisions into templates.

Operational grant access ends exactly at `PromoGrant.effective_to`. `grace_period_days` is a recovery and paid-continuation window only and is never added to the effective operational-access period. Request-time resolution derives active, recovery, or expired state without mutating the immutable grant history or tenant data.

Consequences

- 1 Existing aligned organizations require a controlled prior classification and

owner-link operation but no fake PendingOrganization.

1 Raw promo code material remains absent from persistence, logs, and audit.

1 Expired, missing, ambiguous, cross-tenant, or conflicting authority fails

closed.

1 Unused branch capacity may cover future normal branch creation, but staff and

teacher capacity is never auto-reconciled.

1 Existing preserved branches always carry explicit active or inactive promo

entitlement evidence.

1 Promo renewal, transfer, communication, automated expiry jobs, and active-promo

early conversion remain deferred.

1 Expired and recovery-period promo authority may enter the separately governed,

provider-authoritative existing-workspace paid continuation in ADR 0024.

Existing Workspace Conversion Read-Only Audit

docs/adr/0021-existing-workspace-conversion-read-only-audit.md

ADR 0021: Existing Workspace Conversion Read-Only Audit

Context

An operational internal sandbox can contain years of tenant data and partial identity or commercial relationships. A future conversion to a real customer must preserve that workspace and cannot rely on a hardcoded tenant script, model-only assumptions, or a partial branch count. Conversion design therefore needs reproducible production evidence before any mutation is authorized.

Decision

M4A introduces a generic query-only service keyed by an explicit SchoolGroup ID, workspace UUID, exact name, and normalized owner email. It reflects the connected database schema, inventories ownership, provisioning, entitlement, paid/demo/promo evidence, traverses branch foreign-key descendants, and reports unconstrained branch-like references. Output is allowlisted and provider references are represented only by presence flags.

The standalone CLI is the only production entry point. It requires PostgreSQL, starts `REPEATABLE READ ONLY`, emits deterministic sanitized JSON or text, and always rolls back and closes. Canonical evidence receives a stable SHA-256 snapshot hash. Exit codes are 0 coherent, 1 execution/configuration failure, 2 manual review, and 3 workspace identity mismatch. Soft-deleted rows remain dependencies; an unmodeled branch foreign key suppresses all archival recommendations. Missing, conflicting, or uncertain evidence fails closed for manual review. The audit never constitutes approval for hard deletion or conversion.

Consequences

- 1 Tenant-specific identifiers remain command inputs and are absent from the reusable service and CLI.
- 1 Schema drift and unknown relationships become visible blockers rather than silent omissions.
- 1 Archival candidate IDs are advisory only and include active branches proven free of direct, transitive, logical, and soft-deleted dependencies.
- 1 The active internal-sandbox entitlement may be reported as expected evidence; paid, demo, promo, ambiguous, or duplicate authority blocks conversion design.
- 1 Branch archival, owner alignment, classification changes, approval tokens, write-mode conversion, Paddle, email, and all data mutation remain outside M4A.

Controlled Existing Workspace Owner Alignment And Activation-Required Conversion

docs/adr/0022-controlled-existing-workspace-conversion.md

Context

M4A can prove the identity, relationships, branch dependencies, intended owner, and commercial state of an existing internal sandbox, but it deliberately has no write authority. TIS needs a controlled bridge that preserves the existing tenant and its data, establishes a real verified customer owner, and removes internal commercial authority without fabricating onboarding or billing state.

Decision

TIS uses a durable conversion operation as both approval ledger and ownership claim. The operation binds exact workspace identity, intended normalized email, M4A evidence, canonical parameters, actors, idempotency, branch/dependency and entitlement snapshots, setup fields, and lifecycle stage. Events are append-only and redacted. A partial unique database index allows one active `tenant_owner` link per SchoolGroup, after migration-time duplicate preflight.

Preparation never creates an account or password. The owner registers and verifies through the existing SaaS authentication architecture, explicitly claims the operation, and is aligned through shared operational-owner identity rules. Cross-tenant, duplicate, inactive, unverified, and platform identities fail closed. A different current owner requires Platform Owner transfer approval. Only legal name, IANA timezone, and educational program are collected.

Final conversion is PostgreSQL-only. It locks the operation and SchoolGroup, performs a fresh M4A audit, and compares stable canonical branch/dependency and sandbox-entitlement evidence to the approved operation. The complete M4A hash is required at preparation; owner alignment and setup are expected to change the later full report, so final validation compares the stored invariant subsets rather than incorrectly requiring the old full hash.

The transaction ends the active internal-sandbox entitlement, preserves its history, and sets classification customer with lifecycle provisioning. It creates no active entitlement, TenantProvisioningLink, PendingOrganization, contract, subscription, demo request, or promo grant. M1 resolves this exact source-free state as `activation_required`; Organization Account remains available and operational access remains blocked.

Consequences

- 1 Every branch and operational record is preserved; branch archival or deletion is outside M4B.
- 1 M3 promo activation can establish the first customer commercial source.
- 1 Existing-workspace Paddle activation is deferred and is not presented as available, while the normal new-onboarding Paddle journey is unchanged.
- 1 Dry-run is default. Write preparation and conversion require exact explicit confirmation phrases, Platform Owner actors, row locks, and idempotency.
- 1 Parameter drift, audit drift, identity conflicts, setup gaps, new dependencies,

or commercial evidence block conversion without partial state.

Existing Workspace Paid Activation Through Paddle

docs/adr/0023-existing-workspace-paid-activation.md

Context

M4B leaves a preserved existing tenant as classification customer, lifecycle provisioning, and commercial state `activation_required`. Promo activation can establish authority, but normal Paddle onboarding requires `PendingOrganization` and tenant provisioning records that must not be fabricated for an existing workspace.

Decision

TIS uses a dedicated `ExistingWorkspacePaidActivation` aggregate anchored directly to `SchoolGroup`, workspace UUID, verified SaaS account, tenant-owner link, plan, interval, branch selection, quote, checkout, payment attempt, contract, and provider identity. Existing checkout/payment tables support either `PendingOrganization` or this activation through strict XOR constraints. Billing profile and payment-customer association can be anchored explicitly to `SchoolGroup`. Each workspace association records the provider address and business selected for that `SchoolGroup`. Account-wide mutable Paddle customer defaults are not sufficient webhook authority when one SaaS account can own multiple workspaces.

Professional and Enterprise AI include every active operational branch and Paddle quantity equals active branch count. Organization-wide staff and teacher counts remain eligibility dimensions, not provider quantity. Starter remains unavailable until restricted branch access is proven across all operational entry points.

Preparation grants no authority. The shared signature-verified Paddle webhook path treats `transaction.paid` as processing only. A matching `transaction.completed` locks the `SchoolGroup` and activation, revalidates all identity, quote, capacity, branch, customer, business/address, price, quantity, currency, interval, transaction, and subscription evidence, then atomically establishes paid commercial authority. No tenant provisioning engine is called.

Plan selection and payment review are distinct states. Organization Account opens the selection surface when the current activation is `draft` or `checkout_ready`; the saved plan may be highlighted and replaced by another eligible plan, producing a fresh quote and superseding only unfinished local checkout authority. This selection work performs no Paddle API call, creates no `PaymentAttempt`, and mutates no branch. After checkout reaches `checkout_started` or `payment_processing`, or the activation is in a manual-review/inconsistent state, replacement fails closed rather than silently changing provider-bound commercial terms.

The successful workspace remains classification customer; paid versus promo is represented by `WorkspaceEntitlement` and `TenantProvisioningLink` source. Lifecycle becomes `active` and operational access resolves through centralized services.

The same aggregate supports a verified owner continuing an existing expired or recovery-period promo workspace. Promo authority remains unchanged through plan selection, checkout, and payment processing. On verified completion, the locked transaction revalidates the exact promo tenant link and entitlement, ends promo entitlement authority, relinks that same tenant link to the paid contract, establishes paid workspace and branch entitlements, and preserves immutable promo evidence. Active promo grants are not eligible for early conversion.

Consequences

- 1 Existing branches, users, teachers, academic records, branding, and tenant identity are preserved.
- 1 Browser return and incomplete provider events cannot activate access.

- 1 Editable pre-checkout drafts can change plan without creating provider or payment authority; in-progress or ambiguous payment states cannot be silently replaced.
- 1 Duplicate launch and webhook handling is idempotent; drift and source conflicts fail closed without partial authority.
- 1 Returned or reused transactions are accepted only when billed with automatic collection and their item, price, quantity, subtotal, interval, currency, customer, address, business, checkout URL, and custom lineage match the current activation quote.
- 1 PostgreSQL row-lock reads refresh mapped state before idempotency decisions, so a concurrent duplicate webhook cannot act on a pre-lock identity-map snapshot.
- 1 Organization Account payment presentation is attempt-authoritative: lifecycle provisioning with no current attempt means Activation required, processing needs a current unexpired checkout-started or payment-processing attempt, and terminal attempts use recovery states.
- 1 Normal onboarding payment and demo-to-paid flows remain separate. Promo-to-paid reuses this aggregate only through the explicit source transition in ADR 0024.
- 1 The additive migration must be applied before deploying the routes and webhook path.

Promo Expiry Recovery And Paid Continuation

docs/adr/0024-promo-expiry-recovery-and-paid-continuation.md

Context

Promo grants already provide a distinct non-Paddle commercial source with immutable effective dates. The product needs an explicit meaning for grace days, source-correct expired messaging, and a way for an expired promo customer to continue as paid without recreating or losing the existing tenant.

Decision

Normal operational access ends exactly at `PromoGrant.effective_to`. `grace_period_days` defines only a commercial recovery interval. During and after that interval, authorized owners may use Organization Account and preserved tenant data, but normal operational routes remain blocked. Grace never changes the grant effective window, restores branch access, or archives/deletes data.

Only recovery-period and expired promo authority is eligible for paid continuation. Active-promo early conversion is blocked because promotional value is not paid credit and TIS will not invent proration. The customer selects an eligible paid plan through the existing-workspace activation aggregate. Quote preparation, checkout launch, pending payment, failure, cancellation, expiry, and abandonment leave promo authority unchanged and do not restore expired access.

A verified `transaction.completed` is the conversion boundary. Under the existing `SchoolGroup` and activation locks, TIS revalidates tenant identity, quote, capacity, provider evidence, the exact promo grant, the active promo workspace entitlement, and the sole promo-sourced tenant link. One transaction then ends the promo entitlement, marks the grant converted while retaining its history, relinks the existing tenant link to the paid contract, creates paid workspace and branch entitlements, and resolves paid authority before commit. A conflict rolls back the authority transition and requires manual review. Duplicate provider confirmation is idempotent.

Commercial identity presentation is centralized in `commercial_badge_service`. Prepared `source/status/plan/icon` tokens drive one reusable badge for Promo, Demo, and Paid authority. Templates do not infer authority from historical rows. Compact mode is available to authorized Organization Account presentation, and full mode is used on the commercial page. The normal operational header does not render commercial identity. Unresolved or source-free states do not claim a commercial badge.

Consequences

- 1 SchoolGroup, workspace UUID, branches, users, teachers, academic data, branding, ownership, and tenant data are preserved through conversion.
- 1 Promo redemption/grant history remains immutable and attributable after conversion.
- 1 There is never simultaneous active promo and paid tenant authority.
- 1 Failed or abandoned checkout cannot create operational access.
- 1 Promo, Demo, and Paid customer messaging and badges remain source-aware.
- 1 Long-term data archival and active-promo conversion remain future decisions.

Capacity-Based Packaging And Common Customer Feature Baseline

docs/adr/0025-capacity-based-packaging-and-common-customer-feature-baseline.md

ADR 0025: Capacity-Based Packaging And Common Customer Feature Baseline

Context

The original subscription entitlement foundation used a temporary feature-tier matrix: Enterprise AI received commercial AI, Professional and Enterprise AI received Advanced Reporting, and other catalog features awaited approval. TIS subsequently established organization-wide branch, staff-user, and teacher capacity as the authoritative self-service plan-sizing model and added paid, promo, and demo commercial sources.

That transitional feature matrix contradicted the approved product model. TIS is sold to organizations, and Starter, Professional, and Enterprise AI represent organization scale rather than progressively unlocked normal product modules.

Decision

Starter, Professional, and Enterprise AI share one normal customer feature baseline. The baseline includes teacher and branch management, observations, hiring, reporting, enabled commercial AI, Advanced Reporting, general exports, and customer-facing cross-branch reporting. `feature.audit_log` is explicitly excluded because the implemented global audit export is Developer-only.

Paid, promo, and active customer-demo authority may use the same baseline only after coherent source and lifecycle resolution. Permissions, tenant isolation, branch entitlement, academic-year scope, and operational prerequisites remain independent mandatory checks. Plan and promo capacity remain unchanged: Starter 1/5/25, Professional 5/20/100, Enterprise AI 25/100/500, and Custom above those ceilings.

The shared feature resolver performs the source-aware decision centrally. PlanEntitlement remains the persisted feature catalog and rollout mechanism; normal baseline values are identical across the three plans, while capacity, beta/disabled features, custom contracts, demo safety, and kill switches remain separate concerns.

Standard, Full, and Custom demos receive the normal customer baseline. Custom demo policy may still configure scope, safety, experimental features, and AI consumption, but cannot simulate a lower paid tier by removing normal modules.

AI feature availability is separate from provider consumption. Enabled AI features share the customer baseline and still require `ai.use`. The existing reservation, idempotency, counters, and operation events remain authoritative. A separate consumption-policy boundary supplies demo allowances now and can later supply paid, promo, rate, budget, or custom-contract limits without changing feature availability. Globally disabled AI features remain disabled.

Superseded Decisions

This ADR supersedes only:

- 1 ADR 0009's delegation of paid customer feature availability to a differentiated plan matrix; and
- 1 ADR 0015's temporary Enterprise-AI-only availability mapping.

Their commercial-source authority, tenant isolation, fail-closed resolution, permission separation, AI accounting, idempotency, and concurrency decisions remain valid.

Consequences

- 1 Normal customer modules are not hidden because a coherent workspace uses Starter, Professional, Enterprise AI, promo, or active demo authority.
- 1 Capacity and Paddle quantity behavior do not change.
- 1 Active promo operational entitlement values require safe idempotent reconciliation; immutable grant/redemption evidence is unchanged.
- 1 Existing paid workspaces need no per-workspace feature backfill because paid entitlements resolve through the current PlanEntitlement matrix.
- 1 Customer feature universality cannot expose Platform, Developer, System Owner, cross-tenant, promo-admin, demo-admin, or global commercial operations.

Related Files

- 1 `saas/customer_feature_policy.py`
- 1 `saas/entitlement_service.py`
- 1 `saas/ai_consumption_policy.py`
- 1 `saas/ai_entitlement_service.py`
- 1 `saas/demo_access_service.py`
- 1 `saas/promo_redemption_service.py`
- 1 `db_migrations.py`
- 1 `authorization.py`

Versioned, Constraint-Based Smart Timetable Generation

docs/adr/0026-versioned-constraint-based-smart-timetable-generation.md

ADR 0026: Versioned, Constraint-Based Smart Timetable Generation

2026-09-02 lifecycle presentation clarification

Release 1 presents Draft ? Validate ? Approve ? Publish while retaining the existing persistence model: successful validation records approval and transitions the Draft to `publication_ready`, and `TimetableActiveVersion` identifies current publication. Active, superseded, and archived versions remain immutable. Copying an eligible immutable version creates a separate working Draft and cannot change publication. Mutation/publication endpoints may add canonical conflict evidence to their existing error envelope while retaining legacy message compatibility.

Teacher-specific timing constraints

Teacher Scheduling Rules are normalized branch/year configuration, not timetable locks and not Planning demand. Schema-v4 snapshots resolve and freeze Must teach, Unavailable, Prefer teaching, and Prefer free rules, including all/selected days, numbered/first/last periods, and assigned-class/grade/section targets. Must teach and Unavailable are hard CP-SAT constraints; preferences are objective terms. The simplified administrator workflow also exposes Schedule within: it restricts existing eligible assigned demand to selected numbered-period slots but does not require every allowed slot to be occupied. An additive semantic discriminator keeps all pre-existing Must-teach rows unchanged. Readiness and the problem builder reject deterministic conflicts, and the OR-Tools-independent validator repeats hard checks. Rule changes invalidate only unpublished Draft authority; active published versions remain immutable. Manual Draft mutation reuses current canonical hard rules to reject teacher-wide Unavailable slots and applicable Schedule-within violations without requiring temporary Must-teach completeness. The complete Draft validator is the lifecycle boundary: it checks Unavailable, Schedule-within, and Must-teach section targets, and approval/publication reuse that result. Soft rules do not block either path. Multiple applicable Schedule-within records compose additively as a per-demand union. Intersecting complete per-rule windows is rejected because it converts two valid allowed-window fragments into an unintended prohibition. The problem builder materializes the combined windows in solver input; CP-SAT and the independent validator consume the same map. A deterministic bipartite capacity check counts grouped demands once and rejects combined teacher-window conflicts, while demand-level checks prove daily-coverage, hard max-per-day/minimum-day, and double-block incompatibilities before solver invocation.

Subject Distribution Rules use a **Partitioned Sessions** contract. Weekly subject placements are partitioned into exactly the configured number of two-period double sessions and one-period single sessions. CP-SAT selects explicit physically-adjacent block starts and explicit singles and channels each occupied period to exactly one selected session. Different sessions may touch: a three-period run may be double plus single, and a four-period run may be two touching doubles. No period may belong to two sessions, and a Break, Prayer, or other physical timeline interruption prevents a double from crossing it. Daily load/session channeling strengthens hard daily coverage; it does not weaken or reinterpret max/day, minimum-day, collision, lock, or teacher rules. The independent validator proves that the placements admit the same exact partition. Block lengths above two remain unsupported and fail closed. Post-solve infeasibility diagnosis uses the same immutable problem and solver but enables controlled hard-family profiles. These profiles are explanatory only: they can identify base collision infeasibility, locks, grouped resources, Subject Distribution Rules, Teacher Scheduling Rules, or subject/teacher interaction, but their placements are never persisted. The original complete solve remains the sole generation authority. Diagnostic timeouts fail closed as inconclusive. Isolation has a dedicated Workflow timeout, `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS`, with a 60-second default per profile; it is not capped by or derived from the primary solver timeout. Profiles stop at the first proven family transition and omit lock/group branches when those inputs are absent.

Context

The original Timetable stored one mutable set of placements per branch and academic year. It had no durable draft/history boundary, active-version authority, reproducible input snapshot, regeneration locks, or generation-run record. Automatic generation therefore could not be introduced safely without first separating Planning authority, timetable history, publication, and future solver execution.

Decision

Timetable conflict evidence uses one canonical domain contract with stable code, legacy source code, HARD/SOFT severity, DURABLE/RECALCULABLE/TRANSIENT evidence class, safe message key/text, optional safe entities/slots/remediation, provenance, and internal correlation. Public serialization removes internal requirement, allocation, and constraint identifiers and redacts unauthorized entities. Existing readiness fields, validator errors, feasibility diagnostics, and generation-run messages remain backward compatible. Infeasibility is never inferred from timeout, cancellation, or execution failure. No universal conflict table is introduced.

Lesson Requirement is a deterministic, read-only timetable projection of Planning authority, not a second demand model. The projection records exact scope, section/subject identity, effective periods, current teacher authority, explicit-versus-legacy provenance, a stable internal requirement identity, and a source fingerprint. Explicit zero/inactive demand suppresses fallback. Schema-v5 snapshots freeze this contract for historical generation; no persisted Lesson Requirement table or public CRUD API is introduced.

The Workflow worker depends on a solver-neutral `TimetableSolver` adapter. The Release 1 production binding is the existing OR-Tools CP-SAT engine, including its bounded infeasibility diagnostics. The adapter is an execution seam only: it does not create another scheduling model or move immutable input building, independent validation, stale-authority checks, or persistence into the solver.

Deterministic readiness establishes only **Configuration Complete**. Before full quality optimization, TIS dispatches a hard-only CP-SAT feasibility run against the exact immutable snapshot and validates the complete candidate independently. The validated placements are persisted by full input fingerprint. Generation is blocked unless that exact scope and fingerprint is verified; changed authority invalidates the result automatically. Full optimization receives the verified placements as a solver hint and may persist them as a validated fallback if its quality search times out. Soft objective terms never participate in the feasibility gate and hard rules are never relaxed.

Draft staleness is orthogonal to current-input structural completeness. `input_changed` keeps the existing Draft stale and outside approval/publication, but when no genuine configuration or allocation blocker remains it is eligible for a new hard-only feasibility check. That check captures current authority in a fresh immutable snapshot; an older fingerprint is never reused. Successful verification permits Regenerate from the protected stale source, producing a new version without rewriting history. Historical run results remain durable but only runs matching current authority may drive current workflow messaging. Source eligibility is lifecycle- and arrangement-based rather than origin-based: the current populated, mutable, unpublished Draft may be manual, generated, or regenerated. Empty starter Drafts remain Generate candidates. Active, superseded, archived, published-history, and older mutable versions are excluded.

Migration `20260828_004_planning_subject_demands_foundation` introduces an additive explicit per-section subject-demand table as the future Planning authority. Its Stage 1 backfill is limited to grade-matched Subjects for Current/New sections in the same branch/year. Stage 2 routes timetable readiness, workspace, immutable snapshot, and snapshot-fed generation input through explicit-first section demand. A missing explicit row alone uses `Subject.weekly_hours` as transitional fallback; an inactive or zero row is authoritative retirement. Published and draft lifecycle semantics do not change.

Planning remains authoritative for section demand, subject requirements, assigned teachers, and HRT fallback. A placement represents one teaching period, not a literal 60-minute hour.

`PlanningSubjectDemand.weekly_periods` supplies explicit per-section requirements, with `Subject.weekly_hours` retained only as missing-row compatibility fallback. HRT fallback resolves the real section-subject demands; no generic HRT lesson replaces them.

Stage 3 introduces a read-only curriculum-adjustment preview before any future late-stage demand mutation. It scopes Current/New sections by grade, explicit IDs, or all active source uses; projects demand, teacher capacity, distribution rules, grouped configuration, and mutable Draft impact; and returns a deterministic authority fingerprint for future stale confirmation. It cannot write demand, reassign teachers, regenerate a Draft, or mutate published history.

Stage 4 makes apply a separate curriculum.adjust capability and one atomic service transaction. It locks and rebuilds the reviewed fingerprint, rejects active generation and unresolved teacher/rule/lock conflicts, changes only selected Current/New demand and explicitly confirmed assignment state, records a durable deduplicated audit, and invalidates-but does not rewrite-the unpublished Draft. Regeneration remains explicit. Published versions and the active pointer stay immutable.

The timetable is a versioned aggregate scoped by SchoolGroup, Branch, and Academic Year. TimetableVersion owns placements, provenance, input authority, staleness, lifecycle, manual-edit, quality, and future solver metadata. Draft and non-active publication-ready versions may be edited. Active, superseded, and archived versions are immutable. TimetableActiveVersion is the sole active-selection authority and enforces one exact-scope pointer; active is not duplicated as a Boolean on a version.

The existing live timetable is imported exactly once per populated scope with origin=imported, a compatibility input snapshot, and an active pointer. Its placement fields are not repaired or normalized. Deterministically detectable inconsistencies are retained and recorded as safe version-level stale evidence. No historical publisher or approval is fabricated. A settings-only scope receives no empty imported version.

Until explicit version UI and publication arrive, the legacy assignment route uses copy-on-write: its first edit copies the active version to a mutable working draft, preserving locks and provenance; subsequent edits reuse that draft. Reads and exports use the operational version, preferring that compatibility draft after an edit while the imported active pointer continues to preserve the historical baseline.

Input snapshots use schema-versioned canonical JSON and SHA-256 component/full fingerprints. They record resolved Planning demand, HRT fallback decisions, timetable settings, canonical slot projection, constraints, and locks without unnecessary personal data. Stage 3.5 makes one composed timeline authoritative: each day starts at the configured shift time, retains the configured number and duration of teaching periods, inserts applicable non-teaching blocks, shifts later periods, and calculates the end. after_period is the preferred placement mode. Existing fixed-time blocks remain stored and compose only when their start is a current timeline boundary; ambiguous overlap fails closed without guessing or rewriting the row.

Locks persist on placements with actor and timestamp. A copied draft retains the intended locks. Later regeneration must treat locked placements as fixed decisions, but Stage 2 executes no regeneration.

Stage 4 exposes locks only on mutable drafts. Lock edits increment edit_revision, return publication-ready drafts to draft, and refresh the immutable input snapshot and authority fingerprint. Invalid locks remain visible and block validation.

Version review is selection-only. The compatibility operational order is the newest mutable non-active draft in the exact scope, otherwise active, otherwise the newest scoped mutable draft; a fresh manual draft therefore becomes the current customer Draft Timetable. Archived and superseded versions are never arbitrary defaults. An explicit copy is required before editing immutable history.

Draft validation checks freshness, complete demand, Planning teacher authority, canonical slots, collisions, placement integrity, and locks without a solver. Successful explicit administrator approval validates the exact draft, records its actor and timestamp, and transitions it to publication_ready; generation alone is not approval. Any later placement or lock mutation returns it to draft. Publication locks the version and exact-scope active pointer, checks edit/pointer revisions, reruns validation, supersedes the previous active version, updates the pointer, and records actor/time atomically. Regeneration preserves locked placements and requires the ceiling of 25 percent of unlocked direct-source placements to change. If that hard diversity target is infeasible, the run terminates with an explicit safe result; existing timetable hard constraints are never weakened and no near-identical fallback is persisted. Active status remains derived from the pointer rather than duplicated on the version.

`TimetableGenerationRun` is the durable PostgreSQL queue and reserves scope, requester, Generate/Regenerate mode, source version/revision, snapshot, solver/seed/diversity metadata, progress, attempts, lease/heartbeat, cancellation audit, safe failure, result, and idempotency. A partial unique index permits only one queued/running/validating/cancel-requested run per exact scope. The production web path dispatches an on-demand Render Workflow task containing only the run public ID. That task claims the exact queued row under a PostgreSQL lock, heartbeats its lease, and rejects save attempts after lease loss. The former `SKIP LOCKED` polling loop remains an optional local fallback, not a production requirement.

Stage 5.1 uses Google OR-Tools CP-SAT 9.15.6755 from a workflow-specific dependency file; no normally loaded web module imports OR-Tools. Schema-v3 immutable snapshots are the sole solve input. CP-SAT enforces exact demand, section and teacher slot exclusivity, canonical teaching slots, fixed locks, and regeneration diversity. Planning/HRT resolution happens before capture and remains subject-specific.

Generate creates a separate generated candidate. Regenerate leaves its source unchanged, fixes locked lessons, excludes the exact source arrangement, and requires the ceiling of the configured diversity percentage of unlocked placements to change (25 percent by default). Seed is recorded only as a secondary reproducibility input. A solver-independent validator checks scope, authority, exact demand, collisions, slots, locks, fingerprints, source revision, and diversity. A final current-input rebuild gates one atomic transaction that creates a publication-ready unpublished version and entries and completes the run. Generation never changes `TimetableActiveVersion` or published history.

Stage 5.2 preserves these internals but makes the newest mutable non-active version the customer Draft Timetable. Delete Draft Timetable permanently removes eligible never-published versions under exact-scope locks, preserves snapshots, runs, and history, removes unpublished dependent versions child-first, reports protected lineage, rejects active generation, and never changes the active pointer. Technical version controls remain in secondary Timetable History.

Official non-management consumption is separate from operational resolution.

`timetable_visibility_service.py` resolves strictly through `TimetableActiveVersion`. `/my-timetable` additionally requires exact-scope `User.user_id == Teacher.teacher_id` identity and filters to that teacher; view-only users cannot consume mutable history.

Academic quality settings are exact-scope explicit subject-code authority captured inside schema-v3 snapshots. CP-SAT implements daily core coverage when demand reaches teaching days, distinct-day and non-consecutive preferences, optional hard ICT one-per-day, and simultaneous configured Swimming groups. Group members may share their configured common teacher in one slot, while unrelated teacher collisions and optional shared-resource capacity remain protected. The independent validator repeats all hard rules. Exports remain version-aware. Future teacher availability, broader rooms/resources, and cross-campus coordination require separately approved stages.

Rejected Alternatives

- 1 Random-only generation: it cannot prove constraint satisfaction or explain infeasibility.
- 1 Overwriting the active timetable during regeneration: it destroys operational history and rollback safety.
- 1 Editing active or superseded versions in place: it makes publication evidence unreliable.
- 1 Treating readiness as proof of solver feasibility: structurally complete input may still be mathematically infeasible.
- 1 Running generation without explicit `SchoolGroup`, `Branch`, and `Academic Year` scope: it violates tenant safety.
- 1 Maintaining separate preview, readiness, export, and future-solver clock calculations: competing time authority causes overlaps and stale fingerprints.

Consequences

- 1 Existing placements remain operational and historically preserved.

- 1 Version numbering and active selection have database-enforced scope guarantees, with service validation for source/snapshot operations.
- 1 Stage 3/3.5 implements composed canonical slot projection and structural readiness on stable snapshots; valid inserted blocks do not consume teaching-period indexes.
- 1 Stage 4 implements version comparison and truthful publication without mutating active history. Stage 5.1 adds generation on this boundary without changing publication authority.
- 1 PostgreSQL row locking plus a per-scope unique key is the version-number allocation authority; SQLite retains the unique guard for supported local tests.
- 1 Stage 5.1 adds durable task execution, real phase polling, and independent validation. Production provisions one Render Workflow task per run and requires no always-on solver worker. Render automatic retries are disabled; TIS terminal state plus Generate Again is the sole retry authority. Stage 5.2 adds simplified workflow and published-only visibility. Academic-quality rules add mapped distribution and grouped activities without changing Workflow or publication authority.

Related Files

- 1 `models.py`
- 1 `db_migrations.py`
- 1 `timetable_snapshot_service.py`
- 1 `timetable_version_service.py`
- 1 `timetable_problem_builder.py`
- 1 `timetable_cp_sat_solver.py`
- 1 `timetable_solution_validator.py`
- 1 `timetable_generation_service.py`
- 1 `timetable_generation_worker.py`
- 1 `timetable_generation_workflow.py`
- 1 `timetable_workflow_dispatch.py`
- 1 `timetable_visibility_service.py`
- 1 `timetable_logic.py`
- 1 `routers/timetable.py`

Academic Calendar History

docs/history/academic-calendar/README.md

This folder tracks meaningful changes to academic calendar workflows, event types, permissions, exports, and branch/year behavior.

Related files:

- 1 routers/academic_calendar.py
- 1 templates/academic_calendar.html
- 1 permission_registry.py

AI Entitlements History

docs/history/ai-entitlements/README.md

2026-08-19 - Availability Separated From Consumption

ADR 0025 supersedes the temporary Enterprise-AI-only availability mapping. Every enabled registered AI feature is commercially available to coherent paid, promo, and active demo workspaces with `ai.use`; globally disabled definitions remain denied. A separate consumption-policy boundary preserves Standard demo allowances, Full/Custom policy, reservations, idempotency, counters, and durable events without inventing a provider billing engine.

2026-07-27 - M8B8 AI Entitlements And Commercial Foundation

M8B8 adds one tenant-safe AI entitlement authority, a controlled feature registry, an assignable `ai.use` permission, and durable usage counters plus reserved/successful/failed operation events. Customer Demo allowance is two successful uses per feature. Internal Sandbox is unlimited and measured separately. Customer Paid access follows the existing verified commercial resolver and `module.ai` plan entitlement; the reviewed foundation mapping is Enterprise AI only.

There are no executable AI routes in the current product. The service and denial payload are reusable boundaries for future implementations; M8B8 does not fabricate an AI tool or add M8B9 reset/override/lifecycle controls.

M8B9 Demo Operations, Notifications, And Testing

docs/history/demo-operations/2026-07-27-m8b9-demo-operations.md

Added generic Platform Owner controls for immediate reversible expiry, reactivation, unlimited-duration custom expiry, final-day reminders, and manual single/global lifecycle execution. Added Standard, Full, and Custom access profiles with workspace defaults and tenant-safe branch overrides.

All operations preserve the Customer Demo workspace and data, retain M8B8 AI usage history, use controlled feature registries, record durable before/after audits, and reuse the M8B7 email and Notification Center architecture. The schema is applied only by the existing pre-deploy migration command.

Common Customer Demo Feature Baseline

[docs/history/demo-operations/2026-08-19-common-customer-demo-baseline.md](#)

Standard, Full, and Custom active customer demos now retain the normal TIS customer feature baseline. Custom policy can still configure branch scope, safety, approved experimental features, and AI consumption allowances, but it cannot remove normal modules to simulate a lower subscription tier.

The migration reconciles only active demo product-feature policy. It preserves duration, lifecycle, expiry, AI allowance/unrestricted configuration, branch overrides, Platform Owner controls, and durable audit. Suspended, expired, or incoherent demos continue to fail closed.

KMS v3 Engineering Handbook

docs/history/engineering-handbook/2026-06-26-kms-v3-engineering-handbook.md

2026-06-26 - KMS v3 Engineering Handbook

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Engineering Handbook

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3A only.

Previous Documented State

The KMS had master context, project state, ADRs, module history, AI context, generated PDF, manifest, and Platform Owner Knowledge Center docs. It did not yet contain a complete engineering handbook for module ownership, repository architecture, and end-to-end flows.

New Documented State

The KMS includes docs/engineering/ with a module map, repository architecture guide, user/system flows, and developer onboarding index. The generated PDF now includes these docs as part of documentation version 3.0.

Reason For Change

New human developers and future Codex/ChatGPT conversations need a practical engineering handbook, not only a bundle of source documents.

User / Business Impact

Improves continuity, onboarding speed, technical review quality, and safer future implementation work.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/README.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/README.md
- 1 docs/CHANGE_HISTORY.md

PDF Regenerated

Yes

Follow-Up Needed

Review PDF readability and handbook completeness before any Phase 3B work.

KMS v3 Phase 3B Engineering Layers

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3b-engineering-layers.md

2026-06-26 - KMS v3 Phase 3B Engineering Layers

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Phase 3B Engineering Layers

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3B only.

Previous Documented State

The engineering handbook included the module map, repository architecture guide, user/system flows, and onboarding index. It did not yet include a database architecture overview, development standards, UI/UX philosophy, or product roadmap.

New Documented State

The engineering handbook includes Phase 3B layers:

- 1 database architecture overview,
- 1 development standards,
- 1 UI/UX design philosophy,
- 1 product roadmap,
- 1 stronger AI/human developer onboarding guidance.

Reason For Change

New senior developers, Codex conversations, ChatGPT conversations, and technical reviewers need stronger system boundaries, design expectations, roadmap context, and development rules before changing TIS.

User / Business Impact

Improves implementation safety, onboarding quality, technical review quality, and continuity across future development sessions.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md
- 1 docs/engineering/DEVELOPMENT_STANDARDS.md
- 1 docs/engineering/UI_UX_DESIGN_PHILOSOPHY.md

- 1 docs/engineering/PRODUCT_ROADMAP.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review handbook completeness and readability before any Phase 3C work.

KMS v3 Phase 3C Governance And AI Traceability

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3c-governance-ai-traceability.md

2026-06-26 - KMS v3 Phase 3C Governance And AI Traceability

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added KMS v3.0 Phase 3C Governance And AI Traceability

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for KMS v3.0 Phase 3C only.

Previous Documented State

The engineering handbook included module maps, repository architecture, user/system flows, database architecture, development standards, UI/UX philosophy, and roadmap. It did not yet document rejected decisions, visual documentation standards, definitive AI guidance, governance, or decision traceability.

New Documented State

The engineering handbook includes Phase 3C layers:

- 1 rejected architectural decisions,
- 1 visual documentation framework,
- 1 AI optimization guide,
- 1 project governance,
- 1 engineering decision traceability.

Reason For Change

Future developers and AI assistants need to understand why TIS became what it is, not only its current structure.

User / Business Impact

Improves continuity, reduces repeated architectural debates, improves AI readiness, and clarifies governance and review expectations.

Technical Impact

Documentation and PDF generation only. No app behavior, SaaS flow, landing page code, database, migration, route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/REJECTED_DECISIONS.md
- 1 docs/engineering/VISUAL_DOCUMENTATION_GUIDE.md
- 1 docs/engineering/AI_OPTIMIZATION_GUIDE.md

- 1 docs/engineering/PROJECT_GOVERNANCE.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Future KMS improvements may add screenshots, diagrams, deployment runbooks, route inventories, test strategy docs, and deeper module guides. Do not implement those without a later approved phase.

KMS v3 Phase 3D Lifecycle Foundation

docs/history/engineering-handbook/2026-06-26-kms-v3-phase-3d-lifecycle-foundation.md

2026-06-26 - KMS v3 Phase 3D Lifecycle Foundation

Module: Engineering Handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Completed KMS v3.0 Phase 3D Lifecycle Foundation

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved as KMS v3.0 Phase 3D final phase.

Previous Documented State

The engineering handbook documented module maps, repository architecture, flows, database architecture, standards, UI/UX philosophy, roadmap, rejected decisions, visual documentation, AI optimization, and governance. It did not yet define a full self-evolving lifecycle from task through deployment.

New Documented State

The engineering handbook now includes:

- 1 knowledge lifecycle,
- 1 documentation automation guide,
- 1 formal KIA standard,
- 1 self-evolving workflow,
- 1 documentation dependency map,
- 1 AI coding workflow,
- 1 future automation roadmap.

This completes the KMS v1.0 lifecycle foundation.

Reason For Change

Future implementation work needs a repeatable lifecycle that keeps KMS synchronized with software changes without automatic source-doc rewriting.

User / Business Impact

Improves long-term governance, reviewer confidence, AI coding discipline, and continuity across future phases.

Technical Impact

Documentation and PDF generation only. No SaaS flow, landing code, database model, migration, `tis.db`, app route, or runtime behavior changed.

Documentation Updated

- 1 docs/engineering/KNOWLEDGE_LIFECYCLE.md

- 1 docs/engineering/DOCUMENTATION_AUTOMATION.md
- 1 docs/engineering/KNOWLEDGE_IMPACT_ASSESSMENT_STANDARD.md
- 1 docs/engineering/SELF_EVOLVING_WORKFLOW.md
- 1 docs/engineering/DOCUMENTATION_DEPENDENCY_MAP.md
- 1 docs/engineering/AI_CODING_WORKFLOW.md
- 1 docs/engineering/FUTURE_AUTOMATION_ROADMAP.md
- 1 core KMS index/context files

PDF Regenerated

Yes

Follow-Up Needed

Review the final handbook for readability. Future improvements should be handled as new approved KMS phases.

Production Memory Stability Guardrails

docs/history/engineering-handbook/2026-06-27-production-memory-stability-guardrails.md

2026-06-27 - Production Memory Stability Guardrails

Module: Engineering Handbook and operational app stability

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-27 - Added Production Memory Stability Guardrails

Related ADRs: None. This is a standards and operational guardrail update, not a new architecture decision.

Reviewer/approval notes: Prepared after a Render restart/502 investigation. Local changes only; no commit, push, migration, database change, or deployment was performed.

Previous Documented State

The KMS required scoped changes, tenant isolation, validation, and documentation updates, but it did not explicitly define a production memory budget or prohibit common memory hazards such as:

- 1 full large-dataset parsing for normal picker requests,
- 1 duplicate template rendering in production routes,
- 1 warning-level diagnostic stage logs during normal traffic,
- 1 unbounded global caches,
- 1 startup-heavy work without memory review.

New Documented State

docs/engineering/DEVELOPMENT_STANDARDS.md now includes a mandatory Production Memory and Render Stability section.

The standards require future work to:

- 1 treat 512 MB Render memory as a hard production budget,
- 1 avoid full large-dataset loads when scoped or streaming lookup is available,
- 1 avoid duplicate template renders in production request paths,
- 1 keep diagnostics opt-in through explicit environment flags,
- 1 filter data before materializing result sets,
- 1 keep caches bounded and justified,
- 1 include memory/performance smoke checks for memory-sensitive changes.

Reason For Change

The 2026-06-27 production investigation found avoidable memory pressure risks after deployment:

- 1 /observations/ had diagnostic logging and extra template pre-renders in the normal route path.
- 1 Global location lookup could parse a 47 MB country/state/city reference dataset into a complete in-memory index for simple picker API calls.

These patterns can trigger process restarts and temporary 502 responses on constrained production instances.

User / Business Impact

The rule reduces the risk that normal navigation causes Render restarts, temporary 502s, or user-visible instability. It also creates a clear review standard for future SaaS onboarding, dashboard, reports, observations, and large-data changes.

Technical Impact

Future changes that add broad data loading, reference datasets, exports, diagnostics, or startup work must include memory-conscious design and validation. The local stabilization patch also reduces current risk by gating observation diagnostics and changing location lookup to scoped loading.

Documentation Updated

- 1 docs/engineering/DEVELOPMENT_STANDARDS.md
- 1 docs/PROJECT_STATE.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/CHANGE_HISTORY.md

PDF Regenerated

Yes

Follow-Up Needed

Review, commit, and deploy the stabilization changes when approved. After deployment, monitor Render memory, restart count, warning logs, and 502 frequency.

Automatic KMS Synchronization Enforcement

docs/history/engineering-handbook/2026-07-21-kms-automatic-enforcement.md

2026-07-21 - Automatic KMS Synchronization Enforcement

Previous State

KMS governance was documented but not mechanically enforced. Developers and AI assistants manually decided impact, edited Markdown, and regenerated artifacts. The Knowledge Center could detect stale hashes, but no pull-request or deployment gate consumed that status.

New State

TIS has a narrow repository enforcement layer:

- 1 `AGENTS.md` makes KMS onboarding and completion rules default for Codex.
- 1 `.kms-impact.yml` records task-level impact, affected areas, updated KMS files, and controlled no-impact overrides.
- 1 `scripts/check_kms_impact.py` compares the declaration with a Git range or local worktree, classifies likely major changes, and validates generated artifacts.
- 1 `scripts/generate_docs_pdf.py --check` verifies source coverage, source hashes, PDF hash/size, version, and manifest consistency without writing.
- 1 GitHub Actions run enforcement for pull requests and dev; production deployment from master depends on the same gate.

Cross-Platform Correction

The initial enforcement release exposed two checkout-dependent differences: Markdown hashes reflected raw CRLF/LF bytes, and dynamic source ordering followed native path comparison. The generator and Knowledge Center now hash normalized UTF-8/LF Markdown. The generator also records repository-relative POSIX paths in a deterministic order while continuing to fail on genuine source-list drift. Impact detection includes deleted files so removing behavior or documentation cannot bypass review.

Task-Boundary Correction

The initial push workflow compared only the previous pushed commit with the new head, while pull requests compared the complete feature branch. Follow-up commits therefore validated a task-level declaration against a commit-level diff. Push checks now derive the task base from the merge base of the default branch and pushed head. Pull-request and push checks consequently enforce the same cumulative KIA scope without allowing undeclared documentation changes.

Phase 6 Unified Command

The initial automation exposed generation, artifact checking, and KIA enforcement as separate commands. Phase 6 adds `scripts/kms.py sync` for KIA preflight, PDF/manifest generation, post-generation freshness validation, and completion reporting. `scripts/kms.py check` is the canonical read-only command used locally and by CI. Both commands delegate to the existing generator and checker; they do not duplicate policy or rewrite Markdown.

Guardrails

Automation does not generate or rewrite Markdown prose. Reviewed Markdown remains authoritative. KMS files must describe engineering design only and must not contain customer, organization, personal, transaction, invoice, production identifier, webhook payload, secret, credential, environment, or database-row data.

Related Files

```
1 AGENTS.md
1 .kms-impact.yml
1 scripts/check_kms_impact.py
1 scripts/generate_docs_pdf.py
1 scripts/kms.py
1 .github/workflows/kms-enforcement.yml
1 .github/workflows/deploy-on-master.yml
1 docs/DOCUMENTATION_UPDATE_POLICY.md
```

KMS Phase 7A Navigation Foundation

docs/history/engineering-handbook/2026-07-21-kms-phase-7a-navigation-foundation.md

2026-07-21 - KMS Phase 7A Navigation Foundation

Previous State

The KMS contained comprehensive core, engineering, ADR, history, marketing, and supporting documentation. Root and engineering README files listed those sources, but readers still had to assemble their own task-specific reading sequence. Three supporting documents also lacked standard title front matter.

New State

docs/KMS_NAVIGATION.md is the canonical role-based and task-based reading guide. It directs human developers, AI assistants, owners, and reviewers to focused document sets with explicit guardrails. Root and engineering indexes now use navigable Markdown links and point to the guide instead of duplicating every reading decision. The location roadmap and both landing-page documents have normalized title metadata.

Scope Boundary

Phase 7A changes reviewed Markdown navigation only. It does not add generated catalog metadata, PDF table-of-contents behavior, PDF bookmarks, Knowledge Center search/filtering, protected document-reader routes, or new KMS enforcement rules.

Related Files

- 1 docs/KMS_NAVIGATION.md
- 1 docs/README.md
- 1 docs/engineering/README.md
- 1 docs/location-data-roadmap.md
- 1 docs/marketing/landing_page_source_of_truth.md
- 1 docs/marketing/tis_landing_page_master_content.md

KMS Phase 7B Professional PDF Navigation

docs/history/engineering-handbook/2026-07-21-kms-phase-7b-pdf-navigation.md

2026-07-21 - KMS Phase 7B Professional PDF Navigation

Previous State

The generated booklet preserved all approved Markdown sources in fixed order and provided page footers, but readers had to move through a long linear document. The manifest tracked source freshness without recording where each source began in the PDF.

New State

The existing ReportLab generator now provides:

- 1 a dedicated "How to Use This Handbook" page,
- 1 a multi-pass table of contents with source-document page numbers,
- 1 stable named destinations and outline entries for every source document,
- 1 child outline entries for H2 major headings,
- 1 each source document's starting pdf_page in the generated manifest,
- 1 strict validation for missing, invalid, or non-increasing source pages.

Bookmark identifiers are derived deterministically from repository-relative source paths and heading context. Existing source ordering, normalized Markdown hashing, source-list comparison, PDF identity checks, and freshness validation remain in force.

Scope Boundary

Phase 7B does not change the Platform Owner Knowledge Center UI, add routes, add dependencies, alter Markdown authority, modify application behavior, or introduce database changes. Phase 7C remains separate.

Related Files

- 1 scripts/generate_docs_pdf.py
- 1 tests/test_kms_automation.py
- 1 static/docs/TIS_Project_Reference_Booklet.pdf
- 1 static/docs/docs_manifest.json

KMS Phase 7D Navigation And Catalog Enforcement

docs/history/engineering-handbook/2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md

2026-07-22 - KMS Phase 7D Navigation And Catalog Enforcement

Module: KMS governance and engineering handbook

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-07-22 - Added Phase 7D Navigation And Catalog Enforcement

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 7D only. Knowledge Center UI, routes, application workflows, database, dependencies, commits, and pushes remain out of scope.

Previous Documented State

KMS enforcement validated KIA declarations, authoritative source coverage, normalized source hashes, deterministic manifest source ordering, generated PDF identity, and positive increasing source page metadata. Phase 7A added task navigation, Phase 7B added PDF navigation, and Phase 7C added the owner catalog, but their title, taxonomy, navigation-target, and real PDF page-bound assumptions were not fully enforced by `scripts/kms.py check`.

New Documented State

The shared dependency-free `kms_catalog.py` defines approved categories, modules, and path classification for both Knowledge Center presentation and validation. The existing read-only check now requires usable source titles, approved taxonomy, exact normalized manifest/source-list equality, valid docs-only navigation links to listed Markdown, and positive increasing source pages that do not exceed the generated PDF page count. Existing hash, freshness, ordering, source coverage, KIA, and artifact checks remain active.

Reason For Change

Navigation and catalog quality must fail during the same local and CI command that already protects KMS freshness, preventing broken onboarding paths or unusable owner metadata from reaching integration.

User / Business Impact

Platform owners, developers, and AI assistants receive a more dependable catalog and navigation guide. Invalid knowledge metadata is blocked before review or deployment rather than discovered while reading the handbook.

Technical Impact

The generator/checker uses dependency-free Markdown metadata and link parsing plus ReportLab PDF page-object counting. Focused tests cover valid and invalid titles, taxonomy, relative links, missing/unlisted targets, source-list drift, and PDF page metadata. The Knowledge Center retains the same routes, UI, permissions, and read-only behavior.

Documentation Updated

- 1 docs/DOCUMENTATION_UPDATE_POLICY.md
- 1 docs/KMS_NAVIGATION.md
- 1 docs/PROJECT_STATE.md

- 1 docs/CHANGE_HISTORY.md
- 1 docs/engineering/DOCUMENTATION_AUTOMATION.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/history/engineering-handbook/README.md
- 1 docs/history/engineering-handbook/2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md

PDF Regenerated

Yes

Follow-Up Needed

Monitor real pull requests for demonstrated false positives. Any future taxonomy expansion must update the shared catalog, relevant docs, and focused tests together.

Claude Code KMS Configuration

docs/history/engineering-handbook/2026-09-03-claude-code-kms-configuration.md

Previous state

TIS had repository-wide AGENTS.md, authoritative KMS onboarding, and automated KIA enforcement, but no repository-native Claude Code entry point, project skill, or specialized subagent definition.

New state

Root CLAUDE.md directs Claude Code to the existing repository and KMS authorities.

.claude/skills/tis-kms/SKILL.md packages the reusable TIS investigation, implementation, validation, KIA, synchronization, and reporting workflow. .claude/agents/tis-kms-developer.md provides the same safeguards for bounded delegated work.

The configuration preserves existing worktree changes, tenant and academic scope, permissions, publication history, audit boundaries, database safety, and the no-commit/no-push default. It does not introduce a competing architecture source or change application, schema, migration, customer, production, or deployment behavior.

Validation contract

Claude Code must run proportional implementation checks, `git diff --check`, KMS sync after authoritative Markdown changes, and final read-only KMS validation. Its completion report includes exact KMS files and the Knowledge Impact Assessment.

Engineering Handbook History

docs/history/engineering-handbook/README.md

This folder tracks meaningful changes to the TIS Engineering Handbook, including module maps, repository architecture, user/system flows, and onboarding guidance for humans and AI coding conversations.

Latest change:

- 1 2026-09-03-claude-code-kms-configuration.md: repository-native Claude Code

entry point, reusable TIS KMS skill, and bounded specialized subagent.

- 1 2026-07-22-kms-phase-7d-navigation-catalog-enforcement.md: strict source-title, catalog, navigation-link, source-inventory, and PDF page-bound validation.

Related files:

- 1 docs/engineering/README.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/engineering/REPOSITORY_ARCHITECTURE.md
- 1 docs/engineering/USER_AND_SYSTEM_FLOWS.md

Landing CTA Consolidation And Demo Domain Policy

docs/history/landing-page/2026-07-25-landing-cta-and-demo-domain-policy.md

The public landing website now uses two clear customer conversion paths:

- 1 Request a Demo: configured SaaS signup URL with `intent=demo`.
- 1 Subscribe Now: configured SaaS signup URL with `intent=subscribe`.

Both URLs are built from `NEXT_PUBLIC_TIS_APP_BASE_URL`; the public site continues to deploy independently from the FastAPI application. The landing page no longer exposes overlapping Open Account, Book a Demo, Request Early Access, or Request Pricing conversion labels.

The selected intent is persisted by the SaaS application through account creation, email verification, School Workspace Setup, and review. At commercial choice, the selected path receives visual emphasis but the customer can choose either available path.

Customer Demo eligibility is enforced by the SaaS application, not by the landing site. It uses a normalized organization domain and a database unique reservation so a second customer demo cannot be created for the same organization after any prior demo request or lifecycle outcome. Internal Sandbox history is excluded. Historical duplicate domains are retained for Platform Owner manual review without deleting, merging, or recreating customer workspaces.

M8 Landing Integration Open Account

docs/history/landing-page/2026-07-25-m8-landing-integration-open-account.md

2026-07-25 - M8 Landing Integration Open Account

Module: Landing Page

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-07-25 - M8 Landing Integration Open Account Entry Points

Related ADRs:

- 1 docs/adr/0001-separate-nextjs-landing-website.md
- 1 docs/adr/0007-landing-page-visual-system-strategy.md

Reviewer/approval notes: M8 final landing integration only. No FastAPI SaaS application, authentication, onboarding backend, payment, provisioning, database, operational module, commit, push, or M9 work.

Previous Documented State

The public Next.js landing website explained TIS, supported demo and early-access interest, and remained separate from the FastAPI application. It did not yet expose the completed customer account setup flow from the public marketing surface.

New Documented State

The public landing website now includes Open Account entry points in:

- 1 the navigation,
- 1 the hero CTA group,
- 1 the final CTA area.

All Open Account links share one destination derived from NEXT_PUBLIC_TIS_APP_BASE_URL and the existing /saas/signup path. The landing project remains a marketing website and redirects visitors to the separately deployed TIS Account signup flow.

Reason For Change

M8A and M8B-1 through M8B-6 completed the SaaS-side customer account, workspace, demo, subscription, and conversion readiness. M8 final integration connects the public website to that existing deployed signup journey without changing the SaaS application.

User / Business Impact

Visitors can move directly from `tisplatform.com` to account setup. Schools still retain demo and early-access paths, while Open Account becomes the direct signup entry point for M8 production validation.

Technical Impact

The change is limited to `tis-landing-website/` plus KMS documentation. The FastAPI app, SaaS routes, authentication, onboarding backend, Paddle, database, APIs, demo workflow, provisioning, and commercial state remain unchanged.

Follow-Up Needed

Set NEXT_PUBLIC_TIS_APP_BASE_URL in the landing website hosting environment, deploy the landing project separately from the FastAPI app, and validate that Open Account navigates to the deployed TIS Account signup flow on desktop, tablet, and mobile.

Landing Page History

docs/history/landing-page/README.md

2026-08-19 - Capacity-Based Feature Copy

The official Next.js pricing cards remain capacity-only. The AI disclaimer no longer implies subscription-tier availability and instead identifies permissions, workspace scope, configured usage allowances, and release stage. Plan names, prices, capacities, layout, and actions are unchanged.

2026-07-29 - Safe Pricing Subscription Entry

Hero Subscribe Now now scrolls to #pricing. The three self-service plans use the same CTA treatment and link to public TIS Account registration with their own allowlisted preferred-plan code. Plan descriptions were removed to keep the cards focused on price, capacity, and action. Custom remains email-only.

2026-07-28 - Returning-Customer Navigation

The Next.js landing now exposes Sign In and Open TIS App beside Request a Demo and Subscribe. Every app link derives from NEXT_PUBLIC_TIS_APP_BASE_URL; authentication and state routing remain in FastAPI.

This folder tracks meaningful changes to the public landing page, marketing positioning, visual system strategy, and source-of-truth boundaries.

Related docs:

- 1 docs/adr/0001-separate-nextjs-landing-website.md
- 1 docs/adr/0007-landing-page-visual-system-strategy.md
- 1 docs/marketing/landing_page_source_of_truth.md
- 1 docs/marketing/tis_landing_page_master_content.md

History entries:

- 1 2026-07-25-landing-cta-and-demo-domain-policy.md: replaces overlapping public conversion CTAs with Request a Demo and Subscribe Now, both using configured deployed SaaS signup URLs and explicit onboarding intent.
- 1 2026-07-25-m8-landing-integration-open-account.md: final M8 Open Account entry points from the public landing website to the deployed TIS Account signup flow.

Platform Owner Knowledge Center

docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

2026-06-26 - Platform Owner Knowledge Center

Module: Platform Knowledge Center

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Added Platform Owner Knowledge Center

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 2C. Regenerate button, public access, SaaS changes, database changes, migrations, landing page changes, commits, and pushes remain out of scope.

Previous Documented State

TIS had KMS Markdown source files, ADRs, module history, a generated PDF booklet, and a manifest, but there was no protected in-app owner utility for viewing KMS status or accessing the booklet.

New Documented State

TIS has a read-only Platform Owner Knowledge Center that shows KMS health, manifest metadata, PDF freshness, source document status, recent change-history entries, ADRs, module history areas, and the Knowledge Impact Assessment checklist.

Reason For Change

Platform owners need a protected internal view of KMS status and a safe way to view/download the generated PDF without linking directly to static public paths.

User / Business Impact

Platform owners can verify whether the KMS snapshot is current and access the reference booklet from inside the app.

Technical Impact

Adds read-only KMS service helpers, owner-protected FastAPI routes, one app-shell template, and an owner-only Platform Console card. No SaaS, database, migration, billing, provisioning, or landing page behavior changes.

Documentation Updated

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/CHANGE_HISTORY.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/history/platform-knowledge/README.md
- 1 docs/history/platform-knowledge/2026-06-26-platform-owner-knowledge-center.md

PDF Regenerated

Yes

Follow-Up Needed

Consider a future explicit owner-only regenerate action after review. The app must not silently rewrite Markdown source docs.

KMS Phase 7C Knowledge Center Navigation

docs/history/platform-knowledge/2026-07-22-kms-phase-7c-knowledge-center-navigation.md

2026-07-22 - KMS Phase 7C Knowledge Center Navigation

Module: Platform Knowledge Center

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-07-22 - Added Phase 7C Knowledge Center Navigation

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 7C only. Existing owner permissions and protected routes remain unchanged. No database, route, dependency, search service, regenerate action, application-data change, commit, or push is included.

Previous Documented State

The owner-only Knowledge Center displayed manifest metadata, freshness, source paths, ADRs, module-history areas, change history, and protected booklet actions. Its source inventory was one path-focused table without search, filters, logical sections, descriptive summaries, or page-specific booklet links.

New Documented State

The manifest remains the sole source inventory. The Knowledge Center derives display titles, summaries, categories, and modules from safe manifest-listed Markdown files; groups sources into Core, Engineering, Decisions, History, Marketing, and Supporting sections; and filters the rendered rows in the browser by search text, category, module, and freshness. Each source can open the existing protected booklet route at its manifest pdf_page. ADRs and module-history areas now surface newest activity first.

Reason For Change

Platform owners need a faster path from KMS status to the exact knowledge they are reviewing while the system remains read-only, dependency-light, and owner protected.

User / Business Impact

Platform owners can find a document by title, summary, module, path, category, or freshness and move directly to its handbook page without exposing the static PDF URL.

Technical Impact

knowledge_service.py adds presentation metadata and deterministic activity ordering for existing manifest sources. templates/platform_knowledge_center.html adds browser-only filtering and protected page-fragment links. No source Markdown is rewritten by the app, and no application state or access boundary changes.

Documentation Updated

- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/README.md

- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/CHANGE_HISTORY.md
- 1 docs/engineering/TIS_MODULE_MAP.md
- 1 docs/history/platform-knowledge/README.md
- 1 docs/history/platform-knowledge/2026-07-22-kms-phase-7c-knowledge-center-navigation.md

PDF Regenerated

Yes

Follow-Up Needed

Phase 7D and any Knowledge Center regenerate, server-side search, or additional route work require separate approval.

Platform Knowledge Center History

docs/history/platform-knowledge/README.md

This folder tracks meaningful changes to the owner-only TIS Knowledge Center, protected documentation PDF access, KMS metadata display, freshness detection, knowledge navigation, and knowledge health reporting.

Latest change:

- 1 2026-07-22-kms-phase-7c-knowledge-center-navigation.md: searchable, filtered, grouped manifest library with protected booklet page links and improved activity ordering.

Related files:

- 1 knowledge_service.py
- 1 templates/platform_knowledge_center.html
- 1 templates/platform_console.html
- 1 main.py

KMS Foundation

docs/history/provisioning/2026-06-26-kms-foundation.md

2026-06-26 - KMS Foundation

Module: Documentation and knowledge management

Related change-history entry: docs/CHANGE_HISTORY.md - 2026-06-26 - Established Knowledge Management System Foundation

Related ADRs: docs/adr/0006-documentation-as-source-knowledge-management-system.md

Reviewer/approval notes: Approved for Phase 2A and Phase 2B only.

Previous Documented State

TIS had Phase 1 documentation source files and a generated PDF booklet, but no formal KMS policy, ADR structure, module history foundation, or AI onboarding context.

New Documented State

TIS now has a KMS foundation with policy, change history, ADRs, module history folders, and AI project context. The PDF generator includes these source docs and produces manifest metadata.

Reason For Change

Future human developers, Codex conversations, ChatGPT conversations, project owners, platform owners, and reviewers need durable project context and history.

User / Business Impact

Improves continuity, reduces repeated rediscovery, and makes project decisions easier to review.

Technical Impact

No runtime app behavior changed. This is documentation and PDF generation foundation only.

Documentation Updated

- 1 docs/CHANGE_HISTORY.md
- 1 docs/DOCUMENTATION_UPDATE_POLICY.md
- 1 docs/AI_PROJECT_CONTEXT.md
- 1 docs/adr/
- 1 docs/history/
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/PROJECT_STATE.md
- 1 docs/README.md

PDF Regenerated

Yes

Follow-Up Needed

Phase 2C can later add a protected Platform Owner Knowledge Center after review.

Provisioning History

`docs/history/provisioning/README.md`

This folder tracks meaningful changes to pending organizations, provisioning jobs, verified-payment readiness, retry behavior, and platform owner provisioning oversight.

Related docs:

- 1 `docs/adr/0005-delayed-tenant-provisioning-after-verified-payment.md`
- 1 `docs/TIS_MASTER_CONTEXT.md`

Related files:

- 1 `saas/provisioning_service.py`
- 1 `saas/service.py`
- 1 `saas/router.py`
- 1 `templates/saas/admin_provisioning.html`

2026-08-18 - SchoolGroup Operational Creation Boundary

Direct operational SchoolGroup creation and deletion now require Platform identity and explicit global SchoolGroup-management capability. Tenant role defaults no longer include these actions, stale permissions cannot cross the handler boundary, and tenant updates accept only the linked SchoolGroup. Approved paid, demo, promo, and conversion provisioning paths are unchanged. A separate PostgreSQL-only read-only provenance audit reports active internal sandboxes missing tenant-link and explicit entitlement evidence for later Platform Owner review; it performs no remediation.

2026-07-22 - Lifecycle-Aware Platform Owner Queue

Platform Owner pending counts and lists now share one query boundary. A record remains pending only while its onboarding status is draft, in progress, changes requested, under review, or ready for checkout and no tenant link, completed provisioning job, or final tenant billing state exists. Active tenants are resolved from coherent payment, active subscription, contract, tenant-link, and active SchoolGroup evidence. Completed provisioning with incomplete or conflicting commercial evidence is retained as Lifecycle Review Required rather than shown as ordinary pending work. No lifecycle rows are rewritten by this owner-facing projection.

TIS Module History

docs/history/README.md

Module history stores deeper area-specific change records. It complements docs/CHANGE_HISTORY.md, which remains the chronological summary.

Use module history when a meaningful area changes and the previous documented state should be preserved.

Module Areas

```
1    ai-entitlements/
1    subscriptions/
1    landing-page/
1    academic-calendar/
1    workforce-planning/
1    saas-onboarding/
1    provisioning/
```

Entry Template

```
# YYYY-MM-DD - Change Title

Module:
Related change-history entry:
Related ADRs:
Reviewer/approval notes:

## Previous Documented State

## New Documented State

## Reason For Change

## User / Business Impact

## Technical Impact

## Documentation Updated

## PDF Regenerated

## Follow-Up Needed
```

SaaS Onboarding History

docs/history/saas-onboarding/README.md

2026-08-19 - Common Customer Feature Baseline

The July 16 feature matrix below remains historical context and is superseded by ADR 0025. Normal customer PlanEntitlement values are now common across Starter, Professional, and Enterprise AI; plan selection remains capacity- and billing-oriented. Existing paid workspace entitlements resolve the new matrix dynamically and require no per-workspace feature backfill.

2026-07-30 - Secure Payment Retry And Checkout Lineage

The observed failure was entirely local and occurred before any Paddle API request:

`_ensure_checkout_launchable()` rejected the legacy `ready_for_checkout` pre-checkout billing state. That state is now prepared safely. Professional Annual remains mapped to USD 790 per active branch; the verified two-branch quote uses quantity 2 and totals USD 1,580 annually.

Secure Payment retry now rebuilds an eligible unpaid checkout when legacy billing state or an obsolete session prevents launch. Plan, interval, quantity, selection, and fingerprint changes supersede both the checkout session and its unfinished payment attempt. The current contract and organization no longer point at that obsolete authority, and its later webhook is blocked before subscription or workspace activation.

Existing started transactions are reused only after remote billed and quote validation. Paddle customer address lookup reuses an active compatible country address, provider failures remain detailed in server logs, and the payment page shows one customer-safe retry alert. Prices, capacity rules, provisioning, and webhook confirmation authority remain unchanged.

2026-07-29 - Organization Profile Save And Compact Account Setup

Organization Profile now accepts the three approved program labels through a required selector and returns customer validation errors inline. Pending logo uploads reuse actual-image validation, the 4 MB limit, minimum dimensions, opaque UUID filenames, and atomic writes. Newly written files are removed if a later save fails; empty upload retains the prior logo and successful replacement safely retires it. Storage and unexpected failures retain transaction rollback and produce server traceback logs without exposing raw errors.

The initial Account Setup state now contains one setup title, one explicit POST start action, and the existing eight-step journey under a smaller official logo. Repeated status, action, context, and help panels are omitted only before the pending organization exists.

Pending logos still use application-local `static/uploads/saas/pending_logos`; durable Render storage remains a separate persistent-disk or object-storage decision.

The saved logo is now visible in Organization Profile and the shared School Workspace setup identity. Existing paid and demo provisioning already copied the pending image into `SchoolGroupLogo`; that path is now explicitly directory-confined, missing referenced files fail activation instead of being silently skipped, and the final logo label uses the organization name for accessible operational-shell rendering. TIS platform branding remains separate.

Branch Setup no longer performs nullable estimate arithmetic in Jinja. Python-normalized totals make empty, legacy-null, and mixed rows safe to render. Customer saves retain required non-negative estimate validation, and new branch records receive explicit zero defaults.

2026-07-29 - Post-Verification Login Method Safety

Fresh verified accounts no longer redirect successful login to the POST-only onboarding creation endpoint. They open the GET Account Setup dashboard, where the established explicit POST action starts School Workspace Setup. Sign-in continuations now accept only known customer GET destinations, and accidental GET navigation to the credential handler redirects to the normal sign-in page instead of returning raw HTTP 405 JSON. Same-email re-registration, subscription intent, and non-authoritative preferred-plan state remain compatible.

2026-07-29 - Public Preferred-Plan Continuity

Public pricing may carry an allowlisted Starter, Professional, or Enterprise AI preference into TIS Account registration. The preference is stored separately from commercial records and does not select a plan or create checkout. Once School Workspace Setup is complete, the existing branch, system-user, and teacher capacity decision either preselects the eligible preference or clears it with customer-safe guidance. Missing and malformed values are safe.

2026-07-28 - Returning Customer State Routing

Successful SaaS login now derives its destination from onboarding, pending demo, durable account-to-workspace, lifecycle, classification, and subscription evidence. Active customers enter the app, incomplete customers resume setup, unpaid customers reach plans, and expired customers receive branded guidance. Customer language uses "TIS team," while internal Platform Owner identifiers and audits remain intact.

2026-07-27 - M8B7 Demo Customer Journey

Normal approval now invokes the existing independently retryable provisioning service. Six branded lifecycle email types use a durable outbox, and Platform Owner events reuse the existing Notification Center. The shared tenant shell displays active Customer Demo status. Day 6 and expiry create communication intents atomically, and coherent expired demos may convert after authoritative confirmed payment by reactivating the same workspace. M8B8 and M8B9 are not included.

This folder tracks meaningful changes to signup, login, account, organization onboarding, contacts, branches, academic setup, review, and account self-service.

2026-07-26 - Platform Owner Historical Demo Eligibility Maintenance

Historical detached SaaS Demo Domain Eligibility rows can outlive the organization/account records that once made them reachable through clean-room reset. Platform Owners now have a separate `/saas-admin/demo-eligibility-maintenance` page that scans the eligibility ledger and explains whether each reservation is safe or protected.

The maintenance analyzer uses authoritative domain normalization and checks for matching pending organizations, SaaS accounts, demo requests, tenant-profile workspaces, paid or demo provisioning, subscription contracts/subscriptions, Demo-to-Paid conversions, and manual-review evidence. Only an exact eligibility ID with no blockers may be deleted. The POST action locks and re-analyzes the row, requires typed-ID and checkbox confirmation, bulk-deletes by primary key only, flushes, verifies absence, and commits or rolls back as one transaction. Successful cleanup is owner-audited.

This administrative repair capability does not modify the customer demo request workflow, the durable one-demo-per-domain rule, the existing test workspace/account reset, schema, foreign keys, or customer-facing behavior.

2026-07-26 - Internal Test Workspace Commercial Clean-Room Reset

The guarded Platform Owner test workspace/account reset now clears only the selected organization's linked demo request, domain-eligibility reservation, review and event history, demo provisioning and lifecycle records, and demo-to-paid conversion history before deleting parent commercial and workspace records. The same email and

organization domain can then complete a new internal M8 journey.

Detached same-domain reservations left by the `ON DELETE SET NULL` relationship are also eligible for removal, but only after the same domain resolver used by Customer Demo submission confirms there is no other organization, request, workspace, or customer account using that domain and the reservation has no historical manual-review evidence. Any conflicting or ambiguous evidence blocks the reset for manual review. This narrow exception does not alter the production Customer Demo policy: normal customer domain reservations remain durable after pending, approved, active, expired, rejected, cancelled, or converted history. Global plans, prices, Paddle records, platform configuration, and other organizations' records remain outside the reset scope. Customer-facing demo wording now identifies the TIS team rather than an internal role.

Reset analysis now publishes the exact detached eligibility IDs that passed those safety checks. The deletion service consumes only that safe list, deletes it before demo-request and parent records, flushes immediately, and verifies that no selected ID remains. Any handoff mismatch or failed verification stops the transaction so the owner route can roll back all changes.

2026-07-25 - Landing Intent Continuity And One Customer Demo Per Organization Domain

The public landing conversion paths now enter TIS Account signup with a validated demo or subscribe intent. The intent is persisted on the SaaS account and copied to School Workspace Setup so the final commercial-choice page can emphasize the customer's original path while still allowing either Request Demo or Subscribe Now.

Customer Demo requests now resolve a normalized organization domain, preferring the organization identity and using a work email only when necessary. Public email providers require an official organization website or domain. A transactionally unique domain-eligibility reservation prevents a second customer demo across pending, approved, active, expired, rejected, cancelled, or converted history. The reservation never causes tenant reprovisioning or Demo-to-Paid workspace replacement, and Internal Sandbox history is excluded.

Migration `20260725_001_demo_domain_eligibility_policy` adds safe historical normalization. It reserves ambiguous duplicate domains for manual review rather than merging, deleting, or guessing across existing customer records. The companion diagnostic command defaults to dry-run and produces the duplicate-domain review list.

2026-07-23 - M8B-6 Demo-To-Paid Workspace Conversion

Eligible active Customer Demo workspaces can now enter the unchanged M7 subscription checkout.

Provider-confirmed payment triggers a dedicated, idempotent conversion service rather than paid tenant provisioning.

The conversion preserves the SchoolGroup, workspace UUID, tenant link row, pending organization, branches, users, permissions, academic records, and all request/demo/audit history. One atomic transaction ends the demo entitlement, creates the confirmed subscription-backed paid entitlement, relinks branch entitlement rows, changes the workspace classification to Customer Paid, and moves the existing tenant link from demo-request evidence to the confirmed subscription contract.

Conversion Requested, Processing, Completed, and Failed states plus audit/internal-notification events are durable. Failure rolls back workspace mutations while preserving confirmed provider payment records for retry. Completed workspaces no longer enter demo reminder or expiration processing. Expired, ambiguous, cross-tenant, internal-sandbox, already-paid, or incoherent workspaces fail closed.

2026-07-23 - M8B-5 Standard Customer Demo Lifecycle

Customer demos now run for exactly seven days from successful workspace activation. The centralized resolver derives Day 6 and Day 7 boundaries in UTC, presents them in the organization timezone, and fails closed on inconsistent lifecycle evidence.

The independently callable processor is dry-run by default. Apply mode creates idempotent internal reminder notifications and atomically ends the demo entitlement and suspends the workspace at expiration. Operational middleware rechecks customer-demo access on every protected request, including existing sessions; web users are routed to subscription guidance and APIs return a safe 403. Workspace users, branches, and all tenant data remain preserved, and Platform Owner inspection remains available.

2026-07-23 - M8B-4 Demo Workspace Provisioning And Activation

Platform Owners can now provision an Approved customer-demo request through a separate, fail-closed action. The demo service revalidates the approval, organization, customer-demo intent, commercial snapshot, entitlement snapshot, and duplicate absence before reusing the shared operational workspace builder.

The transaction creates the customer-demo SchoolGroup, branches, academic year, owner user, permissions, account link, explicit demo entitlement, and demo-sourced tenant link, then activates the workspace and entitlement. No Paddle, payment, paid subscription, subscription contract, checkout, or email record is created. Failures roll back all workspace changes, preserve the Approved request, and retain a safe retryable outcome plus audit event.

2026-07-22 - M8B-3 Demo Request Workflow

Completed onboarding now presents Request Demo and Subscribe Now. Subscribe Now preserves the existing plan-selection and Paddle lifecycle. Request Demo validates the verified customer and complete onboarding record, stores a review-only commercial snapshot, and prevents duplicate pending requests.

Customers can view request status and withdraw a pending request. Platform Owners have a searchable/filterable review queue and may approve, reject with a mandatory reason, or cancel. Approval creates review evidence only; no SchoolGroup, entitlement, checkout, payment, provisioning, activation, or email is created. Durable audit and internal-notification events record every transition.

2026-07-16 - M7 Phase 2 Read-Only Subscription Management Portal

Verified SaaS customers now have a read-only Subscription Management page at `/saas/subscription`. The page presents the confirmed plan, safe subscription health, billing interval, next billing date, paid and active branch quantities, remaining capacity, grouped feature entitlements, and a compact plan comparison. All commercial data is supplied by `saas.entitlement_service`; the portal does not query Paddle or independently calculate subscription state or branch capacity.

Upgrade, branch-addition, billing-history, invoice, pending-change, and subscription-management controls are visible only as disabled Coming Soon options. This phase adds no subscription mutations, Paddle calls, proration, refunds, cancellation, or plan and quantity editing.

2026-07-16 - M7 Phase 1 Subscription Entitlement Foundation

Commercial access now resolves through `saas.entitlement_service` from the provisioned SchoolGroup's tenant link, paid operational contract, and one confirmed active Paddle subscription. Onboarding selections, checkout quotes, pending payment attempts, and page values are not entitlement sources. Missing, mismatched, or ambiguous subscription relationships fail closed as `manual_review`.

The initial normalized matrix authoritatively maps only existing plan metadata: Enterprise AI receives `module.ai`; Professional and Enterprise AI receive `feature.advanced_reporting`; Starter does not receive either. Paid active-branch capacity is derived only from `PaymentSubscription.quantity`. Teacher management, branch management, observations, hiring, core reporting, general exports, audit logs, and cross-branch reporting remain `owner_approval_required` until commercial rules are approved.

Pilot enforcement is limited to allocation-plan PDF/XLSX exports. Both the existing `reports.export` user permission and `feature.advanced_reporting` subscription entitlement must succeed. Platform Owner and Developer

identities do not bypass subscription entitlements and must operate in a selected tenant scope.

Upgrades, downgrades, proration, refunds, Paddle subscription changes, branch-specific plans, and customer subscription-management UI remain later M7 work.

2026-07-26 - Test Workspace Reset Subscription-Change Dependency

The Platform Owner-only test workspace/account reset now deletes `subscription_change_requests` scoped by the selected operational `school_group_id` before deleting that workspace's users. It deletes selected entitlement values and branch-entitlement children before the selected workspace entitlement and final `SchoolGroup`. These narrow ordering rules prevent foreign-key deletion failures without changing lifecycle rules, customer behavior, or the preservation of records from other workspaces. The reset remains one transaction with the existing validation blocks, rollback behavior, pre-analysis counts, and structured diagnostics.

2026-07-14 - M6 Phase 3 Abandoned Draft Cleanup

Automatic cleanup applies only to unpaid, unprovisioned SaaS drafts. A draft becomes eligible after the globally configured inactivity period (30 days by default) and only after its final reminder was sent successfully for the current activity cycle. Any later meaningful activity restarts the lifecycle and prevents deletion.

Before deleting, the processor locks and rechecks each account and pending organization in its own transaction. Payment success or processing, subscription evidence, provisioning or tenant links, operational identities, protected accounts, shared ownership, and unresolved provider relationships block cleanup. Ambiguous records are retained for manual review. Successful payment evidence, Paddle webhooks and remote Paddle records, global plans/prices, reference data, and unrelated accounts or tenants are always preserved.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --batch-size 100
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --batch-size 100
PYTHONPATH=. python scripts/process_abandoned_draft_cleanup.py --dry-run --account-email draft@example.com
```

`--max-inactivity-days` is available only for local testing and is rejected in production-like environments. For Render, configure a daily Cron Job using the deployed service environment and `DATABASE_URL`; dry-run should be used before enabling the live command. Each account commits independently, failures roll back completely, concurrent workers skip locked rows, and durable external audit events record deleted, skipped, manual-review, recovery, and rolled-back outcomes.

Platform Owner lifecycle analytics remain future M6 scope.

2026-07-14 - M6 Phase 2 Draft Onboarding Reminder Engine

Draft retention remains inactivity-based. The reminder engine sends at most one first, second, and final reminder per activity cycle using the globally configured lifecycle thresholds (defaults: 24 hours, 7 days, and 25 days). The final reminder shows the deletion-eligibility date derived from the configured retention period (default: 30 days). Meaningful customer activity continues to flow through `draft_lifecycle_service.record_meaningful_activity(...)`, which starts a new reminder cycle.

Run the bounded processor from the repository root:

```
PYTHONPATH=. python scripts/process_draft_reminders.py --batch-size 100
PYTHONPATH=. python scripts/process_draft_reminders.py --dry-run
PYTHONPATH=. python scripts/process_draft_reminders.py --stage final
```

For Render, use a Cron Job with the service's `DATABASE_URL`, `RESEND_API_KEY`, `EMAIL_FROM`, `EMAIL_REPLY_TO`, and `TIS_PUBLIC_BASE_URL`. `TIS_SUPPORT_EMAIL` is optional and falls back to `EMAIL_REPLY_TO` in reminder content. An hourly schedule is recommended. PostgreSQL row locking prevents overlapping workers from sending the

same reminder stage.

Automatic draft deletion and Platform Owner lifecycle analytics are not enabled in Phase 2.

2026-06-27 - Subscription And Workspace Activation Guided Journey Phase 3C

Phase 3C subscription, payment, and activation page redesign is accepted.

What changed:

- 1 Subscription Selection, Secure Payment summary, Payment Return, Payment Cancel, Subscription Status, and Workspace Activation status pages now use the Phase 3A shared shell and Phase 3B guided style.
- 1 Each page now has one shared-shell primary CTA and keeps secondary actions visually secondary.
- 1 Secure Payment pages now clearly explain that browser return from checkout does not itself confirm payment.
- 1 Subscription and activation status pages now use concise customer-safe cards for payment, subscription, activation, and TIS Platform access state.
- 1 Customer-facing pages now consistently explain that TIS Platform access becomes available after Workspace Activation.

Scope notes:

- 1 Payment behavior, billing behavior, provisioning behavior, webhook logic, checkout start/launch behavior, stored statuses, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal /saas route names, and admin views were not changed.

Related files:

- 1 `saas/router.py`
- 1 `templates/saas/plan_selection.html`
- 1 `templates/saas/checkout_summary.html`
- 1 `templates/saas/checkout_return.html`
- 1 `templates/saas/checkout_cancel.html`
- 1 `templates/saas/account_billing.html`
- 1 `templates/saas/billing_status.html`
- 1 `tests/test_saas_phase1.py`

2026-06-27 - School Workspace Setup Guided Wizard Phase 3B

Phase 3B School Workspace Setup onboarding page redesign is accepted.

What changed:

- 1 Organization Profile, Branch Setup, Academic Setup, Primary Contact, and Review School Workspace Setup now use a consistent guided wizard structure on top of the Phase 3A shared shell.
- 1 Each onboarding page now has one shared-shell primary CTA and keeps Back/Save Draft actions visually secondary.
- 1 Organization Profile groups identity, logo upload, program/location, and estimated scale fields.
- 1 Branch Setup uses compact branch panels instead of heavy repeated blank blocks.
- 1 Academic Setup and Primary Contact use focused single-step sections with concise guidance.

- 1 Review School Workspace Setup now presents a clearer ready-to-continue summary before Subscription Selection.

Scope notes:

- 1 Form actions, field names, routes, validation behavior, draft behavior, onboarding progression, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, OAuth behavior, internal /saas route names, and admin views were not changed.
- 1 Subscription/payment/status pages remain future Phase 3 work.

Related files:

- 1 `saas/router.py`
- 1 `templates/saas/base.html`
- 1 `templates/saas/onboarding_organization.html`
- 1 `templates/saas/onboarding_branches.html`
- 1 `templates/saas/onboarding_academic_setup.html`
- 1 `templates/saas/onboarding_contacts.html`
- 1 `templates/saas/onboarding_review.html`
- 1 `tests/test_saas_phase1.py`

2026-06-27 - TIS Account Guided Setup Framework Phase 3A

Phase 3A shared guided setup framework is accepted.

What changed:

- 1 The shared customer account shell now supports a guided setup console when setup context is provided.
- 1 The TIS Account page now uses an 8-step customer journey: TIS Account, Email Verification, School Workspace Setup, Review & Confirmation, Subscription Selection, Secure Payment, Workspace Activation, and Enter TIS Platform.
- 1 The account page now focuses on the current step, one primary next action, concise account/workspace context, and guidance that TIS Platform access becomes available after Workspace Activation.
- 1 The old customer account dashboard statistics and session detail blocks were removed from the account landing page.
- 1 Journey state is calculated from existing account, onboarding, billing, payment, and activation data without changing stored statuses.

Scope notes:

- 1 Onboarding forms, subscription/payment pages, billing/status pages, payment behavior, billing behavior, provisioning behavior, database schema, migrations, operational modules, the Next.js landing website, internal /saas route names, admin views, and Google/Microsoft login were not changed.
- 1 This phase prepares the shared framework for later Phase 3 onboarding and payment page redesign work.

Related files:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/base.html`

1 templates/saas/account.html
1 tests/test_saas_phase1.py

2026-06-27 - TIS Account Customer-Facing Wording And Logo Cleanup

Phase 2 customer-facing wording cleanup is accepted.

What changed:

- 1 Customer account and school workspace setup pages now use professional labels such as "TIS Account", "Account Dashboard", "School Workspace Setup", "Organization Profile", "Branch Setup", "Academic Setup", "Subscription Setup", "Secure Payment", and "Workspace Activation".
- 1 Customer views use customer-safe display labels for internal onboarding, billing, payment, and activation statuses instead of exposing raw tenant/provisioning/checkout status values.
- 1 Customer-facing billing and subscription views hide provider transaction/subscription IDs, attempt UUIDs, checkout session internals, plan IDs, and school group IDs.
- 1 The shared customer account shell now includes the official full-color horizontal TIS logo image so inherited customer account/setup pages carry official branding.
- 1 TIS Account transactional emails use an existing official dark-blue TIS wordmark asset.
- 1 Activation email copy now uses "School Workspace", "Workspace Activation", and "TIS Account" language.

Scope notes:

- 1 Internal /saas route/module/model names and stored statuses were not renamed.
- 1 Payment, billing, provisioning behavior, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.
- 1 Phase 3 account setup UI redesign was not implemented.

Related files:

1 saas/router.py
1 saas/service.py
1 saas/provisioning_service.py
1 email_templates.py
1 templates/saas/
1 tests/test_saas_phase1.py
1 tests/test_saas_phase5.py
1 tests/test_email_templates.py

2026-06-27 - TIS Account Email Verification Recovery And Setup Gate

Phase 1 TIS Account email verification recovery is accepted.

What changed:

- 1 Valid email verification links now mark the SaaS account email verified/active and redirect to TIS Account login with a professional success notice.

- 1 Expired or invalid verification links now show a recovery page with a resend verification option.
- 1 Resend verification supports unverified accounts, already verified accounts, and unknown-email cases with safe customer-facing messaging that does not reveal account existence.
- 1 Password-based accounts that remain unverified are blocked from starting or continuing school workspace setup.
- 1 New visible wording in this verification flow uses "TIS Account" and "school workspace setup".

Scope notes:

- 1 Payment, billing, provisioning, database schema, migrations, operational modules, and the Next.js landing website were not changed.
- 1 Google/Microsoft login remains future work and was not implemented.
- 1 Phase 2 customer-facing wording cleanup and Phase 3 account setup UI redesign were not implemented as part of this change.

Related files:

- 1 `saas/router.py`
- 1 `saas/service.py`
- 1 `templates/saas/`
- 1 `docs/adr/0002-separate-saas-identity-and-operational-users.md`

Subscription History

docs/history/subscriptions/README.md

2026-08-19 - Capacity-Based Packaging

ADR 0025 makes the normal customer feature matrix identical for Starter, Professional, and Enterprise AI while preserving 1/5/25, 5/20/100, and 25/100/500 organization capacity. The source-aware feature resolver accepts coherent active paid, promo, and demo authority only after permission and scope. Active promo operational feature snapshots are reconciled without modifying exact grant capacity, immutable promo evidence, Paddle, or expired authority. Priority support remains a separate service-level distinction.

2026-08-06 - Existing Workspace Paid Activation Through Paddle

Activation-required existing customer workspaces can now choose Professional or Enterprise AI from Organization Account, confirm a SchoolGroup billing identity, review an operational-branch quote, and launch Paddle without creating onboarding or tenant records. A dedicated activation aggregate preserves quote, branch, checkout, payment, contract, and provider lineage. Starter remains unavailable pending complete restricted-branch enforcement.

Only verified `transaction.completed` evidence creates paid commercial authority. The atomic transition preserves the existing SchoolGroup and operational data, keeps classification customer, activates the workspace lifecycle, and creates the paid subscription, workspace/branch entitlements, and paid tenant link. Promo redemption, demo conversion, and new PendingOrganization checkout remain unchanged.

Organization Account **Choose a Plan** reopens plan selection while the activation is draft or checkout_ready. The saved plan is highlighted but may be replaced by another eligible plan, which recalculates the authoritative branch quote without a Paddle call, PaymentAttempt, or branch mutation. Once checkout starts, payment is processing, or activation evidence requires manual review, replacement fails closed. Only verified payment completion can establish paid commercial authority.

Final PostgreSQL validation proved migration rollback/idempotency, one unresolved activation, one transaction launch, duplicate and out-of-order webhook safety, atomic drift rollback, and exactly one winner in a paid-versus-promo source race. Workspace-specific provider address/business lineage and complete transaction validation prevent stale customer defaults or incomplete provider responses from becoming activation authority.

Organization Account derives its activation badge from the current payment attempt, not from `workspace_lifecycle_status=provisioning`. No attempt means Activation required; a current unexpired checkout-started or processing attempt means Payment processing; failed, cancelled, and expired attempts use recovery presentation.

2026-08-05 - Secure Promo Definition Foundation

M2 adds Platform-managed Starter, Professional, and Enterprise AI promo definitions with exact positive capacity bounded by the established catalog. Controlled scopes, one expiry policy, validity, redemption-policy metadata, branch snapshots, Draft/Active/Paused/Revoked lifecycle, duplication, replacement linking, row locking, and durable redacted audit are available.

Raw codes are TIS-generated, use at least 100 bits of secure randomness, appear once under `Cache-Control: no-store`, and are never persisted or audited. The database stores dedicated-secret HMAC-SHA256 lookup authority plus a safe mask. Only Platform Owners activate or revoke; authorized Developers may manage non-terminal definitions. Promo redemption, tenant grants, M1 authority, Paddle, payment, onboarding, provisioning, and current subscription behavior remain unchanged.

2026-08-04 - Unified Operational Commercial Capacity Authority

One facade now resolves commercial access, capacity source, confirmed plan and quantity, current branch/staff/teacher usage, remaining capacity, violations, minimum eligible plan or Custom, and safe recovery. It composes the existing M7/M8 authorities and fails closed on missing, ambiguous, or contradictory relationships. Demo and Internal Sandbox are explicitly unmetered in M1.

Staff capacity now correctly counts every distinct active tenant operational User in the workspace regardless of role, title, position, or internal-test attribution. An operational owner counts. A teacher-position User counts as staff, and an associated active Teacher record separately counts as a teacher. Platform users, inactive users, other tenants, and account-only SaaS identities do not count. Known teacher IDs deduplicate across branches and academic data; blank legacy identities never merge.

Capacity-increasing writes lock the SchoolGroup and perform permission/scope validation, authority resolution, recount, proposed-final-state evaluation, mutation, and commit as one transaction. This covers branch creation and reactivation, bulk branch status changes, user growth/reactivation, teacher creation/year-copy, academic-year activation, and paid/demo provisioning final validation. Existing over-capacity tenants preserve records and access but may not further increase an exceeded dimension.

2026-07-29 - Per-Branch System-User And Teacher Capacity

Plan eligibility combines persisted `max_branches`, `max_system_users`, and `max_teachers`. Starter supports 1/5/25, Professional 5/20/100, and Enterprise AI 25/100/500. Required non-negative estimates are stored per onboarding branch and summed organization-wide. Legacy organization totals are assigned to the primary branch when no branch estimates exist.

Before payment, system-user and teacher authority separately uses the greater of estimated and actual same-workspace counts; paid operations use actual counts. This milestone originally excluded teacher-position and internal-test login records from system-user usage. M1 supersedes that operational rule: every active tenant operational User now consumes staff capacity, while active Teacher records separately consume teacher capacity. Inactive, platform, account-only, other-tenant, inactive-branch, and inactive-year data remains excluded as applicable.

The lowest eligible plan is recommended. The shared decision protects selection through payment reconciliation and includes both people counts in quote fingerprints. Estimate changes clear only an undersized plan and supersede stale checkout lineage. Paid system-user and teacher growth plus downgrades fail before mutation when capacity would be exceeded. The always-visible Custom card never creates Paddle checkout and is emphasized above any Enterprise AI maximum.

The current Teacher model has no inactive/reactivation state and no teacher import endpoint exists. Current post-activation enforcement covers individual teacher creation and preflights academic-year copying atomically; future import or reactivation workflows must adopt the same capacity boundary.

2026-07-28 - Authoritative Plan Branch-Capacity Enforcement

Initial self-service selection now enforces the persisted plan limits: Starter one active billable branch, Professional five, and Enterprise AI twenty-five. All higher plans remain eligible below their capacity, while organizations above twenty-five receive the custom-plan contact path. Actual active branches still determine the per-branch total.

The same fail-closed capacity decision protects selection, quote generation, checkout preparation and launch, and payment validation. If branch editing makes the selected plan undersized before payment, its selection and checkout lineage are superseded; an eligible higher plan remains selected. Tenant-scoped branch counting prevents another organization from affecting eligibility.

2026-07-28 - Pre-Payment Branch Editing And Quote Supersession

Onboarding branches remain editable until confirmed payment. The existence of an unpaid demo SchoolGroup, tenant link, Paddle customer, checkout session, or billed-but-unpaid attempt is not paid provisioning evidence. Once payment

is confirmed, branch changes move to Subscription Management.

Pre-payment branch changes invalidate the old quote and checkout lineage, supersede incomplete attempts, and force a new immutable transaction using the new active branch count. Late events for the old transaction cannot activate, convert, or reprovision the workspace.

2026-07-28 - Authoritative Fixed Branch Quantity In Paddle Checkout

Initial checkout now keeps the TIS-calculated billable branch quantity fixed through payment. The server creates an automatically collected Paddle transaction with customer, address, catalog price, and exact quantity; verifies its item, subtotal, and quote-fingerprint evidence; then marks the ready transaction billed before releasing its transaction ID to Paddle.js.

Quantities of one, two, or more remain data-driven. Branch changes continue only through TIS branch and subscription-quantity workflows. No shared catalog price is mutated and no quantity-specific price is created. Automatic payment completion remains the recurring-subscription, webhook, and conversion authority.

The public launcher accepts only the unique locally attributable transaction after Paddle confirms billed and automatic collection. It rejects draft, ready, canceled, past-due, unrelated, and mismatched transactions, while retry clicks reuse the existing started checkout rather than creating another transaction.

2026-08-18 - Promo-Aware Commercial Access Portal

Organization Account Billing & Subscription now resolves the selected workspace's authoritative commercial source before choosing a customer view. Promo-backed workspaces render a dedicated read-only Commercial Access page composed from M1 capacity authority and attributable immutable promo metadata. It shows the plan, safe access status and period, masked reference, and authoritative capacity without calling Paddle or exposing recurring billing, invoice, plan-change, quantity-change, or cancellation controls. Paid and demo portals are unchanged. Promo-to-paid conversion remains deferred.

2026-08-18 - Promo Expiry Recovery And Paid Continuation

Promo access is dynamically active only before `PromoGrant.effective_to`. At expiry, operations fail closed even when persisted entitlement rows still say active. `grace_period_days` identifies a recovery interval for customer presentation and paid continuation only; it never extends operational access or mutates tenant data.

Expired and recovery-period promo owners can prepare the existing-workspace paid activation quote and launch Paddle checkout without replacing the SchoolGroup. Promo remains the sole tenant source until verified `transaction.completed` evidence. Completion locks and revalidates current authority, then atomically ends promo entitlement, relinks the existing tenant source to the paid contract, establishes paid workspace/branch entitlements, and records immutable conversion history. Failed, abandoned, or conflicting attempts retain promo authority and no paid entitlement. Active promo conversion is deliberately deferred to avoid treating unused promo value as paid credit.

A shared authority-driven badge component now renders Promo, Demo, and Paid identity with source icon, lifecycle status, and plan treatment in compact and full modes.

2026-08-05 - Promo Commercial Grants

Approved promo definitions can now become non-Paddle commercial authority only through verified owner activation. The transaction creates immutable redemption/grant snapshots, promo workspace entitlement, explicit active or inactive branch entitlement, and a promo-sourced tenant link. It supports both completed onboarding and separately aligned existing tenants without creating fake payment, subscription, contract, demo, or pending-organization records.

Promo capacity is not a Paddle quantity. Branch selection determines covered operational branches; organization-wide staff and teacher overage blocks activation without mutating people records. Pending sessions grant no access, expiry fails closed, and paid subscription history and provider behavior are unchanged.

2026-08-17 - Explicit Promo Branch Evidence And Reconciliation

Promo activation now separates selectable active branches from the complete preserved workspace branch inventory. Selected branches receive one assignment and active entitlement; every unselected branch receives an inactive entitlement even when operationally inactive. Branch creation and individual or bulk reactivation consume unused promo capacity and update assignment, entitlement, and operational status atomically. A dry-run-first PostgreSQL reconciliation command repairs only deterministically missing inactive evidence and leaves contradictory states for manual review.

2026-07-28 - Expired-Demo Checkout Identity Repair

Paddle sandbox recovery now recognizes the existing active tenant owner as authoritative when the authenticated account, owned organization, demo source, tenant link, SchoolGroup, operational user, and email resolve uniquely. A stale account-user link from a deleted or inactive account is safely reassigned to that account/workspace relationship. Unrelated tenants, active former accounts, multiple identities, and live email-only recovery remain blocked. Failed checkout preparation exposes no internal diagnostics and shows a retry state; the same workspace UUID, SchoolGroup, and branches remain unchanged.

2026-07-28 - Expired Demo Subscription Continuity

Expired demo subscription selection now reads the preserved SchoolGroup and its active operational branches, presents real public plans and billing intervals, and continues through existing Paddle checkout. Confirmed payment still converts the same workspace; missing configuration produces safe support guidance rather than unavailable placeholders.

2026-07-20 - M7 Phase 5 Lifecycle Policy, Cancellation, And Reversal

The Subscription Management portal now resolves customer-visible state and allowed actions through `saas.subscription_lifecycle_service`. Upgrade, downgrade, branch increase/reduction, cancellation, and cancellation reversal controls are exposed only when authorization, provider subscription state, pending requests, effective dates, and local relationships permit them.

Authorized billing administrators can schedule cancellation at period end and reverse it before the effective boundary. Current paid access remains active until authoritative provider evidence confirms the end of the paid period. Provider conflicts, unknown outcomes, and local/provider mismatches fail closed to manual review.

2026-07-18 - M7 Payment Lifecycle And Reconciliation Protections

Initial checkout and post-activation subscription-change transactions are resolved as distinct payment lifecycles. Webhook processing remains signature-verified and idempotent, validates attributable provider subscription/transaction evidence, and prevents change transactions from replaying initial provisioning transitions.

Diagnostics and the sandbox-guarded finalized-lifecycle reconciliation script expose sanitized evidence and default to dry-run behavior. Reconciliation applies only when stored completed webhook evidence and authoritative Paddle transaction data agree; unexpected or ambiguous state is blocked rather than guessed.

2026-07-17 - M7 Phase 4 Plan Upgrades And Scheduled Downgrades

Authorized billing administrators can preview and submit plan transitions from `/saas/subscription`. Paddle supplies monetary previews and proration outcomes. Upgrades use immediate provider proration with payment-failure prevention and do not become local entitlement truth before verified completion. Downgrades are scheduled for the next billing period and preserve the current plan until provider/webhook confirmation at the effective boundary.

Customers can cancel or replace an eligible scheduled plan change before it becomes effective. TIS sends complete retained recurring items and enters manual review on provider/local ambiguity.

2026-07-20 - M7 Phase 6 Billing History and Invoice Management

The customer Subscription Management page now retrieves billing history directly from Paddle's paginated GET `/transactions` API, scoped to the customer's confirmed provider subscription. TIS displays provider transaction totals, statuses, origins, invoice numbers, and returned credit/refund adjustments without creating local financial records or calculating replacement amounts.

Eligible invoice downloads use Paddle's GET `/transactions/{transaction_id}/invoice` API. TIS reauthorizes the current billing user, resolves the invoice against the confirmed customer subscription, and requests a fresh expiring provider URL at download time. The provider URL is not stored locally. No billing-history cache, schema change, migration, or webhook behavior is introduced; temporary provider failures render customer-safe retry states.

2026-07-16 - M7 Phase 3 Active Branch Quantity Management

Authorized organization billing administrators can now preview and submit paid branch-quantity changes from `/saas/subscription`. Paddle remains the sole source of financial calculations: TIS sends the complete retained subscription item list to Paddle's subscription-update preview endpoint and stores only customer-safe charge, credit, net, recurring-total, and effective-date summaries.

Increases use `prorated_immediately` with `on_payment_failure=prevent_change`. Local paid capacity does not increase until `verified_subscription.updated` and successful `transaction.completed` evidence confirms the requested quantity and subscription-update payment. Reductions use `prorated_next_billing_period`, issue no immediate refund, and remain locally scheduled until a renewal-boundary subscription webhook confirms the reduced quantity. Reductions below active operational branch usage are rejected, and scheduled reductions can be restored before their effective date.

`PaymentSubscription.quantity` remains the only entitlement-capacity authority. Branch creation and individual or bulk reactivation now fail closed for provisioned SaaS tenants when confirmed paid capacity is exhausted. Provider mismatches, unsupported state, stale previews, or ambiguous ownership enter a customer-safe blocked/manual-review path without exposing provider diagnostics.

This folder tracks meaningful changes to TIS subscription plans, pricing, billing status, payment behavior, checkout assumptions, and provider boundaries.

Related docs:

- 1 docs/adr/0003-paddle-payment-architecture.md
- 1 docs/adr/0004-webhook-only-payment-confirmation.md
- 1 docs/TIS_MASTER_CONTEXT.md

2026-07-11 - Paddle Transaction Payment Launcher

Paddle transaction checkout now uses a dedicated public SaaS payment launcher page at `/saas/payment` instead of the app root or operational login page. Server-side checkout still creates Paddle transactions through the existing payment service and still redirects to Paddle's returned `transaction.checkout.url`; `PADDLE_CHECKOUT_BASE_URL` should point to `https://app.tisplatform.com/saas/payment` so Paddle appends `_ptxn` to the launcher page.

The launcher page loads Paddle.js from the official Paddle CDN, initializes Paddle with the public `PADDLE_CLIENT_TOKEN`, uses `PADDLE_ENVIRONMENT` for sandbox/live mode, reads `_ptxn`, and opens checkout for the transaction. It does not require SaaS or operational login, does not expose `PADDLE_API_KEY`, and does not change webhook-confirmed payment state, subscription activation, provisioning, pricing, or billing transitions.

2026-06-30 - Paddle Initial Checkout Price Mapping Configuration

Initial Paddle checkout now has a script-based configuration path for mapping TIS subscription plan prices to Paddle provider price IDs. The runtime source of truth remains `subscription_plan_prices.provider_price_id`; Paddle credentials and endpoints remain environment variables.

Added configuration support:

```
1    scripts/sync_paddle_price_ids.py
1    config/paddle/paddle_prices.sandbox.example.json
1    config/paddle/paddle_prices.production.example.json
```

The sync script validates the six required plan/interval mappings: Starter monthly, Starter annual, Professional monthly, Professional annual, Enterprise AI monthly, and Enterprise AI annual. Real local, sandbox, and production mapping files are ignored by Git so sandbox and live Paddle price IDs remain separated.

If a selected plan price still lacks a Paddle provider price ID, checkout remains fail-closed before calling Paddle. Customers see a support-oriented Secure Payment message while internal diagnostics retain plan code, billing interval, and currency context.

Smart Timetable Stage 2 Version Foundation

[docs/history/workforce-planning/2026-08-19-smart-timetable-stage-2-version-foundation.md](#)

Stage 2 converted timetable storage from one mutable live set into a durable versioned aggregate without adding automatic generation. It added input snapshots, timetable versions, one exact-scope active pointer, future generation-run records, version-owned placements, lock metadata, and version-scoped collision constraints.

Existing populated branch/year timetables are migrated exactly into one imported compatibility version and made operational through an active pointer. Deterministically inconsistent rows remain unchanged and produce safe stale evidence. Empty settings-only scopes are not versioned.

The current assignment experience remains available through a copy-on-write bridge. The first edit of an active imported timetable creates a working draft; later legacy edits reuse that draft. Active history remains immutable, while page views and XLSX/PDF exports resolve the operational version.

No solver, worker, generation endpoint, generation UI, room/resource model, availability model, or new timetable capacity authority was added.

Smart Timetable Stage 3 Readiness And Slot Semantics

[docs/history/workforce-planning/2026-08-19-smart-timetable-stage-3-readiness-and-slot-semantics.md](#)

Stage 3 introduced one deterministic projection from existing timetable settings. Teaching periods stay fixed. Full-period/all-day blocks disable covered slots, every-day rules expand across configured days, between-period blocks consume no lesson slot, and partial overlaps are explicit configuration errors.

The read-only `TimetableReadinessService` evaluates one selected organization, branch, and academic year against configuration, Planning demand/allocation, HRT, authoritative teacher capacity, slot sanity, locks, staleness, and active runs. Only `generation_ready` is ready, and it does not guarantee solver feasibility.

The page shows readiness and corrective links. Unavailable slots cannot be assigned through either the route or mutation service; existing placements are preserved as stale. Versions and exports remain compatible. No schema, solver, worker, generation endpoint, availability, rooms/resources, or Stage 4 publication UI was introduced.

Smart Timetable Stage 3.5 Composed Timeline

[docs/history/workforce-planning/2026-08-21-smart-timetable-stage-35-composed-timeline.md](#)

Stage 3.5 corrected the configuration engine so teaching clocks are no longer calculated independently of non-teaching blocks. One canonical service now composes each working day from shift start, teaching-period count and duration, and applicable blocks. After-period blocks insert at a selected teaching boundary and shift every later period. Fixed-time rows remain compatible only when their stored start is a live boundary; ambiguous rows remain unchanged and produce an explicit blocker.

The calculated end, configuration preview, main timetable bands, readiness, assignment validation, snapshot fingerprints, future solver-ready teaching slots, and XLSX/PDF rows consume the same projection. The migration adds placement mode, after-period boundary, and duration columns idempotently and defaults legacy rows to fixed time. Controlled block types were expanded without removing existing keys.

This stage added no solver, OR-Tools package, Generate/Regenerate endpoint, availability, room/resource, or soft-constraint configuration.

Smart Timetable Stage 4 Version Publication

[docs/history/workforce-planning/2026-08-21-smart-timetable-stage-4-version-publication.md](#)

Stage 4 made the Stage 2 lifecycle visible without adding automatic generation. The timetable page identifies the selected version, lifecycle, origin, freshness, manual changes, and active state; lists same-scope history; and permits explicit historical review without changing the active timetable.

Immutable history can be copied to a new draft. Draft lessons can be locked or unlocked under a dedicated permission, and lock changes refresh snapshot authority. Version-specific solver-independent validation checks current authority, demand completion, Planning teachers, canonical slots, collisions, stale placements, and locks. Generation readiness remains a separate scope-input evaluation.

Publishing a fresh validated draft locks and revalidates the version and revisioned active pointer, preserves the previous active version as superseded history, records the actor/time, and makes the published version immutable. Same-scope comparison, selected-version XLSX/PDF export, and safe draft/superseded archive behavior are also available. Placement permissions are aligned separately for create, replace, and clear. No schema, solver, worker, generation endpoint, availability, rooms/resources, commit, push, deployment, or production action was introduced.

Smart Timetable Stage 5.1 Generator

[docs/history/workforce-planning/2026-08-22-smart-timetable-stage-51-generator.md](#)

Stage 5.1 added real automatic timetable generation and regeneration. Google OR-Tools CP-SAT 9.15.6755 is isolated to `requirements-worker.txt` and the dedicated `python -m timetable_generation_worker` process. Web requests capture immutable schema-v3 inputs and use the existing PostgreSQL `generation-run` table as a durable queue with claim locking, leases, heartbeat, bounded recovery, cancellation, progress, idempotency, and one active run per SchoolGroup/Branch/Academic Year.

The problem contract preserves exact section-subject-teacher weekly demand, Planning and subject-specific HRT authority, composed Stage 3.5 teaching slots, locks, fingerprints, and regeneration source state. CP-SAT enforces exact demand, section/teacher exclusivity, locks, and explicit diversity. An OR-Tools-independent validator and a final current-input/source-revision comparison gate persistence.

Successful generation atomically creates a separate unpublished publication-ready version and all placements while leaving `TimetableActiveVersion` and published history unchanged. `Generate/Regenerate` actions, real phase polling, safe terminal messages, and `timetable.generate` are exposed now. Stage 5.2 simplification, `Delete Working Timetable`, `Timetable History simplification`, `teacher My Timetable`, and published-only teacher visibility remain deferred. No production action occurred.

On 2026-08-26, the production execution host changed to an on-demand Render Workflow task. The solver pipeline and all durable safety described here remain unchanged; `python -m timetable_generation_worker` is now only an optional local fallback and `requirements-workflow.txt` is the production task dependency set.

On-Demand Timetable Generation Workflow

<docs/history/workforce-planning/2026-08-26-on-demand-timetable-generation-workflow.md>

Production timetable generation now provisions compute per durable run through a Render Workflow task. Generate and Regenerate first commit the existing `TimetableGenerationRun` and immutable snapshot. The server then starts the configured task with only the run public ID; PostgreSQL remains authoritative for input, scope, progress, cancellation, failure, and result.

The focused `timetable_generation_workflow.py` task claims exactly that queued run and invokes the existing single-run executor. Existing leases and heartbeats reject duplicate or late executors, and the atomic persistence service still rechecks staleness and creates at most one unpublished candidate without changing the active pointer. The solver, validator, regeneration diversity, locks, and Stage 5.2 publication/deletion/visibility rules are unchanged.

The web service uses `render==1.0.1` without OR-Tools. The Workflow runtime installs `requirements-workflow.txt`, which adds pinned OR-Tools. Render task retries are set to zero; deterministic and infrastructure outcomes become durable TIS terminal state, and the user may Generate Again. Immediate dispatch failure ends an unclaimed run safely. Cooperative database cancellation is retained; calling Render cancellation is not required for this MVP.

Future Render setup is manual because Workflows are not currently represented by Blueprints: configure the same repository/revision, `DATABASE_URL`, the TIS solver settings, and task entry `timetable_generation_workflow:app`. Configure the web with the server-only `RENDER_API_KEY` and `TIS_TIMETABLE_WORKFLOW_TASK_SLUG`. No Render resource or production database was changed by this work.

Smart Timetable Stage 5.2 Simplified Workflow And Published Visibility

<docs/history/workforce-planning/2026-08-26-smart-timetable-stage-52-simplified-workflow.md>

Stage 5.2 simplified the school workflow to Configure, Generate Draft, Review/Edit Draft, and Publish to Users while preserving all Stage 2-5.1 version and solver internals. Customer language replaces technical lifecycle state on the main page; version selection, comparison, exports, and archive remain in History.

The 2026-08-28 academic-quality extension keeps this workflow intact while adding branch/year settings for mapped core daily distribution, spread preferences, ICT, grouped Swimming activities, and regeneration diversity. The settings are captured in immutable generation snapshots and do not change approval/publication semantics.

Delete Draft Timetable permanently removes an eligible current mutable unpublished candidate under scope locks. It removes version placements, preserves snapshots, runs, audit fields, official publication, and history, and uses `timetable.delete_versions`. Historical Archive remains a separate History-only action.

Timetable History permits permanent deletion of any non-active, never-published candidate regardless of origin or internal draft/archive state. Unpublished dependent versions are removed child-first, generation source/result links are nulled where optional, shared snapshots and generation audit records remain, and protected published lineage or active generation is reported rather than silently removed. The dedicated assignable `timetable.delete_versions` permission controls this action and is included in the Administrator defaults. History also provides exact-scope Delete All Unpublished Timetables.

Authorized users may drag an unlocked lesson within a mutable Draft Timetable to a canonical teaching slot. Empty destinations produce an immediate move; occupied destinations attempt one atomic swap. Non-teaching periods, active publication, locked lessons, class collisions, and teacher collisions fail without a partial edit. Successful edits increment the existing revision and return the candidate to the normal re-check workflow. Publishing now requires explicit first-publication or replacement confirmation and presents Published to Users instead of technical active version language; publication permissions and transaction semantics are unchanged.

The Published Timetable management view offers Edit This Timetable, which copies the official timetable into an editable draft, and Create New Timetable, which creates a fresh empty draft from current Planning/configuration authority. Both preserve the official timetable and active pointer until the draft is explicitly published.

Approve Draft replaces Check Timetable in customer-facing workflow language. It validates the exact selected draft and records the administrator and approval time. Generated and regenerated results begin unapproved; placement, drag/drop, swap, lock, regeneration, or stale-authority changes clear approval. Publish requires a still-current approval and repeats validation before the existing atomic pointer swap. The normal Draft workspace groups Regenerate and Approve Draft before approval, then shows Publish Timetable after approval. Destructive, comparison, archive, and version-specific export controls remain in History. Regeneration now requires 25 percent of unlocked direct-source placements to change, rounded up, and reports a controlled failure when no valid timetable can reach that diversity.

The published-only service resolves strictly from `TimetableActiveVersion`. `/my-timetable` uses exact-scope teacher identity and exposes only that teacher's official lessons. A general published page supports other view-only users. Management drafts, history, solver details, readiness, locks, and controls are not exposed there. No schema, migration, CP-SAT constraint, deployment, or production action occurred.

Explicit Planning Subject Demand Foundation

`docs/history/workforce-planning/2026-08-28-explicit-planning-subject-demand-foundation.md`

Stage 1 adds `planning_subject_demands` as the normalized future authority for one Planning section's subject and weekly periods. Rows are branch/year scoped, retain active or retired state, and carry normal timestamps and optional actor metadata. Composite section and Subject foreign keys prevent scope drift. A partial unique index permits one active row per section and subject while preserving retired rows.

Migration `20260828_004_planning_subject_demands_foundation` backfills only Current and New Planning sections. It joins Subjects by branch, academic year, and grade, copies the current weekly hours, and inserts only when no prior section-subject demand exists so reruns neither duplicate demand nor reactivate retirement evidence.

`planning_subject_demand_service.py` resolves explicit rows before legacy grade and weekly-hours fallback. An inactive explicit row is retirement evidence and suppresses fallback. No existing consumer changes authority in Stage 1: teacher allocation, reports, readiness, timetable snapshots and generation, drafts, publication, and UX remain unchanged. No production or local `tis.db` operation occurred.

Planning Subject Demand Consumer Transition

docs/history/workforce-planning/2026-08-28-planning-subject-demand-consumers.md

On 2026-08-28, Stage 2 moved live required-period consumers from direct grade-level `Subject . weekly_hours` inference to exact-scope, explicit-first `PlanningSubjectDemand` resolution. Planning totals, teacher workload, timetable readiness/workspace/snapshot/generation input, Subject Scheduling Rule arithmetic, and required-hours reporting now honor per-section active, inactive, and zero demand.

Sections without an explicit row retain the Stage 1 legacy fallback during the transition. Teacher assignment identity, solver behavior, timetable presentation, and published timetable history were not redesigned or mutated.

Atomic Curriculum Adjustment Apply

docs/history/workforce-planning/2026-08-29-curriculum-adjustment-apply.md

Stage 4 turns an already-reviewed preview into one exact-scope transaction. Dedicated `curriculum.adjust` authorization, preview-fingerprint revalidation, row locks, active-generation rejection, explicit teacher decisions, qualification/capacity, scheduling rules, and Draft locks all gate mutation.

Successful application retires or updates source demand, makes target demand explicit, applies only confirmed assignment choices, retires an active zero-source section rule, marks the unpublished Draft stale, clears approval, and inserts one durable audit. A unique scope/fingerprint prevents duplicate application. Every affected write rolls back together on failure.

No timetable regeneration occurs. Existing snapshots and placements are preserved, and published versions plus the active pointer are never mutated.

Read-Only Curriculum Adjustment Preview

docs/history/workforce-planning/2026-08-29-curriculum-adjustment-preview.md

Stage 3 adds a non-mutating preview for late subject-demand changes after Planning, teacher allocation, and Draft Timetable data exist. It supports one grade, selected Current/New sections, or all active source uses within one exact tenant/branch/year.

The result projects source-to-target periods, current teachers, suggested but uncommitted teacher choices, load/capacity, normalized scheduling-rule arithmetic, grouped legacy warnings, and Draft stale/regeneration consequences. A deterministic fingerprint binds the analyzed authority for future stale-confirmation checks.

No demand or assignment is written, no timetable is regenerated, and published history remains untouched. Apply and atomic rollback behavior are not part of this stage.

Curriculum Transfer and Subjects Display Correction

docs/history/workforce-planning/2026-08-29-curriculum-transfer-and-subjects-display-fix.md

The curriculum adjustment contract now preserves the administrator's requested transfer count from the guided UI through preview fingerprinting and atomic apply. Each section derives source and target results from its current effective Planning demand. Non-positive transfers are rejected, requests larger than a section's source demand block preview, partial transfers keep the source active, and only a zero result retires it.

The Subjects list now reads explicit-first PlanningSubjectDemand for Current/New sections in the selected branch and academic year. Uniform grade-subject demand is shown as the effective Planning weekly-period value. When sections differ, the list shows **Varies** and exposes each section's value. The catalog Subject .weekly_hours value remains unchanged and serves only as fallback/default context.

Guided Curriculum Adjustment UI

[docs/history/workforce-planning/2026-08-29-guided-curriculum-adjustment-ui.md](#)

Stage 5 introduced the administrator-facing workflow for late-stage curriculum changes. Permissioned users can select a Current/New Planning scope, choose source and target subjects, inspect the deployed preview, make an explicit teacher decision per section, and confirm the atomic apply request.

The UI clearly separates blockers and warnings and describes Draft regeneration in customer language. If the reviewed state changes, it refreshes the preview before any apply attempt can proceed. Successful application provides navigation to Timetable and a regeneration CTA without starting generation. Backend transaction ownership, solver behavior, and published timetable history remain unchanged.

Reduce-Only Curriculum Adjustment

[docs/history/workforce-planning/2026-08-29-reduce-only-curriculum-adjustment.md](#)

The guided curriculum workflow now distinguishes period transfer from source-only reduction. Reduce-only carries an authoritative requested reduction, requires no target subject or teacher decision, and calculates each selected section from explicit-first Planning demand. Partial results stay active; zero results retire the source demand and its invalid section-level scheduling rule.

Preview and atomic apply retain scope locking, stale fingerprint checks, Draft lock validation and invalidation, rollback, audit, and immutable published history. Subject Details presents effective Current/New Planning periods and offers a `curriculum.adjust-gated`, grade-specific prefilled reduction action. Original `Subject.weekly_hours` remains catalog/default data. Multi-grade aggregation is not added because the established transaction reviews one exact subject code and subject codes are grade-specific.

Teacher Scheduling Rules

[docs/history/workforce-planning/2026-08-30-teacher-scheduling-rules.md](#)

Implemented tenant-scoped teacher timing configuration under Timetable Settings. Administrators use the dedicated `timetable.manage_teacher_rules` permission to create, edit, and remove Must teach, Unavailable, Prefer teaching, and Prefer free rules for configured days, numbered/first/last periods, and assigned-class, grade, or Current/New section targets.

Migration `20260830_001_teacher_scheduling_rules` creates normalized rule, slot, and target tables after establishing the composite teacher scope required by PostgreSQL. The tables are deferred from baseline metadata creation so the repository's production migration order remains safe and idempotent.

Schema-v4 snapshots include canonical rules in timetable constraint authority. Required rules are hard CP-SAT constraints, preferences affect only optimization, and the independent validator repeats hard checks. Readiness/problem construction reports deterministic rule, workload, availability, target, and lock conflicts. Rule changes stale unpublished Drafts and clear approval; published timetable history, Planning demand, and teacher allocation are unchanged. No configured rules retains the previous generation behavior.

The administrator form was subsequently simplified to Teacher, Rule, Days, Periods, optional Sections, and Save. It directly displays P1?Pn, uses Current/New section labels with Select All, and removes normal grade, class-scope, and first/last controls. Schedule within these periods was added as a distinct hard semantic through migration `20260830_002_teacher_scheduling_window_semantics`: existing assigned lessons must fit inside the selected window, but selected slots do not create or imply extra lessons. Existing Must-teach, first/last, grade-target, and preference rows retain their original meaning.

Final administrator polish made Select All bidirectionally synchronize the exact visible Current/New section checkboxes, restored those targets during edit, and made saved summaries display grade-section names, All assigned sections, or All classes as appropriate. Selected-section targets remain intact through snapshots, the problem builder, CP-SAT, and independent validation.

Manual Draft edits now reject hard Unavailable placements and lessons outside an applicable Schedule-within window. Missing Must-teach occupancy remains permissible while editing, but complete Draft validation detects it along with unavailable, window, and selected-section target violations. That shared hard validation blocks approval and publication; soft preferences remain non-blocking and published history is unchanged.

An infeasibility investigation then found that multiple Schedule-within records covering the same demand were each prohibiting every slot outside their own row. That intersected the rows in CP-SAT even though readiness checked them separately, allowing a zero-blocker scope to fail as infeasible. Applicable rows now compose as a per-demand union consistently in manual edits, complete-Draft validation, problem construction, CP-SAT, and independent solution validation.

Pre-solve diagnostics now use exact teacher demand-to-slot matching over combined windows and unavailable periods, counting grouped activity teacher occupancy once. They also identify provable Must-teach/window, daily-coverage, hard maximum-per-day, minimum-teaching-day, and double-block conflicts with required-versus-available guidance. No hard rule is relaxed and no Planning, allocation, lock, Draft, or published-history authority is changed.

For infeasibility that remains solver-only, the Workflow now performs bounded diagnostic solves over isolated hard-constraint families and persists the result on the existing Generation Run. The diagnostic differentiates base collisions, locks, grouped activities/resources, subject rules, teacher rules, and combined subject/teacher-rule conflicts, while recording lock and grouped-activity counts. Relaxed diagnostic candidates are never saved and the complete hard model remains the only generation authority. A six-section, forty-slot-per-section regression with 240 periods, exact double blocks, daily coverage, hard maximum/day, and scoped teacher rules proves the full-utilization model itself remains feasible.

Production showed that the initial diagnostic's fixed ten-second cap could itself time out before proving a family. Isolation now reads the independent `TIS_TIMETABLE_DIAGNOSTIC_TIMEOUT_SECONDS` Workflow setting, defaults to 60 seconds per attempted profile, and may be raised without changing the main solve timeout. Zero-lock and zero-group scopes skip those profiles, and successful results include safe section, demand, teacher, rule, period, and selected-window counts where available.

Solver-backed feasibility gate

Timetable readiness now separates configuration completeness from global feasibility. A hard-only CP-SAT Workflow verifies and independently validates one complete timetable before Generate is enabled. Its placements are retained by the exact snapshot fingerprint for reuse as the optimization hint and timeout fallback; any teacher-rule or other timetable-authority change requires a new verification.

Advanced Timetable Release 1 Reconciliation And Solver Adapter

[docs/history/workforce-planning/2026-09-01-advanced-timetable-r1-reconciliation.md](#)

Milestone 0 reconciled the governed timetable design with the implemented versioned scheduling pipeline. The existing SchoolGroup/Branch/Academic Year scope, Planning demand authority, immutable snapshots and fingerprints, version-relative placements, revision-guarded mutation/publication, exclusive generation runs, CP-SAT execution, independent validation, permissions, and Render Workflow boundary remain the implementation foundation.

The first additive Release 1 slice introduced `timetable_solver.py`. The Workflow worker now calls a solver-neutral contract for solving and bounded infeasibility diagnosis. `CpSatTimetableSolver` delegates to the existing OR-Tools model without changing the problem or result contracts. No alternate solver, scoring authority, schema, API, UI state, migration, or service was introduced.

Repository evidence confirms that Planning's explicit-first `PlanningSubjectDemand` projection already supplies timetable demand; a separate academic-demand authority must not be added. It also confirms that locks remain placement metadata, while teacher Unavailable and Schedule-within behavior belongs to normalized teacher rules. General Room/Classroom/Campus authority and generic availability remain absent and require separate approval before implementation.

Structured Timetable Conflict Evidence

[docs/history/workforce-planning/2026-09-01-structured-timetable-conflicts.md](#)

Advanced Timetable Release 1 now uses `timetable_conflicts.py` as its canonical conflict boundary. The contract records a stable machine code, legacy source code, HARD/SOFT severity, DURABLE/RECALCULABLE/TRANSIENT evidence class, safe message, optional safe entities and slots, remediation, provenance, and internal requirement, allocation, or constraint correlation.

Public conflict serialization deliberately replaces internal correlation values with presence flags. Internal Lesson Requirement hashes and solver clauses are never emitted. Same-scope entity references may reuse information already exposed by the authorized workflow; unauthorized or cross-tenant references are generic and redacted.

Readiness retains its existing blocker/warning fields and adds canonical conflicts. Feasibility retains stored diagnostics and adds durable conflict projections. Generation-run responses derive durable conflict evidence from existing terminal status, failure category, safe message, and finish time. The independent validator retains legacy errors and adds recalculable canonical conflicts without importing CP-SAT.

Infeasibility, timeout, cancellation, stale authority, independent validation failure, and solver/execution failure remain distinct. Recalculable readiness and validation evidence is not persisted. No universal conflict table, migration, Room, generic availability, Resource, co-teaching, or frontend redesign was added.

Timetable Lesson Requirement Projection

[docs/history/workforce-planning/2026-09-01-timetable-lesson-requirement-projection.md](#)

Advanced Timetable Release 1 now has one internal domain contract between Planning demand and scheduling. `timetable_requirement_projection.py` resolves the exact SchoolGroup/Branch/Academic Year, Planning section, subject, effective weekly periods, existing teacher assignment or HRT fallback, and demand source. It consumes `PlanningSubjectDemand` first and uses `Subject.weekly_hours` only when no explicit row exists. Explicit zero and retired rows remain authoritative and never reactivate fallback.

The internal requirement identity is deterministic over scope and governing Planning source. A separate source fingerprint detects effective-period, active-state, and teacher-authority changes without changing the logical identity of an unchanged explicit source. These values are internal correlation and freshness evidence, not customer-facing business identifiers.

Schema-v5 snapshots store the requirement identity, source fingerprint, demand authority, and source ID. The existing problem builder remains compatible with schema-v3/v4 snapshots, while schema-v5 fails closed if projection provenance is missing. Readiness and snapshot creation now share the projection; feasibility and generation consume it through immutable snapshots, and the independent validator checks captured provenance without depending on CP-SAT.

Grouped Swimming remains its existing specialized equal-demand/common-teacher synchronization path over projected requirements. This work does not introduce general co-teaching, a Lesson Requirement table, a CRUD API, Room, generic availability, or generic Resource authority.

Professional Timetable Lifecycle UX

docs/history/workforce-planning/2026-09-02-professional-timetable-lifecycle-ux.md

The timetable workspace now exposes the existing Release 1 lifecycle in customer language: Draft, Validate, Approve, and Publish. This is not a new persisted state. Successful validation records approval and moves the Draft to `publication_ready`; the scoped active pointer identifies the official published version.

Version/history cards show origin/source, created or generated time, validation, approval, publication, mutability, and current-publication identity. Permissioned actions create independent working Drafts from current published or immutable history, while starting an empty Draft is separate. Generate and Regenerate are explicitly Draft operations.

Stale guidance explains that older inputs are neither corruption nor proof of infeasibility. Mutation/publication responses retain their message and status behavior while adding canonical conflicts for revision races, stale validation, approval, immutability, lifecycle errors, blocked slots, counts, and collisions. The UI offers a keyboard-accessible refresh action for stale concurrency evidence.

No migration, solver/Workflow behavior, Room, generic availability, Resource, co-teaching, partial regeneration, AI, or opaque score was added.

Planning And Subject Delete Dependency Guards

docs/history/workforce-planning/2026-09-03-planning-and-subject-delete-dependency-guards.md

Deleting a Planning section or a Subject previously either raised an unhandled `IntegrityError` (Internal Server Error) once enough dependent authoritative data existed, or in the Subject case silently succeeded for reference types no existing check covered. Both routes now perform a read-only, scope-safe dependency check before any mutation and block deletion with a specific, customer-safe explanation naming every blocking category and a real next action, rather than crashing, deleting silently, or leaving the administrator with no path forward.

Planning section delete (`routers/planning.py:delete_planning_section`) checks for `TeacherSectionAssignment`, `PlanningSubjectDemand`, `TimetableEntry`, `TeacherSchedulingRuleTarget`, `CalendarEvent/CalendarEventSectionTarget`, and section-scoped `SubjectDistributionRule` references. `TeacherSectionAssignment` is no longer silently deleted alongside the section - it is now a blocker like every other category, since no documented product rule required the previous cascade; the admin removes it through the existing Edit Planning Section save flow and retries. Multiple simultaneous blockers are listed individually with the specific action that resolves each one.

Correction on 2026-09-04: an inactive, zero-period `PlanningSubjectDemand` with `updated_by_user_id IS NULL` is the setup-only suppression created by Remove Subject Requirement, not operational demand or history. It no longer blocks an otherwise-empty section. The delete route removes only those exact rows, within the selected section/branch/year and in the same transaction as the section delete. If any other dependency blocks, the suppression is not cleaned. Active demands, Curriculum-Adjustment-touched rows, and every other dependency below remain blockers; no general cascade was introduced. Customer-facing remediation uses "Remove Subject Requirement", not "Remove demand".

Planning subject demand is split into two honest cases rather than being treated as uniformly permanent. Curriculum Adjustment (`curriculum_adjustment_apply_service._set_demand`) always stamps `updated_by_user_id` on a row it creates or modifies, while the one-time setup backfill migration never sets it. A demand row with no `updated_by_user_id` has therefore never been acted on by an admin - it is pure setup scaffolding - and a new, narrowly scoped action, `GET /planning/subject-demand/delete/{demand_id}` (surfaced as "Remove demand" next to the subject on the Planning page), hard-deletes exactly that row so the section can then be deleted. A row Curriculum Adjustment has ever touched, active or retired, is genuine history TIS preserves permanently (retirement never deletes the row), so it remains a permanent blocker with an honest explanation instead of a false "remove and retry" promise. Timetable placements follow the equivalent split implicitly: only entries in a mutable Draft can be removed, while published/active/superseded/archived placements are permanent history.

Subject delete and Bulk Delete (`routers/subjects.py`) consolidate what were previously two narrower checks into one `_get_subject_delete_blockers` scan covering `Teacher.subject_code`, `TeacherSubjectAllocation`, `TeacherSectionAssignment`, `PlanningSubjectDemand` (using the same removable/permanent split), `TimetableEntry.subject_code`, `CurriculumAdjustmentAudit` source/target codes, and `SubjectDistributionRule.subject_code`. The last three had no prior application-level check at all - a Subject referenced only by timetable placements, Curriculum Adjustment history, or a distribution-rule override could previously be deleted, silently orphaning that historical reference. Bulk Delete is atomic from the administrator's perspective: if any selected Subject is blocked, nothing in the batch is deleted, and every blocked Subject is named with its specific reason so the admin can resolve it (using the same Planning-page Remove-demand action where applicable) and retry. An unforeseen dependency reaching the database commit is still caught by an `IntegrityError` rollback rather than surfacing a raw error.

No archive/closed/inactive lifecycle, broad schema change, migration, or cascade deletion was introduced. The one new capability is a narrow, permission-gated hard delete of an untouched `PlanningSubjectDemand` row - using an existing, unused-until-now column (`updated_by_user_id`) as the sole eligibility signal - so every blocker shown to an admin now has either a real removal path or an honest permanent-history explanation.

Return-To Reopen-On-Load UI Pattern

docs/history/workforce-planning/2026-09-03-return-to-reopen-on-load-pattern.md

Server-rendered actions inside an expanded Planning section (native `<details>`) previously redirected to a fresh `/planning` render with no memory of which section was open, forcing the administrator to re-expand it for every follow-up action. This adds a small, reusable fix rather than an SPA conversion, by finishing wiring together capability that already existed in the codebase separately.

`redirect_utils.py` is a new shared module holding `safe_redirect_path()`, `redirect_with_notice()`, and `redirect_with_error()` - moved out of `main.py`, which now imports them under their original private names so every existing caller (`_safe_redirect_path`, `_redirect_with_notice`, `_redirect_with_error`) keeps working unchanged. `safe_redirect_path()` is the single open-redirect guard: it accepts only a path starting with exactly one `/`, rejecting `http(s)://` and protocol-relative `//` targets, and otherwise preserves the given `?query#fragment` unchanged. The notice/error helpers additionally inject their query parameter while re-appending a trailing `#fragment` after it, since a fragment must stay after the query string.

`static/js/reopen-on-load.js`, loaded globally from `base.html`, runs on `DOMContentLoaded`. It resolves a target element id from `window.location.hash` (the redirect case) or an optional inline `window.__tisReopenTargetId` a page can render for a non-redirect, in-place response, looks it up with `document.getElementById` (which never throws, unlike a raw CSS-selector lookup), opens it if it is a `<details>`, and scrolls it into view. A missing, stale, or renamed id is a silent no-op by design.

Planning is the first and, for now, only consumer. Each section's `<details class="subject-assignment-details">` now carries a stable `id="planning-section-{{ record.id }}"`. The "Remove demand" link passes `return_to=/planning#planning-section-<id>` (URL-encoded); `routers/planning.py: delete_planning_subject_demand` reads `return_to`, validates it with `safe_redirect_path(return_to, default="/planning")`, and redirects there on every outcome that returns to the list (success, and the not-found case), falling back to plain `/planning` when `return_to` is absent or unsafe. The in-place render outcomes that stay on the current request (permanent-demand refusal, the `IntegrityError` fallback, and `delete_planning_section`'s own blocked-dependency render) instead set `open_section_id` on `_render_planning_page`, which the template turns into the inline `window.__tisReopenTargetId` the shared script also understands - the same reopening behavior without a redirect. Permission-denied and not-authenticated outcomes are unchanged and never consult `return_to`.

Subjects and Teachers were not touched. This is scoped to Planning as the first concrete use case; other pages may adopt the same `return_to + #fragment + shared-script` convention later without needing a new mechanism.

Planning Subject Requirement Removal (Single And Bulk)

docs/history/workforce-planning/2026-09-04-planning-subject-requirement-removal.md

The prior round's "Remove demand" action (GET /planning/subject-demand/ delete/{demand_id}) only ever worked for a requirement with an explicit, untouched PlanningSubjectDemand row. A purely legacy-fallback requirement - resolved only from the Subject catalog's weekly_hours, with no explicit row at all - had no removal action, and the button was simply absent with no explanation. Reported live example: for one teacher assigned to three subjects in the same section, only the subject with an explicit row showed "Remove demand"; the other two, resolved by fallback, showed nothing. The condition controlling visibility was never about the teacher - it was whether an explicit PlanningSubjectDemand row existed at all for that section+subject pair, and if so, whether Curriculum Adjustment had ever touched it.

_get_planning_requirement_removal_status (routers/planning.py) now classifies every requirement using resolve_section_subject_demands - the exact same explicit-first/legacy-fallback authority already used to render the page, so classification can never diverge from what is displayed:

1 removable: an explicit row exists and updated_by_user_id IS NULL

(never touched by Curriculum Adjustment) - deletable.

1 permanent: an explicit row exists and has been touched - genuine

history, shown as "Protected (Curriculum Adjustment history)" instead of a hidden or missing action.

1 fallback: no explicit row exists; the requirement is resolved only from

the Subject catalog - now removable by creating an explicit is_active=False/weekly_periods=0 suppression row. Unlike a Curriculum Adjustment zero-out retirement, this row is stamped with only the acting admin's created_by_user_id (audit trail); updated_by_user_id is left NULL. Because updated_by_user_id IS NULL is exactly the signal every removability check in this module treats as "never touched by Curriculum Adjustment", the suppression row stays classified removable/setup-only instead of becoming permanent - see the correction below.

1 not_found: no active requirement for that pair at all.

POST /planning/subject-requirements/remove handles both the single quick action ("Remove Subject Requirement", one target) and checkbox-selected bulk removal ("Bulk Remove Subject Requirements"). Targets are "<planning_section_id>:<subject_code>" strings; checkboxes use the form="planningBulkRemoveForm" attribute so selection works across every section rendered on the page in a single request, not only within one section - achievable without a frontend redesign since the whole Planning table is already server-rendered on one page load. Every target is scope-checked (section and subject must belong to the caller's branch/ academic year) and classified before any mutation; if any target is permanent, not_found, unparseable, or out of scope, nothing in the batch is removed and every blocked target is named with its specific reason, matching the same all-or-nothing contract established for Subject bulk delete. On success, a single-section batch reopens that section via the existing return_to/open_section_id reopen-on-load pattern; a multi-section batch returns to the plain Planning list, since there is no single section to reopen. Confirmation dialogs precede both single and bulk removal, naming the subject/count being removed.

TeacherSectionAssignment, TimetableEntry, and CurriculumAdjustmentAudit rows are never touched by this action - verified directly against a teacher assigned to all three example subjects (one explicit, two fallback), where the assignment rows remained intact after removing both fallback requirements. The prior round's GET /planning/subject-demand/delete/ {demand_id} route remains fully functional; it is no longer linked from the rendered page for cases the new route already covers, but it is still the supported way to fully clear a leftover setup-only suppression row (see correction below) so that Planning section/Subject deletion can proceed.

No schema or migration change: the suppression row reuses the existing `PlanningSubjectDemand` table and the previously-unused `created_by_user_id/updated_by_user_id` columns. Subjects and Teachers pages were not changed. A teacher-based quick-select filter for bulk selection was considered but left out as an unnecessary frontend addition for this round; the admin selects the relevant checkboxes directly.

Correction (2026-09-04): setup-only suppression must not look like Curriculum Adjustment history

The first version of this change stamped a fallback suppression row with `created_by_user_id/updated_by_user_id` both set to the acting admin - mirroring a genuine Curriculum Adjustment retirement exactly. Since `updated_by_user_id IS NOT NULL` is the sole signal `_get_planning_requirement_removal_status`, `_get_planning_section_demand_status`, and `routers.subjects._get_subject_demand_status` use to classify a row as permanent/Curriculum-Adjustment-touched, that stamp made the row permanently undeletable the instant a plain admin cleanup created it - an admin removing a fallback requirement to unblock deleting the section could never actually get there, breaking the remove requirement -> remove section -> remove subject workflow this action exists for.

Fix (`_apply_planning_requirement_removal` in `routers/planning.py`): the suppression row now sets `created_by_user_id=<acting admin>` (audit trail of who suppressed it survives) but leaves `updated_by_user_id=None`. No schema change. This does not conflict with any existing invariant: `curriculum_adjustment_apply_service._set_demand` remains the only code path that ever sets `updated_by_user_id`, so `IS NULL` continues to mean exactly "not touched by Curriculum Adjustment" regardless of who created the row or why. A row becomes permanent only when Curriculum Adjustment itself later touches it - never merely by being suppressed from this action.

Verified end-to-end retry flow: a fallback requirement is removed (status becomes `not_found`, a setup-only row now exists) -> the Planning section is still blocked from deletion by that leftover row, but the row itself is still removable (not permanent) -> an admin clears it via the existing `GET /planning/subject-demand/delete/{demand_id}` route -> the Planning section deletes successfully -> the Subject deletes successfully. A row Curriculum Adjustment has genuinely touched (`updated_by_user_id` set directly) stays permanent throughout and is unaffected by this fix. Bulk removal of several fallback requirements in one request produces only setup-only rows (`updated_by_user_id IS NULL` on every one). No customer-facing wording changed: "Remove Subject Requirement" / "Bulk Remove Subject Requirements" / "Protected (Curriculum Adjustment history)" are unchanged, and no new lifecycle terminology was introduced - an admin never needs to know a "suppression row" exists.

Bulk-action UI completion (2026-09-04)

The first bulk UI exposed row checkboxes but placed its only submit button below the complete Planning table and provided no Select All control. Each expanded section now presents a section-scoped **Select all** checkbox beside the action it drives. The action uses the shared trash icon and reflects the live selection in its visible label: "Remove Subject Requirement" for one or "Remove N Subject Requirements" for several. Partial section selection gives the Select All control an indeterminate state, and removal controls are disabled with no selection. The existing cross-section form, atomic endpoint, permission boundary, confirmation, and persistence behavior are unchanged.

Empty-section deletion correction (2026-09-04)

After removing every fallback requirement, the visible section was empty but its inactive zero-period setup suppression rows still triggered the generic Planning-demand blocker. Section deletion now excludes only rows matching all safe artifact conditions (`is_active=False`, `weekly_periods=0`, and `updated_by_user_id IS NULL`) from the blocker check, then deletes those exact branch/year-scoped rows in the same transaction as the otherwise-allowed section delete. Cleanup does not run when another blocker exists. Active setup requirements, Curriculum Adjustment history, teacher assignments, timetable and calendar references, and scheduling rules remain protected. This is a narrow artifact cleanup, not cascade deletion.

Workforce Planning History

docs/history/workforce-planning/README.md

- 1 [2026-09-04 Planning subject requirement removal (single and bulk)](2026-09-04-planning-subject-requirement-removal.md)
- 1 [2026-09-03 return-to reopen-on-load UI pattern](2026-09-03-return-to-reopen-on-load-pattern.md)
- 1 [2026-09-03 Planning and Subject delete dependency guards](2026-09-03-planning-and-subject-delete-dependency-guards.md)
- 1 [2026-09-02 professional timetable lifecycle UX](2026-09-02-professional-timetable-lifecycle-ux.md)
- 1 [2026-09-01 structured timetable conflict evidence](2026-09-01-structured-timetable-conflicts.md)
- 1 [2026-09-01 timetable Lesson Requirement projection](2026-09-01-timetable-lesson-requirement-projection.md)
- 1 [2026-09-01 Advanced Timetable Release 1 reconciliation and solver adapter](2026-09-01-advanced-timetable-r1-reconciliation.md)
- 1 [2026-08-30 teacher scheduling rules](2026-08-30-teacher-scheduling-rules.md)
- 1 [2026-08-29 reduce-only curriculum adjustment](2026-08-29-reduce-only-curriculum-adjustment.md)
- 1 [2026-08-29 curriculum transfer and Subjects display correction](2026-08-29-curriculum-transfer-and-subjects-display-fix.md)
- 1 [2026-08-29 guided curriculum adjustment UI](2026-08-29-guided-curriculum-adjustment-ui.md)

This folder tracks meaningful changes to teacher information, workload planning, staffing gaps, planning sections, timetable relationships, and related reports.

Related files:

- 1 routers/teachers.py
- 1 routers/planning.py
- 1 routers/timetable.py
- 1 teacher_capacity.py
- 1 timetable_logic.py

History entries:

- 1 2026-09-04-planning-subject-requirement-removal.md - "Remove Subject

Requirement" and checkbox-selected, cross-section "Bulk Remove Subject Requirements" now cover legacy-fallback requirements (no explicit demand row) in addition to untouched explicit rows, fixing the inconsistency where the same teacher's subjects could differ in removability for reasons unrelated to the teacher; protected/Curriculum-Adjustment-touched rows show an explicit explanation instead of a hidden action. Same-day correction: the suppression row created for a fallback requirement sets only `created_by_user_id`, not `updated_by_user_id`, so it stays removable instead of instantly becoming permanent, preserving the remove requirement -> remove section -> remove subject workflow.

- 1 2026-09-03-return-to-reopen-on-load-pattern.md - a shared `redirect_utils.py`

`open-redirect` guard plus `static/js/reopen-on-load.js` finish wiring the existing `return_to/fragment` convention so a Planning section stays open and scrolled into view after "Remove demand" and related in-place actions, without converting TIS into an SPA.

1 2026-09-03-planning-and-subject-delete-dependency-guards.md - read-only

dependency checks before Planning section and Subject deletion, with every blocking category named; untouched setup-only Planning demand is now actually removable through a new narrow "Remove demand" action, while demand Curriculum Adjustment has touched and published timetable data stay honestly permanent instead of promising a fix that cannot work.

1 2026-08-29-curriculum-adjustment-apply.md - permissioned fingerprint guard,

atomic apply/audit, explicit teacher decisions, and Draft invalidation.

1 2026-08-29-curriculum-adjustment-preview.md - read-only scoped transfer preview,

teacher/capacity options, rule/grouped warnings, Draft impact, and stale guard.

1 2026-08-28-planning-subject-demand-consumers.md - live Planning, workload,

timetable, scheduling-rule, and required-hours report consumers adopt explicit per-section demand authority with missing-row legacy fallback.

1 2026-08-28-explicit-planning-subject-demand-foundation.md - normalized future

section-subject demand, safe backfill, scoped integrity, retirement state, and transitional legacy compatibility.

1 2026-08-21-smart-timetable-stage-4-version-publication.md - visible version history, draft locks/validation, comparison, explicit export, archive, and atomic publication.

1 2026-08-21-smart-timetable-stage-35-composed-timeline.md - automatic per-day teaching/block composition, calculated end time, explicit placement modes, and shared slot authority.

1 2026-08-19-smart-timetable-stage-3-readiness-and-slot-semantics.md - canonical fixed-period slot semantics, assignment protection, and solver-independent readiness.

1 2026-08-19-smart-timetable-stage-2-version-foundation.md - durable timetable versions, snapshots, active pointer, locks, migration, and legacy edit/export compatibility.

Workspace Classification History

docs/history/workspace-classification/README.md

This folder tracks the evolution of workspace identity, classification, lifecycle, intent, and later approved commercial separation rules.

2026-07-22 - M8B-1 Foundation

Added stable Workspace UUID, constrained Workspace Classification and Lifecycle metadata, onboarding Workspace Intent, SaaS Account Purpose, and internal-test identity attribution. Added validation-only services, a commercial-state skeleton, Platform Owner read-only display, a relationship diagnostic, and a controlled one-time backfill for confirmed existing test data.

M8B-1 deliberately leaves demo workflows, expiration, entitlements, branch entitlements, conversions, memberships, AI-Andalus migration, and commercial-state decisions unimplemented.

2026-07-22 - M8B-2 Commercial Resolution Foundation

Added normalized workspace entitlement envelopes, typed feature/limit values using the existing entitlement catalog, optional branch inherit/active/inactive records, and read-only effective workspace, branch, and commercial-state resolvers. Paid workspaces continue to depend on M7 confirmed local subscription evidence. No customer enforcement, demo lifecycle, conversion, membership, role, Paddle, onboarding, or AI-Andalus behavior changed.

Related docs:

- 1 docs/adr/0008-workspace-classification-foundation.md
- 1 docs/adr/0009-commercial-state-and-entitlement-resolution.md
- 1 docs/TIS_MASTER_CONTEXT.md
- 1 docs/engineering/DATABASE_ARCHITECTURE_OVERVIEW.md

TIS Landing Page Source of Truth

docs/marketing/landing_page_source_of_truth.md

Landing Page Source of Truth

Starter, Professional, and Enterprise AI pricing presentation is capacity-based. Normal customer feature availability is common across the three plans; AI consumption allowances may be configured separately. Public copy must not imply that ordinary modules or enabled AI availability require a higher plan.

The public header and relevant conversion areas expose Request a Demo, Subscribe, Sign In, and Open TIS App. All app links are constructed here from `NEXT_PUBLIC_TIS_APP_BASE_URL`; Sign In targets `/saas/login`, Open TIS App targets `/login`, and the landing site does not duplicate authentication or customer-state routing.

The hero Subscribe Now action remains within the marketing page and scrolls to `#pricing`. Starter, Professional, and Enterprise AI pricing actions use the configured application base URL and public `/saas/signup` route with distinct allowlisted preferred-plan codes. The application treats those codes as non-authoritative preferences and revalidates capacity after organization setup. Custom remains an email/contact action and never enters signup or checkout.

The official source of truth for the public TIS landing website is:

`tis-landing-website/`

Public Website

- 1 **Domain:** `https://tisplatform.com`
- 1 **Runtime:** Next.js / Node
- 1 **Local testing URL:** `http://localhost:3000`

All future marketing landing page changes must be made inside `tis-landing-website/`.

Application Portal

The TIS application portal remains separate from the public landing website:

- 1 **Domain:** `https://app.tisplatform.com`
- 1 **Runtime:** FastAPI / Python with PostgreSQL

Legacy Landing Page Files

The former FastAPI/Jinja landing page files are now legacy:

- 1 `templates/landing.html`
- 1 `static/landing/landing.css`

Codex and other developers must not modify these legacy files unless explicitly instructed.

TIS Landing Page Master Content

docs/marketing/tis_landing_page_master_content.md

TIS Landing Page Master Content Approved Marketing Foundation

1 Main Positioning

TIS is a developing SaaS academic operations platform designed to connect school leadership, supervisors, teachers, branches, calendars, observations, staffing data, and decision-making inside one integrated system. The platform helps schools organize teacher information, analyze teaching loads, identify staffing gaps, coordinate shared academic calendars, manage structured observation cycles, support supervisor-teacher communication, and build clearer visibility across academic operations. As TIS continues to grow, advanced AI-powered features will be introduced through subscription-based plans, including curriculum-driven assessment generation, academic analytics, planning support, and intelligent recommendations based on verified school data.

1 Hero Section

Hero Badge Built for Schools Moving Beyond Spreadsheets Main Headline When Academic Decisions Are Scattered, Schools Lose Time, Clarity, and Control. Subheadline TIS brings teacher data, workloads, staffing gaps, academic calendars, observations, and leadership decisions into one connected SaaS platform - helping schools see what is happening, what is missing, and what needs action. CTA Buttons Request Early Access See How TIS Works Small Supporting Line A developing academic operations platform with advanced AI capabilities planned through subscription-based growth.

1 Problem Section

Section Label The Problem Main Heading Schools Are Not Short of Data. They Are Drowning in Disconnected Files. Section Text In many schools and educational organizations, every academic process becomes another spreadsheet. Teacher loads, subject coverage, observations, calendars, reports, staffing needs, and follow-up tasks are often managed in separate files, created by different people, in different formats. Over time, this creates a hidden operational burden. Staff members spend more time updating files than understanding the data. Leaders receive information late, inconsistently, or from multiple disconnected sources. In multi-branch organizations, the problem becomes even larger: every branch may work differently, every file may tell a different story, and every decision requires extra checking. Problem Cards

1 Too Many Spreadsheets

Academic work is scattered across different files, formats, and personal tracking systems.

1 Inconsistent Data

Each person or branch may structure information differently, making comparison and reporting difficult.

1 Heavy Staff Workload

Teachers, supervisors, and coordinators carry the burden of repeatedly entering and updating data.

1 Delayed Visibility

Leaders often discover staffing gaps, missing coverage, or academic issues after the problem has already grown.

1 Disconnected Decisions

Calendars, observations, staffing, reports, and academic planning are managed separately instead of working as one connected system.

1 Solution Section

Section Label The Solution Main Heading One Connected Platform to Bring Academic Operations Back Under Control.
Section Text TIS brings the scattered pieces of school operations into one structured academic ecosystem. Instead of chasing separate spreadsheets, comparing different file formats, and waiting for updates from multiple sources, school leaders can work from a clearer, more connected view of what is happening across their school or organization. Teacher data, workloads, subject coverage, academic calendars, observations, supervision follow-up, branch structures, and leadership decisions become part of one system. This allows schools to reduce manual confusion, improve visibility, standardize academic processes, and make decisions with greater confidence. For multi-branch organizations, TIS helps create consistency across campuses while still allowing each branch to manage its own academic operations within a secure, organized, and tenant-isolated platform. **Solution Cards**

1 One Source of Academic Truth

Bring teacher data, workloads, observations, calendars, and staffing information into one connected system.

1 Clearer Leadership Visibility

Help principals, academic leaders, and supervisors see what is happening, what is missing, and what requires action.

1 Standardized Academic Processes

Reduce personal spreadsheet formats and create a more consistent way to manage academic operations across branches.

1 Smarter Staffing Decisions

Identify teaching load issues, uncovered hours, subject gaps, and staffing needs before they become larger problems.

1 Connected School Communication

Support better coordination between leadership, supervisors, teachers, and branches through shared academic workflows.

1 Ready for Future Intelligence

Build the structured data foundation needed for advanced AI-powered features as the platform continues to grow.

1 Core Platform Capabilities

Section Label Core Platform Capabilities Main Heading Everything Your Academic Operation Needs - Connected in One Platform. **Section Intro** TIS is designed to bring the most important academic operations into one structured SaaS environment. From teacher planning and branch management to observations, calendars, dashboards, and future AI-powered intelligence, the platform helps schools move from scattered work to connected academic control. **Capability Cards**

1 Academic Workforce Planning

Plan teacher assignments, analyze weekly loads, identify uncovered teaching hours, and understand staffing needs with greater clarity.

1 Academic Calendars & Shared Coordination

Coordinate academic events, timelines, school priorities, and branch-level planning through shared calendars that keep teams aligned.

1 Observation & Supervision Workflows

Manage structured teacher observations, supervisor feedback, shared records, and signing workflows inside one organized process.

1 Multi-Branch Organization Management

Support school groups, campuses, branches, users, roles, and tenant-isolated structures from one scalable SaaS platform.

1 Dashboards & Decision Visibility

Give leaders a clearer view of coverage, staffing gaps, teacher utilization, subject sufficiency, and academic follow-up needs.

1 Future AI-Powered Intelligence

Prepare for advanced subscription-based AI capabilities, including curriculum-driven assessment generation, answer keys, planning support, analytics, and intelligent recommendations based on verified academic data.

1 Teacher Workforce Planning Engine

Section Label Teacher Workforce Planning Engine Main Heading Know Exactly Who Is Teaching What - and Where Support Is Needed. Section Text One of the strongest current capabilities of TIS is its ability to help schools understand their teaching workforce with clarity. Instead of manually calculating teacher loads, subject coverage, and staffing needs across separate files, TIS brings this information into one structured planning engine. School leaders can track teacher qualifications, majors, subject specialties, weekly teaching loads, assigned subjects, uncovered hours, and additional staffing requirements. This helps academic teams identify gaps earlier, use existing teachers more effectively, and make staffing decisions based on clear data rather than assumptions. For schools and multi-branch organizations, this creates a more reliable way to plan academic coverage, compare staffing needs, and understand whether each subject, grade, and branch has the right teaching support. **Key Capabilities**

1 Teacher Load Visibility

See teacher workloads clearly and understand whether teaching hours are balanced, underused, or overloaded.

1 Subject Coverage Analysis

Identify which subjects are fully covered and which still have uncovered teaching hours.

1 Staffing Gap Detection

Detect shortages early before they affect schedules, instruction, or academic quality.

1 Qualification-Based Planning

Use teacher degrees, majors, and subject specialties to support smarter assignment decisions.

1 New Teacher Requirement Insights

Understand how many additional teachers may be needed based on uncovered hours and school workload requirements.

1 Multi-Branch Workforce Clarity

Support better staffing visibility across branches, campuses, and larger educational organizations.

1 Observation & Supervision Workflows

Section Label Observation & Supervision Workflows Main Heading Turn Teacher Observation into a Clear, Shared, and Accountable Process. Section Text TIS is designed to help schools move teacher observation away from scattered forms, disconnected files, and informal follow-up. Observation records, supervisor feedback, teacher responses, signatures, and approval steps can become part of one structured academic workflow. This gives supervisors a clearer way to document classroom visits, teachers a more transparent way to receive and respond to feedback, and school leaders a stronger view of supervision quality across departments and branches. As the platform continues to grow, advanced AI-assisted supervision features are planned to support supervisors by suggesting improvement plans based on observation notes, evaluation results, teacher performance evidence, and school-approved criteria. Key Capabilities

1 Structured Observation Records

Create organized observation forms and records that follow a clear school-approved process.

1 Shared Teacher-Supervisor Visibility

Allow teachers and supervisors to access relevant observation records, feedback, and follow-up actions.

1 Signing and Approval Workflow

Support formal documentation through review, acknowledgement, signing, and approval steps.

1 Supervision Follow-Up

Track feedback, recommendations, and improvement actions after classroom observations.

1 AI-Suggested Improvement Plans

Future AI-assisted tools may help supervisors generate suggested teacher improvement plans based on observation notes, evaluation data, and approved academic criteria.

1 Leadership Oversight

Give academic leaders better visibility over observation activity, supervision quality, and follow-up completion.

1 Reduced Observation Paperwork

Replace scattered observation files with a more consistent, traceable, and connected supervision system.

1 Academic Calendars & Shared Coordination

Section Label Academic Calendars & Shared Coordination Main Heading Turn the Academic Calendar into a Shared Planning Engine. Section Text TIS is designed to help schools move beyond static calendars and disconnected event planning. Academic leaders, supervisors, teachers, and branches can work from a shared academic calendar connected to the selected academic year, helping the whole school coordinate priorities, events, activities, observations, deadlines, and academic follow-up. Future calendar capabilities can support default worldwide and international academic events, allowing schools to plan meaningful activities around important global occasions. This gives international, national, private, and public schools a stronger way to connect school events with classroom learning, curriculum objectives, student engagement, and whole-school awareness. As TIS continues to grow,

AI-assisted calendar planning can help schools generate suggested activities for selected events, including classroom activities, outdoor activities, entertainment activities, curriculum-integrated tasks, and lesson-based ideas. The calendar vision also includes support for Sustainable Development Goals awareness planning, helping schools design year-round activities connected to SDGs, global citizenship, and curriculum integration. Key Capabilities

1 Academic-Year-Based Calendar

Build and manage school calendars according to the selected academic year, such as 2026-2027.

1 Shared School Coordination

Connect leaders, supervisors, teachers, and branches through one shared academic planning calendar.

1 International Event Awareness

Support worldwide and international academic events that schools can use for activities, campaigns, and curriculum connections.

1 AI-Suggested Activity Planning

Future AI-assisted tools may suggest activity plans based on selected events, school needs, grade levels, and curriculum links.

1 Curriculum-Integrated Events

Help schools connect calendar events with classroom learning, lesson planning, and academic objectives.

1 SDG Awareness Planning

Support year-round planning for Sustainable Development Goals awareness, student engagement, and global citizenship activities.

1 Branch-Level Visibility

Give school groups and multi-branch organizations a clearer view of academic events, priorities, and planning across campuses.

1 Multi-Branch Organization Management

Section Label Multi-Branch Organization Management Main Heading Built for Schools That Operate Across Branches, Campuses, and Teams. Section Text TIS is designed to support schools and educational organizations that need structure beyond a single location. Whether an organization manages one school, several branches, multiple campuses, or different academic pathways, TIS provides a scalable environment where users, roles, branches, and academic operations can be organized with greater clarity. School leadership and organization administrators can manage branches, users, permissions, branding, academic structures, and operational workflows inside a tenant-isolated SaaS environment. This helps every branch work within a shared organizational framework while still maintaining its own operational identity and academic needs. For educational groups, TIS can support stronger management-level visibility across branches. Leaders can monitor branch progress, compare completion levels, identify delayed tasks, and recognize where follow-up is needed across areas such as curriculum planning, exams, observations, calendars, reporting, and academic operations. Key Capabilities

1 Organization and Branch Structure

Manage organizations, schools, branches, campuses, and academic pathways inside one scalable platform.

1 Role-Based User Management

Assign users according to their responsibilities, such as organization administrator, branch leader, academic leader, supervisor, teacher, or future custom roles.

1 Tenant-Isolated Environment

Keep each organization's data, users, branding, and academic operations securely separated.

1 Branch Progress Intelligence

Give management-level users a clearer view of each branch's completion, progress, delays, and operational follow-up needs.

1 Branch Comparison Indicators

Compare branch performance across connected academic areas such as curriculum tasks, exams, observations, calendars, planning, and reports.

1 Custom Branding by Organization

Support organization and branch branding, including logos, visual identity, and school-specific presentation.

1 Scalable SaaS Structure

Prepare schools and educational groups for growth, future modules, subscription plans, and advanced AI-powered capabilities.

1 Dashboards & Decision Visibility

Section Label Dashboards & Decision Visibility Main Heading See Academic Progress, Risks, and Priorities Before They Become Problems. Section Text TIS dashboards are designed to give school leaders more than static reports. They provide a connected view of academic operations, helping leadership understand what has been completed, what is delayed, what needs follow-up, and where intervention may be required. As the platform continues to grow, dashboards can reflect progress across annual plans, weekly plans, curriculum coverage, assessment blueprints, quizzes, exams, academic calendar deadlines, teacher observations, supervision follow-up, staffing gaps, and branch-level performance. For branch management, dashboards can show the academic health of one school or campus. For organization-level leadership, dashboards can compare branches, highlight delayed tasks, identify performance gaps, and support faster, evidence-based decisions. Key Capabilities

1 Planning Completion Indicators

Track progress for annual plans, weekly plans, lesson planning, curriculum coverage, and completed academic tasks.

1 Assessment Progress Visibility

Monitor assessment blueprints, quizzes, exams, term-based assessment requirements, and calendar-linked due dates.

1 Observation & Supervision Insights

See observation progress, teacher follow-up status, supervisor actions, and areas requiring improvement plans.

1 Academic Calendar Integration

Connect events, deadlines, assessments, activities, and required tasks to dashboard indicators.

1 Teacher & Academic Performance Indicators

Support leadership with visual indicators related to teacher follow-up, academic progress, and school-level performance trends.

1 Branch and Organization Dashboards

Provide branch-level dashboards for school management and organization-level dashboards for comparing progress across multiple campuses.

1 Risk and Delay Alerts

Highlight delayed tasks, incomplete requirements, missing academic evidence, and branches or departments that need attention. Supporting Line: From reports to real-time academic visibility.

1 Future AI-Powered Intelligence

Section Label Future AI-Powered Intelligence Main Heading AI That Works Inside the Academic Workflow - Not Outside It. Section Text TIS is being developed with a future AI vision that goes beyond simple content generation. Instead of using AI as a separate tool, future AI-powered capabilities are planned to work inside the platform's academic workflows, supporting schools through the data already stored, reviewed, and approved in TIS. This future intelligence layer can support academic calendars, observation follow-up, curriculum planning, teacher support, assessment generation, dashboards, branch progress indicators, and leadership decision-making. At the center of this vision is a curriculum-to-classroom workflow. Uploaded curriculum data, standards, learning outcomes, annual plans, weekly plans, lesson plans, and confirmed instructional coverage can become the foundation for guided planning, AI-assisted recommendations, assessment generation, answer keys, and academic analytics. Future AI capabilities will be introduced progressively across the normal TIS customer product, subject to permissions, workspace scope, configured consumption allowances, and release readiness. Key Capabilities

1 Curriculum-to-Classroom AI Support

Assist teachers and academic leaders in moving from uploaded curriculum to annual plans, weekly plans, lesson plans, and verified coverage.

1 AI-Assisted Assessment Generation

Support future generation of exams, quizzes, assessment blueprints, and answer keys based on approved curriculum data and confirmed instructional coverage.

1 AI-Supported Observation Follow-Up

Help supervisors generate suggested teacher improvement plans based on observation notes, evaluation results, and school-approved criteria.

1 AI-Powered Calendar Planning

Suggest activities for international events, SDG awareness, curriculum links, classroom activities, and school-wide academic events.

1 Academic Analytics and Recommendations

Provide future intelligent insights related to planning progress, teacher follow-up, assessment readiness, curriculum coverage, and branch performance.

1 Managed AI Growth

Introduce advanced AI features progressively while controlling provider consumption, safety, and release readiness independently from plan capacity.

1 From Curriculum to Classroom - With Guided AI Support

Section Label From Curriculum to Classroom Main Heading Support Teachers Without Adding Another Burden. Section Text TIS is being developed to support teachers, not overload them. Instead of asking teachers to build annual plans, weekly plans, lesson plans, and curriculum tracking documents from scratch, future AI-assisted planning tools will help guide them through structured steps using approved templates, uploaded curriculum data, learning outcomes, standards, objectives, and school-specific requirements. Teachers will be able to review, adjust, complete, and submit their plans through the system. Academic leaders can then review and approve the work, turning planning documents into reliable, official academic data inside TIS. This helps reduce teacher workload, improve planning consistency, and give school leadership clearer visibility over what has been planned, approved, taught, and covered.

1 Subscription Plans Preview

Section Label Subscription Plans Main Heading Choose the Organization Capacity Your School Needs. Section Text TIS is a subscription-based organization SaaS platform. Starter, Professional, and Enterprise AI share the normal customer product baseline and primarily represent different organization capacities. Permissions, workspace scope, operational prerequisites, and separately configured AI consumption policy continue to control safe use. Plan Cards

1 Starter

For organizations within the Starter branch, staff-user, and teacher capacity ceilings.

1 Professional

For organizations that require the Professional branch, staff-user, and teacher capacity ceilings.

1 Enterprise AI

For larger organizations within the Enterprise AI branch, staff-user, and teacher capacity ceilings. Supporting Note AI-powered capabilities will be introduced progressively and remain subject to permissions, workspace scope, configured usage allowances, and platform development stage. CTA Request Early Access Request Pricing

1 Final Call-to-Action

Section Label Start Your TIS Journey Main Heading Ready to Bring Clarity, Control, and Intelligence to Your Academic Operations? Section Text TIS is being built for schools and educational organizations that want to move beyond scattered files, disconnected workflows, and delayed decisions. Start exploring how one connected academic operations platform can support your leadership team, supervisors, teachers, branches, and future AI-powered growth. CTA Buttons Request Early Access Book a Demo Supporting Line TIS is currently in active development, with selected features and advanced AI capabilities planned for progressive release through subscription-based plans.

TIS Location Data Roadmap

docs/location-data-roadmap.md

Phase 1: Global Form Usability

Status: implemented in the current working tree.

Phase 1 intentionally keeps location data attached directly to an organization or branch. It does not create shared locality records.

Included:

- 1 Controlled country selection from the local dataset.
- 1 Controlled region/state/province selection when dataset coverage exists.
- 1 Dataset-backed city/locality selection.
- 1 Other / manual entry fallback for missing regions and cities.
- 1 Optional district and neighborhood fields, stored separately from city.
- 1 Local cached APIs; no external location API dependency at runtime.
- 1 Existing text location fields remain compatible with legacy Saudi data.

Manual region and city values are record-level values. They are not published to other tenants and are not added to the imported dataset.

Deferred Architecture

The following phases are roadmap items only. They are not part of Phase 1 and must not be inferred from the current text-based storage model.

Locality Registry

Create stable TIS locality identifiers and canonical locality records that are independent from vendor dataset identifiers. Organizations and branches would eventually reference these records while retaining display-name snapshots for audit and migration compatibility.

Locality Aliases

Support native names, translated names, transliterations, spelling variants, and historical names. Alias matching should be region-aware and must not merge places solely because their normalized names match.

GeoNames and Official Dataset Enrichment

Build an offline enrichment process using GeoNames country dumps and, where available, official national gazetteers. Source priority, licensing, attribution, place classification, and duplicate resolution must be defined before combining records.

Tenant-Private City Additions

Allow an authorized tenant user to suggest a missing locality. New records should initially be visible only to that tenant and must be constrained to the tenant's organization scope.

Platform Moderation

Provide a Platform Owner moderation queue for reviewing proposed localities, potential duplicates, coordinates, place type, aliases, and source evidence.

City Verification and Promotion

Support explicit states such as tenant-private, pending, verified, deprecated, and merged. Promotion to the global catalog must be an audited platform action, never an automatic consequence of a tenant entering free text.

Dataset Refresh and ETL Workflow

Introduce versioned, offline imports with source checksums, staging tables, coverage metrics, diff review, attribution records, and rollback support. Referenced localities should be deprecated or merged rather than deleted.

Guardrails for Future Work

- 1 Preserve tenant isolation for all tenant-created locality data.
- 1 Keep stable internal IDs separate from external source IDs.
- 1 Retain source and license provenance for every imported record.
- 1 Do not call public geocoding services from normal page requests.
- 1 Do not modify the vendor JSON to store tenant-entered values.
- 1 Migrate existing organization and branch text values without data loss.